Realized Volatility
Estimation, Forecasting, and ML

A Learning Guide

Ryan Vincent

Last compiled: May 14, 2026

Contents

Part I

What Is Volatility and How Do You Measure It?

Chapter 1

Returns, Variance, and Why Volatility Matters

Why This Chapter

Volatility is the central quantity in quantitative finance. Options pricing, risk management, portfolio construction, and trade execution all depend on accurate volatility estimates. Every chapter in this guide builds on the concepts introduced here. All five project directions (Chapter 18) start from the returns and variance foundations covered in this chapter.

Financial markets produce a stream of prices. To do anything quantitative with those prices, you first need to convert them into *returns*, measure how much those returns vary, and understand *why* that variation matters. This chapter covers that ground.

1.1 What Are Returns?

A stock price on its own tells you almost nothing about performance. Knowing that Apple closed at \$187 today is useless without knowing where it closed yesterday. *Returns* solve this: they express price changes as fractions (or percentages), making different assets and time periods comparable.

To compare price movements across different assets and time periods, we need to express changes as fractions rather than absolute dollar amounts. There are two standard ways to do this.

Natural Logarithm Properties

The natural logarithm $\ln(\cdot)$ is the inverse of the exponential function: $\ln(e^x) = x$. Three properties matter here:

- $\ln(A/B) = \ln A - \ln B$ (quotients become differences)
- $\ln(AB) = \ln A + \ln B$ (products become sums)
- For small x: $\ln(1+x) \approx x$ (so log returns $\approx$ simple returns when returns are small)

1.1.1 Simple Returns

We want a single number that tells us "what fraction of my investment did I gain or lose in one period?" The simple (arithmetic) return does exactly this:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} \tag{1.1}$$

- R_t: simple return from period $t-1$ to t
- P_t: price at the end of period t
- P_{t-1}: price at the end of the previous period

In Plain English

This equation says: "take the price change (today minus yesterday) and divide by yesterday's price to get the fractional change." The result is a dimensionless number: 0.02 means a 2% gain, -0.03 means a 3% loss. It's the same calculation as "what percentage did my investment grow?"

Why This Matters

Returns are the raw input to everything in this project. Realized volatility is computed by summing *squared* returns, so every RV estimate you build in later chapters starts from this quantity. If you don't understand what a return measures, the entire volatility pipeline won't make sense.

A simple return of 0.05 means the price rose by 5%.

1.1.2 Log Returns

The simple return works fine in many contexts, but for volatility research we almost always use a different version: the log return. Instead of computing the fractional change directly, we take the natural logarithm of the price ratio:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln P_t - \ln P_{t-1} \tag{1.2}$$

- r_t: log return from period $t-1$ to t
- $\ln(\cdot)$: natural logarithm

In Plain English

This equation says: "take the ratio of today's price to yesterday's price, then take the natural log of that ratio." For small price changes (under 5%), the log return is almost identical to the simple return. The reason we use it instead is purely mathematical convenience: log returns add up over time, which makes the math for volatility much cleaner.

Why This Matters

Every volatility model in this guide uses log returns as its input. When you compute realized volatility in Chapter 2, you will square log returns and sum them. The time-additivity property below is what makes realized volatility estimators mathematically valid.

Why Log Returns?

Log returns have two properties that make them the default in volatility research:

1. **Time additivity.** The multi-period log return is the sum of single-period log re-

turns: $r_{1:T} = r_1 + r_2 + \cdots + r_T$.
Why this works: the 2-day log return is $\ln(P_2/P_0) = \ln\big((P_1/P_0) \cdot (P_2/P_1)\big) = \ln(P_1/P_0) + \ln(P_2/P_1) = r_1 + r_2$. The log property $\ln(AB) = \ln A + \ln B$ does all the work. Simple returns compound multiplicatively instead: the 2-day simple return is $(1 + R_1)(1 + R_2) - 1$, not $R_1 + R_2$, which makes variance formulas much more complicated.

2. **Approximate symmetry.** If you gain $+0.05$ log return and then lose -0.05 log return, the total is $\ln(P_{\text{final}}/P_{\text{start}}) = 0.05 + (-0.05) = 0$, meaning $P_{\text{final}} = P_{\text{start}}$ exactly. Simple returns don't cancel this way: $\$100 \times 1.05 \times 0.95 = \$99.75 \neq \$100$.

Throughout this guide, r_t always means the log return unless stated otherwise.

Worked Example:

Computing Returns A stock closes at \$100.00 on Monday, \$105.00 on Tuesday, and \$102.00 on Wednesday.

Tuesday (price rises from \$100 to \$105):

$$R_{\text{Tue}} = \frac{105 - 100}{100} = 0.0500 \quad (+5.00\%)$$

$$r_{\text{Tue}} = \ln\left(\frac{105}{100}\right) = \ln(1.05) = 0.0488 \quad (+4.88\%)$$

Wednesday (price falls from \$105 to \$102):

$$R_{\text{Wed}} = \frac{102 - 105}{105} = -0.0286 \quad (-2.86\%)$$

$$r_{\text{Wed}} = \ln\left(\frac{102}{105}\right) = \ln(0.9714) = -0.0290 \quad (-2.90\%)$$

Two-day log return (time additivity in action):

$$r_{\text{Mon}\to\text{Wed}} = r_{\text{Tue}} + r_{\text{Wed}} = 0.0488 + (-0.0290) = 0.0198$$

Check directly: $\ln(102/100) = \ln(1.02) = 0.0198$. ✓
The simple returns are close to the log returns because the magnitudes are small (under 5%). For daily equity returns, which are typically well under 1%, the two are nearly identical.

1.2 Variance and Standard Deviation

With returns in hand, the next question is: how spread out are they? Variance answers this by measuring the average squared deviation from the mean return.

Expected Value Notation

Throughout this guide, $\mathbb{E}[X]$ means the **expected value** of X: the long-run average you would get if you could observe X infinitely many times. For example, $\mathbb{E}[r_t]$ is the average return you'd observe over an infinite number of trading days. When you see $\mathbb{E}[\cdot]$ in a formula, read it as "the average of what's inside the brackets."

Sample Variance and Standard Deviation

Given T log returns $r_1, r_2, \ldots, r_T$ with sample mean $\bar{r} = \frac{1}{T}\sum_{t=1}^{T} r_t$, we want a single number that captures "how spread out are these returns?" The sample variance answers this by measuring the average squared distance of each return from the mean:

$$\hat{\sigma}^2 = \frac{1}{T-1}\sum_{t=1}^{T}\left(r_t - \bar{r}\right)^2 \tag{1.3}$$

- $\hat{\sigma}^2$: sample variance (the hat denotes an estimate)
- T: number of observations
- r_t: log return at time t
- $\bar{r}$: sample mean of returns
- $T-1$: Bessel's correction (dividing by $T-1$ instead of T gives an unbiased estimate)

In Plain English

This equation says: "for each return, measure how far it is from the average return, square that distance (so negatives don't cancel positives), then average all those squared distances." A large variance means returns are spread far from the mean (volatile); a small variance means they cluster tightly around the mean (calm). Squaring is what makes this sensitive to outliers: one large move contributes disproportionately.

Why This Matters

Variance is the quantity you are forecasting in this entire project. Realized volatility (Chapter 2) is essentially this formula applied to high-frequency returns within a single day. Every model from GARCH to deep learning is trying to predict tomorrow's version of this number.

The sample standard deviation is $\hat{\sigma} = \sqrt{\hat{\sigma}^2}$.

Why $T-1$?

Dividing by $T-1$ instead of T corrects for the fact that you estimated $\bar{r}$ from the same data. Using $\bar{r}$ instead of the true mean μ systematically shrinks the deviations, so dividing by the smaller number $T-1$ compensates. This is called Bessel's correction. For large T the difference is negligible.

1.2.1 Annualizing Volatility

Raw daily standard deviations are tiny numbers (around 0.01 for a typical equity index), so practitioners report volatility on an annual scale. The key insight: if daily returns are independent, variances add. The n-day return is $r_1 + r_2 + \cdots + r_n$ (time additivity), and for independent random variables, $\mathrm{Var}(r_1 + r_2 + \cdots + r_n) = \mathrm{Var}(r_1) + \mathrm{Var}(r_2) + \cdots + \mathrm{Var}(r_n) = n\sigma^2$. Taking the square root: the n-day standard deviation is $\sqrt{n}\,\sigma$.

$$\hat{\sigma}_{\text{annual}} = \hat{\sigma}_{\text{daily}} \times \sqrt{252} \tag{1.4}$$

- 252: approximate number of trading days per year in U.S. equity markets
- $\sqrt{252} \approx 15.87$

In Plain English

This equation says: "multiply the daily volatility by about 16 to get the annual volatility." The $\sqrt{252}$ comes from the fact that if daily returns are independent, variances add over time (252 trading days), so standard deviations scale by the square root. A stock with 1% daily volatility has roughly 16% annual volatility.

Why This Matters

Practitioners quote volatility in annualized terms. When someone says "the VIX is at 20," they mean 20% annualized volatility. Your model's output (daily RV forecasts) will need to be converted to this scale for comparison with implied volatility and industry benchmarks.

The Square-Root-of-Time Rule

The $\sqrt{252}$ scaling assumes returns are independent and identically distributed. When volatility clusters (Section 1.4), this assumption fails, and multi-day volatility may be higher or lower than the square-root rule predicts. Chapters 5 and 6 develop models that handle this.

Worked Example:

Annualized Volatility in Context Five consecutive daily S&P 500 log returns give a daily standard deviation of $\hat{\sigma}_{\text{daily}} = 0.50\%$. Annualizing: $\hat{\sigma}_{\text{annual}} = 0.50\% \times \sqrt{252} = 0.50\% \times 15.87 = 7.9\%$.
For context, the S&P 500's long-run annualized volatility is roughly 15–20%, so this five-day window happened to be calm. The key insight: daily returns are tiny numbers (fractions of a percent); the $\sqrt{252}$ multiplier transforms them into the annual percentage figures that practitioners actually discuss.

1.3 Why Volatility Matters

You now know how to compute returns and measure their spread. The next question: why does that spread matter so much? Variance and standard deviation sit at the center of four major problems in finance.

Finance Concepts Preview

The following four applications are covered briefly here to motivate the rest of the guide. You do not need to understand them fully yet; each connects to a later chapter.

Options pricing. An option gives the holder the right (not obligation) to buy or sell an asset at a fixed price by a future date. The value of this right depends critically on how much the underlying price might move, which is volatility. The Black-Scholes model (Black and Scholes, 1973) takes volatility as an *input* and produces an option price as output. If your volatility estimate is wrong, your price is wrong. Chapter 8 explores this connection.

Risk management. Value-at-Risk (VaR) estimates the worst-case loss over a holding period at a given confidence level. In the simplest version, VaR is directly proportional to volatility: if volatility doubles, your risk bound roughly doubles. For a 99% daily VaR, the worst expected

loss is about 2.33 times the daily volatility (e.g., if daily vol is 1%, the worst-case daily loss at 99% confidence is roughly 2.33%).

Portfolio construction. Mean-variance optimization (Markowitz, 1952) and risk-parity strategies both require a covariance matrix as input. Volatility is the diagonal of that matrix. Better volatility forecasts produce better-diversified portfolios.

Trade execution. When a desk needs to buy a large position, it splits the order over time to reduce market impact. The participation rate (fraction of daily volume per time slice) depends on intraday volatility: higher volatility means wider price swings, which changes optimal execution speed.

What Makes Volatility Special

Volatility is one of the few quantities in finance that is (a) directly observable from intraday data (Chapter 2), (b) economically central to pricing, hedging, and risk control, and (c) forecastable with meaningful accuracy (Chapters 5–14). That combination makes it the right place to apply ML carefully.

Preview: What Is Realized Volatility?

The central concept of this guide is **realized volatility** (RV): the sum of squared intraday returns over one day, $\text{RV}_t = \sum_{i=1}^{M} r_{t,i}^2$, where M is the number of intraday observations (e.g., 78 five-minute returns per trading day). Instead of using a model to *estimate* what volatility was, RV directly *measures* it from high-frequency data. Chapter 2 develops this fully; for now, just know that RV gives you a daily number that tells you "how volatile was the market today," and your project's goal is to forecast tomorrow's RV given today's information.

1.4 Stylized Facts of Financial Returns

Before building models, you need to know what patterns actually appear in return data. Cont (2001) catalogued a set of "stylized facts": statistical regularities observed across equities, foreign exchange, and commodities, across decades and geographies. Four of these facts matter most for volatility modeling.

1.4.1 Fact 1: Returns Are Approximately Uncorrelated

Autocorrelation

Autocorrelation at lag k measures how correlated a time series is with itself shifted k periods into the past. A value near $+1$ means "if it was high today, it will tend to be high k days from now"; near 0 means "no relationship"; near -1 means "high today predicts low k days later." When we say "return autocorrelations are zero," we mean knowing today's return tells you nothing about tomorrow's.

Daily return autocorrelations are statistically indistinguishable from zero for lags beyond a few minutes (Cont, 2001). In plain terms: knowing today's return tells you almost nothing about tomorrow's return. This is consistent with market efficiency; if returns were predictable, traders would exploit the pattern until it disappeared.

No Free Lunch in Means

If positive returns reliably followed positive returns, everyone would buy after an up day, driving the price up immediately and eliminating the pattern. Competition among traders keeps return autocorrelations near zero. Volatility, by contrast, is *not* a quantity you can directly trade on the same way, so its autocorrelation can persist.

1.4.2 Fact 2: Volatility Clusters

Although returns themselves are uncorrelated, *squared* returns and *absolute* returns show strong, slowly decaying positive autocorrelation (Cont, 2001; Mandelbrot, 1963). Large moves tend to follow large moves, and calm periods tend to follow calm periods.

Engle (1982) formalized this observation with the ARCH model: the variance of next period's return depends on recent squared returns. Chapter 5 develops this family of models in detail.

Cont (2001), Stylized Fact: Volatility Clustering

The autocorrelation of absolute returns remains significantly positive over lags of several weeks and decays slowly (approximately as a power law with exponent $\beta \in [0.2, 0.4]$), a pattern that is remarkably stable across asset classes and time periods.

What "Power Law Decay" Means

A **power law decay** means the autocorrelation drops as $\text{lag}^{-\beta}$ — much more slowly than exponential decay. With $\beta \approx 0.3$, the autocorrelation at lag 100 is still about $100^{-0.3} \approx 0.25$: volatility 100 days ago is still informative about today's volatility. This "long memory" is why simple models that only look at yesterday fail. The HAR model (Chapter 6) addresses this by explicitly including weekly and monthly vol averages as predictors.

Figure 1.1 shows volatility clustering visually: returns alternate between calm and turbulent stretches.

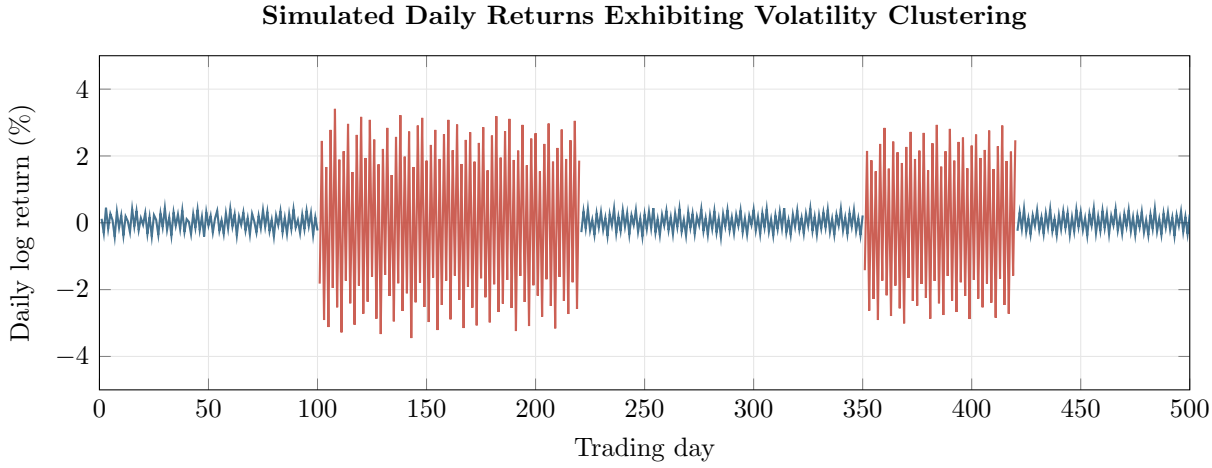


Figure 1.1: Simulated daily log returns exhibiting volatility clustering. Calm periods (blue, days 1–100, 221–350, 421–500) have returns within ±0.5%; turbulent periods (red, days 101–220, 351–420) reach ±3%. The clustering is visible by eye: large returns beget large returns.

Figure 1.2 quantifies this. Return autocorrelations hover near zero at all lags, but squared-return autocorrelations remain positive for many lags, confirming that volatility is persistent.

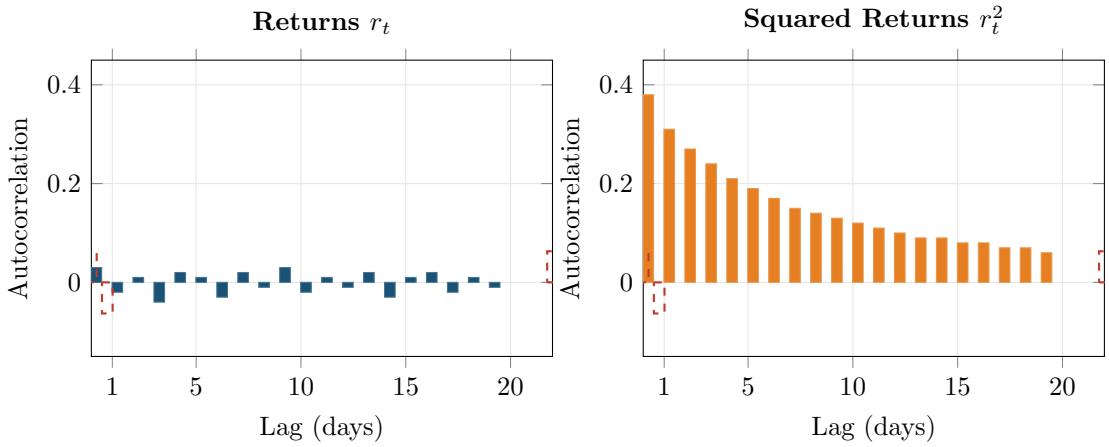


Figure 1.2: Autocorrelation functions for returns (left) and squared returns (right), based on representative daily equity index data. Returns show no significant autocorrelation at any lag (all bars within the dashed 95% significance bands). Squared returns show strong autocorrelation that decays slowly, reflecting volatility clustering.

1.4.3 Fact 3: Fat Tails

If returns were normally distributed, a daily move of 4 standard deviations would occur about once every 126 years. In practice, moves this large happen several times per year. The return distribution has "fat tails" (also called heavy tails): extreme returns are far more frequent than a Gaussian model predicts.

Mandelbrot (1963) first documented this for cotton prices, proposing that returns follow a stable Paretian distribution rather than a Gaussian. Fama (1965) confirmed the finding for stock returns, observing leptokurtic (heavy-tailed) distributions across U.S. equities.

Kurtosis

We need a way to measure "how fat are the tails?" compared to a normal distribution. Kurtosis does this by looking at the fourth power of deviations (which amplifies extreme values even more than squaring):

$$\kappa = \frac{\mathbb{E}\left[(r_t - \mu)^4\right]}{\left(\mathbb{E}\left[(r_t - \mu)^2\right]\right)^2} \tag{1.5}$$

- κ: kurtosis
- $\mu = \mathbb{E}[r_t]$: population mean
- For a normal distribution, $\kappa = 3$
- *Excess kurtosis* $= \kappa - 3$; values above zero indicate fatter tails than normal

In Plain English

This equation says: "take each return's deviation from the mean, raise it to the 4th power (which massively amplifies outliers), average those, and normalize by the squared variance." The 4th power is the key: if extreme returns are rare (thin tails), raising them to the 4th power doesn't add much. If extreme returns are common (fat tails), the 4th power blows them up and the ratio exceeds 3. The further above 3, the fatter the tails.

Why This Matters

Fat tails mean that extreme vol events (crashes, spikes) are far more likely than a normal model predicts. Your vol forecasting model must handle these outliers gracefully: if you assume normality, you will systematically underestimate the risk of large moves, and your model will appear to work in calm markets but fail catastrophically during crises.

Typical daily equity index returns have excess kurtosis in the range 5–10 (Cont, 2001).

Figure 1.3 illustrates fat tails with a **Q–Q (quantile-quantile) plot**. Here is how it works: sort your actual returns from smallest to largest, then ask "if these returns *were* normally distributed, what values would I expect at each position in the sorted list?" Plot the expected-if-normal values on the x-axis and the actual values on the y-axis. If the data is truly normal, expected equals actual at every point, so you get a straight diagonal line. When the dots curve away from the line at the extremes, it means the tails of your actual distribution are fatter than normal: extreme moves happen more often than the bell curve predicts.

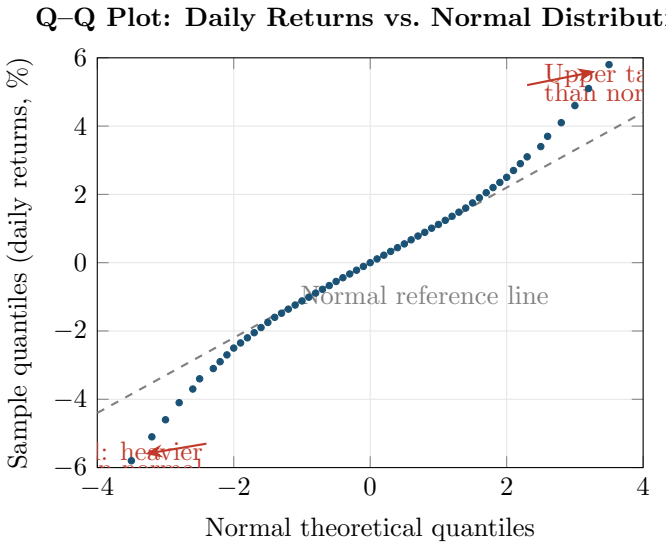


Figure 1.3: Q–Q plot of representative daily equity returns against a normal distribution. If returns were Gaussian, the dots would lie on the dashed diagonal. The S-shaped departure, first documented by Fama (1965), shows that both tails of the empirical distribution are heavier than normal: extreme moves (positive and negative) occur much more often than a Gaussian model predicts.

Why Fat Tails Matter for Models

Any model that assumes normally distributed returns will underestimate the probability of large moves. This affects risk management (VaR breaches), option pricing (mispricing of out-of-the-money options), and backtesting (understating drawdowns). Chapters 4 and 5 introduce models that accommodate fat tails.

1.4.4 Fact 4: The Leverage Effect

Negative returns tend to increase future volatility more than positive returns of the same magnitude. Black (1976) first noted this asymmetry and proposed a mechanism: when a stock price

falls, the firm's equity value shrinks relative to its debt, so leverage (debt/equity) rises, making the stock riskier and more volatile.

Leverage Effect

The leverage effect is the negative correlation between an asset's return and changes in its subsequent volatility:

$$\text{Corr}(r_t, \sigma^2_{t+1}) < 0 \tag{1.6}$$

- r_t: return at time t
- σ^2_{t+1}: conditional variance at time $t+1$
- The negative sign means: price drops $\rightarrow$ volatility rises

In Plain English

This equation says: "when today's return is negative (a price drop), tomorrow's volatility tends to be higher; when today's return is positive (a price rise), tomorrow's volatility tends to be lower." The relationship is asymmetric: markets get more volatile when they fall, but don't calm down as much when they rise. Think of it as "fear is louder than greed": a crash makes everyone nervous, but a rally doesn't make everyone equally calm.

Why This Matters

The leverage effect means your vol forecasting model should treat negative and positive returns differently. A symmetric model (basic GARCH) that treats +2% and −2% as equally informative about tomorrow's vol will systematically underpredict vol after selloffs and overpredict after rallies. This is why GJR-GARCH and EGARCH (Chapter 5) add an asymmetry term, and why it often improves forecast accuracy.

In equity markets, the leverage effect is pronounced. In foreign exchange and commodity markets, the asymmetry is weaker or sometimes reversed (Cont, 2001). Chapter 5 introduces GJR-GARCH and EGARCH models that capture this asymmetry directly.

1.4.5 Summary of Stylized Facts

Four Facts That Shape Every Volatility Model

1. **Uncorrelated returns:** tomorrow's return direction is unpredictable from today's.
2. **Volatility clustering:** large moves follow large moves, small follow small.
3. **Fat tails:** extreme returns occur far more often than a Gaussian model predicts.
4. **Leverage effect:** negative returns raise future volatility more than positive returns.

Any useful volatility model must reproduce (at minimum) facts 2 and 3. A good model also captures fact 4.

1.5 Conditional vs. Unconditional Volatility

Everything so far has treated volatility as a single number: the standard deviation of the full return sample. That number is the *unconditional* volatility. It answers the question "what is the typical size of a daily return, averaging over all market conditions?"

Traders need a different answer: given what has happened up to today, how volatile will the

market be *tomorrow*? That is the *conditional* volatility.

Unconditional vs. Conditional Variance

Unconditional variance:

$$\sigma^2 = \mathrm{Var}(r_t) \tag{1.7}$$

A single number, constant over time. It is the long-run average variance.

Conditional variance:

$$\sigma_t^2 = \mathrm{Var}(r_t \mid \mathcal{F}_{t-1}) \tag{1.8}$$

- σ_t^2: variance of the return at time t, conditional on past information
- $\mathcal{F}_{t-1}$: the **information set** available at time $t-1$. Read this as "everything you could possibly know as of yesterday": all past returns, prices, volumes, news, and any other observable data up through time $t-1$. The fancy script-$\mathcal{F}$ notation comes from probability theory (where it is called a "filtration"); all it means in practice is "given what we knew yesterday." This notation appears throughout the entire guide whenever a quantity is conditioned on past information.

The conditional variance changes from day to day as new information arrives.

Weather Analogy

Unconditional volatility is like the average annual rainfall for your city: a useful summary, but it does not tell you whether to carry an umbrella tomorrow. Conditional volatility is the weather forecast: it uses recent data (yesterday's pressure, today's cloud cover) to predict tomorrow's conditions. This guide is about building the forecast, not computing the average.

Why This Matters

Your entire project is about forecasting *conditional* variance: given everything you know up to today, what will volatility be tomorrow? The unconditional variance is just a naive baseline (always predict the long-run average). Every model you build, from HAR to neural networks, is trying to produce a better estimate of σ_t^2 than that naive baseline by using the information in $\mathcal{F}_{t-1}$.

Worked Example:

Conditional vs. Unconditional in Practice Over 2024, the S&P 500's annualized volatility was about 16% (the unconditional estimate). But during a single turbulent week in August, realized daily vol spiked to 35% annualized; during a calm stretch in November, it dropped to 9%. The unconditional model always says "16%." A conditional model notices the recent spike and says "tomorrow will be around 30%" (or notices the calm and says "around 10%"). Computing the conditional variance requires a model: for example, GARCH uses a weighted average of recent squared returns (Chapter 5), and realized volatility directly measures it from intraday data (Chapter 2).

The unconditional variance and conditional variance are connected by a standard probability result called the **law of total variance**. You do not need to derive it; just understand what it says about how the long-run average relates to the time-varying quantity:

$$\sigma^2 = \mathbb{E}[\sigma_t^2] + \mathrm{Var}\big(\mathbb{E}[r_t \mid \mathcal{F}_{t-1}]\big) \tag{1.9}$$

- $\mathbb{E}[\sigma_t^2]$: average of the conditional variances over time

- $\mathrm{Var}\big(\mathbb{E}[r_t \mid \mathcal{F}_{t-1}]\big)$: variance of the conditional mean (small for equities, since expected returns are nearly constant at daily frequency)

In Plain English

This equation says: "the long-run average variance equals the average of all the daily conditional variances, plus a small correction for the fact that expected returns also fluctuate slightly." For equities at daily frequency, the second term is negligible because expected returns barely change day to day. So in practice: the unconditional variance is just what you get if you average all the conditional variances over a long period.

Why This Matters

This equation tells you that the long-run average vol is a meaningful "anchor" for your forecasts. Many models (GARCH, HAR) build in mean-reversion: after a vol spike, forecasts should gradually drift back toward this long-run level. If your model's forecasts don't average out to something close to the unconditional variance over long horizons, something is likely wrong.

When the conditional mean is approximately constant (a reasonable assumption for daily equity returns), the unconditional variance is simply the time average of the conditional variances. The gap between the two is what makes volatility modeling interesting.

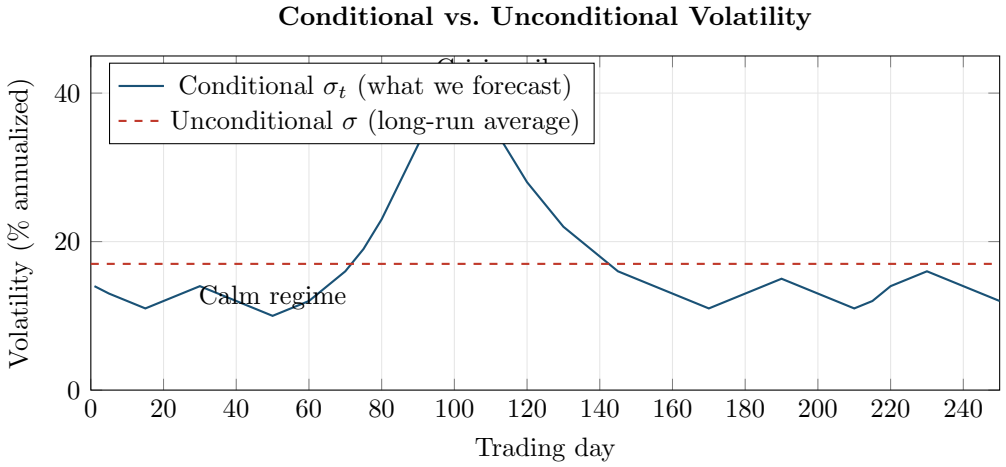


Figure 1.4: The unconditional volatility (dashed red) is a single long-run average: it tells you nothing about which regime you're in today. The conditional volatility (solid blue) varies over time, spiking during crises and falling during calm periods. Your project's goal is to forecast the blue line one step ahead, not just know the red line.

The Central Goal of This Guide

The goal of every model in Chapters 5–14 is to produce accurate estimates of the conditional variance σ_t^2. Chapter 2 shows how to measure the realized conditional variance from high-frequency data. Chapters 5–14 show how to forecast it. Chapter 17 shows how to tell whether your forecasts are any good.

Summary

- **Returns** convert raw prices into comparable, analyzable quantities. Log returns ($r_t = \ln P_t - \ln P_{t-1}$) are preferred because they are additive over time.

- **Simple vs. log returns** are nearly identical for small magnitudes (daily equities); they diverge for large moves.
- **Sample variance** ($\hat{\sigma}^2$) measures dispersion around the mean return; standard deviation ($\hat{\sigma}$) is its square root.
- **Annualized volatility** scales daily volatility by $\sqrt{252}$, assuming independent returns.
- **Volatility matters** because it is a direct input to options pricing, risk management (VaR), portfolio optimization, and trade execution.
- **Stylized fact 1:** Returns are approximately uncorrelated (no predictability in the mean).
- **Stylized fact 2:** Squared and absolute returns are strongly autocorrelated (volatility clusters).
- **Stylized fact 3:** Return distributions have fat tails (excess kurtosis of 5–10 for daily equity returns).
- **Stylized fact 4:** The leverage effect makes volatility respond asymmetrically to negative vs. positive returns.
- The **unconditional** variance is a single long-run number; the **conditional** variance σ_t^2 varies over time and is what we want to forecast.
- The **central goal** of this guide is to estimate and forecast the conditional variance σ_t^2 using both classical and ML methods.
- Cont (2001) provides the canonical reference for stylized facts across asset classes.
- Mandelbrot (1963) and Fama (1965) established that financial returns have fat tails, not Gaussian distributions.
- Engle (1982) introduced ARCH, the first formal model of time-varying conditional variance, covered in Chapter 5.

Table 1.1: Key results referenced in this chapter.

Paper	Result	Relevance
Mandelbrot (1963)	Daily commodity returns follow a stable Paretian (fat-tailed) distribution, not Gaussian; sample variance does not converge.	First evidence that normal-distribution assumptions fail for financial data.
Fama (1965)	Confirmed fat tails (leptokurtosis) in daily U.S. stock returns via S-shaped Q–Q departures from normality.	Extended Mandelbrot's finding to equities; supported the stable distribution hypothesis.
Black (1976)	Documented negative correlation between stock returns and subsequent volatility changes (the leverage effect).	Motivates asymmetric volatility models (GJR-GARCH, EGARCH) in Chapter 5.
Engle (1982)	Introduced ARCH: conditional variance is a function of past squared residuals; applied to UK inflation.	Founded the entire field of conditional volatility modeling (Chapter 5).
Cont (2001)	Canonical enumeration of stylized facts (absence of return autocorrelation, volatility clustering with slow power-law decay, fat tails, leverage effect) across equities, FX, and commodities.	Benchmark reference for the empirical properties any volatility model must match.

Chapter 2

Realized Volatility

Why This Chapter

Realized volatility is the target variable for every project direction in this guide. Chapters 3 and 4 refine the estimator (handling noise and jumps), Chapter 6 builds the HAR forecasting model around it, and Chapter 10 uses RV-derived features. You need to understand RV at an intuitive level (what it measures, how it is constructed, why 5-minute sampling) before any of that.

Chapter 1 introduced variance as a measure of how spread out returns are. That chapter computed variance from daily returns over weeks or months, producing a single number for the whole sample: unconditional volatility. This chapter moves to a different question: how volatile was the market *today*? Answering that requires looking inside the trading day at high-frequency (intraday) returns.

2.1 The Theoretical Target: Integrated Variance

Before building an estimator, you need to know what you are estimating. The theoretical quantity is called *integrated variance*.

Integrals as Accumulated Area

If $f(x)$ is a non-negative function, the integral $\int_a^b f(x)\,dx$ equals the area under the curve f between a and b. When f varies over time, the integral accumulates all of those local values into a single total. You can think of $\int_a^b f(x)\,dx$ as "add up all the tiny contributions of f across the interval $[a, b]$."

Price Processes (Informal)

In continuous-time finance, the log price $p_t = \ln P_t$ evolves according to

$$dp_t = \mu_t\,dt + \sigma_t\,dW_t$$

You do not need stochastic calculus to follow this chapter. Here is what the equation says in plain English: over any tiny time interval, the change in the log price is made up of two pieces:

- $\mu_t\,dt$: a small predictable nudge (the drift, or expected return per unit time). At intraday horizons this is negligibly small and we ignore it.

- $\sigma_t \, dW_t$: a random shock whose size is controlled by σ_t (the instantaneous volatility). W_t is a Wiener process (continuous random walk), so dW_t is a tiny random push that is equally likely to be positive or negative.

The key takeaway: σ_t is the quantity that controls how much the price wiggles at each instant, and it can change from moment to moment within the day.

Time Notation Convention

Throughout this guide, t is an integer day counter. Day t runs from the market close on day $t-1$ to the market close on day t. Inside day t, we use s (in integrals) or i (in sums) to index moments or sub-intervals within that day. So σ_s is the instantaneous volatility at some moment s during the day, while $r_{t,i}$ is the return over the i-th intraday interval on day t.

The intuition is straightforward. At every instant during the trading day, there is a local volatility level σ_t describing how rapidly the price fluctuates. This level is not constant; it changes as news arrives, liquidity shifts, and traders adjust positions. *Integrated variance* adds up all of these instantaneous variance levels across the day to produce a single summary of total price variation.

Think of it by analogy: if you drive a car at varying speeds throughout the day, the total distance driven is the integral of your speed over time. Integrated variance is the "total distance" of price fluctuation.

Integrated Variance

For a log-price process p_t with instantaneous variance σ_t^2, the integrated variance over day t (from the close of day $t-1$ to the close of day t) is:

$$\mathrm{IV}_t = \int_{t-1}^{t} \sigma_s^2 \, ds \tag{2.1}$$

- IV_t: integrated variance over day t
- σ_s^2: instantaneous variance at time s within the day
- ds: an infinitesimal increment of time
- The integral sums the instantaneous variances from the previous close $(t-1)$ to today's close (t)

Integrated variance is a *latent* quantity: you never observe σ_s^2 directly. The entire point of this chapter is to estimate IV_t from observable intraday price data.

In Plain English

This equation says: "add up the instantaneous variance at every moment throughout the trading day." Imagine volatility as a dial that moves up and down throughout the day as news arrives and liquidity shifts. Integrated variance is the total accumulated reading on that dial from open to close. A day with high IV_t is one where the dial was turned up for most of the day (lots of wiggling, even if the price ended flat); a day with low IV_t is one where the dial stayed low (calm, smooth drift).

Why This Matters

Integrated variance is the *true* volatility you are trying to forecast. Every model in this guide (HAR, GARCH, neural networks) is ultimately trying to predict tomorrow's IV_{t+1}. You never observe IV_t directly, so you estimate it with realized variance (next section), but this is the theoretical gold standard your estimates are aiming at.

2.2 Realized Variance as an Estimator

You now know the target: integrated variance IV_t. The question is how to estimate it from data you can actually observe. The answer, developed by Andersen et al. (2001) and Barndorff-Nielsen and Shephard (2002), is remarkably simple: sum up squared intraday returns.

2.2.1 Construction

Divide the trading day into n equal intervals. At the end of each interval, record the price. Compute the log return over each interval. Square each return and sum.

Realized Variance

Given n intraday log returns $r_{t,1}, r_{t,2}, \ldots, r_{t,n}$ within day t, the realized variance is:

$$\text{RV}_t = \sum_{i=1}^{n} r_{t,i}^2 \tag{2.2}$$

- RV_t: realized variance for day t
- $r_{t,i}$: the i-th intraday log return on day t; that is, $r_{t,i} = p_{t,i} - p_{t,i-1}$, where $p_{t,i}$ is the log price at the end of the i-th interval
- n: the number of intraday intervals (e.g., $n = 78$ for 5-minute intervals over a 6.5-hour U.S. equity trading day)
- No mean subtraction: the sample mean of intraday returns is so close to zero that omitting it improves finite-sample performance (Andersen et al., 2003)

In Plain English

This equation says: "take every intraday return, square it, and add them all up." Squaring serves two purposes: it makes every return positive (so up moves and down moves both contribute), and it captures the *magnitude* of price movement rather than the direction. Each squared return is a noisy snapshot of how volatile the market was during that interval; summing them accumulates these snapshots into a total for the day.

Why Squaring Works (Deeper)

From the price process equation above ($dp_t = \mu_t\, dt + \sigma_t\, dW_t$), each small intraday return is approximately the local volatility times a random draw: $r_{t,i} \approx \sigma_{t,i}\, \epsilon_i$, where $\sigma_{t,i}$ is the volatility during interval i and ϵ_i is a random variable with mean zero and variance one (coming from the Wiener process increments). Squaring gives $r_{t,i}^2 \approx \sigma_{t,i}^2\, \epsilon_i^2$. Since ϵ_i has variance 1 and mean 0, $\mathbb{E}[\epsilon_i^2] = \text{Var}(\epsilon_i) + (\mathbb{E}[\epsilon_i])^2 = 1 + 0 = 1$. So on average, each squared return equals the local variance: $\mathbb{E}[r_{t,i}^2] \approx \sigma_{t,i}^2$. Summing across the day accumulates these local variance snapshots. As intervals shrink ($n \to \infty$), the random fluctuations in each ϵ_i^2 average out, and the sum converges to the integrated variance.

Why This Matters

This is the formula you will compute every single day in your project. RV is your dependent variable: the thing you're trying to forecast. When you build a HAR model (Chapter 6) or train a neural network (Chapter 13), the y in your regression is RV_{t+1} (or $\ln \text{RV}_{t+1}$), and your goal is to predict it from today's information. Understanding what this sum of squared returns actually measures is essential because your model's accuracy depends on the quality of this estimate.

2.2.2 Why It Works: Convergence to Quadratic Variation

We've defined RV as the sum of squared returns. But why should summing squared returns give us anything meaningful? Why not sum absolute returns, or cubed returns? The answer comes from a deep result in probability theory: for processes like stock prices, the sum of squared increments converges to a specific quantity called the **quadratic variation**, and this quantity equals the integrated variance we're trying to measure. This subsection explains that result.

Quadratic Variation (Informal)

For a continuous-time stochastic process X_t, the quadratic variation $[X]_t$ measures the cumulative squared increments of the path up to time t. It is defined as the limit of the sum of squared increments as the partition becomes infinitely fine:

$$[X]_t = \lim_{n\to\infty} \sum_{i=1}^{n} (X_{t_i} - X_{t_{i-1}})^2$$

For a process with continuous paths (no jumps), the quadratic variation equals the integrated variance: $[X]_t = \int_0^t \sigma_s^2 \, ds$. If the process has jumps, the quadratic variation also captures the sum of squared jumps.
The chain to remember: RV (what you compute from data) $\to$ QV (the limit as sampling gets infinitely fine) = IV (the true volatility you want, if there are no jumps). Quadratic variation is the theoretical bridge connecting your finite-sample computation to the true quantity of interest.

The central result is this: as you sample more frequently (as $n \to \infty$ and the length of each interval $\Delta \to 0$), the sum of squared returns converges to the quadratic variation of the log-price process (Andersen et al., 2001; Barndorff-Nielsen and Shephard, 2002):

$$\text{RV}_t \xrightarrow{p} [p]_t \quad \text{as } \Delta \to 0 \tag{2.3}$$

- $\xrightarrow{p}$: convergence in probability (meaning the estimate gets arbitrarily close to the true value as sampling gets finer)
- $[p]_t$: quadratic variation of the log-price process on day t
- $\Delta = 1/n$: the length of each sampling interval (as a fraction of the trading day)

In Plain English

This equation says: "if you chop the trading day into finer and finer intervals, compute the return over each tiny interval, square it, and sum them all up, the result converges to a specific number called the quadratic variation." Think of it like measuring the total wobble of a drunk person's walk: the more finely you measure their steps, the more zigzags you capture. As your measurement gets infinitely fine, you converge on the *true*

total wobble. That true total wobble is the quadratic variation $[p]_t$.

Why This Matters

This convergence result is the entire theoretical foundation for using RV as a volatility measure. It tells you that what you're computing (sum of squared returns) is not just an arbitrary statistic: it is a *consistent estimator* of a well-defined theoretical quantity. Without this result, RV would have no theoretical grounding, and you'd have no reason to believe it measures anything meaningful about the price process.

Now, what exactly is this quadratic variation equal to? That depends on whether the price path had any sudden jumps during the day.

If the price path is continuous (no jumps), then the quadratic variation equals the integrated variance, exactly:

$$[p]_t = \text{IV}_t \qquad \text{(no jumps)} \tag{2.4}$$

In Plain English

When there are no jumps, this equation says: "the total wobble of the price path equals the total accumulated instantaneous variance." This is the ideal case. It means your sum of squared returns converges to exactly the thing you want to measure (IV_t), with no contamination from other sources.

If the price path has jumps (sudden discontinuous moves, like a price gap after an earnings announcement), the quadratic variation picks up an extra component:

$$[p]_t = \text{IV}_t + \sum_{s \le t} (J_s)^2 \qquad \text{(with jumps)} \tag{2.5}$$

- J_s: the jump in log price at time s (zero if no jump occurs at s). Some papers write this as Δp_s; we use J_s here to avoid confusion with Δ (sampling interval length) used elsewhere.
- The sum looks like it runs over infinitely many times, but in practice there are only a few jumps per day (or none), so the sum has at most a handful of nonzero terms.
- Separating jumps from continuous variation is the subject of Chapter 4.

In Plain English

This equation says: "when there are jumps, the total wobble (quadratic variation) equals the smooth continuous wiggling (IV_t) *plus* the squared sizes of all the sudden jumps." Your RV estimate picks up both components. If you want to measure only the smooth, continuous volatility (which is more persistent and forecastable), you need to strip out the jumps. That's what Chapter 4 does.

Why This Matters

In practice, jumps are relatively rare but can be large. Including jump variation in your forecasting target adds noise: jumps are hard to predict (they're often driven by surprise events), so your model gets a noisier signal. Many of the best forecasting models (HAR-CJ, SHAR) separate the jump and continuous components and model them separately, because the continuous part is far more persistent and predictable.

Andersen et al. (2001), Barndorff-Nielsen and Shephard (2002): RV Consistency

Under mild regularity conditions, realized variance RV_t is a consistent estimator of quadratic variation $[p]_t$. In the absence of jumps, RV_t consistently estimates integrated variance IV_t. This result holds regardless of the specific form of σ_t; the volatility process can be stochastic, path-dependent, or driven by latent factors.

Figure 2.1 summarizes the full chain of relationships between the quantities introduced so far.

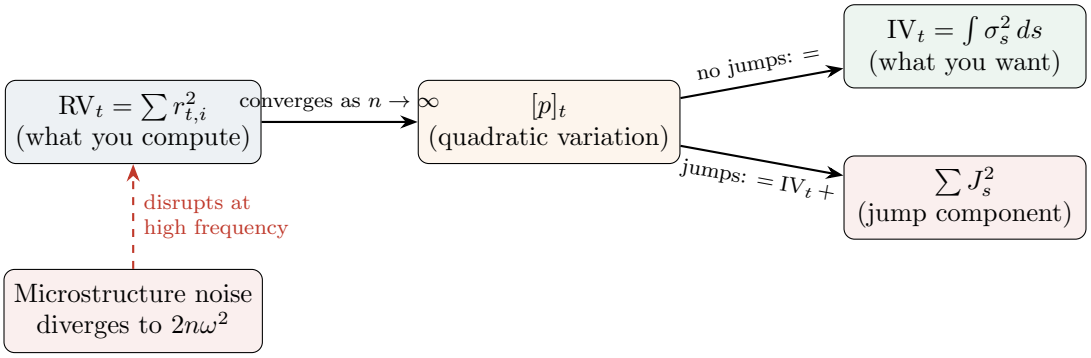


Figure 2.1: The conceptual chain connecting the three key quantities. **RV** (what you compute from data) converges to **QV** (a mathematical limit) as sampling gets finer. Without jumps, QV equals **IV** (the true volatility you want). With jumps, QV includes both IV and a jump component (Chapter 4 separates them). Microstructure noise disrupts the convergence at high sampling frequencies (Chapter 3 fixes this).

Figure 2.2 illustrates the convergence idea: as you chop the day into more intervals, the sum of squared returns gets closer to the true integrated variance.

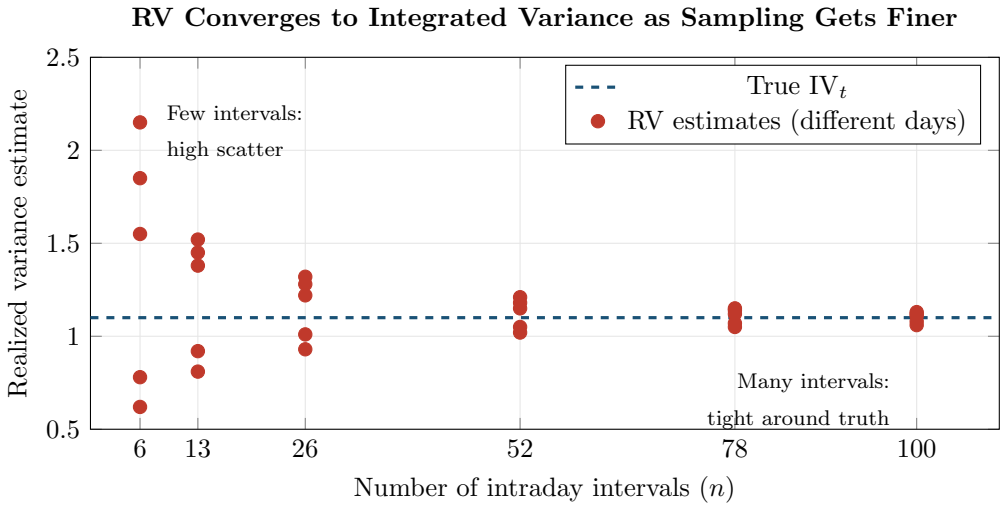


Figure 2.2: Convergence of RV to integrated variance. Each red dot is an RV estimate from a different simulated day. With only 6 intervals (hourly sampling), estimates are scattered widely around the true IV_t (dashed blue). With 78 intervals (5-minute sampling), estimates cluster tightly. This is the convergence result of Equation (2.3) in action. (In practice, microstructure noise prevents going much beyond 78 intervals.)

2.2.3 Diagram: Building RV from a Price Path

Figure 2.3 shows how RV is constructed from a single day's price data.

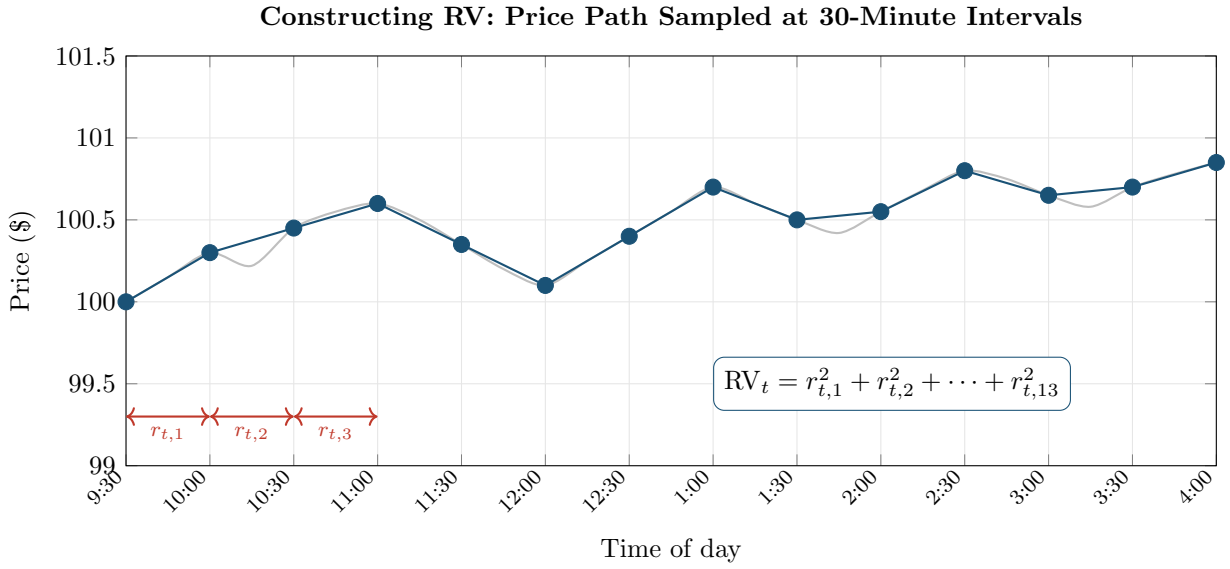


Figure 2.3: Constructing realized variance from a price path. Gray: the continuous intraday price. Blue dots: prices sampled at half-hourly intervals ($n = 13$ intervals). Each interval produces a log return $r_{t,i}$. RV is the sum of all squared returns. Sampling more frequently (e.g., every 5 minutes instead of 30) produces more terms in the sum and a more precise estimate.

Worked Example:

Computing RV from Half-Hourly Returns A stock opens at $100.00 and you record its price every 30 minutes over a shortened trading day (6 intervals for simplicity). The prices and resulting log returns are:

Time	Price ($)	Log return $r_{t,i}$	$r_{t,i}^2$
9:30 (open)	100.00		
10:00	100.30	$\ln(100.30/100.00) = +0.003000$	9.000×10^{-6}
10:30	100.10	$\ln(100.10/100.30) = -0.001998$	3.992×10^{-6}
11:00	100.50	$\ln(100.50/100.10) = +0.003992$	15.936×10^{-6}
11:30	100.20	$\ln(100.20/100.50) = -0.002992$	8.952×10^{-6}
12:00	100.60	$\ln(100.60/100.20) = +0.003988$	15.904×10^{-6}
12:30 (close)	100.45	$\ln(100.45/100.60) = -0.001491$	2.223×10^{-6}

Step 1: Sum the squared returns.

$$\mathrm{RV}_t = (9.000 + 3.992 + 15.936 + 8.952 + 15.904 + 2.223) \times 10^{-6} = 56.007 \times 10^{-6}$$

Step 2: Convert to realized volatility.

$$\sqrt{\mathrm{RV}_t} = \sqrt{56.007 \times 10^{-6}} = 0.00748 \quad (0.75\% \text{ daily})$$

Step 3: Annualize.

$$\text{Annualized realized volatility} = 0.00748 \times \sqrt{252} = 0.119 \quad (11.9\%)$$

For context, 11.9% annualized is a bit below the S&P 500's long-run average (roughly 15–20%). With only 6 intervals, this estimate is noisy. Sampling every 5 minutes would give 78 intervals over a 6.5-hour day, producing a much more precise estimate.

2.3 How Frequently to Sample

You now have an estimator (RV_t) and a convergence result (it approaches $[p]_t$ as $n \to \infty$). The obvious next step is to sample as frequently as possible: every second, every tick, every trade. In theory, this is optimal. In practice, it fails.

2.3.1 The Theory: More Is Better

The convergence result in Equation (2.3) says that RV becomes a better estimator as Δ shrinks. More precisely, Barndorff-Nielsen and Shephard (2002) showed that the estimation error has a central limit theorem:

The convergence result tells you RV approaches IV_t as you sample more frequently, but how *fast* does it converge? How precise is your estimate with, say, 78 five-minute intervals versus 390 one-minute intervals? Barndorff-Nielsen and Shephard (2002) showed the estimation error follows a central limit theorem:

$$\sqrt{n}\,(\text{RV}_t - \text{IV}_t) \xrightarrow{d} \mathcal{N}\left(0,\ 2\int_{t-1}^{t} \sigma_s^4\, ds\right) \tag{2.6}$$

- $\xrightarrow{d}$: convergence in distribution (the error becomes approximately normally distributed)
- $\sqrt{n}$: the estimation error shrinks at rate $1/\sqrt{n}$
- $\text{RV}_t - \text{IV}_t$: the gap between your estimate and the truth
- $2\int_{t-1}^{t} \sigma_s^4\, ds$: the asymptotic variance of the error (it depends on the "quarticity," the integral of σ^4, which measures how variable the volatility itself was during the day)

In Plain English

This equation says: "the error in your RV estimate is approximately bell-shaped, centered on zero, and shrinks as $1/\sqrt{n}$." Doubling the number of intervals (going from 5-minute to 2.5-minute sampling) cuts your standard error by a factor of $\sqrt{2} \approx 1.41$.
Why does σ^4 appear? Each squared return $r_{t,i}^2$ estimates the local variance $\sigma_{t,i}^2$, but with some error. The variance of that estimation error is proportional to $\sigma_{t,i}^4$ (because computing the variance of a squared quantity involves squaring something that is already squared). Summing these error variances across the day gives $\int \sigma_s^4\, ds$, called the "quarticity." On days when volatility was itself highly variable within the day, quarticity is large, and your RV estimate is less precise.

Why This Matters

This CLT tells you how noisy each day's RV estimate is, and that directly affects your forecasting model. RV is your dependent variable (the y you're predicting); if some days' y values are noisier than others, your model trains on noisy labels, which degrades performance. The HARQ model (Chapter 6) explicitly accounts for this: it estimates each day's RV precision from the CLT and downweights noisy observations. Days with few intraday observations (holiday-shortened sessions) or wildly variable intraday vol have noisier RV, and a smart model adjusts for this.

So with perfect data, you would sample every millisecond and get an essentially noise-free estimate of integrated variance.

2.3.2 The Practice: Microstructure Noise

Real transaction prices are not clean readings of a frictionless price process. They are contaminated by *microstructure noise*: the cumulative effect of bid-ask bounce, discrete price grids, order-processing delays, and other trading frictions.

Bid-Ask Bounce

When you buy a stock, you pay the *ask* price (slightly above the "true" value). When you sell, you receive the *bid* price (slightly below). The difference between bid and ask is the *spread*. Even if the true value is perfectly constant, alternating buys and sells cause the transaction price to bounce between bid and ask. This artificial oscillation creates spurious volatility in very high-frequency returns.

The standard model for microstructure noise (Andersen et al., 2001; Barndorff-Nielsen and Shephard, 2002) captures this with a simple additive structure: the price you actually observe is the "true" price plus some random noise from trading frictions.

$$p^*_{t,i} = p_{t,i} + \varepsilon_{t,i} \tag{2.7}$$

- $p^*_{t,i}$: observed (noisy) log price
- $p_{t,i}$: true (latent) efficient log price
- $\varepsilon_{t,i}$: microstructure noise, typically assumed i.i.d. with $\mathbb{E}[\varepsilon_{t,i}] = 0$ and $\text{Var}(\varepsilon_{t,i}) = \omega^2$

In Plain English

This equation says: "the price you see on your screen is the true underlying price plus some random jitter from the mechanics of trading." The jitter (ε) averages to zero (it doesn't systematically push prices up or down) but it adds random noise to every price observation. The noise variance ω^2 is a property of the market: liquid stocks with tight bid-ask spreads have small ω^2; illiquid stocks have larger ω^2.

Why This Matters

When you download high-frequency price data for your project, you are downloading $p^*_{t,i}$, not $p_{t,i}$. This noise is baked into your data and cannot be removed by cleaning alone. The entire next chapter (Chapter 3) is about building RV estimators that work correctly despite this contamination.

When you compute returns from noisy prices, each observed return picks up the noise: $r^*_{t,i} = r_{t,i} + (\varepsilon_{t,i} - \varepsilon_{t,i-1})$. The noise part $(\varepsilon_{t,i} - \varepsilon_{t,i-1})$ has variance $\text{Var}(\varepsilon_{t,i}) + \text{Var}(\varepsilon_{t,i-1}) = 2\omega^2$ (since the two noise terms are independent). When you sample very frequently, the true returns $r_{t,i}$ become tiny (the price barely moves in a millisecond), but each noise contribution is still $2\omega^2$ regardless of the interval length. So the noise dominates, and RV diverges:

$$\text{RV}_t^{(\text{noisy})} \to 2n\omega^2 \quad \text{as } n \to \infty \tag{2.8}$$

In Plain English

This equation says: "if you sample too frequently, your RV estimate doesn't converge to the true volatility; it explodes to infinity." Each noisy return contributes roughly $2\omega^2$ of garbage on average (from the noise term bouncing back and forth), and you're summing n of these. As n grows, the garbage accumulates linearly while the true signal grows more slowly, so the noise wins. This is why sampling every tick or every second gives you a

wildly inflated RV.

Why This Matters

This divergence result is the reason you cannot simply use the highest-frequency data available. It creates the central practical challenge of RV estimation: you want high frequency for precision, but too-high frequency is destroyed by noise. The 5-minute convention and the noise-robust estimators in Chapter 3 are both responses to this problem.

Sampling Too Fast Destroys the Estimate

With tick-by-tick or second-by-second data, the bid-ask bounce inflates realized variance far above the true integrated variance. Sampling more frequently makes the estimate *worse*, not better. This is the fundamental tension of realized variance estimation.

2.3.3 The Bias-Variance Tradeoff

The result is a tradeoff:

- **Sample too infrequently** (e.g., hourly): few observations, high estimation variance, but little noise contamination.
- **Sample too frequently** (e.g., every second): many observations, low estimation variance in theory, but massive upward bias from microstructure noise.

There is an optimal sampling frequency that balances these two forces. The concept is often visualized with a *volatility signature plot*.

2.3.4 The Volatility Signature Plot

Volatility Signature Plot

A volatility signature plot displays the average realized variance (or realized volatility), computed across many days, as a function of the sampling frequency. The x-axis shows the return interval (e.g., 1 second, 1 minute, 5 minutes, ...), and the y-axis shows the resulting average RV_t.

Figure 2.4 shows the typical shape.

The plot has three regions:

1. **High-frequency region** (left): RV rises sharply, contaminated by bid-ask bounce and microstructure effects.
2. **Intermediate region** (center): RV flattens near the true integrated variance. This is the practical sweet spot.
3. **Low-frequency region** (right): RV drifts downward and becomes noisy. The downward bias occurs because with fewer sample points, you miss many intraday price wiggles. Returns computed over long intervals smooth out the zigzags between your sparse sample points, producing smaller squared returns and hence a lower RV estimate.

The Fundamental Tradeoff

In theory, sample as fast as possible. In practice, noise wins beyond about 1-minute returns for most liquid assets. The volatility signature plot is a diagnostic tool: if average RV is still rising as you increase frequency, you are in the noise-contaminated region and

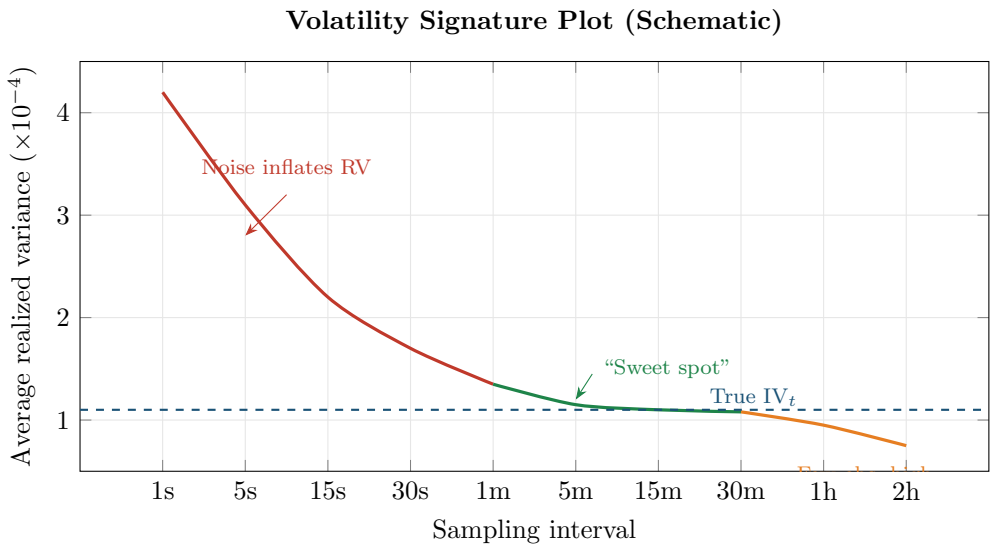


Figure 2.4: Schematic volatility signature plot. At very high frequencies (left, red), microstructure noise inflates realized variance. At very low frequencies (right, orange), too few observations make the estimate imprecise and biased downward. The green region (roughly 1–15 minutes) is the "sweet spot" where RV is closest to the true integrated variance (dashed blue line). Chapter 3 develops noise-robust estimators that work at higher frequencies.

need to back off (or use a noise-robust estimator from Chapter 3).

2.4 5-Minute RV: The Practical Workhorse

Given the tradeoff from the previous section, what sampling frequency should you use in practice? The answer, for most applications, is 5 minutes.

This is not a number derived from first principles (the optimal frequency depends on the noise-to-signal ratio, which varies by asset and time period). It is an empirical finding, and a remarkably robust one.

Liu et al. (2015): Does Anything Beat 5-Minute RV?

Liu et al. (2015) conducted a large-scale comparison of approximately 400 realized volatility estimators (including subsampled, kernel-based, pre-averaged, and multi-scale estimators) applied to 31 assets across 5 asset classes (equities, equity indices, exchange rates, bonds, and commodities). Their conclusion: for the purpose of *forecasting* future volatility, "it is difficult to significantly outperform" the simple 5-minute realized variance. More sophisticated estimators sometimes produce marginally more accurate daily estimates, but these gains do not reliably translate into better forecasts.

Why 5 minutes? Two practical reasons:

1. For liquid assets (major equity indices, G10 FX pairs, actively traded stocks), the volatility signature plot typically flattens by the 5-minute mark. At this frequency, microstructure noise is small relative to genuine price variation.
2. 5-minute sampling produces 78 intraday observations per day for U.S. equities (6.5 hours $\times$ 12 intervals per hour), which provides enough observations for the central limit theorem in Equation (2.6) to give a reasonable approximation.

The Default Choice

Unless you have a specific reason to do otherwise (illiquid assets, tick-level analysis, or a noise-robust estimator from Chapter 3), use 5-minute returns to compute RV. This is the baseline in most empirical volatility research and the starting point for every forecasting model in this guide.

5 Minutes Is Not Universal

For illiquid assets (small-cap stocks, emerging-market currencies, less-traded commodities), the noise-contaminated region can extend to 15 or even 30 minutes. Always check the volatility signature plot for your specific asset before committing to a sampling frequency.

2.5 Realized Volatility vs. Realized Variance

This chapter has used "realized variance" for $\text{RV}_t = \sum r_{t,i}^2$ and "realized volatility" for $\sqrt{\text{RV}_t}$. Not every paper follows this convention.

Variance vs. Volatility: A Factor-of-10 Error

The S&P 500's typical daily realized variance is around 1×10^{-4}. Its typical daily realized volatility is $\sqrt{1 \times 10^{-4}} = 0.01$ (1%). Confusing the two is a factor-of-100 error in the daily number, which becomes roughly a factor of 10 when annualized ($0.01 \times \sqrt{252} \approx 0.16$ vs. $0.0001 \times \sqrt{252} \approx 0.0016$). Whenever you read a paper, check whether the authors report RV_t or $\sqrt{\text{RV}_t}$, and note the units (decimal, percent, or basis points).

Conventions in This Guide

Throughout this guide:

- RV_t (and "RV") always refers to *realized variance*: $\text{RV}_t = \sum_{i=1}^{n} r_{t,i}^2$
- Realized *volatility* refers to $\sqrt{\text{RV}_t}$, the square root of realized variance
- When annualizing, multiply $\sqrt{\text{RV}_t}$ by $\sqrt{252}$ (as in Section 1.2.1 of Chapter 1)

These conventions follow Andersen et al. (2003) and Barndorff-Nielsen and Shephard (2002).

Some papers (particularly in the HAR literature, Chapter 6) work with the natural logarithm of realized variance, $\ln(\text{RV}_t)$. The log transform has two practical benefits:

1. RV_t is right-skewed and bounded below by zero; $\ln(\text{RV}_t)$ is approximately Gaussian (Andersen et al., 2001, 2003), making standard regression assumptions more plausible.
2. Forecasting $\ln(\text{RV}_t)$ automatically ensures that predictions of the variance level are positive (since $e^x > 0$ for all x).

When you encounter $\ln(\text{RV}_t)$ in later chapters, this is why.

Summary

- **Integrated variance** $\text{IV}_t = \int_{t-1}^{t} \sigma_s^2 \, ds$ is the theoretical target: the total variance accumulated within a single day.
- **Realized variance** $\text{RV}_t = \sum_{i=1}^{n} r_{t,i}^2$ is a model-free estimator of integrated variance, constructed by summing squared intraday returns.

- **Convergence**: as the sampling frequency increases ($n \to \infty$), RV converges in probability to the quadratic variation of the log-price process (Andersen et al., 2001; Barndorff-Nielsen and Shephard, 2002).
- In the **absence of jumps**, quadratic variation equals integrated variance, so RV consistently estimates IV_t.
- With **jumps**, quadratic variation equals integrated variance plus the sum of squared jumps. Chapter 4 addresses separating the two components.
- **Microstructure noise** (bid-ask bounce, discrete prices) corrupts very high-frequency returns, causing RV to diverge as sampling frequency increases.
- The **volatility signature plot** (average RV vs. sampling interval) visualizes the bias-variance tradeoff: too fast means noise contamination, too slow means imprecise estimates.
- **5-minute RV** is the standard benchmark. Liu et al. (2015) showed it is hard to beat for forecasting across 31 assets and ∼400 competing estimators.
- For **illiquid assets**, 5 minutes may still be in the noise-contaminated region; always check the volatility signature plot.
- **Convention**: this guide uses RV_t for realized variance and $\sqrt{\text{RV}_t}$ for realized volatility. Confusing the two is roughly a factor-of-10 annualized error.
- The **log transform** $\ln(\text{RV}_t)$ is approximately Gaussian and guarantees positive forecasts, which is why it appears frequently in later chapters.
- Chapter 3 develops noise-robust estimators that can safely use higher frequencies; Chapter 4 separates continuous and jump variation.

Table 2.1: Key results referenced in this chapter.

Paper	Result	Relevance
Andersen et al. (2001)	Realized variance converges to quadratic variation as sampling frequency increases; the distribution of $\ln(\text{RV}_t)$ is approximately Gaussian.	Foundational result establishing RV as a consistent, model-free volatility estimator.
Andersen et al. (2003)	Formal econometric theory for realized variance: consistency, asymptotic distribution, practical implementation with 5-minute FX returns, and the result that mean subtraction is unnecessary.	Provided the CLT for RV and demonstrated that dropping the intraday mean improves finite-sample accuracy.
Barndorff-Nielsen and Shephard (2002)	Parallel asymptotic theory for realized variance; introduced bipower variation (BPV) to separate jumps from continuous variation.	Second foundational contribution; BPV is the basis for jump detection in Chapter 4.
Liu et al. (2015)	Compared ∼400 realized estimators across 31 assets in 5 asset classes; simple 5-minute RV is very hard to beat for volatility forecasting.	Justifies 5-minute RV as the default benchmark for all forecasting models in this guide.

Chapter 3

Microstructure Noise and Robust Estimators

Why This Chapter

Microstructure noise is the reason you cannot simply sample tick-by-tick and compute realized variance. This chapter teaches when noise matters, how it biases RV, and the family of robust estimators that correct for it. Chapter 10 uses these estimators as features. Project 2 (LOB-based intraday) works directly in the high-frequency regime where noise is most severe.

Chapter 2 ended with a paradox. The theory says "sample as fast as possible": more intraday returns means a more precise estimate of integrated variance. But the volatility signature plot showed that sampling faster than about 5 minutes makes the estimate *worse*, not better, because microstructure noise inflates the sum of squared returns. This chapter explains exactly where that noise comes from, quantifies the damage it does, and develops four estimators that fix the problem: TSRV, MSRV, the realized kernel, and pre-averaging.

3.1 The Microstructure Noise Problem

Chapter 2 introduced the noise model $p^*_{t,i} = p_{t,i} + \varepsilon_{t,i}$ (Equation 2.7) and showed that noise causes RV to diverge. This section digs into *where* the noise comes from, because understanding the source tells you when it matters and when it does not.

Bid, Ask, Mid, and Spread

At any moment, a stock has two prices: the *bid* (the highest price a buyer will pay) and the *ask* (the lowest price a seller will accept). The *mid price* is the average, $(P_{\text{bid}} + P_{\text{ask}})/2$. The *spread* is $P_{\text{ask}} - P_{\text{bid}}$. Transaction prices alternate between bid and ask depending on who initiates the trade (a market buy hits the ask; a market sell hits the bid).

Sampling Frequency Terminology

"Sampling at 1-second frequency" means computing one return per second. A 6.5-hour U.S. equity trading day has $6.5 \times 3{,}600 = 23{,}400$ one-second intervals. "Tick-by-tick" means using every single trade as a price observation; for liquid stocks, this can mean thousands of observations per minute.

There are three main sources of microstructure noise.

Source 1: Bid-ask bounce. Even if the true value of a stock is perfectly constant at \$50.00, alternating buys and sells produce transaction prices that bounce between \$49.99 (bid) and \$50.01 (ask). Computing returns from these prices generates a sequence of $+0.02$, -0.02, $+0.02$, . . . that has nothing to do with actual volatility. Squaring and summing these fake returns inflates RV. Bid-ask bounce is the dominant source of noise for most liquid assets (Hansen and Lunde, 2006).

Figure 3.1 illustrates how bid-ask bounce creates spurious volatility even when the true price is constant.

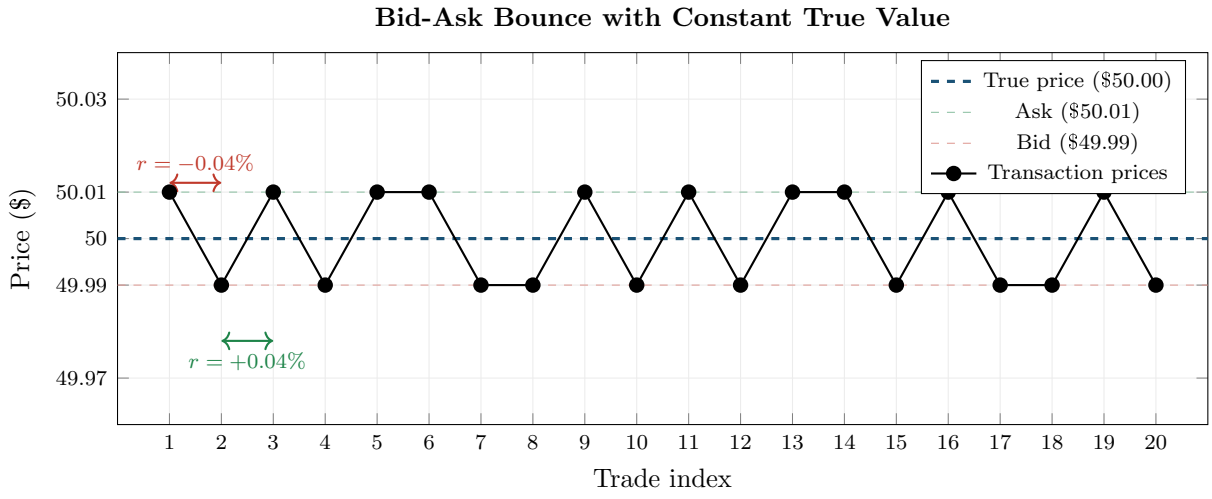


Figure 3.1: Bid-ask bounce creates spurious volatility. The true price (blue dashed) is constant at \$50.00, but transaction prices alternate between the bid (\$49.99) and ask (\$50.01). Each trade generates a return of $\pm 0.04\%$ that is pure noise. Squaring and summing these fake returns inflates RV far above zero.

Source 2: Discrete tick sizes. Prices on exchanges are quoted in discrete increments (one cent for U.S. equities). When the true efficient price is \$50.003, the observed price must round to either \$50.00 or \$50.01. This rounding adds a small, mean-zero error to every observation. At very high frequencies, these rounding errors accumulate into a non-trivial noise term.

Source 3: Price staleness. Illiquid stocks may not trade every second. If you sample the "last trade price" at regular intervals, you often repeat the same stale price. The resulting return is zero, which underestimates true volatility. When a new trade finally arrives, the return is artificially large. This creates spurious autocorrelation in returns (Aït-Sahalia et al., 2005).

Why Noise Kills High-Frequency RV

At very high frequencies, observed prices alternate between bid and ask. Squaring these back-and-forth moves inflates the sum far beyond the true volatility. The signal (real price moves) scales with the time interval Δ, but the noise (bid-ask bounce) has a roughly constant variance ω^2 per observation. As you shrink Δ, the noise-to-signal ratio explodes: each return is mostly noise, and you are summing $n = 1/\Delta$ of them.

Source 4: Adverse selection (information asymmetry). Beyond mechanical noise, spreads reflect a deeper economic force. Glosten and Milgrom (1985) showed that market makers widen spreads because some counterparties possess private information. The **bid-ask spread** must

compensate for losses to informed traders:

$$s_t \propto \alpha \cdot \sigma_v$$

where α is the probability the counterparty is informed and σ_v is the volatility of the information signal. When volatility is high, information asymmetry increases (there is more to know), so spreads widen mechanically. This creates a feedback loop: high vol leads to wider spreads, which creates more noise in transaction prices, which creates more biased RV estimates.

This means microstructure noise is not merely a nuisance to filter; it carries information about market quality and **informed-trading intensity** that can itself predict future volatility (Chapter 10).

A related insight from Kyle (1985): the first-order autocovariance of price changes satisfies $\text{Cov}(\Delta p_t, \Delta p_{t+1}) = -s^2/4$. This connects the noise-robust estimators developed later in this chapter (which exploit autocovariance structure) to a liquidity interpretation.

3.1.1 Quantifying the Bias

The standard noise model assumes the observed log price is the true (efficient) price plus i.i.d. noise:

$$p^*_{t,i} = p_{t,i} + \varepsilon_{t,i}, \qquad \varepsilon_{t,i} \overset{\text{iid}}{\sim} (0, \omega^2)$$

The fundamental question is: if you compute RV from noisy prices, how far off is the answer? Hansen and Lunde (2006) and Aït-Sahalia et al. (2005) showed that, under this model, the expected value of noisy RV decomposes into signal plus a noise term that scales with the number of observations:

$$\mathbb{E}[\text{RV}_t^{(\text{noisy})}] = \text{IV}_t + 2n\omega^2 \tag{3.1}$$

- $\text{RV}_t^{(\text{noisy})}$: realized variance computed from observed (noisy) prices
- IV_t: true integrated variance (the target)
- n: number of intraday returns
- ω^2: variance of the microstructure noise per observation
- $2n\omega^2$: the noise-induced bias, linear in n

In Plain English

On average, noisy RV equals the true volatility plus a junk term that grows in proportion to how many returns you use. Every return you add contributes a little bit of noise variance ($2\omega^2$), so using more returns makes the contamination worse, not better. This is the opposite of what you would expect from classical statistics, where more data means more precision.

Why This Matters

This equation is the reason your vol forecasting project uses 5-minute returns instead of tick-by-tick data. At 5-minute frequency ($n \approx 78$ for U.S. equities), the bias $2n\omega^2$ is small enough to ignore for most liquid stocks. If you ever move to higher-frequency data, you will need the noise-robust estimators developed later in this chapter.

The bias $2n\omega^2$ grows linearly with the number of observations. As $n \to \infty$ (tick-by-tick), the bias term dominates and $\text{RV}_t^{(\text{noisy})} \to \infty$. The IV_t term becomes a negligible fraction of the total.

The Bias Is Not Just Upward

The i.i.d. noise model predicts a purely upward bias. In practice, noise can also be negatively autocorrelated (alternating buys and sells) or positively autocorrelated (momentum in order flow), which can shift the bias in either direction. Hansen and Lunde (2006) found that the simple i.i.d. model is a useful first approximation for liquid stocks, but the actual noise structure is more complex. Always inspect the volatility signature plot for your specific asset.

3.2 The Volatility Signature Plot

Chapter 2 introduced the volatility signature plot (Figure 2.4). This section explains it as a *diagnostic tool*: the first thing you should plot before choosing an estimator or sampling frequency.

Volatility Signature Plot (Formal)

The volatility signature plot displays $\bar{\text{RV}}(\Delta)$, the average realized variance computed across T days, as a function of the sampling interval Δ:

$$\bar{\text{RV}}(\Delta) = \frac{1}{T}\sum_{t=1}^{T} \text{RV}_t(\Delta)$$

- Δ: sampling interval (e.g., 1 second, 1 minute, 5 minutes, . . .)
- $\text{RV}_t(\Delta)$: realized variance for day t computed at interval Δ
- T: number of trading days in the sample

The x-axis shows Δ (often on a log scale), and the y-axis shows $\bar{\text{RV}}(\Delta)$.

Figure 3.2 illustrates the three regimes.

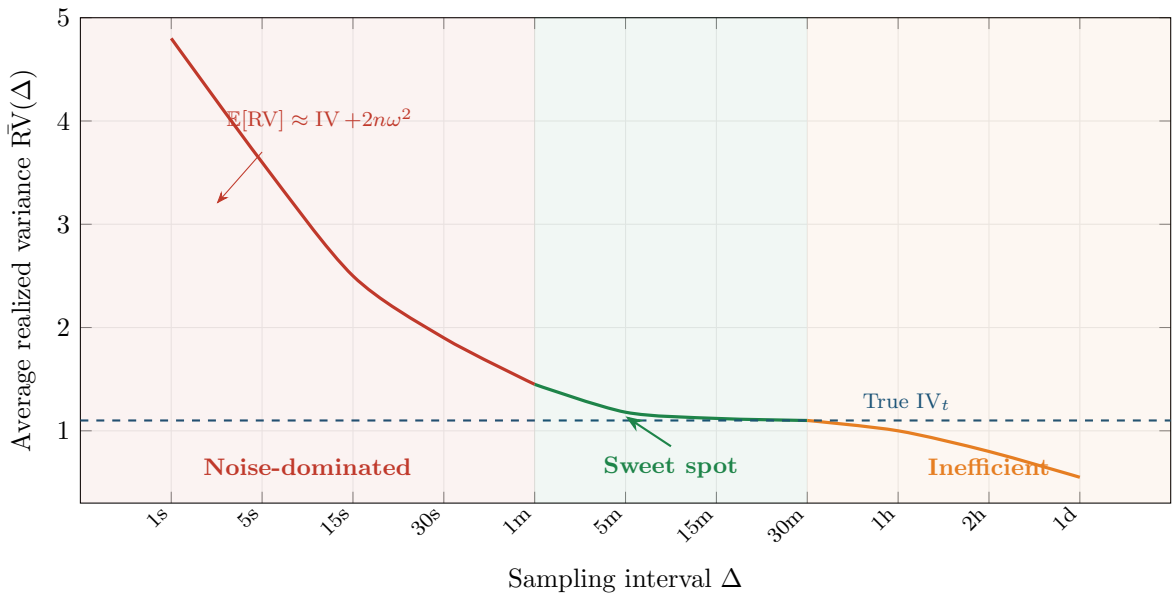


Figure 3.2: Volatility signature plot with three regimes. **Left (red):** noise dominates; RV inflated by $2n\omega^2$. **Center (green):** sweet spot where RV is close to true IV_t. **Right (orange):** too few observations; high estimation variance. The dashed blue line is the true integrated variance. The curve flattens around 1–15 minutes for liquid U.S. equities.

The Volatility Signature Plot Is a Diagnostic, Not a Conclusion

The shape of the volatility signature plot tells you where noise begins to bite for your specific asset. Before computing RV for any analysis, plot $\bar{\text{RV}}(\Delta)$ for a range of frequencies. If the curve is still rising at your chosen frequency, you are in the noise-contaminated region and must either reduce frequency or use a noise-robust estimator from this chapter.

The Sweet Spot Varies by Asset

The 5-minute rule of thumb holds for liquid large-cap equities and major FX pairs. For less liquid instruments (small-cap stocks, emerging-market currencies, corporate bonds), the noise-contaminated region can extend to 15 or even 30 minutes (Hansen and Lunde, 2006). For extremely liquid futures (e.g., E-mini S&P 500), the curve may flatten by 1 minute. Always check empirically.

3.3 Two-Scales Realized Volatility (TSRV)

The volatility signature plot shows that standard RV is badly biased at high frequencies. One approach (Chapter 2) is to avoid the problem by sampling at 5 minutes. A better approach is to *correct* for the noise and use all the high-frequency data. The Two-Scales Realized Volatility (TSRV) estimator, developed by Zhang et al. (2005), is the simplest noise-robust estimator.

The Two-Scales Idea

Compute RV at two different frequencies: a "fast" scale (e.g., every 1 minute) and a "slow" scale (e.g., every 30 minutes). The fast-scale RV picks up both signal and noise: $\text{RV}^{(\text{fast})} \approx \text{IV}_t + 2n_{\text{fast}}\omega^2$. The slow-scale RV also picks up signal and noise, but with far fewer observations: $\text{RV}^{(\text{slow})} \approx \text{IV}_t + 2n_{\text{slow}}\omega^2$. However, the key insight is that the *noise per observation* is the same at both scales. By taking a specific weighted difference of the two, you can cancel the noise term and isolate the signal.

3.3.1 Construction

TSRV works with subsampled realized variances. Instead of computing a single RV on a non-overlapping grid, you average over all possible starting points.

Subsampling

A "subsampled" RV at scale K takes the original n tick prices and computes K separate realized variances, each on a non-overlapping grid offset by one tick. For example, with $K = 3$, you compute RV using ticks $\{1, 4, 7, \ldots\}$, then $\{2, 5, 8, \ldots\}$, then $\{3, 6, 9, \ldots\}$, and average the three. This "average RV" uses all the data while spacing returns K ticks apart.

Denote the subsampled (averaged) RV at scale K as $\text{RV}_t^{(\text{avg},K)}$. TSRV uses a fast scale $K = 1$ (every tick) and a slow scale $K = K_{\text{slow}}$ (spaced-out returns):

Two-Scales Realized Volatility (TSRV)

$$\widehat{\text{IV}}_t^{\text{TSRV}} = \text{RV}_t^{(\text{avg},K_{\text{slow}})} - \frac{\bar{n}_{K_{\text{slow}}}}{n}\,\text{RV}_t^{(\text{all})} \tag{3.2}$$

- $\text{RV}_t^{(\text{avg},K_{\text{slow}})}$: the subsampled (averaged) RV computed with returns spaced K_{slow}

ticks apart
- $\mathrm{RV}_t^{(\mathrm{all})}$: realized variance computed from all n tick-by-tick returns (the "fast scale")
- $\bar{n}_{K_{\mathrm{slow}}} = (n - K_{\mathrm{slow}} + 1)/K_{\mathrm{slow}}$: the average number of returns per subsample at the slow scale
- n: total number of tick-level returns
- The ratio $\bar{n}_{K_{\mathrm{slow}}}/n$ is a small number; the second term removes the noise bias from the first

Why the Subtraction Works

Both $\mathrm{RV}_t^{(\mathrm{avg},K_{\mathrm{slow}})}$ and $\mathrm{RV}_t^{(\mathrm{all})}$ contain a noise bias proportional to $2\omega^2 \times$ (number of returns). The subsampled slow-scale RV has bias $\approx 2\bar{n}_{K_{\mathrm{slow}}}\omega^2$. The all-tick RV has bias $\approx 2n\omega^2$. Subtracting $(\bar{n}_{K_{\mathrm{slow}}}/n) \times \mathrm{RV}_t^{(\mathrm{all})}$ removes exactly $2\bar{n}_{K_{\mathrm{slow}}}\omega^2$ from the slow-scale estimate, canceling the noise.

Why This Matters

TSRV is the conceptual foundation for all noise-robust estimators. Even if you use 5-minute RV for your HAR model, understanding TSRV tells you exactly what you are giving up: the ability to use all available tick data. If you extend the project to intraday forecasting or LOB-based features, TSRV is the simplest estimator to implement as a first noise-robust baseline.

Figure 3.3 illustrates the two scales on the same price path.

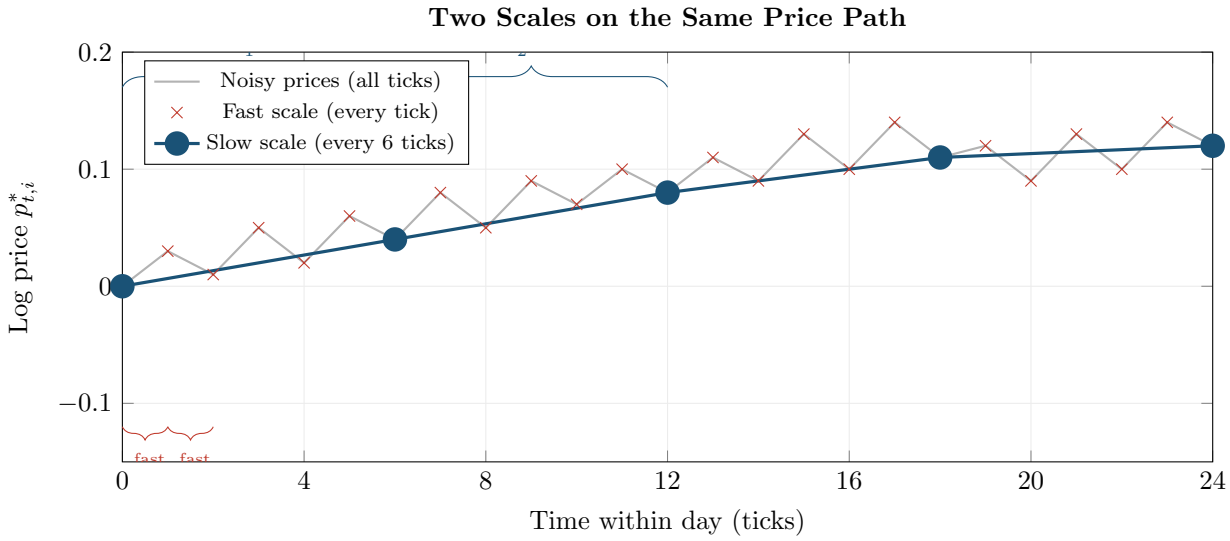


Figure 3.3: TSRV uses two scales on the same price path. **Red:** fast scale (every tick), picks up both signal and noise. **Blue:** slow scale (every 6 ticks), picks up signal with less noise but fewer observations. TSRV subtracts a noise correction derived from the fast scale to debias the slow-scale estimate.

3.3.2 Convergence Rate

Zhang et al. (2005): TSRV Convergence

Under the i.i.d. noise model, the optimal choice of K_{slow} yields a convergence rate of $n^{-1/6}$ for the TSRV estimator:

$$\widehat{\text{IV}}_t^{\text{TSRV}} - \text{IV}_t = O_p(n^{-1/6})$$

For comparison, the standard 5-minute RV (without noise correction) converges at rate $n^{-1/2}$ but only *if you ignore noise*. With noise, standard RV does not converge at all. TSRV is the first estimator that is consistent in the presence of noise.

The $n^{-1/6}$ rate is slower than the noise-free $n^{-1/2}$. This is the price you pay for not knowing the noise level ω^2 a priori. Subsequent estimators (MSRV, realized kernel) improve this rate.

Worked Example:

TSRV on a 6.5-Hour Trading Day Suppose you have tick-by-tick data for a liquid stock with $n = 23{,}400$ ticks (one per second over 6.5 hours). You choose a slow scale of $K_{\text{slow}} = 390$ (equivalent to spacing returns 390 seconds, or 6.5 minutes, apart).

Step 1: Fast-scale RV. Compute $\text{RV}_t^{(\text{all})}$ from all 23,400 one-second returns. Say you get $\text{RV}_t^{(\text{all})} = 3.2 \times 10^{-4}$. This is inflated by noise.

Step 2: Slow-scale subsampled RV. Compute 390 separate RVs, each from a non-overlapping grid of $23{,}400/390 = 60$ returns spaced 390 seconds apart, offset by one tick each time. Average them: $\text{RV}_t^{(\text{avg},390)} = 1.3 \times 10^{-4}$.

Step 3: Noise correction. The average number of returns per subsample is $\bar{n}_{390} = (23{,}400 - 390 + 1)/390 \approx 59$. The correction factor is $\bar{n}_{390}/n = 59/23{,}400 \approx 0.00252$.

Step 4: Combine.

$$\widehat{\text{IV}}_t^{\text{TSRV}} = 1.3 \times 10^{-4} - 0.00252 \times 3.2 \times 10^{-4} = 1.3 \times 10^{-4} - 0.008 \times 10^{-4} \approx 1.29 \times 10^{-4}$$

The correction is small here because the slow scale already removes most noise. In cases where the slow scale is less aggressive (e.g., $K_{\text{slow}} = 30$), the correction is larger. The annualized volatility is $\sqrt{1.29 \times 10^{-4}} \times \sqrt{252} \approx 18.0\%$.

3.4 Multi-Scale Realized Volatility (MSRV)

TSRV uses two scales. A natural extension is to use *many* scales simultaneously. Zhang (2006) developed the Multi-Scale Realized Volatility (MSRV) estimator, which combines information from multiple time scales to achieve a faster convergence rate.

From Two Scales to Many

Think of TSRV as using a single pair of binoculars: one lens zoomed in (fast scale) and one zoomed out (slow scale). MSRV uses an entire array of zoom levels, weighting each one optimally. More zoom levels means more information about how the noise distorts each scale, which means a more precise correction.

Multi-Scale Realized Volatility (MSRV)

MSRV takes a weighted combination of subsampled RVs at J different scales $K_1 < K_2 < \cdots < K_J$:

$$\widehat{\text{IV}}_t^{\text{MSRV}} = \sum_{j=1}^{J} a_j \ \text{RV}_t^{(\text{avg},K_j)} \tag{3.3}$$

- $\text{RV}_t^{(\text{avg},K_j)}$: subsampled RV at scale K_j (same construction as in TSRV)
- a_j: weights chosen to (i) cancel the noise bias and (ii) minimize variance
- J: number of scales; Zhang (2006) showed that the optimal J grows with n
- The weights satisfy $\sum_j a_j = 1$ (so the estimator targets IV_t) and a second constraint that cancels the ω^2 bias

In Plain English

MSRV says: instead of looking at the data through just two zoom levels (fast and slow), look through many zoom levels and combine them with carefully chosen weights. The weights are picked so that all the noise cancels out, while squeezing as much precision as possible from each scale. It is a weighted average of the same subsampled RVs that TSRV uses, but with more of them and smarter weights.

Why This Matters

MSRV proves that the $n^{-1/4}$ rate is achievable and establishes the theoretical efficiency frontier for noise-robust estimation. In practice, the realized kernel (next section) is easier to tune and achieves the same rate, so MSRV is more important as a theoretical benchmark than as a tool you would implement for your vol forecasting pipeline.

Zhang (2006): MSRV Achieves the Optimal Rate

The MSRV estimator achieves the convergence rate $n^{-1/4}$:

$$\widehat{\text{IV}}_t^{\text{MSRV}} - \text{IV}_t = O_p(n^{-1/4})$$

This is the best possible rate for any estimator under the i.i.d. noise model without knowing ω^2. For context: with $n = 23{,}400$ one-second observations, $n^{-1/4} \approx 0.081$, compared to $n^{-1/6} \approx 0.188$ for TSRV. MSRV is roughly 2.3 times more precise than TSRV for the same data.

In practice, MSRV is straightforward to implement (it is a weighted sum of subsampled RVs), but the optimal weight selection requires choosing J and the scale grid $\{K_j\}$. Zhang (2006) provides explicit formulas. For most applied work, the realized kernel (next section) is preferred because it achieves the same $n^{-1/4}$ rate with a simpler tuning procedure.

3.5 The Realized Kernel

The realized kernel, developed by Barndorff-Nielsen et al. (2008), is the most widely used noise-robust estimator in the volatility literature. It achieves the optimal $n^{-1/4}$ convergence rate (same as MSRV) with a clean, intuitive construction.

Noise as Spurious Autocorrelation

Under the i.i.d. noise model, the *observed* returns $r^*_{t,i}$ have negative first-order autocorrelation: if the noise pushes the price up this tick, the next return is more likely to be negative (reverting the noise). This spurious autocorrelation shows up in the autocovariances of returns. The realized kernel idea: compute the autocovariance function of observed returns and use a kernel weighting to remove the noise-induced autocorrelation while keeping the genuine signal.

Autocovariance

The autocovariance at lag h of a sequence $\{x_1, \ldots, x_n\}$ measures the linear association between observations h steps apart:

$$\hat{\gamma}_h = \frac{1}{n} \sum_{i=1}^{n-h} (x_i - \bar{x})(x_{i+h} - \bar{x})$$

For returns, lag 0 autocovariance is the variance. Lag 1 autocovariance measures whether today's return predicts tomorrow's (or this tick's return predicts the next).

Kernel Functions

A kernel function $k(x)$ is a smooth weighting function satisfying $k(0) = 1$ and $k(x) \to 0$ as $|x| \to \infty$. It assigns full weight to the center (lag 0) and smoothly downweights observations further away. Common examples: the Bartlett kernel $k(x) = 1 - |x|$ for $|x| \leq 1$ (a triangle), and the Parzen kernel (a piecewise cubic that is smoother at the boundary).

3.5.1 Construction

The realized kernel takes the autocovariances of observed returns and weights them with a kernel function.

Realized Kernel

$$\widehat{K}_t = \sum_{h=-H}^{H} k\left(\frac{h}{H+1}\right) \hat{\gamma}_h \tag{3.4}$$

- $\hat{\gamma}_h = \sum_{i=1}^{n-|h|} r^*_{t,i}\, r^*_{t,i+|h|}$: the h-th sample autocovariance of observed (noisy) returns; note $\hat{\gamma}_0 = \mathrm{RV}_t^{(\text{noisy})}$
- $k(\cdot)$: a kernel function with $k(0) = 1$, typically the Parzen kernel
- H: bandwidth (number of lags included); controls the bias-variance tradeoff
- The sum runs from $-H$ to H, so both positive and negative lags are included
- $k(h/(H+1))$: the weight assigned to autocovariance at lag h; lags near zero get high weight, distant lags get low weight

In Plain English

The realized kernel starts with the naive RV (the lag-0 autocovariance) and then adds in weighted autocovariances at nearby lags. The negative autocovariances at lag ± 1, caused by bid-ask bounce, partially cancel the noise that inflated the lag-0 term. The kernel function smoothly tapers the weights so that distant lags (which are mostly estimation

noise) contribute very little. The result is an estimate that keeps the true volatility signal while subtracting out the spurious noise component.

Why This Matters

The realized kernel is the default noise-robust estimator in empirical finance and the one you are most likely to encounter in published vol forecasting papers. If you extend your project beyond 5-minute RV (for example, to compute more accurate daily IV estimates or to work with tick-level LOB data), the realized kernel from the `highfrequency` R package is the standard tool. Using it as a feature alongside 5-minute RV in your HAR model could capture information about intraday noise intensity.

The "flat-top" property is important: Barndorff-Nielsen et al. (2008) require $k'(0) = 0$ (the kernel is flat at the origin), which ensures that the noise bias is removed at the correct rate. The Parzen kernel satisfies this.

Figure 3.4 shows how the Parzen kernel assigns weights to different lags.

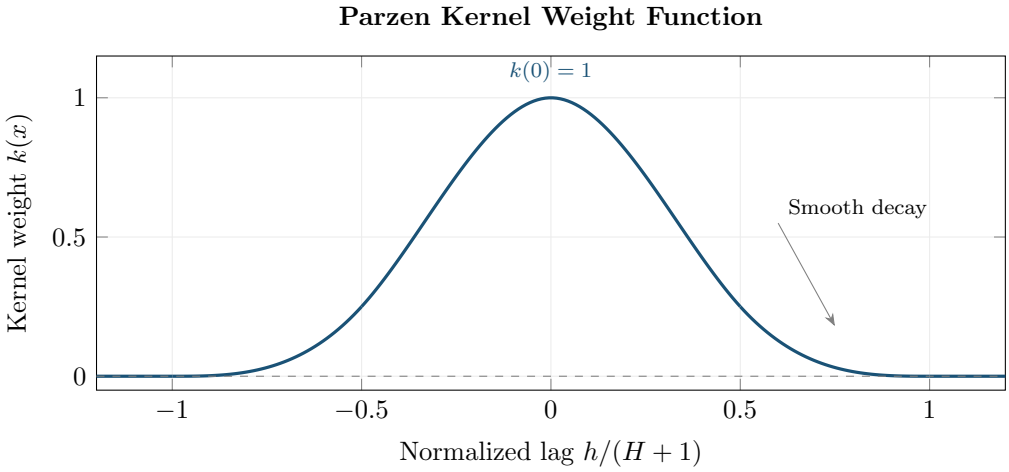


Figure 3.4: The Parzen kernel assigns full weight ($k(0) = 1$) to the zero-lag autocovariance (which is just RV) and smoothly downweights higher lags. Lags beyond H receive zero weight. The flat top ($k'(0) = 0$) ensures correct noise removal.

3.5.2 Why It Works

The logic is:

1. **Lag 0**: The zero-lag autocovariance $\hat{\gamma}_0 = \text{RV}_t^{(\text{noisy})}$ contains the signal plus noise bias.
2. **Lags 1, 2, . . .**: The noise-induced autocorrelation appears at lag ± 1 (and possibly a few more lags for dependent noise). These autocovariances are negative on average, reflecting the bid-ask bounce reversal.
3. **Kernel weighting**: By including these negative autocovariances with appropriate weights, their sum partially cancels the positive noise bias in $\hat{\gamma}_0$.
4. **Higher lags**: Beyond a few lags, the noise-induced autocorrelation dies out, and genuine autocorrelation (if any) is captured.

The Realized Kernel in Practice

The realized kernel is the most widely used noise-robust estimator in empirical finance. It achieves the optimal convergence rate ($n^{-1/4}$), handles moderately dependent noise (not just i.i.d.), has well-understood asymptotic theory, and is implemented in standard packages (`highfrequency` in R, `realized` in MATLAB). If you need a noise-robust daily RV estimate, the realized kernel is the default choice.

3.5.3 Bandwidth Selection

The bandwidth H controls how many autocovariance lags are included.

- **Too small** H: not enough lags to fully remove the noise bias; the estimator is biased upward.
- **Too large** H: too many noisy autocovariance estimates are included; the estimator has high variance.

Barndorff-Nielsen et al. (2008) showed that the optimal bandwidth scales as $H^* \propto n^{3/5}$. For $n = 23{,}400$ one-second observations, this gives $H^* \approx 23{,}400^{3/5} \approx 400$. Data-driven bandwidth selection methods (analogous to those used in spectral density estimation) are available and recommended for applied work.

3.6 Pre-Averaging

Pre-averaging, developed by Jacod et al. (2009), takes a different approach to the noise problem. Instead of correcting the autocovariance structure (as the realized kernel does), it smooths the noise out of the price path *before* computing squared returns.

Smoothing Out the Bounce

If bid-ask bounce makes individual prices noisy, averaging a few adjacent prices should cancel some of the noise (positive and negative noise terms offset). Pre-averaging does exactly this: replace each price $p^*_{t,i}$ with a local average of nearby prices, then compute returns and sum their squares. The local averaging shrinks the noise while preserving the signal.

3.6.1 Construction

Pre-Averaged Realized Variance

Choose a block length L and a weight function $g : [0,1] \to \mathbb{R}$ (typically $g(x) = \min(x, 1-x)$). Define the pre-averaged price:

$$\bar{p}^*_{t,i} = \sum_{j=1}^{L-1} g\left(\frac{j}{L}\right) \Delta p^*_{t,i+j} \tag{3.5}$$

where $\Delta p^*_{t,i+j} = p^*_{t,i+j} - p^*_{t,i+j-1}$ is the tick-by-tick return. The pre-averaged realized variance is:

$$\widehat{\mathrm{IV}}^{\mathrm{PA}}_t = \frac{1}{L\psi_2} \sum_{i=0}^{n-L} (\bar{p}^*_{t,i})^2 - \frac{\psi_1}{2L\psi_2}\, \mathrm{RV}^{(\mathrm{all})}_t \tag{3.6}$$

- $\bar{p}^*_{t,i}$: the pre-averaged return at position i (a weighted sum of $L-1$ tick returns)
- $g(j/L)$: weight function; the standard choice $g(x) = \min(x, 1-x)$ gives a triangular shape that ramps up then ramps down

- $\psi_1 = \int_0^1 [g'(x)]^2\,dx$ and $\psi_2 = \int_0^1 [g(x)]^2\,dx$: constants determined by the weight function
- $\mathrm{RV}_t^{(\mathrm{all})}$: tick-by-tick RV (used for a small bias correction, analogous to the TSRV correction)
- L: block length; the optimal choice is $L \propto n^{1/2}$

In Plain English

Pre-averaging works in two steps. First, it replaces each noisy price with a local moving average of nearby prices (Equation 3.5), which smooths out the bid-ask bounce in the same way that averaging poll results smooths out sampling error. Second, it computes a sum of squared "pre-averaged returns" (Equation 3.6), with a small bias correction subtracted (the second term), playing the same debiasing role as the fast-scale correction in TSRV.

Why This Matters

Pre-averaging is particularly relevant if your project uses tick-level order book data, because it provides a natural way to construct "clean" returns from noisy transaction prices before feeding them into ML features. The block length L is the single tuning parameter, and its optimal value $L \propto n^{1/2}$ means you can set it automatically from the number of ticks in a day.

Figure 3.5 compares raw and pre-averaged returns.

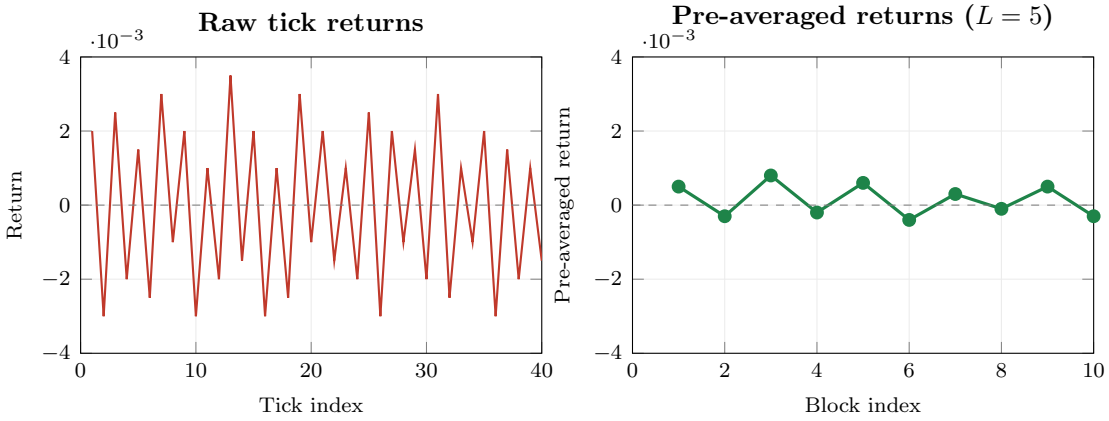


Figure 3.5: Raw tick returns (left, red) versus pre-averaged returns (right, green) with block length $L = 5$. The raw returns show large, alternating moves (bid-ask bounce). Pre-averaging smooths out the noise, producing smaller, more stable returns that better reflect true price variation.

Jacod et al. (2009): Pre-Averaging Convergence

With the optimal block length $L \propto n^{1/2}$, the pre-averaged estimator achieves the optimal convergence rate $n^{-1/4}$, the same as MSRV and the realized kernel. The pre-averaging approach also provides a feasible central limit theorem, enabling confidence intervals for integrated variance.

3.7 Other Estimators

The four estimators above (TSRV, MSRV, realized kernel, pre-averaging) are the most cited. Several other approaches exist and are worth brief mention.

Subsampled/Averaged RV. Before TSRV was developed, practitioners used averaged RV (computing multiple RVs on offset grids and averaging them) to reduce noise. This is the slow-scale component of TSRV without the bias correction. It reduces noise variance but does not eliminate the bias. It remains a useful quick fix when the noise level is low.

Fourier Estimator. Malliavin and Mancino (2002, 2009) proposed estimating integrated variance via the Fourier coefficients of the price process. The idea: the variance is related to the squared magnitude of low-frequency Fourier coefficients, while noise concentrates at high frequencies. By truncating high-frequency components, you filter out the noise. The Fourier estimator is less widely used in practice but has elegant theoretical properties and does not require equally spaced observations.

Quasi-Maximum Likelihood Estimator (QMLE). Xiu (2010) proposed treating the noisy price observations as a state-space model: the true price is the latent state, and the observed price adds Gaussian noise. Applying the Kalman filter produces a quasi-maximum likelihood estimate of integrated variance and the noise variance ω^2 jointly. The QMLE achieves the optimal $n^{-1/4}$ rate and provides a natural estimate of ω^2 as a byproduct.

All Roads Lead to $n^{-1/4}$

Under the standard i.i.d. noise model, the best possible convergence rate is $n^{-1/4}$. All of the modern estimators (MSRV, realized kernel, pre-averaging, QMLE) achieve this rate. They differ in implementation details, robustness to dependent noise, and ease of tuning, but their asymptotic efficiency is the same.

3.8 Which Estimator to Use

You now have a menu of noise-robust estimators. The practical question is: which one should you use? The answer depends on your goal.

Figure 3.6 shows how the different estimators behave on the volatility signature plot. The key visual takeaway: noise-robust estimators produce a flat line across frequencies, while naive RV diverges.

3.8.1 Comparison Table

Table 3.1 summarizes the key properties.

3.8.2 Decision Flowchart

Figure 3.7 provides a practical decision tree.

Practical Guidance

For daily RV forecasting (Chapters 6 and 10), simple 5-minute RV suffices. Liu et al. (2015) showed that noise-robust estimators rarely produce better *forecasts* of future volatility, even though they produce more accurate *estimates* of today's integrated variance. For estimation accuracy, intraday analysis, or when working with tick data in Project 2, use

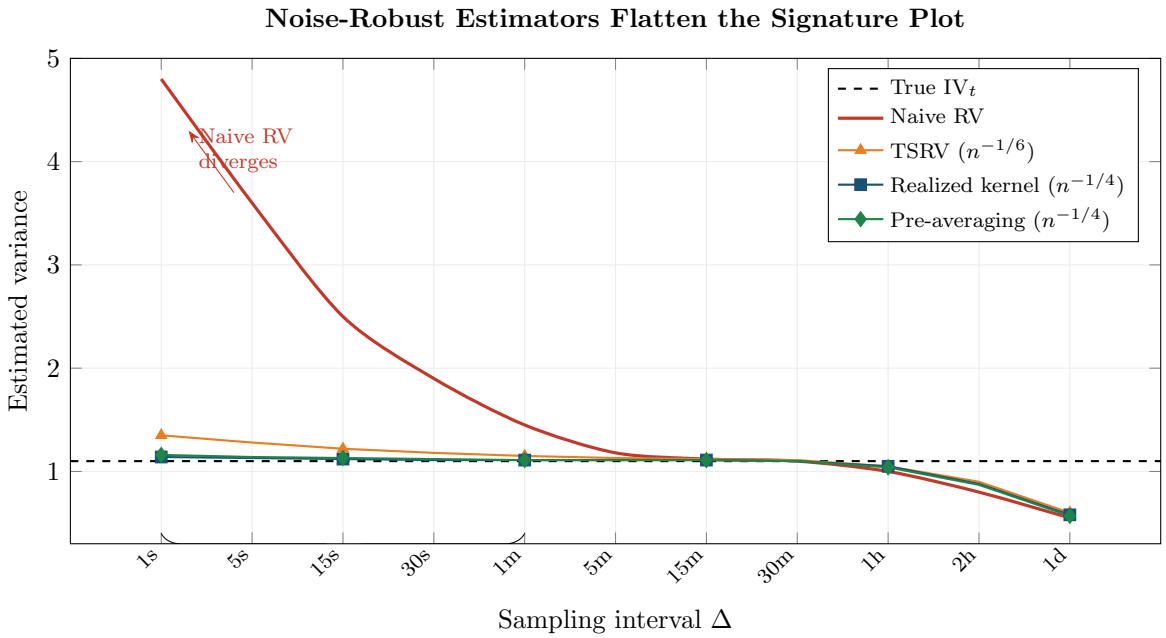


Figure 3.6: Volatility signature plot comparing estimators. Naive RV (red) diverges at high frequencies due to noise. TSRV (orange) is consistent but has a slower convergence rate, producing slight upward bias at the highest frequencies. The realized kernel (blue) and pre-averaging (green) achieve the optimal $n^{-1/4}$ rate and remain close to the true IV_t across all frequencies. At very low frequencies (right side), all estimators lose precision due to too few observations.

Table 3.1: Comparison of noise-robust realized variance estimators.

Estimator	**Rate**	**Noise assumption**	**Practical notes**
5-min RV	$n^{-1/2}$ (no noise)	Avoids noise	Simple; hard to beat for forecasting (Liu et al., 2015)
TSRV	$n^{-1/6}$	i.i.d.	First consistent estimator with noise; easy to implement
MSRV	$n^{-1/4}$	i.i.d.	Optimal rate; weight selection requires tuning
Realized kernel	$n^{-1/4}$	Dependent OK	Most widely used; Parzen kernel standard; bandwidth data-driven
Pre-averaging	$n^{-1/4}$	Dependent OK	Intuitive; provides feasible CLT; block length $L \propto n^{1/2}$
QMLE	$n^{-1/4}$	Gaussian noise	Joint estimate of IV and ω^2; state-space framework

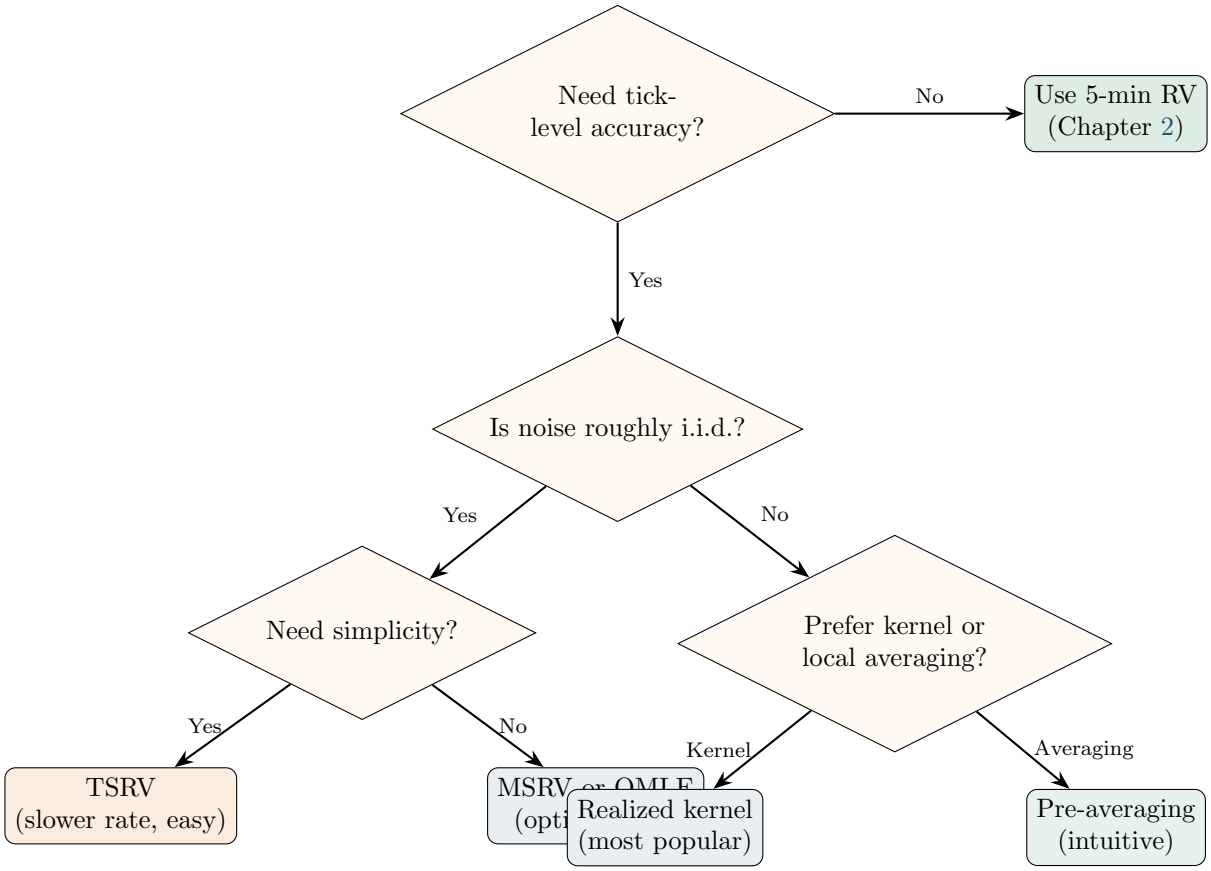


Figure 3.7: Decision tree for choosing a noise-robust estimator. For daily volatility forecasting, 5-min RV is the starting point. For intraday analysis or when estimation accuracy matters, use the realized kernel (default) or pre-averaging. TSRV is the simplest noise-robust estimator and a good starting point for learning.

the realized kernel as the default.

Why This Matters

For your HAR-based vol forecasting project, the practical takeaway from this chapter is threefold. First, 5-minute RV is your primary target variable, and it works well precisely because it sidesteps the noise problem. Second, the *difference* between noise-robust RV (e.g., realized kernel) and 5-minute RV on the same day is itself a signal: it measures how much microstructure noise is present, which proxies for liquidity and can be used as an ML feature. Third, if you extend the project to higher-frequency forecasting or LOB data, you now know exactly which estimators to reach for and why.

Summary

- **Three noise sources**: bid-ask bounce (dominant for liquid assets), discrete tick sizes, and price staleness each corrupt high-frequency prices.
- **Noise model**: observed log price $p^* = p + \varepsilon$, where ε is mean-zero noise with variance ω^2. Under this model, noisy RV has bias $2n\omega^2$, which diverges as $n \to \infty$.
- **Volatility signature plot**: plot average RV vs. sampling frequency. If the curve is still rising at your chosen frequency, you are in the noise-contaminated region. Always check this diagnostic before computing RV.
- **TSRV** (Zhang et al., 2005): combines a fast-scale and slow-scale RV to cancel noise. First consistent estimator with noise. Convergence rate $n^{-1/6}$.
- **MSRV** (Zhang, 2006): extends TSRV to multiple scales, achieving the optimal convergence rate $n^{-1/4}$.
- **Realized kernel** (Barndorff-Nielsen et al., 2008): weights the autocovariances of observed returns with a flat-top kernel (e.g., Parzen). Handles dependent noise. Rate $n^{-1/4}$. The most widely used noise-robust estimator.
- **Pre-averaging** (Jacod et al., 2009): smooths prices over local blocks before computing squared returns. Rate $n^{-1/4}$. Provides a feasible CLT for inference.
- **Other approaches**: subsampled/averaged RV (simple but biased), Fourier estimator (Malliavin and Mancino, 2002), QMLE (Xiu, 2010).
- **All optimal estimators converge at** $n^{-1/4}$ under i.i.d. noise. They differ in robustness to dependent noise, implementation complexity, and tuning requirements.
- **For forecasting**, 5-minute RV is hard to beat (Liu et al., 2015). Noise-robust estimators improve *estimation* accuracy but these gains rarely translate into forecast improvements.
- **For estimation and intraday work**, the realized kernel is the default choice. Pre-averaging is a close second.
- Chapter 4 addresses a separate contamination: jump variation mixed into the continuous component.
- Chapter 10 uses both 5-minute RV and noise-robust estimators as ML features.

Table 3.2: Key results referenced in this chapter.

Paper	**Result**	**Relevance**
Hansen and Lunde (2006)	Characterized the effect of microstructure noise on realized variance; showed noise causes divergence as sampling frequency increases; documented that the i.i.d. noise model is a useful first approximation.	Foundational empirical analysis of the noise problem; motivates all robust estimators.
Aït-Sahalia et al. (2005)	Showed that optimal sampling frequency balances bias (from noise) and variance (from fewer observations); derived the optimal Δ^* under the i.i.d. noise model.	Theoretical foundation for the volatility signature plot and the bias-variance tradeoff.
Zhang et al. (2005)	Introduced TSRV: first estimator consistent for integrated variance in the presence of noise. Convergence rate $n^{-1/6}$.	The simplest noise-robust estimator; foundational for MSRV and other extensions.
Zhang (2006)	Extended TSRV to multiple scales (MSRV), achieving the optimal convergence rate $n^{-1/4}$.	Proved the $n^{-1/4}$ efficiency bound for noise-contaminated estimation.
Barndorff-Nielsen et al. (2008)	Developed the realized kernel: flat-top kernel weighting of autocovariances. Rate $n^{-1/4}$, handles dependent noise.	The most widely used noise-robust estimator in practice.
Jacod et al. (2009)	Introduced pre-averaging: smooth prices locally before computing RV. Rate $n^{-1/4}$, feasible CLT.	Intuitive alternative to the realized kernel.
Xiu (2010)	Proposed QMLE via Kalman filter on a state-space model. Rate $n^{-1/4}$, jointly estimates IV and ω^2.	Useful when you also need a noise variance estimate.
Liu et al. (2015)	Compared ∼400 estimators across 31 assets: noise-robust estimators rarely improve *forecasts* over 5-min RV.	Key practical finding: estimation accuracy ≠ forecast accuracy.

Chapter 4

Jumps and Continuous Variation

Why This Chapter

Separating jump variation from continuous variation is critical for forecasting. The HAR-J and HAR-CJ extensions (Chapter 6) split the RV signal into components with different persistence and predictability. Signed jump features appear in Chapter 10. Projects 1 and 4 use jump-decomposed features.

Chapter 2 showed that realized variance RV_t converges to the quadratic variation of the log-price process. When the price path is continuous (no jumps), quadratic variation equals integrated variance, and RV_t is a clean estimator of IV_t. But prices do jump. Earnings surprises, central bank announcements, and flash crashes all produce sudden, large price moves that do not come from the smooth diffusion process. When jumps are present, RV_t picks up *both* continuous and jump variation (Equation 2.5 in Chapter 2). This chapter develops the tools to separate them.

4.1 Why Prices Jump

Prices move for two fundamentally different reasons. Most of the time, they drift and fluctuate continuously as traders gradually digest information, adjust positions, and provide liquidity. This smooth fluctuation is the *diffusion* component. Occasionally, a piece of news is so large or so sudden that the price discontinuously leaps to a new level. This is a *jump*.

Concrete examples of jump triggers:

- **Earnings announcements:** A company reports revenue 15% above consensus after the close. The stock gaps up 8% at the next open.
- **Macro releases:** Non-farm payrolls (NFP) come in at +350k vs. a +180k forecast. Treasury yields spike 15 basis points in seconds.
- **Central bank decisions:** The Fed surprises with a 50bp rate cut when 25bp was priced. Equity indices jump 2% in one tick.
- **Flash crashes:** The May 6, 2010 flash crash sent the Dow down nearly 1,000 points (roughly 9%) in minutes before recovering.
- **Geopolitical shocks:** The Swiss National Bank abandoned its EUR/CHF floor on January 15, 2015. EUR/CHF dropped roughly 30% in seconds.

Diffusion vs. Jump

Normal price movements are many small moves (diffusion). A jump is a sudden, large move that cannot be explained by the smooth diffusion process. An earnings surprise that moves a stock 8% in one tick is a jump. A stock drifting 0.05% per 5-minute bar over the course of an afternoon is diffusion. The mathematical distinction is sharp: diffusion is a continuous path, while a jump is a discontinuity in the path.

The Jump-Diffusion Price Process

Chapter 2 introduced the pure-diffusion log-price process: $dp_t = \mu_t\,dt + \sigma_t\,dW_t$. Adding jumps extends this to:

$$dp_t = \mu_t\,dt + \sigma_t\,dW_t + J_t\,dN_t$$

where N_t is a counting process (it increases by 1 each time a jump occurs) and J_t is the jump size (a random variable, possibly negative). You do not need to work with this process directly. The key takeaway: the price path now has two sources of variation, and the quadratic variation captures both:

$$[p]_t = \underbrace{\int_{t-1}^{t} \sigma_s^2\,ds}_{\text{continuous}} + \underbrace{\sum_{s\in(t-1,t]} J_s^2}_{\text{jumps}}$$

This chapter is about estimating each piece separately.

4.1.1 Diagram: Continuous vs. Jump Price Paths

Figure 4.1 shows two price paths over the same day. The left path evolves smoothly (pure diffusion). The right path is identical except for a single large jump at 2:15pm.

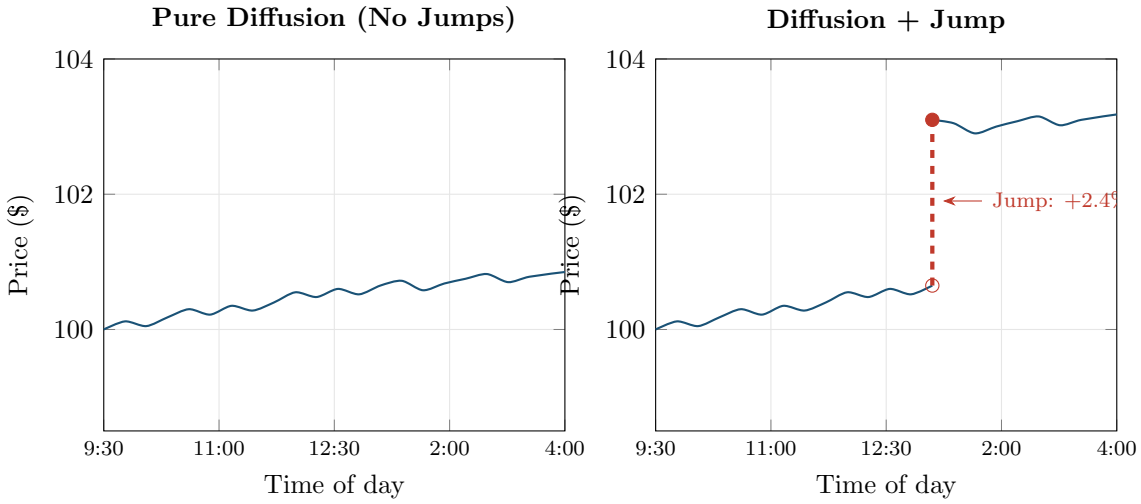


Figure 4.1: Two price paths over the same trading day. **Left:** pure diffusion with no jumps. The path is continuous and smooth. **Right:** diffusion plus a single jump at 2:00pm (red dashed line). The open circle marks the pre-jump price; the filled circle marks the post-jump price. Both paths have similar diffusion volatility, but the right path has much higher quadratic variation due to the jump.

4.2 Bipower Variation

You now know that RV_t captures both continuous and jump variation. The next step is to build an estimator that captures *only* the continuous part. The key idea, due to Barndorff-Nielsen and Shephard (2004), is bipower variation.

Why Multiplying Neighbors Kills Jumps

RV_t squares each return, so a single jump return of +2% contributes $(0.02)^2 = 4 \times 10^{-4}$ to the sum. Bipower variation instead multiplies each return's absolute value by its neighbor's absolute value. A jump return of +2% is large, but its neighbor (one interval before or after) is typically a normal-sized diffusion return, say 0.05%. The product $|0.02| \times |0.0005| = 1 \times 10^{-5}$ is far smaller than the squared jump 4×10^{-4}. The jump's contribution is "diluted" by the normal-sized neighbor. This is the core mechanism: jumps are isolated events, so a product of consecutive returns dampens them.

Bipower Variation

We need an estimator that converges to integrated variance even when jumps are present. The idea: instead of squaring each return (which amplifies jumps), multiply consecutive absolute returns so that an isolated jump is dampened by its normal-sized neighbor.

$$\text{BPV}_t = \frac{\pi}{2} \sum_{i=2}^{n} |r_{t,i}| \, |r_{t,i-1}| \tag{4.1}$$

- BPV_t: bipower variation for day t
- $|r_{t,i}|$: absolute value of the i-th intraday log return
- $|r_{t,i-1}|$: absolute value of the adjacent (previous) intraday log return
- The sum runs from $i = 2$ to n (you need pairs of consecutive returns, so you lose one observation)
- $\frac{\pi}{2} \approx 1.5708$: a scaling constant that corrects for the fact that $\mathbb{E}[|Z|] = \sqrt{2/\pi}$ when $Z \sim \mathcal{N}(0,1)$

Why This Matters

BPV_t is the workhorse estimator you will use to construct the continuous variation component C_t for HAR-J and HAR-CJ models (Chapter 6). Without it, your forecasting model receives RV_t contaminated by unpredictable jumps, which dilutes the persistent signal that drives forecast accuracy.

4.2.1 Why the $\pi/2$ Scaling Factor

The scaling factor deserves its own explanation, because it looks arbitrary until you see where it comes from.

Mean Absolute Value of a Standard Normal

If $Z \sim \mathcal{N}(0,1)$, then $\mathbb{E}[|Z|] = \sqrt{2/\pi} \approx 0.7979$. This follows from integrating the folded normal density. The important consequence: $\mathbb{E}[|Z|]^2 = 2/\pi$, which is *not* equal to $\mathbb{E}[Z^2] = 1$. Taking absolute values and then multiplying introduces a bias relative to squaring, and the scaling factor corrects for it.

Consider two consecutive diffusion returns with no jumps. Each return is approximately $r_{t,i} \approx$

$\sigma_{t,i}\,\epsilon_i$, where ϵ_i is standard normal. The product of absolute values is:

$$|r_{t,i}|\,|r_{t,i-1}| \approx \sigma_{t,i}\,\sigma_{t,i-1}\,|\epsilon_i|\,|\epsilon_{i-1}|$$

Taking expectations:

$$\mathbb{E}\big[|r_{t,i}|\,|r_{t,i-1}|\big] \approx \sigma_{t,i}\,\sigma_{t,i-1}\,\big(\mathbb{E}[|\epsilon|]\big)^2 = \sigma_{t,i}\,\sigma_{t,i-1}\cdot\frac{2}{\pi}$$

To make the sum converge to $\int \sigma_s^2\,ds$ (integrated variance), you need to undo the $2/\pi$ factor. Multiplying by $\pi/2$ does exactly that.

Barndorff-Nielsen and Shephard (2004): BPV Consistency

Under mild regularity conditions, bipower variation converges in probability to integrated variance even in the presence of finite-activity jumps:

$$\mathrm{BPV}_t \xrightarrow{p} \mathrm{IV}_t = \int_{t-1}^{t} \sigma_s^2\,ds \quad \text{as } n \to \infty$$

This result holds regardless of whether jumps occurred during day t. In contrast, RV_t converges to integrated variance *plus* the sum of squared jumps.

4.2.2 Diagram: How RV and BPV Respond to a Jump

Figure 4.2 shows the key mechanism. A sequence of six 5-minute returns includes one jump return (r_4). The left panel shows each return's contribution to RV_t (its squared value). The right panel shows each return's contribution to BPV_t (its absolute value times the previous absolute value, scaled by $\pi/2$).

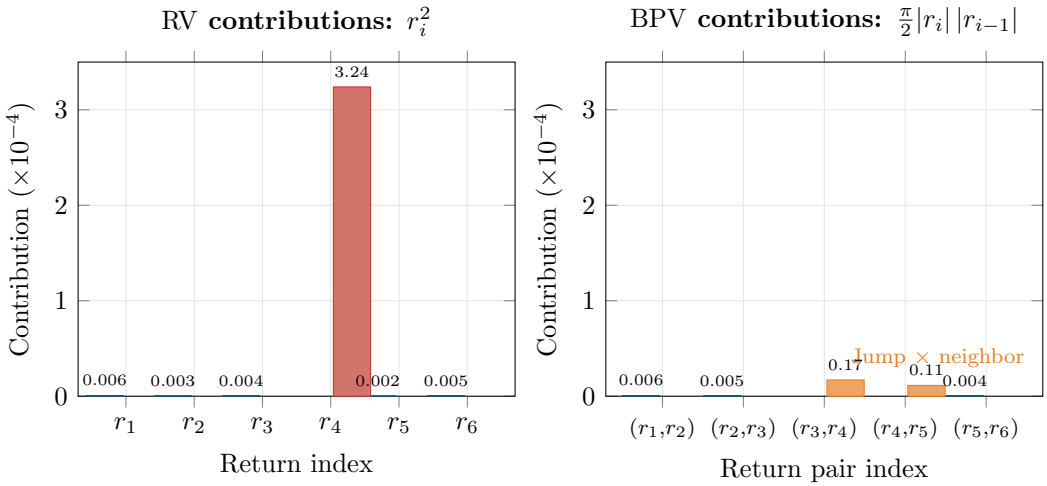


Figure 4.2: How a jump affects RV vs. BPV. **Left:** the jump return r_4 contributes 3.24×10^{-4} to RV (via r_4^2), completely dominating all other terms. **Right:** in BPV, the jump is multiplied by its normal-sized neighbors. The two terms involving r_4 contribute only 0.17 and 0.11×10^{-4}, roughly 50× smaller than the squared jump. BPV "sees through" the jump and recovers the continuous variation.

Worked Example:

Computing RV and BPV for a Day with a Jump A stock is sampled every 5 minutes. You observe the following six returns (a shortened day for illustration):

Index	**Return** r_i	r_i^2	$\lvert r_i\rvert\,\lvert r_{i-1}\rvert$	$\frac{\pi}{2}\,\lvert r_i\rvert\,\lvert r_{i-1}\rvert$
1	$+0.0008$	0.64×10^{-6}	—	—
2	-0.0005	0.25×10^{-6}	4.00×10^{-7}	6.28×10^{-7}
3	$+0.0006$	0.36×10^{-6}	3.00×10^{-7}	4.71×10^{-7}
4	$+0.0180$	324.0×10^{-6}	10.80×10^{-6}	16.96×10^{-6}
5	-0.0004	0.16×10^{-6}	7.20×10^{-6}	11.31×10^{-6}
6	$+0.0007$	0.49×10^{-6}	2.80×10^{-7}	4.40×10^{-7}

Return $r_4 = +0.0180$ (a 1.8% move in a single 5-minute bar) is the jump. The other returns are typical: 4–8 basis points each.

Step 1: Compute RV_t**.**

$$\mathrm{RV}_t = \sum_{i=1}^{6} r_i^2 = (0.64 + 0.25 + 0.36 + 324.0 + 0.16 + 0.49)\times10^{-6} = 325.90\times10^{-6}$$

The jump return contributes $324.0/325.9 = 99.4\%$ of total RV.

Step 2: Compute BPV_t**.**

$$\mathrm{BPV}_t = \frac{\pi}{2}\sum_{i=2}^{6} \lvert r_i\rvert\,\lvert r_{i-1}\rvert = (6.28 + 4.71 + 16{,}960 + 11{,}310 + 4.40)\times10^{-7}\times\frac{1}{10}$$

Summing the $\frac{\pi}{2}|r_i||r_{i-1}|$ column:

$$\mathrm{BPV}_t = (0.628 + 0.471 + 16.96 + 11.31 + 0.440)\times10^{-6} = 29.81\times10^{-6}$$

Step 3: Estimate the jump component.

$$J_t^2 = \mathrm{RV}_t - \mathrm{BPV}_t = 325.90\times10^{-6} - 29.81\times10^{-6} = 296.09\times10^{-6}$$

Interpretation. RV_t is inflated by the jump: $\sqrt{325.9\times10^{-6}} = 1.81\%$ daily vol. BPV_t strips the jump and recovers the diffusion component: $\sqrt{29.81\times10^{-6}} = 0.55\%$ daily vol. The jump component $J_t^2 = 296\times10^{-6}$ accounts for 91% of the quadratic variation, consistent with a single large jump in an otherwise quiet day.

Note: BPV_t is not zero in the jump terms because the jump (r_4) is still multiplied by its (normal) neighbors. But those products are far smaller than r_4^2. With 78 returns instead of 6, the effect would be even more diluted.

4.3 The BNS Jump Test

BPV_t gives you an estimate of continuous variation, and $\mathrm{RV}_t - \mathrm{BPV}_t$ gives you an estimate of jump variation. But both are estimates, subject to sampling noise. On any given day, $\mathrm{RV}_t - \mathrm{BPV}_t$ will be slightly positive even if no jump occurred, simply due to estimation error. You need a formal statistical test to determine whether the difference is large enough to declare a jump day.

The Logic of the Jump Test

If no jumps occurred, RV_t and BPV_t are both consistent estimators of the same quantity (IV_t), so their difference should be close to zero. If the difference is "too large" relative to sampling noise, you reject the null and declare that a jump occurred. The test statistic measures how many standard deviations the difference is from zero.

Barndorff-Nielsen and Shephard (2006) proposed the following test.

BNS Jump Test Statistic

We know $\text{RV}_t - \text{BPV}_t$ estimates the jump component, but on any given day this difference is noisy. We need a way to ask: "Is this difference large enough to be statistically significant, or is it just estimation noise?" The BNS test standardizes the difference by its sampling variability.

$$Z_{\text{BNS},t} = \frac{\text{RV}_t - \text{BPV}_t}{\sqrt{\vartheta \, \max\left(\frac{1}{n}\,\text{RQ}_t,\; 10^{-10}\right)}} \tag{4.2}$$

- $Z_{\text{BNS},t}$: the test statistic for day t
- $\text{RV}_t - \text{BPV}_t$: the estimated jump component (the difference between realized variance and bipower variation)
- $\vartheta = (\pi^2/4) + \pi - 5 \approx 0.6090$: a constant from the asymptotic theory
- $\text{RQ}_t = n\frac{\pi^2}{4}\frac{1}{3}\sum_{i=3}^{n} |r_{t,i}|\,|r_{t,i-1}|\,|r_{t,i-2}|$: realized quarticity, a consistent estimator of integrated quarticity $\int \sigma_s^4\, ds$, needed to scale the variance of the numerator
- n: number of intraday returns
- $\max(\cdot,\, 10^{-10})$: a numerical safeguard to prevent division by zero
- Under the null hypothesis of no jumps, $Z_{\text{BNS},t} \xrightarrow{d} \mathcal{N}(0,1)$ as $n \to \infty$

In Plain English

This is a signal-to-noise ratio. The numerator measures how much jump variation you think occurred (the gap between RV and BPV). The denominator measures how noisy that estimate is (derived from realized quarticity, which captures the variability of the variability). If the ratio exceeds a standard normal critical value, the jump is real, not a statistical artifact.

Why This Matters

The BNS test gives you a daily binary jump indicator: did a statistically significant jump occur on day t? In your HAR-J model, this indicator determines whether you include the jump component $J_t^2 = \max(\text{RV}_t - \text{BPV}_t, 0)$ as a separate regressor. Days without significant jumps get $J_t^2 = 0$, keeping the jump feature clean and preventing estimation noise from leaking in.

How to Use the BNS Test

Compare $Z_{\text{BNS},t}$ to standard normal critical values. At the 5% level (one-sided), declare a jump day if $Z_{\text{BNS},t} > 1.645$. At the 1% level, use 2.326. If $Z_{\text{BNS},t} \leq 1.645$, you cannot reject the null of no jumps, so you treat RV_t as pure continuous variation for that day.

Quarticity

Just as variance is the integral of σ^2, *quarticity* is the integral of σ^4:

$$\int_{t-1}^{t} \sigma_s^4 \, ds$$

This quantity controls the variance of the $\mathrm{RV}_t - \mathrm{BPV}_t$ difference. You never need to compute it by hand. The realized quarticity estimator RQ_t does it for you using products of three consecutive absolute returns, analogous to how BPV uses products of two.

Worked Example:

Applying the BNS Jump Test Suppose you compute the following quantities for a single day using 78 five-minute returns ($n = 78$):

- $\mathrm{RV}_t = 2.50 \times 10^{-4}$
- $\mathrm{BPV}_t = 1.80 \times 10^{-4}$
- $\mathrm{RQ}_t = 5.00 \times 10^{-8}$

Step 1: Compute the numerator.

$$\mathrm{RV}_t - \mathrm{BPV}_t = 2.50 \times 10^{-4} - 1.80 \times 10^{-4} = 0.70 \times 10^{-4}$$

Step 2: Compute the denominator.

$$\sqrt{\vartheta \cdot \frac{1}{n} \mathrm{RQ}_t} = \sqrt{0.6090 \times \frac{5.00 \times 10^{-8}}{78}} = \sqrt{0.6090 \times 6.41 \times 10^{-10}}$$

$$= \sqrt{3.90 \times 10^{-10}} = 1.975 \times 10^{-5}$$

Step 3: Compute the test statistic.

$$Z_{\mathrm{BNS},t} = \frac{0.70 \times 10^{-4}}{1.975 \times 10^{-5}} = \frac{7.00 \times 10^{-5}}{1.975 \times 10^{-5}} = 3.54$$

Step 4: Compare to critical value. $Z_{\mathrm{BNS},t} = 3.54 > 2.326$ (1% critical value). Reject the null of no jumps at the 1% level. This day has a statistically significant jump component.

The continuous variation estimate is $\mathrm{BPV}_t = 1.80 \times 10^{-4}$, and the estimated jump variation is 0.70×10^{-4}, roughly 28% of the total quadratic variation.

BNS Test Size Distortion in Finite Samples

The BNS test has known size distortions in finite samples: with 78 returns per day, it can over-reject the null (detect jumps that are not there) or under-reject (miss small jumps). Huang and Tauchen (2005) documented these problems and proposed a ratio form of the test statistic that has better finite-sample properties. For production use, consider the ratio variant or the corrected versions in Barndorff-Nielsen and Shephard (2006).

4.4 Lee-Mykland: Detecting Individual Jumps

The BNS test answers: "Did a jump occur at any point during day t?" It does not tell you *when* the jump happened or how large it was. If you want intraday jump timing and sizes, you need the Lee and Mykland (2008) test.

From Daily to Intraday Jump Detection

The BNS test is like checking whether a patient had a fever at some point during the day. The Lee-Mykland test is like checking the patient's temperature every hour and flagging the specific times when fever occurred. It tests each individual return against a local volatility estimate to see if that return is "too large" to be diffusion.

The Lee-Mykland test works as follows. For each intraday return $r_{t,i}$, compute a local volatility estimate from a window of recent returns (excluding the return being tested). Then standardize the return by this local volatility. If the standardized return exceeds a threshold derived from extreme-value theory, flag it as a jump.

Lee-Mykland Test Statistic

To pinpoint exactly which returns within the day are jumps, we standardize each return by a local estimate of diffusion volatility. If a return is far too large to have come from the local diffusion process, it must be a jump.

$$\mathcal{L}_{t,i} = \frac{|r_{t,i}|}{\hat{\sigma}_{t,i}} \tag{4.3}$$

- $\mathcal{L}_{t,i}$: the test statistic for the i-th return on day t
- $|r_{t,i}|$: absolute value of the return being tested
- $\hat{\sigma}_{t,i}$: local volatility estimate from a window of K returns centered around (but excluding) return i, typically computed as $\hat{\sigma}_{t,i} = \sqrt{\text{BPV}_K / K}$, where BPV_K is the bipower variation over the K-return window

A return is classified as a jump if the statistic exceeds a critical value from the Gumbel distribution (extreme-value theory), after centering and scaling by C_n and S_n (constants that depend on the number of observations n):

$$\frac{\mathcal{L}_{t,i} - C_n}{S_n} > \beta_\alpha \tag{4.4}$$

where β_α is the $(1 - \alpha)$ quantile of the standard Gumbel distribution.

In Plain English

The Lee-Mykland statistic asks: "How many local standard deviations is this return?" A normal diffusion return might be 1 or 2 local standard deviations. A jump will be 5, 10, or more. The Gumbel critical value accounts for the fact that with many returns per day, even under pure diffusion you expect a few moderately large ones by chance. Only returns that exceed this multiple-testing-adjusted threshold are flagged as jumps.

Why This Matters

Lee-Mykland gives you jump *times and sizes*, not just a daily indicator. This is the basis for signed jump features: you can separately measure "bad jumps" (large negative returns) and "good jumps" (large positive returns) within the day. Signed jump variation feeds directly into the SHAR model and the asymmetric jump features in Chapter 10, where negative jumps predict higher future volatility than positive jumps of the same magnitude.

Lee and Mykland (2008): Intraday Jump Detection

The Lee-Mykland test provides both jump detection and jump timing at intraday frequency. It identifies the specific returns within the day that are jumps, along with their sizes. At standard significance levels, the test correctly identifies large jumps (those exceeding roughly 3–5 local standard deviations) with high power, while maintaining reasonable size control.

4.4.1 Diagram: Intraday Jump Detection

Figure 4.3 shows an intraday return path with detected jump returns highlighted.

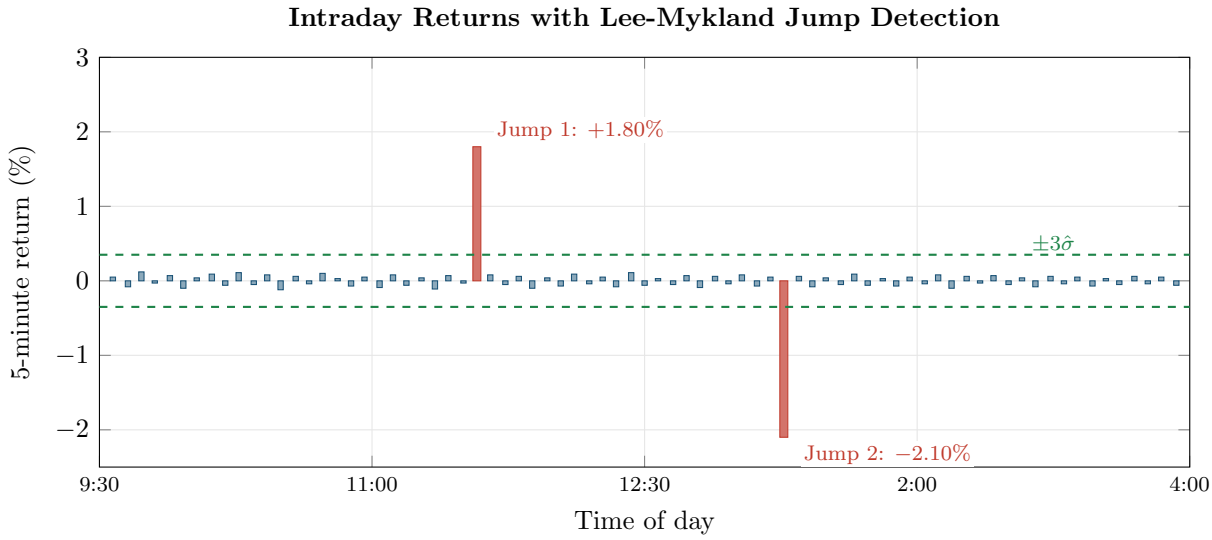


Figure 4.3: Intraday 5-minute returns with Lee-Mykland jump detection. Blue bars: normal diffusion returns (typically ±0.1%). Red bars: detected jump returns (far exceeding the local $\pm 3\hat{\sigma}$ band, shown as green dashed lines). Jump 1 at 12:15pm (+1.80%) might correspond to a macro release. Jump 2 at 2:00pm (−2.10%) is a large negative jump. The Lee-Mykland test identifies both the times and sizes of jumps, information that the BNS test (which only flags the day) cannot provide.

BNS vs. Lee-Mykland: When to Use Which

Use the BNS test when you need a daily-level binary indicator: "did a jump occur today?" This is sufficient for most forecasting applications (e.g., HAR-J in Chapter 6). Use Lee-Mykland when you need intraday jump timing (e.g., studying how volatility behaves in the minutes after a jump) or when you want to construct signed jump features (Chapter 10).

4.5 Aït-Sahalia-Jacod Test

Both the BNS test and the Lee-Mykland test assume that microstructure noise is negligible at the sampling frequency you use. As Chapter 2 discussed, this is a reasonable assumption at 5-minute frequency for liquid assets, but it fails at higher frequencies. Aït-Sahalia and Jacod (2009) proposed an alternative that is robust to microstructure noise.

Power Variation at Two Scales

The idea is to compare realized power variation computed at two different sampling frequencies. If no jumps are present, the ratio of these two quantities converges to a known constant (that depends only on the power used, not on the volatility level). If jumps are present, the ratio shifts. The test detects this shift.

Power Variation

Instead of always squaring returns, we can raise absolute returns to any power p. This single parameter p controls whether the estimator "listens to" jumps or ignores them.

$$V_t^{(p)} = \sum_{i=1}^{n} |r_{t,i}|^p \tag{4.5}$$

- $p = 2$: this reduces to RV_t (realized variance)
- $p = 1$: sum of absolute returns
- The key property: for $p < 2$, the power variation is dominated by diffusion (jumps contribute negligibly). For $p = 2$, jumps contribute fully. For $p > 2$, jumps dominate.

In Plain English

Think of the power p as a volume knob for jump sensitivity. Small returns (diffusion) and large returns (jumps) are both present in the data. When p is low (say $p = 1$), raising everything to p compresses the big returns more than the small ones, so jumps fade into the background. When p is high (say $p = 4$), the large returns get amplified disproportionately, so jumps scream. At $p = 2$ (standard RV), you get an honest count of both.

Why This Matters

The Aït-Sahalia-Jacod test uses power variation to detect jumps at higher sampling frequencies where microstructure noise corrupts BPV-based methods. If your data source provides tick-level or 1-minute data and you want to push to higher frequency for better estimation, this framework gives you a noise-robust alternative to the BNS test for deciding which days contain jumps.

The Aït-Sahalia and Jacod (2009) test statistic compares $V_t^{(p)}$ computed at two sampling frequencies, Δ and $k\Delta$ (e.g., 5-minute and 15-minute). Under the null of no jumps, the ratio converges to a constant $k^{p/2-1}$. Under the alternative (jumps present), the ratio is different.

Aït-Sahalia and Jacod (2009): Power-Variation-Ratio Test

The test statistic

$$S_t = \frac{V_t^{(p)}(\Delta)}{V_t^{(p)}(k\Delta)}$$

converges to $k^{p/2-1}$ under the null of no jumps. Significant deviation from this value indicates jump activity. Because the test uses ratios of power variations at different frequencies, microstructure noise affects numerator and denominator similarly, making the test more robust than BNS in noisy settings.

The practical advantage of this test is that you can apply it at higher sampling frequencies (1-minute or even 30-second) without the noise distortions that affect BNS. The cost is additional

complexity and the need to choose k and p. For a first pass, the BNS test at 5-minute frequency is usually sufficient.

4.6 Threshold and Truncation Methods

The BNS approach estimates continuous variation indirectly: compute BPV_t, and the jump component is the residual $\text{RV}_t - \text{BPV}_t$. Threshold methods take the opposite approach: directly classify each return as either a jump or a diffusion return, then compute realized variance using only the diffusion returns.

Truncation: The Simplest Decomposition

Any return larger than a threshold ϑ_i is classified as a jump. Continuous variation is computed by summing squared returns only for returns below the threshold. This is conceptually simpler than bipower variation and naturally extends to the HAR-CJ decomposition used in Chapter 6.

Threshold Realized Variance

Instead of dampening jumps implicitly (as BPV does), we can remove them explicitly: classify each return as "jump" or "not jump" based on a size cutoff, then sum squared returns only for the non-jump returns.

$$\text{TRV}_t = \sum_{i=1}^{n} r_{t,i}^2 \cdot \mathbf{1}\{|r_{t,i}| \leq \vartheta_i\} \tag{4.6}$$

- TRV_t: threshold realized variance for day t
- $r_{t,i}^2$: squared i-th intraday return
- $\mathbf{1}\{\cdot\}$: indicator function (equals 1 if the condition is true, 0 otherwise)
- ϑ_i: threshold for return i, typically set as $\vartheta_i = c\,\hat{\sigma}_i\,\Delta^{\varpi}$, where c is a constant (e.g., $c = 3$), $\hat{\sigma}_i$ is a local volatility estimate, and $\varpi \in (0, 1/2)$ controls how the threshold scales with sampling frequency

In Plain English

Threshold realized variance is a filter. It looks at each 5-minute return and asks: "Is this return small enough to be normal diffusion, or is it suspiciously large?" Returns below the cutoff are kept and squared as usual (they estimate continuous variation). Returns above the cutoff are discarded (they are presumed jumps). The result is a "cleaned" version of RV_t with jump contamination removed.

Why This Matters

The threshold approach is the basis of the HAR-CJ model (Corsi et al., 2010), one of the strongest baselines for your vol forecasting project. HAR-CJ enters $C_t = \text{TRV}_t$ and $J_t = \text{RV}_t - \text{TRV}_t$ as separate features, letting the model learn that continuous variation is persistent (high coefficient) while jump variation is transient (low coefficient). If your ML model cannot beat HAR-CJ, the decomposition itself is doing most of the heavy lifting.

Mancini (2009) established the theoretical foundation for threshold estimators. Corsi et al. (2010) combined the threshold approach with the HAR forecasting model to produce the HAR-CJ decomposition. Their key contribution was showing that the threshold-based continuous and jump components improve forecasting performance when entered separately into the HAR

model, because they have different persistence.

Corsi et al. (2010): Threshold HAR-CJ

Splitting RV_t into a continuous component ($C_t = \text{TRV}_t$) and a jump component ($J_t = \text{RV}_t - \text{TRV}_t$) using the threshold approach, and entering them separately into the HAR model, produces significantly better volatility forecasts than the baseline HAR. The improvement comes from allowing the model to assign different persistence parameters to continuous and jump variation.

4.7 Why the Decomposition Matters for Forecasting

You have now seen four ways to separate jumps from continuous variation: bipower variation, the BNS test, Lee-Mykland, and threshold truncation. Why does this separation matter? The answer is forecasting: continuous and jump variation behave very differently over time.

Different Persistence

Continuous volatility is highly persistent. If a stock had high diffusion volatility today, it will very likely have high diffusion volatility tomorrow, and next week, and even next month. This is the long-memory property that the HAR model (Chapter 6) exploits.
Jump variation is the opposite. A jump today tells you almost nothing about whether a jump will occur tomorrow. Earnings announcements, FOMC decisions, and geopolitical shocks do not cluster in the same way that diffusion volatility does. The jump component is nearly unpredictable.

This difference has a direct implication for forecasting models. If you feed raw RV_t into a model, you are mixing a highly persistent signal (continuous variation) with transient noise (jump variation). The model has to learn a single set of persistence parameters that compromises between the two. Separating them lets the model learn different dynamics for each component.

4.7.1 Diagram: The Decomposition Pipeline

Figure 4.4 summarizes the full decomposition pipeline from raw intraday returns to the separated components that feed into forecasting models.

Forecasting with Separated Components

In the HAR-J and HAR-CJ models (Chapter 6), the continuous component enters with high persistence coefficients (it is highly predictable), while the jump component enters with a small, often statistically insignificant coefficient (it is nearly unpredictable). The forecasting improvement comes from the continuous side: by removing jump contamination, you get a cleaner, more persistent signal that the model can exploit more effectively (Andersen et al., 2007).

There is an additional asymmetry. Bollerslev et al. (2009a) and Patton and Sheppard (2015a) found that "bad jumps" (large negative returns) are more informative for future volatility than "good jumps" (large positive returns). A 5% daily drop signals elevated volatility going forward; a 5% daily rally does not, to the same degree. This connects to the signed volatility decompositions (SHAR) covered in Chapter 6 and the signed jump features in Chapter 10.

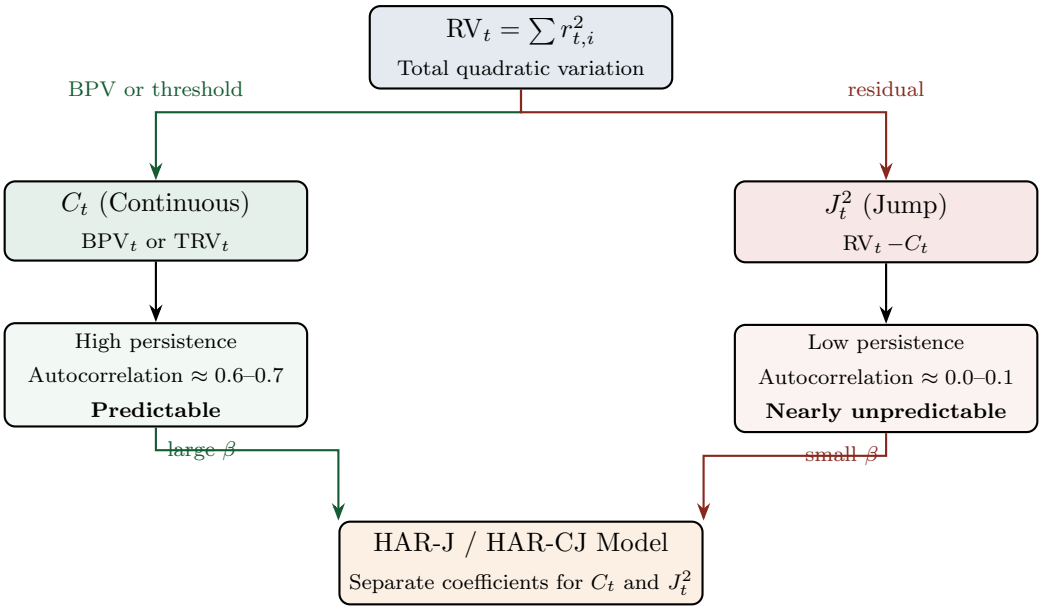


Figure 4.4: The decomposition pipeline. RV_t is split into a continuous component C_t (estimated via BPV or threshold truncation) and a jump component J_t^2 (the residual). The continuous component is highly persistent and drives forecasting accuracy. The jump component is nearly unpredictable but still included to prevent it from contaminating the continuous signal. In the HAR-J and HAR-CJ models, each component receives its own coefficient, letting the model exploit the different persistence.

Do Not Treat Jump and Continuous Components as Interchangeable

The continuous and jump components of RV_t have different statistical properties: different persistence, different distributional shapes, and different predictive content. Lumping them together (as raw RV_t does) throws away information. Treating the jump component as if it were as persistent as continuous variation will lead to overstated volatility forecasts after jump days. Always decompose before forecasting.

Summary

- **Jumps** are sudden, large price moves caused by discrete news events (earnings, macro releases, geopolitical shocks). They are distinct from the smooth diffusion that drives normal price variation.
- **Quadratic variation** equals integrated variance plus the sum of squared jumps: $[p]_t = \mathrm{IV}_t + \sum J_s^2$. Standard RV_t captures both; this chapter develops tools to separate them.
- **Bipower variation** $\mathrm{BPV}_t = \frac{\pi}{2} \sum_{i=2}^{n} |r_{t,i}|\,|r_{t,i-1}|$ multiplies consecutive absolute returns, dampening the contribution of isolated jumps (Barndorff-Nielsen and Shephard, 2004).
- **BPV consistency**: BPV_t converges to IV_t even in the presence of finite-activity jumps. $\mathrm{RV}_t - \mathrm{BPV}_t$ consistently estimates the jump component.
- The $\frac{\pi}{2}$ scaling factor corrects for the fact that $\mathbb{E}[|Z|]^2 = 2/\pi$ when $Z \sim \mathcal{N}(0,1)$.
- The **BNS jump test** formalizes this: $Z_{\mathrm{BNS},t} = (\mathrm{RV}_t - \mathrm{BPV}_t)/\sqrt{\vartheta \cdot \mathrm{RQ}_t / n}$ is asymptotically standard normal under the null of no jumps (Barndorff-Nielsen and Shephard, 2006). Compare to 1.645 (5%) or 2.326 (1%).
- The BNS test has **finite-sample size distortions** (it can over-reject). The ratio variant or Huang-Tauchen correction improves performance (Huang and Tauchen, 2005).
- The **Lee-Mykland test** detects individual jumps within the day, providing jump timing

and sizes, not just a daily binary indicator (Lee and Mykland, 2008).

- The **Aït-Sahalia-Jacod test** uses power-variation ratios at two frequencies and is more robust to microstructure noise than the BNS test (Aït-Sahalia and Jacod, 2009).
- **Threshold methods** classify each return as jump or diffusion based on a size cut-off. Threshold realized variance sums squared returns only for below-threshold returns (Mancini, 2009; Corsi et al., 2010).
- **Continuous variation is persistent**; jump variation is transient. Separating them improves forecasting by allowing models to assign different persistence to each component.
- **Bad jumps** (negative) are more informative for future volatility than good jumps (positive), motivating signed jump features in Chapter 10 (Bollerslev et al., 2009a; Patton and Sheppard, 2015a).
- The HAR-J and HAR-CJ models (Chapter 6) exploit this decomposition directly; Projects 1 and 4 use jump-decomposed features.

Table 4.1: Key results referenced in this chapter.

Paper	Result	Relevance
Barndorff-Nielsen and Shephard (2004)	Bipower variation BPV_t converges to integrated variance IV_t even in the presence of finite-activity jumps.	Foundational estimator for separating continuous and jump variation.
Barndorff-Nielsen and Shephard (2006)	BNS jump test: $Z_{\text{BNS},t}$ is asymptotically $\mathcal{N}(0,1)$ under no jumps; formal test for jump days.	Standard daily jump test used in HAR-J models.
Lee and Mykland (2008)	Intraday jump detection: identifies individual jump times and sizes using local volatility and extreme-value theory.	Enables signed jump features and intraday jump analysis.
Aït-Sahalia and Jacod (2009)	Power-variation-ratio test for jump activity, robust to microstructure noise.	Alternative to BNS for noisy high-frequency data.
Corsi et al. (2010)	Threshold-based HAR-CJ decomposition improves volatility forecasts by separating persistent continuous and transient jump components.	Direct input to HAR-CJ forecasting model (Chapter 6).
Andersen et al. (2007)	Jump component of RV has near-zero persistence; forecasting improves when continuous and jump components are modeled separately.	Motivates the jump-continuous decomposition for all forecasting in this guide.

Part II

Forecasting Volatility with Classical Models

Chapter 5

The GARCH Family

Why This Chapter?

GARCH models forecast volatility using only daily returns, without intraday data. They are the historical workhorse of volatility modeling and remain widely used in risk management systems. Understanding GARCH is necessary background for appreciating why the HAR model (Chapter 6), which uses realized volatility from intraday data, represents a significant improvement. Realized GARCH and the HEAVY model bridge these two worlds. Project 1 (HARQ-X) uses GARCH as a comparison baseline.

Chapters 2–4 built volatility estimators from intraday data. This chapter steps back in time: before high-frequency data was widely available, how did people forecast volatility? The answer is the GARCH family, a set of models that extract volatility dynamics entirely from daily returns.

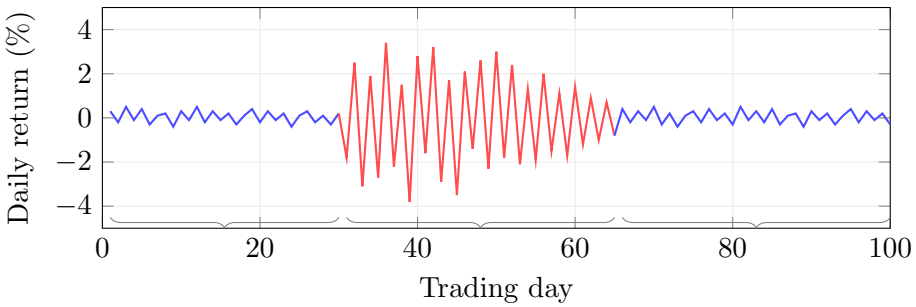
5.1 The ARCH Model

We start with the simplest idea: yesterday's return tells you something about today's volatility.

Conditional vs. Unconditional Variance

The *unconditional* variance of a return series is a single number computed over the full sample: $\text{Var}(r_t)$. The *conditional* variance $\sigma_t^2 = \text{Var}(r_t \mid \mathcal{F}_{t-1})$ is the variance you would forecast for period t, given everything you know up through period $t-1$ (the information set $\mathcal{F}_{t-1}$). GARCH-family models are models for this conditional variance.

Volatility clustering: calm periods alternate with turbulent periods



Why Squared Returns?

If a stock fell 3% yesterday, you expect higher volatility today. If it barely moved (0.1%), you expect calm. Squaring the return strips away the sign and gives a rough measure of how "big" the move was. ARCH formalizes this: the conditional variance is a weighted sum of past squared returns.

Engle (1982) proposed the ARCH(q) model. In its simplest form, ARCH(1):

$$\sigma_t^2 = \omega + \alpha_1 r_{t-1}^2 \tag{5.1}$$

- σ_t^2: conditional variance for period t (the quantity we forecast)
- $\omega > 0$: baseline variance level, ensuring σ_t^2 stays positive even when $r_{t-1} = 0$
- $\alpha_1 \geq 0$: sensitivity to the most recent squared return
- r_{t-1}^2: squared return from the previous period, a noisy proxy for realized variance

In Plain English

Tomorrow's variance equals a floor level plus a correction based on how big today's move was. If the market barely moved, the forecast stays near the floor. If it moved a lot, the forecast jumps. This is the simplest possible "volatility responds to recent news" model.

Why This Matters

ARCH is the conceptual ancestor of every conditional variance model you will compare against. Its core idea (past squared returns predict future variance) reappears in the HAR model, which replaces squared returns with realized variance at multiple horizons.

The general ARCH(q) model adds more lags:

$$\sigma_t^2 = \omega + \sum_{i=1}^{q} \alpha_i r_{t-i}^2 \tag{5.2}$$

- q: number of lags included
- $\alpha_i \geq 0$: weight on the squared return from i days ago

In Plain English

ARCH(q) generalizes the same idea: instead of looking only at yesterday, you look at the last q days of squared returns and take a weighted average. More lags let the model capture longer memory in volatility, but each lag adds a free parameter.

ARCH's Parameter Problem

Volatility is persistent: a shock today still affects volatility weeks later. Capturing this with ARCH requires many lags ($q = 10$ or more), each with its own parameter. This makes estimation fragile and overfitting likely. GARCH solves this.

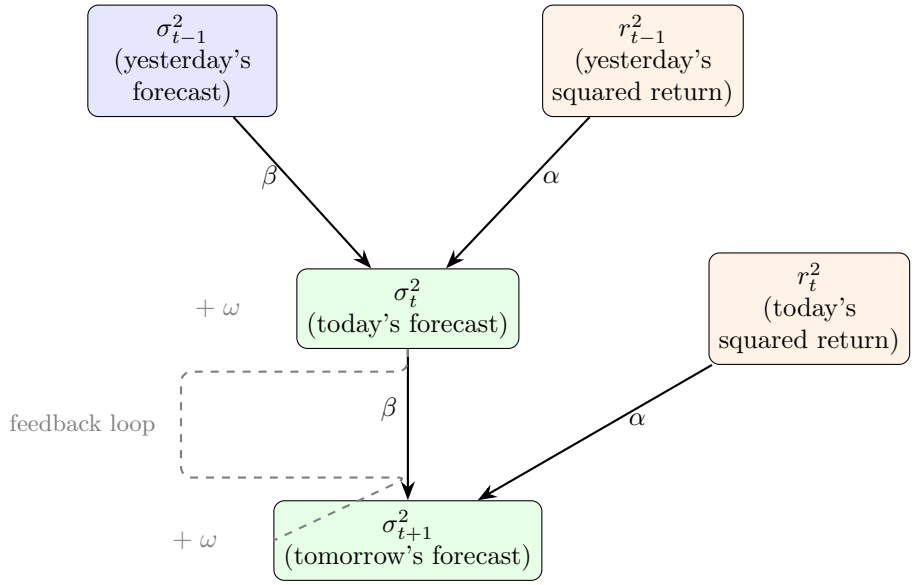
5.2 GARCH(1,1): The Workhorse Model

ARCH needs many parameters to capture persistence. The key insight of GARCH is to add a single feedback term: let yesterday's *conditional variance* help predict today's.

GARCH as Exponential Smoothing

Think of a weather forecast. Tomorrow's temperature forecast depends on two things: (1) what actually happened today (the "news"), and (2) what you predicted yesterday. GARCH works the same way: the new volatility forecast blends today's squared return (news) with yesterday's forecast (memory). A single memory term replaces an infinite history of squared returns.

The following diagram shows how information flows in GARCH(1,1). Today's variance forecast feeds into tomorrow's, creating a self-reinforcing loop that captures persistence.



Bollerslev (1986) introduced the GARCH(1,1) model:

$$\sigma_t^2 = \omega + \alpha\, r_{t-1}^2 + \beta\, \sigma_{t-1}^2 \tag{5.3}$$

- σ_t^2: conditional variance for period t
- $\omega > 0$: intercept, related to the long-run average variance
- $\alpha \geq 0$: reaction coefficient; how strongly the model reacts to new information (yesterday's squared return)
- $\beta \geq 0$: persistence coefficient; how much of yesterday's variance forecast carries forward
- r_{t-1}^2: yesterday's squared return (the "shock" or "news")
- σ_{t-1}^2: yesterday's conditional variance forecast (the "memory")

In Plain English

Tomorrow's variance is a weighted average of three things: the long-run average variance (ω), how big today's surprise was ($\alpha\, r_{t-1}^2$), and what you already thought variance was ($\beta\, \sigma_{t-1}^2$). The memory term $\beta\, \sigma_{t-1}^2$ is the key innovation over ARCH: it lets one parameter do the work of many lagged squared returns, because yesterday's forecast already encodes the entire history.

Why This Matters

GARCH(1,1) is your simplest conditional-variance baseline. In a model horse race, any ML model or HAR variant that cannot beat GARCH(1,1) out of sample is not worth the added complexity. Typical $\alpha + \beta \approx 0.98$ for equity indices, meaning volatility shocks are extremely persistent, the same stylized fact that HAR captures with its weekly and

monthly RV components.

Stationarity Condition for GARCH(1,1)

The process is covariance-stationary if and only if $\alpha + \beta < 1$. When this holds, the unconditional (long-run) variance is:

$$\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \beta} \tag{5.4}$$

- $\bar{\sigma}^2$: the unconditional variance, i.e., the level to which σ_t^2 mean-reverts over time
- $\alpha+\beta$: total persistence; values close to 1 mean slow mean-reversion (volatility shocks are long-lived)

If $\alpha+\beta = 1$, the model becomes IGARCH (Integrated GARCH): shocks never decay, and the unconditional variance is undefined (Engle and Bollerslev, 1986).

In Plain English

The unconditional variance is the "gravity" level that the conditional variance always pulls toward. When $\alpha + \beta$ is close to 1, the pull is very weak: after a volatility spike, it takes weeks or months for the forecast to drift back to normal. This is mean reversion in volatility, and the speed of that reversion is $1 - \alpha - \beta$.

Why This Matters

When you forecast RV at the 22-day horizon, mean-reversion speed matters enormously. A GARCH model with $\alpha+\beta = 0.98$ will still forecast elevated volatility three weeks after a shock, while HAR's monthly component captures the same persistence more flexibly. Comparing the implied half-lives of GARCH versus HAR forecasts is a simple diagnostic for your project.

Worked Example:

GARCH(1,1) One-Step Forecast Suppose you have estimated parameters $\omega = 0.00001$, $\alpha = 0.08$, $\beta = 0.90$. Yesterday's return was $r_{t-1} = -0.03$ (a 3% loss), and yesterday's conditional variance was $\sigma_{t-1}^2 = 0.0004$ (implying $\sigma_{t-1} = 0.02$, or 2% daily vol).

Step 1: Compute each component.

$$\begin{aligned} \text{Baseline:} \quad & \omega = 0.00001 \\ \text{News:} \quad & \alpha \cdot r_{t-1}^2 = 0.08 \times (-0.03)^2 = 0.08 \times 0.0009 = 0.000072 \\ \text{Memory:} \quad & \beta \cdot \sigma_{t-1}^2 = 0.90 \times 0.0004 = 0.00036 \end{aligned}$$

Step 2: Sum the components.

$$\sigma_t^2 = 0.00001 + 0.000072 + 0.00036 = 0.000442$$

Step 3: Convert to daily volatility.

$$\sigma_t = \sqrt{0.000442} \approx 0.0210 = 2.10\%$$

The 3% loss (larger than the 2% daily vol) pushes the variance forecast up from 0.0004 to 0.000442, a 10.5% increase. Most of the forecast (81%) comes from the memory term $\beta\sigma_{t-1}^2$, reflecting the high persistence typical of financial volatility.

Step 4: Check long-run variance.

$$\bar{\sigma}^2 = \frac{0.00001}{1 - 0.08 - 0.90} = \frac{0.00001}{0.02} = 0.0005 \quad \Rightarrow \quad \bar{\sigma} \approx 2.24\%$$

With $\alpha + \beta = 0.98$, mean-reversion is very slow: it takes roughly $1/(1 - 0.98) = 50$ days for the conditional variance to close half the gap to $\bar{\sigma}^2$.

GARCH(1,1) Is Usually Enough

Hansen and Lunde (2005) compared 330 GARCH-type models on exchange rate and equity data. For exchange rates, no model significantly outperformed GARCH(1,1). For equities, models with a leverage effect (Section 5.3) did better, but higher-order specifications like GARCH(2,1) or GARCH(1,2) provided negligible improvement. In practice, GARCH(1,1) captures the bulk of conditional variance dynamics. The gains from asymmetric extensions come from modeling leverage, not from adding lags.

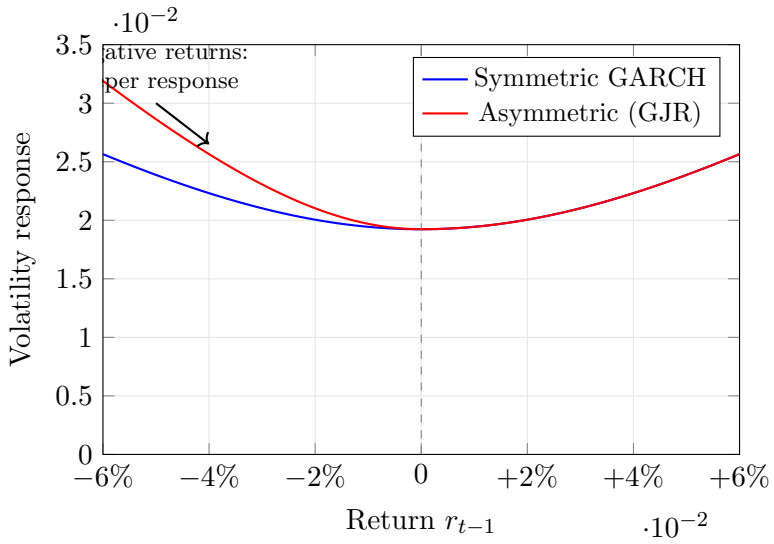
5.3 The Leverage Effect

GARCH(1,1) treats positive and negative returns symmetrically: a +3% return and a −3% return produce the same $r_{t-1}^2 = 0.0009$ and thus the same volatility forecast. In real equity markets, this is wrong.

Why Negative Returns Increase Volatility More

Black (1976) documented this asymmetry and proposed an explanation. When a stock drops, the firm's equity value falls while its debt stays fixed. The firm becomes more leveraged (higher debt-to-equity ratio), making its equity riskier, so volatility rises. A positive return has the opposite effect but to a smaller degree. This "leverage effect" is one of the most robust stylized facts in equity markets.

The following diagram illustrates the asymmetry. For the same magnitude of return, negative returns produce a larger volatility response than positive returns.



Standard GARCH(1,1) cannot capture this asymmetry because it depends on r_{t-1}^2, which discards the sign of the return. The next two sections present models that fix this.

5.4 GJR-GARCH: A Simple Asymmetric Extension

The most direct way to capture leverage is to add an indicator function that activates only for negative returns.

Glosten et al. (1993) proposed:

$$\sigma_t^2 = \omega + \alpha\, r_{t-1}^2 + \gamma\, \mathbf{1}_{\{r_{t-1}<0\}}\, r_{t-1}^2 + \beta\, \sigma_{t-1}^2 \tag{5.5}$$

- ω, α, β: same roles as in GARCH(1,1) (Equation 5.3)
- $\gamma \geq 0$: the asymmetry (leverage) parameter
- $\mathbf{1}_{\{r_{t-1}<0\}}$: indicator function equal to 1 when $r_{t-1} < 0$, and 0 otherwise

The effect is straightforward:

- After a *positive* return: the effective coefficient on r_{t-1}^2 is just α.
- After a *negative* return: the effective coefficient is $\alpha + \gamma$, giving a stronger volatility response.

In Plain English

GJR-GARCH is just GARCH(1,1) with a bonus penalty for bad news. After a positive return, the equation is identical to standard GARCH. After a negative return, the indicator switches on and adds extra sensitivity (γ) to the squared return. The model has two "gears": a calm gear for up days and a reactive gear for down days.

Why This Matters

The leverage effect is one of the strongest predictable patterns in equity volatility. Adding an asymmetry term (whether in GJR-GARCH or as a signed-return feature in your ML model) typically improves QLIKE by 5–15%. If your HAR baseline does not include a negative-return indicator, GJR-GARCH shows exactly why it should.

Worked Example:

GJR-GARCH: Positive vs. Negative Shocks Using $\omega = 0.00001$, $\alpha = 0.04$, $\gamma = 0.08$, $\beta = 0.90$, and $\sigma_{t-1}^2 = 0.0004$:
Case A: $r_{t-1} = +0.03$ **(3% gain).**

$$\begin{aligned}\sigma_t^2 &= 0.00001 + 0.04 \times 0.0009 + 0 + 0.90 \times 0.0004 \\ &= 0.00001 + 0.000036 + 0.00036 = 0.000406\end{aligned}$$

$$\sigma_t = \sqrt{0.000406} \approx 2.01\%$$

Case B: $r_{t-1} = -0.03$ **(3% loss).**

$$\begin{aligned}\sigma_t^2 &= 0.00001 + 0.04 \times 0.0009 + 0.08 \times 0.0009 + 0.90 \times 0.0004 \\ &= 0.00001 + 0.000036 + 0.000072 + 0.00036 = 0.000478\end{aligned}$$

$$\sigma_t = \sqrt{0.000478} \approx 2.19\%$$

The negative return produces a forecast of 2.19% versus 2.01% for the same-size positive return. The 3% loss roughly doubles the "news" contribution to variance (from 0.000036 to 0.000108), reflecting the leverage effect.

When GJR Matters

GJR-GARCH is most useful for equities, where the leverage effect is strong. For exchange rates, the effect is weaker and often statistically insignificant; symmetric GARCH(1,1) is typically adequate (Hansen and Lunde, 2005).

5.5 EGARCH: Log-Variance and Guaranteed Positivity

GJR-GARCH has a practical annoyance: you need parameter constraints ($\omega > 0$, $\alpha \geq 0$, $\alpha + \gamma \geq 0$, $\beta \geq 0$) to ensure $\sigma_t^2 > 0$. Unconstrained optimization sometimes produces estimates that violate these bounds.

Nelson (1991) proposed EGARCH, which models the *log* of variance. Since $\exp(\cdot)$ is always positive, the variance is automatically positive regardless of parameter signs.

$$\ln \sigma_t^2 = \omega + \beta \ln \sigma_{t-1}^2 + \alpha \left(\frac{|r_{t-1}|}{\sigma_{t-1}} - \sqrt{\frac{2}{\pi}} \right) + \gamma \frac{r_{t-1}}{\sigma_{t-1}} \tag{5.6}$$

- $\ln \sigma_t^2$: log conditional variance (the model's target)
- ω: intercept on the log-variance scale (can be any real number)
- β: persistence of log-variance; $|\beta| < 1$ for stationarity
- $|r_{t-1}|/\sigma_{t-1}$: the absolute value of the standardized return, measuring shock magnitude
- $\sqrt{2/\pi}$: the expected value of $|z|$ when $z \sim \mathcal{N}(0,1)$; subtracting it centers the magnitude term at zero
- α: the size effect; controls how strongly large shocks (of either sign) increase variance
- r_{t-1}/σ_{t-1}: the signed standardized return
- γ: the sign effect (asymmetry parameter); $\gamma < 0$ means negative returns increase log-variance

How the Sign Effect Works

The γ term adds $\gamma \cdot z_{t-1}$ to log-variance, where $z_{t-1} = r_{t-1}/\sigma_{t-1}$ is the standardized return. If $\gamma = -0.10$ and the standardized return is $z = -2$ (a two-standard-deviation loss), the contribution is $(-0.10)(-2) = +0.20$: log-variance rises. If $z = +2$ (a two-standard-deviation gain), the contribution is $(-0.10)(+2) = -0.20$: log-variance falls. The same shock magnitude produces opposite effects depending on sign.

Why This Matters

EGARCH's log-variance formulation has a practical benefit for your project: working in logs stabilizes estimation and makes the variance automatically positive. Many ML models for realized volatility also work in log space ($\log \mathrm{RV}_t$) for the same reason. If you include EGARCH as a baseline, its sign-effect coefficient γ gives you a direct comparison point for how well your ML model captures asymmetry.

EGARCH Forecasting Complication

Multi-step forecasts from EGARCH require computing $\mathbb{E}[\exp(\cdot)]$, which does not simplify as cleanly as for linear GARCH. In practice, simulation-based forecasts are used for horizons beyond one step.

5.6 Long Memory in Volatility: FIGARCH

Volatility autocorrelations in financial data decay very slowly. If you compute the autocorrelation of daily squared returns (or absolute returns) at lags 1, 5, 20, 100, you find that the autocorrelation is still positive even at lag 100. Standard GARCH(1,1) implies *exponential* decay of autocorrelations, which is too fast.

Long Memory and Fractional Integration

A time series has *long memory* if its autocorrelations decay at a hyperbolic (power-law) rate rather than an exponential rate. In the context of ARIMA, a non-integer differencing parameter $d \in (0, 0.5)$ produces this behavior: the series is neither stationary ($d = 0$) nor unit-root ($d = 1$), but something in between. The same idea can be applied to the variance equation of a GARCH model.

Baillie et al. (1996) introduced FIGARCH, which replaces the integer-differencing implicit in GARCH with fractional differencing.

FIGARCH Differencing Parameter

The FIGARCH model introduces a parameter $d \in (0, 1)$ governing the rate at which past shocks influence current variance:

- $d = 0$: equivalent to GARCH; shock impact decays exponentially (short memory)
- $d = 1$: equivalent to IGARCH; shock impact never decays (unit root in variance)
- $0 < d < 1$: shock impact decays hyperbolically (long memory); past shocks matter, but their influence eventually fades

The full FIGARCH(1,d,1) specification is written using the lag operator L:

$$(1 - \beta L)\sigma_t^2 = \omega + \left[1 - \beta L - (1 - \phi L)(1 - L)^d\right] r_t^2 \tag{5.7}$$

- L: lag operator ($L\, x_t = x_{t-1}$)
- $(1 - L)^d$: fractional differencing operator with parameter d
- ϕ: an additional ARCH-side parameter
- β: persistence parameter, same role as in GARCH

The key point is not the algebra but the implication: FIGARCH matches the slow decay of volatility autocorrelations far better than GARCH.

In Plain English

Standard GARCH forgets shocks exponentially: a volatility spike from two months ago has almost no influence on today's forecast. FIGARCH forgets shocks more slowly, following a power law. The parameter d controls how slowly: $d = 0$ is normal GARCH (fast forgetting), and d closer to 1 means shocks linger almost indefinitely. This matches what we see empirically, where volatility autocorrelations are still detectable at lags of 100 days or more.

Why This Matters

Long memory in volatility is precisely why HAR works: its weekly and monthly components act as a discrete approximation to the slow-decaying memory that FIGARCH models continuously. If your ML residual model finds that long-lag features improve forecasts, FIGARCH's d parameter gives you a theoretical explanation for why.

FIGARCH vs. Rough Volatility

Both FIGARCH and rough volatility models (Chapter 7) address the same empirical fact: volatility has long memory. FIGARCH does so within the discrete-time GARCH framework by adding the parameter d. Rough volatility uses continuous-time fractional Brownian motion with Hurst parameter $H \approx 0.1$. The two approaches are complementary views of the same phenomenon.

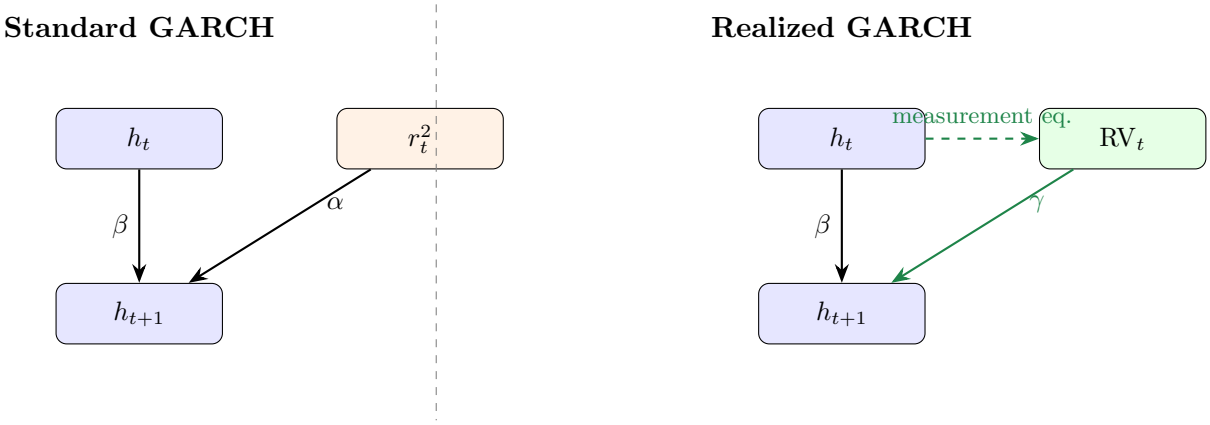
5.7 Realized GARCH: Bringing Intraday Data into GARCH

Standard GARCH uses only daily returns to infer volatility. But if you have access to intraday data, you can compute realized volatility (RV_t, as defined in Chapter 2), a far more precise measure of what actually happened. Realized GARCH incorporates RV_t directly into the GARCH framework.

The Measurement Problem

In GARCH, the conditional variance σ_t^2 is a latent (unobserved) variable. The model infers it from the noisy signal r_t^2 (the squared daily return). But r_t^2 is an extremely noisy estimator of daily variance: on average, the noise-to-signal ratio exceeds 5. Realized volatility RV_t is computed from many intraday returns and is orders of magnitude more precise. Realized GARCH exploits this better signal.

The following diagram shows how information flows in Realized GARCH versus standard GARCH. The key addition is the measurement equation linking RV_t to the latent conditional variance h_t.



Hansen et al. (2012) specify three equations. The return equation is standard:

$$r_t = \sqrt{h_t}\, z_t, \qquad z_t \sim \mathcal{N}(0,1) \tag{5.8}$$

- r_t: daily return
- h_t: conditional variance (the latent variable we model)
- z_t: standardized innovation, assumed i.i.d. standard normal

This is the standard return-generating process: the daily return is volatility times a random shock. It is shared with standard GARCH and is not where Realized GARCH innovates.

The measurement equation links the observed RV_t to the latent h_t:

$$\log \text{RV}_t = \xi + \delta \log h_t + \tau(z_t) + u_t \tag{5.9}$$

- $\log \text{RV}_t$: log realized volatility (observed from intraday data)

- ξ: intercept allowing for a level difference between RV_t and h_t
- δ: loading of $\log h_t$ on $\log \text{RV}_t$; typically close to 1
- $\tau(z_t)$: a leverage function of the standardized return, capturing the asymmetry between positive and negative returns (often specified as $\tau_1 z_t + \tau_2(z_t^2 - 1)$)
- u_t: measurement noise, independent of z_t

The GARCH equation uses RV_t instead of r_t^2:

$$\log h_{t+1} = \omega + \beta \log h_t + \gamma \log \text{RV}_t \tag{5.10}$$

- ω: intercept on the log-variance scale
- β: persistence of the conditional variance
- γ: sensitivity to the realized measure; this is where intraday information enters the model

In Plain English

Realized GARCH is a three-part system. (1) Returns are driven by latent variance, as usual. (2) The measurement equation says "realized volatility is a noisy, possibly biased reading of that latent variance, with an asymmetry correction." (3) The GARCH equation says "tomorrow's latent variance depends on today's latent variance (persistence) plus today's realized volatility (new information)." The key upgrade over standard GARCH is in step (3): instead of feeding in the noisy squared return r_t^2, you feed in RV_t, which is orders of magnitude more precise.

Why This Matters

Realized GARCH is the bridge between the GARCH world and the RV world your project lives in. It uses RV_t directly as an input, just as your HAR and ML models do. Including Realized GARCH as a baseline lets you ask: "Does my ML model add value beyond what a well-specified parametric model can extract from the same RV_t data?" If your ML model only matches Realized GARCH, the complexity may not be justified.

Realized GARCH Outperforms Standard GARCH

Hansen et al. (2012) find that Realized GARCH with log-linear specification substantially outperforms standard GARCH(1,1) in both in-sample fit and out-of-sample forecasting on Dow Jones Industrial Average stocks. Squared returns become statistically insignificant once a realized measure is included. The leverage function $\tau(z_t)$ is highly significant, confirming the importance of asymmetry.

5.8 The HEAVY Model

Realized GARCH is not the only way to combine daily and intraday information. Shephard and Sheppard (2010) proposed the HEAVY (High-frEquency-bAsed VolatilitY) model, which takes a different but related approach: instead of treating RV_t as a noisy measurement of a latent h_t, HEAVY models both the conditional variance of returns and the conditional expectation of RV_t as a joint system.

The HEAVY system has two equations. The first governs the conditional variance of daily returns:

$$\sigma_{R,t}^2 = \omega_R + \alpha_R \, \text{RV}_{t-1} + \beta_R \, \sigma_{R,t-1}^2 \tag{5.11}$$

- $\sigma_{R,t}^2$: conditional variance of daily returns
- RV_{t-1}: yesterday's realized variance (replaces r_{t-1}^2 from GARCH)

- α_R: sensitivity to the realized measure
- β_R: persistence of the return variance

The second governs the conditional expectation of realized variance:

$$\mu_{M,t} = \omega_M + \alpha_M \, \mathrm{RV}_{t-1} + \beta_M \, \mu_{M,t-1} \tag{5.12}$$

- $\mu_{M,t}$: conditional expectation of RV_t (what you expect today's realized variance to be)
- α_M, β_M: analogous to GARCH parameters but for the realized measure

In Plain English

HEAVY runs two GARCH-like equations side by side. The first forecasts the variance of daily returns, and the second forecasts realized variance itself. Both use yesterday's RV as input instead of r^2. Think of it as two parallel trackers: one for "what will the daily return variance be?" and one for "what will intraday volatility look like?" Because RV reacts to intraday information immediately, the HEAVY model adjusts to regime changes (like a volatility spike) much faster than standard GARCH, which must wait for large daily returns to accumulate.

Why This Matters

The second HEAVY equation (Equation 5.12) is essentially forecasting RV_t using an AR(1) in RV with GARCH-style persistence, making it a close relative of the HAR model's daily component. Comparing HEAVY's $\mu_{M,t}$ forecasts against your HAR forecasts at the one-day horizon reveals how much the multi-horizon structure of HAR adds beyond a simple autoregressive specification.

HEAVY Model Performance

Shephard and Sheppard (2010) find that the HEAVY model outperforms GARCH both in-sample and out-of-sample across a range of asset classes. Forecast gains are most pronounced at short horizons (one to five days). The model adjusts quickly to structural breaks in the volatility level because RV_{t-1} responds immediately to intraday price variation, while GARCH must wait for the slow accumulation of squared daily returns.

5.9 Comparison: GARCH vs. Realized GARCH vs. HEAVY

The table below summarizes the three approaches. The key distinction is the data input: standard GARCH uses only daily returns, while Realized GARCH and HEAVY incorporate intraday information through RV_t.

	GARCH(1,1)	Realized GARCH	HEAVY
Data input	Daily returns only	Daily returns + RV_t	Daily returns + RV_t
Volatility driver	r_{t-1}^2	$\log \text{RV}_t$	RV_{t-1}
Latent variance	Yes (σ_t^2)	Yes (h_t), linked to RV_t via measurement eq.	Two: $\sigma_{R,t}^2$ (returns) and $\mu_{M,t}$ (RV)
Leverage effect	Not in basic form; needs GJR or EGARCH	Built in via $\tau(z_t)$	Not in basic form; extensions exist
Structural breaks	Slow adjustment	Faster (uses RV_t)	Fastest (direct RV input)
Estimation	MLE on daily returns	Joint MLE	Equation-by-equation or joint MLE
Use when	No intraday data available	Intraday data available; want a single integrated framework	Intraday data available; want fast response to regime changes

GARCH Is Not a Straw Man

When intraday data is unavailable (many asset classes, historical periods, or emerging markets), GARCH remains the best option. Even with intraday data, GARCH serves as a useful baseline: any more complex model should demonstrably outperform it. Andersen and Bollerslev (1998) showed that GARCH forecasts appear poor only when evaluated against noisy proxies like r_t^2; evaluated against RV_t, they perform respectably.

5.10 Summary

- ARCH (Engle, 1982) models conditional variance as a function of past squared returns but requires many lags to capture persistence.
- GARCH(1,1) (Bollerslev, 1986) adds a single feedback term ($\beta\sigma_{t-1}^2$), capturing persistence with just three parameters (ω, α, β).
- The stationarity condition is $\alpha + \beta < 1$; the unconditional variance is $\omega/(1-\alpha-\beta)$.
- Higher-order GARCH(p, q) rarely improves on GARCH(1,1) in practice (Hansen and Lunde, 2005).
- The leverage effect (Black, 1976): negative returns increase volatility more than positive returns of the same magnitude.
- GJR-GARCH (Glosten et al., 1993) adds an indicator term $\gamma\, \mathbf{1}_{\{r<0\}}\, r_{t-1}^2$ to capture leverage simply.
- EGARCH (Nelson, 1991) models log-variance, guaranteeing positivity without parameter constraints and capturing both sign and size effects.
- FIGARCH (Baillie et al., 1996) introduces fractional integration ($d \in (0,1)$) to match the slow, hyperbolic decay of volatility autocorrelations.
- Realized GARCH (Hansen et al., 2012) links RV_t to the latent conditional variance via a measurement equation, substantially outperforming standard GARCH.

- The HEAVY model (Shephard and Sheppard, 2010) builds a joint system for daily return variance and RV_t, adjusting quickly to structural breaks.
- GARCH remains the right baseline when intraday data is unavailable.
- The HAR model (Chapter 6) offers a simpler, non-parametric alternative for RV forecasting that often matches or beats Realized GARCH.

Key Results

Result	Source	Finding
ARCH model	Engle (1982)	Conditional variance depends on past squared returns; Nobel Prize–winning framework
GARCH(1,1)	Bollerslev (1986)	Single memory term replaces many ARCH lags; three parameters capture the bulk of variance dynamics
Leverage effect	Black (1976)	Negative returns increase volatility more than positive returns of the same magnitude
GJR-GARCH	Glosten et al. (1993)	Indicator-based asymmetry; effective coefficient on negative shocks is $\alpha + \gamma$
EGARCH	Nelson (1991)	Log-variance specification; no positivity constraints; separates sign and size effects
GARCH(1,1) benchmark	Hansen and Lunde (2005)	Among 330 models, no significant improvement from higher-order lags; leverage matters for equities
FIGARCH	Baillie et al. (1996)	Fractional integration parameter d captures hyperbolic decay of volatility autocorrelations
Realized GARCH	Hansen et al. (2012)	Incorporating RV_t via measurement equation substantially outperforms daily-return-only GARCH
HEAVY model	Shephard and Sheppard (2010)	Joint return/RV system; fast adjustment to regime changes; strongest gains at short horizons

Chapter 6

The HAR Model and Its Extensions

Why This Chapter

HAR is THE benchmark for volatility forecasting. Every ML model in Chapters 11–14 must beat HAR to justify its complexity. This chapter must be crystal clear because every subsequent forecasting chapter references it. Projects 1, 4, and 5 all use HAR variants as their primary baseline.

Chapter 5 forecast volatility using only daily returns. That approach works when intraday data is unavailable, but it uses a noisy proxy (r_t^2) for what actually happened during the day. Chapter 2 showed that realized variance $\text{RV}_t = \sum r_{t,i}^2$, computed from intraday returns, is a far more precise measure of daily volatility. The natural next question: can you forecast tomorrow's RV directly from past RV values, without the latent-variable machinery of GARCH?

The answer is yes, and the model that does it is remarkably simple: three OLS coefficients.

6.1 The Heterogeneous Market Hypothesis

Mean Reversion

A quantity *mean-reverts* if it tends to return to a long-run average after being pushed away. If today's value is unusually high, mean reversion says tomorrow's value is more likely to be lower (and vice versa). The speed of mean reversion determines how long departures persist.

Start with an observation about who trades in financial markets. Not everyone operates at the same frequency. Day traders react to what happened in the last few hours. Portfolio managers at mutual funds or pension funds rebalance weekly. Large institutional allocators (sovereign wealth funds, endowments) adjust positions monthly or quarterly.

Each type of participant pays attention to volatility at their own characteristic horizon. A day trader cares about yesterday's realized volatility. A weekly rebalancer looks at the average volatility over the past week. A monthly allocator responds to the volatility level over the past month.

Müller et al. (1993) formalized this as the *Heterogeneous Market Hypothesis* (HMH): the market is a superposition of participants operating at different time scales, and these participants interact. When monthly allocators adjust positions, it creates volatility that daily traders react to, and vice versa.

Three Clocks Running Simultaneously

Imagine three clocks ticking at different speeds. Clock 1 (daily) ticks every day and captures fast-moving volatility reactions: earnings surprises, Fed announcements, flash crashes. Clock 2 (weekly) ticks every week and captures medium-term dynamics: multi-day volatility clusters, the weekly options expiration cycle. Clock 3 (monthly) ticks every month and captures the slow-moving background volatility level: business-cycle shifts, prolonged risk-on/risk-off regimes. Observed volatility is a mixture of all three clocks. The HAR model puts one predictor per clock.

Figure 6.1 illustrates how the three participant types feed into observed market volatility.

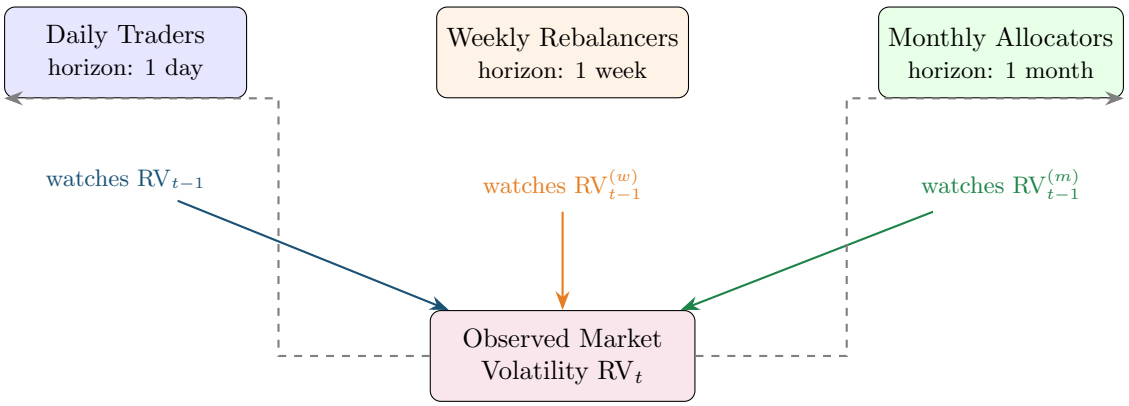


Figure 6.1: The Heterogeneous Market Hypothesis. Three types of market participants operate at different horizons, each responding to volatility averaged over their characteristic time scale. Their collective actions produce the observed realized volatility. Dashed arrows indicate feedback: today's volatility feeds back into each participant's future decisions.

Why This Matters for Forecasting

If the market were homogeneous (all participants on the same clock), a simple AR(1) model would suffice. The heterogeneity means that volatility dynamics involve multiple time scales simultaneously. You need a model that captures daily, weekly, and monthly persistence in a single equation. That model is HAR.

6.2 The HAR Model

OLS Regression

Ordinary Least Squares (OLS) fits a linear equation $y = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k + \varepsilon$ by choosing $\beta_0, \ldots, \beta_k$ to minimize the sum of squared residuals $\sum \varepsilon_t^2$. The resulting coefficients are the best linear unbiased estimators under standard assumptions (Gauss–Markov theorem). If you can set up a forecasting problem as a linear regression, OLS gives you the answer.

The insight from Section 6.1 suggests three predictors: yesterday's RV, the average RV over the past week, and the average RV over the past month. Corsi (2009) turned this directly into a regression. The key move is defining the weekly and monthly averages.

Weekly and Monthly Realized Variance

The weekly average realized variance on day t is:

$$\mathrm{RV}_t^{(w)} = \frac{1}{5}\sum_{i=0}^{4} \mathrm{RV}_{t-i} \tag{6.1}$$

The monthly average realized variance on day t is:

$$\mathrm{RV}_t^{(m)} = \frac{1}{22}\sum_{i=0}^{21} \mathrm{RV}_{t-i} \tag{6.2}$$

- $\mathrm{RV}_t^{(w)}$: the arithmetic mean of the five most recent daily realized variances (including today)
- $\mathrm{RV}_t^{(m)}$: the arithmetic mean of the 22 most recent daily realized variances (including today)
- 5 trading days = 1 week; 22 trading days ≈ 1 month
- These are backward-looking averages, not forecasts

In Plain English

These are just moving averages of past volatility over two different windows. $\mathrm{RV}^{(w)}$ smooths out the last five days to give a "medium-term" volatility reading, and $\mathrm{RV}^{(m)}$ smooths over the last 22 days to give a "long-term" reading. The daily value captures what happened yesterday; the averages capture what has been happening recently and over the past month.

Why This Matters

When you build your feature matrix for any ML model, you will always include these three inputs: daily, weekly average, and monthly average RV. They are the core HAR features, and every extension in this chapter simply adds to them.

Now the model. The idea is as simple as it sounds: regress tomorrow's realized variance on yesterday's daily, weekly, and monthly values.

The HAR-RV Model (,)

$$\mathrm{RV}_{t+1} = \beta_0 + \beta_d\, \mathrm{RV}_t + \beta_w\, \mathrm{RV}_t^{(w)} + \beta_m\, \mathrm{RV}_t^{(m)} + \varepsilon_{t+1} \tag{6.3}$$

- RV_{t+1}: tomorrow's realized variance (the target you are forecasting)
- β_0: intercept, related to the long-run average level of RV
- β_d: coefficient on yesterday's daily RV; captures the short-term reaction
- RV_t: yesterday's realized variance
- β_w: coefficient on the weekly average; captures medium-term persistence
- $\mathrm{RV}_t^{(w)}$: average of the five most recent daily RV values (Equation 6.1)
- β_m: coefficient on the monthly average; captures long-term persistence
- $\mathrm{RV}_t^{(m)}$: average of the 22 most recent daily RV values (Equation 6.2)
- ε_{t+1}: forecast error

Estimation: OLS. No maximum likelihood, no iterative optimization, no latent variables.

HAR Mimics Long Memory with Three Coefficients

Volatility autocorrelations decay slowly (Chapter 5, Section 5.6). A pure AR model would need many lags to capture this. HAR achieves the same effect with a trick: the weekly average $\text{RV}^{(w)}$ implicitly includes lags 1 through 5, and the monthly average $\text{RV}^{(m)}$ implicitly includes lags 1 through 22. Three coefficients, but through the averages, information from 22 past days enters the forecast. The result is autocorrelation decay that closely approximates a long-memory process, with none of the estimation complexity of FIGARCH (Corsi, 2009).

Why This Matters

HAR is YOUR primary baseline. Every ML model you build must beat HAR to be worth anything. It predicts tomorrow's vol from a weighted combination of yesterday's vol, last week's average vol, and last month's average vol. When you report results, your first table column is always HAR.

Log or Level?

Many papers estimate HAR on ln(RV) rather than RV in levels. As noted in Chapter 2 (Section 2.5), $\ln(\text{RV}_t)$ is approximately Gaussian, which makes OLS residuals better behaved and guarantees positive forecasts (since $e^x > 0$). The log specification generally forecasts better. Throughout this chapter, all equations are written in levels for clarity; in practice, use logs unless you have a specific reason not to.

6.2.1 Typical Coefficient Values

What do the coefficients look like on real data? Corsi (2009) estimates HAR on S&P 500 5-minute RV (1990–2003) and reports approximate values of $\beta_d \approx 0.36$, $\beta_w \approx 0.28$, $\beta_m \approx 0.28$. All three components contribute meaningfully. The monthly term's coefficient is similar to the daily term's, confirming that long-horizon persistence matters.

The in-sample R^2 is typically 0.40–0.60 for daily-horizon forecasts on equity index RV. This is high for a financial time series, and it comes from just three predictors.

Worked Example:

Computing a HAR Forecast You have 22 days of daily realized variance for the S&P 500 (in units of 10^{-4}, so 1.0 means 1% annualized vol corresponds to roughly 0.40×10^{-4} daily):

Day	RV_t	**Day**	RV_t	**Day**	RV_t
1	0.80	9	1.50	17	1.80
2	0.90	10	1.30	18	2.20
3	1.10	11	1.20	19	2.50
4	1.00	12	1.40	20	2.00
5	1.20	13	1.60	21	1.70
6	1.50	14	1.50	22	1.90
7	1.40	15	1.70		
8	1.60	16	1.90		

All values are in units of 10^{-4}.

Step 1: Compute the daily component. $\text{RV}_{22} = 1.90$

Step 2: Compute the weekly average.

$$\mathrm{RV}_{22}^{(w)} = \frac{1}{5}(\mathrm{RV}_{22} + \mathrm{RV}_{21} + \mathrm{RV}_{20} + \mathrm{RV}_{19} + \mathrm{RV}_{18}) = \frac{1.90 + 1.70 + 2.00 + 2.50 + 2.20}{5} = \frac{10.30}{5} = 2.06$$

Step 3: Compute the monthly average.

$$\mathrm{RV}_{22}^{(m)} = \frac{1}{22}\sum_{i=1}^{22} \mathrm{RV}_i = \frac{0.80 + 0.90 + \cdots + 1.90}{22} = \frac{32.70}{22} = 1.486$$

Step 4: Apply pre-fitted coefficients. Using $\beta_0 = 0.05$, $\beta_d = 0.36$, $\beta_w = 0.28$, $\beta_m = 0.28$ (all in 10^{-4} units):

$$\begin{aligned}\widehat{\mathrm{RV}}_{23} &= 0.05 + 0.36 \times 1.90 + 0.28 \times 2.06 + 0.28 \times 1.486 \\ &= 0.05 + 0.684 + 0.577 + 0.416 \\ &= 1.727\end{aligned}$$

The forecast for day 23 is 1.727×10^{-4}, or equivalently a daily volatility of $\sqrt{1.727 \times 10^{-4}} = 1.31\%$.

Notice how the forecast blends three time scales. The daily term ($\mathrm{RV}_{22} = 1.90$) pulls the forecast toward recent conditions. The weekly average (2.06) is higher than the monthly average (1.486) because the last five days were volatile, so the weekly term pulls the forecast up. The monthly average anchors the forecast closer to the longer-run level.

Figure 6.2 summarizes the HAR cascade: how the three components at different time scales feed into a single forecast.

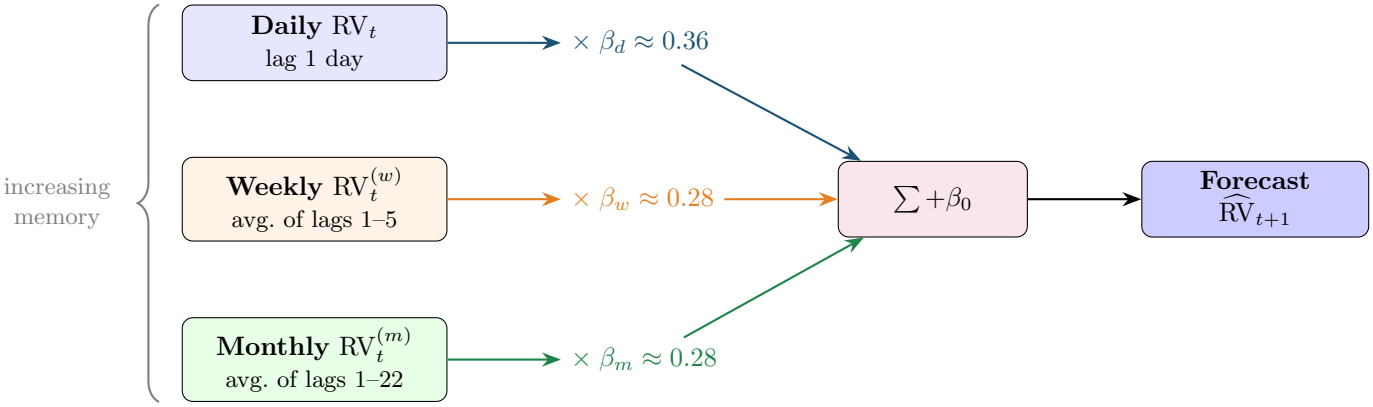


Figure 6.2: The HAR cascade. Three components at daily, weekly, and monthly time scales are weighted by OLS coefficients and summed to produce the forecast. Through the weekly and monthly averages, information from 22 past days enters the model with only three coefficients, approximating long-memory decay. Typical coefficient values are from Corsi (2009) on S&P 500 data.

6.3 HAR-J and HAR-CJ: Adding Jumps

The basic HAR uses total realized variance RV_t as the predictor. But as discussed in Chapter 2 (Equation 2.5), RV captures both the continuous component of price variation and any jumps that occurred during the day. A natural question: do jumps and continuous variation have different forecasting power?

Bipower Variation

Bipower variation (BPV) is an estimator of integrated variance that is robust to jumps. It is defined as:

$$\text{BPV}_t = \frac{\pi}{2} \sum_{i=2}^{n} |r_{t,i}| \, |r_{t,i-1}| \tag{6.4}$$

- $|r_{t,i}|$: absolute value of the i-th intraday return
- The product of consecutive absolute returns averages out jump contributions because it is unlikely that two consecutive intervals both contain a jump
- $\pi/2$: a scaling constant that ensures BPV_t converges to integrated variance IV_t (not quadratic variation) as sampling frequency increases

The key property: $\text{BPV}_t \to \text{IV}_t$ even when jumps are present, whereas $\text{RV}_t \to \text{IV}_t + \sum(\text{jumps})^2$. The difference $\text{RV}_t - \text{BPV}_t$ therefore isolates the jump component. BPV was introduced by Barndorff-Nielsen and Shephard (2004) and is covered in detail in Chapter 4.

In Plain English

BPV estimates the "smooth" part of volatility by multiplying consecutive absolute returns together. The idea is that a genuine jump is a one-off spike that is unlikely to hit two adjacent intervals, so the product of neighbors filters jumps out while keeping the continuous volatility signal.

Why This Matters

BPV is the tool you use to separate continuous volatility from jumps when constructing features for HAR-J and HAR-CJ. If your data source provides intraday returns, computing BPV alongside RV gives you a richer feature set at zero additional model complexity.

6.3.1 HAR-J

Andersen et al. (2007) proposed the HAR-J model, which adds a jump component as an additional predictor:

$$\text{RV}_{t+1} = \beta_0 + \beta_d \, \text{RV}_t + \beta_w \, \text{RV}_t^{(w)} + \beta_m \, \text{RV}_t^{(m)} + \beta_J \, J_t + \varepsilon_{t+1} \tag{6.5}$$

- $J_t = \max(\text{RV}_t - \text{BPV}_t, 0)$: the estimated jump component on day t; the max ensures non-negativity, since measurement error can make $\text{RV}_t - \text{BPV}_t$ slightly negative even on jumpless days
- β_J: coefficient on jumps; typically estimated as negative and small, meaning jumps have a weak or transient effect on future volatility
- All other terms are the same as in Equation (6.3)

In Plain English

HAR-J takes the standard HAR and adds one extra input: how much of yesterday's volatility came from jumps (sudden price spikes) rather than normal trading. If yesterday had a big jump, the model can treat that spike differently from steady high volatility, because jumps tend not to repeat.

Why This Matters

HAR-J adds the jump component as a separate feature. The continuous component is more persistent and predictable than jumps, so letting the model see them separately improves forecasts. In your ML pipeline, $J_t = \max(\text{RV}_t - \text{BPV}_t, 0)$ is a feature you should always compute when you have intraday data.

Andersen et al. (2007): Jumps Matter Less Than You Expect

Andersen et al. (2007) find that the jump component J_t is statistically significant but economically small in forecasting next-day RV. Most of the predictive power comes from the continuous component. Jumps are largely transient: a jump on day t does not meaningfully raise volatility on day $t+1$.

6.3.2 HAR-CJ

Corsi et al. (2010) take the separation further with the HAR-CJ model, which replaces total RV with the continuous and jump components at all three horizons:

$$\text{RV}_{t+1} = \beta_0 + \beta_d^C C_t + \beta_w^C C_t^{(w)} + \beta_m^C C_t^{(m)} + \beta_d^J J_t + \beta_w^J J_t^{(w)} + \beta_m^J J_t^{(m)} + \varepsilon_{t+1} \tag{6.6}$$

- $C_t = \text{BPV}_t$: the continuous component of day t's variation
- $J_t = \max(\text{RV}_t - \text{BPV}_t, 0)$: the jump component of day t
- $C_t^{(w)}, C_t^{(m)}$: weekly and monthly averages of the continuous component, constructed like $\text{RV}^{(w)}$ and $\text{RV}^{(m)}$
- $J_t^{(w)}, J_t^{(m)}$: weekly and monthly averages of the jump component

The HAR-CJ model has six slope coefficients instead of three. The empirical finding is consistent with HAR-J: continuous coefficients (β^C) are large and significant; jump coefficients (β^J) are small and often insignificant at weekly and monthly horizons (Corsi et al., 2010).

In Plain English

HAR-CJ goes further than HAR-J by building separate daily, weekly, and monthly averages for the continuous part and the jump part. Instead of asking "how volatile was the market?" at each horizon, it asks two questions: "how volatile was the smooth trading activity?" and "how big were the jumps?" The consistent finding is that only the continuous answers matter for forecasting; the jump answers contribute almost nothing beyond the daily horizon.

Why This Matters

HAR-CJ gives you six features instead of three: $C_t, C_t^{(w)}, C_t^{(m)}, J_t, J_t^{(w)}, J_t^{(m)}$. When you feed these to an ML model, the model can learn that continuous components carry the forecasting signal and jumps are noise, rather than you having to hard-code that decision.

Jumps Are Noise for Forecasting

A large jump (e.g., from a surprise rate cut) spikes today's RV but typically does not persist into tomorrow. The continuous component, which reflects the background level of trading activity and uncertainty, is far more persistent and forecastable. This is why separating C from J can help: it prevents a one-day spike from inflating the forecast.

Figure 6.3 shows how HAR-CJ decomposes realized variance before forecasting, compared to the standard HAR approach.

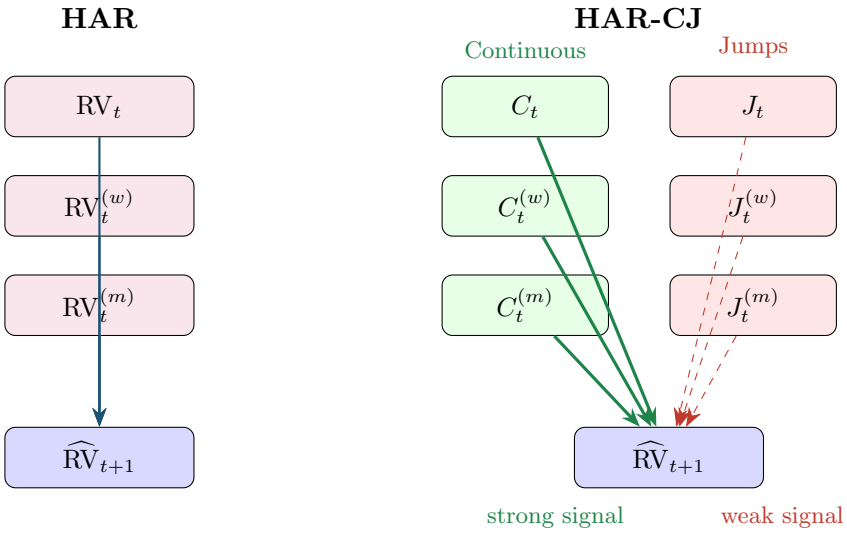


Figure 6.3: HAR vs. HAR-CJ. HAR feeds total RV at three horizons into the forecast (left). HAR-CJ first decomposes RV into continuous and jump components at each horizon (right). Thick green arrows indicate strong forecasting power from continuous components; thin dashed red arrows indicate the weak contribution of jumps.

6.4 SHAR: Good Volatility, Bad Volatility

HAR and its jump extensions treat all returns symmetrically: a large up move and a large down move both increase RV by the same amount (both r^2 terms are positive). But Chapter 5 documented the leverage effect (Section 5.3): negative returns increase volatility more than positive returns of the same magnitude. Can you bring this asymmetry into the HAR framework?

Realized Semivariance

Realized semivariance decomposes realized variance into contributions from positive and negative intraday returns:

$$RS_t^+ = \sum_{i=1}^{n} r_{t,i}^2 \, \mathbf{1}_{\{r_{t,i}>0\}}, \qquad RS_t^- = \sum_{i=1}^{n} r_{t,i}^2 \, \mathbf{1}_{\{r_{t,i}<0\}} \tag{6.7}$$

- RS_t^+: positive semivariance, the sum of squared positive intraday returns
- RS_t^-: negative semivariance, the sum of squared negative intraday returns
- $\mathbf{1}_{\{r_{t,i}>0\}}$: indicator function, equal to 1 if $r_{t,i} > 0$
- $RS_t^+ + RS_t^- = \mathrm{RV}_t$: the two halves sum to total realized variance

In Plain English

Realized semivariance splits total volatility into "upside choppiness" and "downside choppiness." RS^+ measures how much the price bounced around on the way up during the day, while RS^- measures how much it bounced around on the way down. Together they add up to total RV, but they carry different information about what comes next.

Patton and Sheppard (2015a): Bad Volatility Is More Persistent

Decomposing realized variance into positive semivariance RS^+ and negative semivariance RS^- substantially improves forecasts. Bad volatility (RS^-, from downward price moves) is significantly more persistent than good volatility (RS^+, from upward moves).

The SHAR (Semivariance HAR) model replaces the daily RV term with RS^+ and RS^-:

$$\text{RV}_{t+1} = \beta_0 + \beta_d^+ RS_t^+ + \beta_d^- RS_t^- + \beta_w \text{RV}_t^{(w)} + \beta_m \text{RV}_t^{(m)} + \varepsilon_{t+1} \tag{6.8}$$

- RS_t^+: yesterday's positive semivariance ("good" volatility)
- RS_t^-: yesterday's negative semivariance ("bad" volatility)
- $\beta_d^- > \beta_d^+$ is the typical finding: negative semivariance has a larger coefficient, meaning bad volatility predicts more future volatility
- The weekly and monthly components remain as total RV averages

Why Downward Moves Are More Informative

Consider two days with identical RV $= 2 \times 10^{-4}$. On Day A, most of the variation came from upward moves ($RS^+ = 1.6$, $RS^- = 0.4$): a rally day with choppy upward movement. On Day B, most came from downward moves ($RS^+ = 0.4$, $RS^- = 1.6$): a selloff with persistent downward pressure.

Day B predicts higher future volatility. Why? Selloffs trigger margin calls, portfolio insurance rebalancing, stop-loss orders, and panic selling, all of which generate additional volatility. Rallies of the same magnitude do not create the same cascading effects. This asymmetry is the leverage effect operating at the intraday level.

Why This Matters

SHAR treats positive and negative semivariance differently, capturing the leverage effect within the HAR framework. For your project, RS_t^+ and RS_t^- are two features that replace the single RV_t feature, and an ML model can learn the asymmetry automatically. SHAR is a strong baseline when forecasting equity index volatility, where the leverage effect is most pronounced.

Patton and Sheppard (2015a) show that SHAR improves forecast accuracy relative to HAR across multiple asset classes, with the largest gains on equity indices where the leverage effect is strongest.

6.5 HARQ: Handling Measurement Error

All the models so far treat every day's RV_t as equally reliable. But it is not. Chapter 2 showed that RV is an *estimate* of integrated variance, subject to estimation error that depends on how volatile volatility was during the day.

Realized Quarticity

The precision of realized variance as an estimator of integrated variance depends on the *integrated quarticity*, the integral of σ_s^4 over the day (see Equation 2.6 in Chapter 2).

Realized quarticity (RQ_t) is a consistent estimator of this quantity:

$$RQ_t = \frac{n}{3} \sum_{i=1}^{n} r_{t,i}^4 \tag{6.9}$$

- $r_{t,i}^4$: the fourth power of the i-th intraday return
- $n/3$: a scaling constant that ensures consistency
- High RQ_t means that volatility itself was volatile during the day, making RV_t a noisy estimate of IV_t
- Low RQ_t means volatility was stable, making RV_t precise

In Plain English

Realized quarticity measures how much volatility itself jumped around during the day. If intraday volatility was steady, fourth powers of returns are small and RQ is low, meaning your RV estimate is trustworthy. If intraday volatility spiked wildly (e.g., a flash crash followed by a recovery), RQ is high, meaning your RV number is noisy and should be taken with a grain of salt.

Why This Matters

RQ_t is the key ingredient that separates HARQ from plain HAR. When you compute your daily features, always compute $\sqrt{RQ_t}$ alongside RV_t. It tells your model how much to trust yesterday's volatility reading, which is the core insight behind the HARQ paper your project builds on (Bollerslev et al., 2016a).

The idea behind HARQ is simple: when yesterday's RV is noisy (high RQ), the model should rely less on it. When it is precise (low RQ), the model should rely more on it.

Bollerslev et al. (2016a): HARQ Adjusts for Measurement Error

Bollerslev et al. (2016a) allow the daily coefficient in the HAR model to vary with realized quarticity RQ_t. On noisy days (high RQ), the daily RV coefficient shrinks toward zero, down-weighting the unreliable estimate. On precise days (low RQ), the coefficient is large, fully exploiting the signal. This simple modification produces statistically significant forecast improvements.

$$\text{RV}_{t+1} = \beta_0 + \left(\beta_d + \beta_{dQ}\sqrt{RQ_t}\right) \text{RV}_t + \beta_w \, \text{RV}_t^{(w)} + \beta_m \, \text{RV}_t^{(m)} + \varepsilon_{t+1} \tag{6.10}$$

- β_d: baseline daily coefficient (the coefficient when $RQ_t = 0$, i.e., the hypothetical noise-free case)
- β_{dQ}: the adjustment coefficient; typically estimated as negative
- $\sqrt{RQ_t}$: square root of realized quarticity, a measure of how noisy RV_t is as an estimate
- $\beta_d + \beta_{dQ}\sqrt{RQ_t}$: the effective daily coefficient, which varies from day to day
- When RQ_t is large (noisy day), $\beta_{dQ}\sqrt{RQ_t}$ is a large negative number, shrinking the effective coefficient toward zero
- When RQ_t is small (precise day), the adjustment is negligible and the coefficient stays near β_d

In Plain English

HARQ adds estimation uncertainty as a feature. On days when yesterday's RV estimate is noisy (high RQ), the model automatically relies less on it and shifts weight to the more stable weekly and monthly averages. On days when yesterday's RV is precise (low RQ), the model trusts it fully. It is a "confidence-weighted" version of HAR.

Why This Matters

HARQ is THE paper your project builds on. The idea that measurement quality should modulate how much weight the model gives to each input is exactly the kind of structure an ML model can generalize. Your project explores whether tree ensembles or neural networks can learn this adaptive weighting from data, potentially extending it to weekly and monthly coefficients or to other noise proxies beyond RQ.

The following diagram shows how the effective daily coefficient changes with measurement noise.

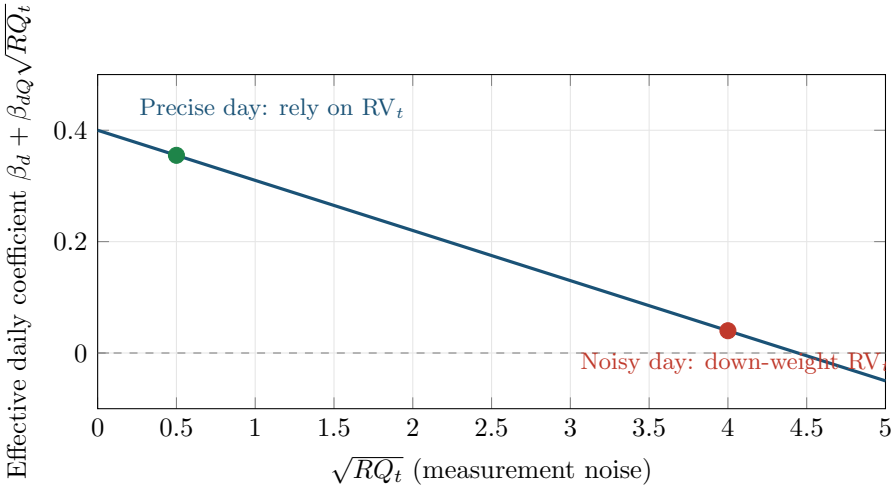


Figure 6.4: HARQ's adaptive coefficient. On days when realized quarticity is low (left), the daily coefficient is large and the model trusts RV_t. On days when quarticity is high (right), the coefficient shrinks toward zero, shifting weight to the more stable weekly and monthly averages.

HARQ Is the Strongest Univariate RV Forecast

Among models that use only past RV and its measurement-error statistics, HARQ is the strongest in the literature (Bollerslev et al., 2016a). It is the bar that any ML model must clear. If your tree ensemble or neural network cannot beat HARQ on out-of-sample QLIKE (Chapter 17), you have not learned useful nonlinear structure; you have likely overfit.

HARQ Needs Realized Quarticity

HARQ requires computing RQ_t from intraday returns. If your data source provides only daily RV without the underlying intraday returns, you cannot construct RQ_t and HARQ is not available. In that case, fall back to SHAR or plain HAR as your benchmark.

6.6 HAR-X and Beyond: Adding Exogenous Predictors

All HAR variants so far use only past RV (and its decompositions) as predictors. But other variables contain information about future volatility: the VIX, lagged signed returns, macroeconomic indicators, trading volume, options-implied information. The HAR-X framework adds these as extra regressors.

HAR-X

$$\mathrm{RV}_{t+1} = \beta_0 + \beta_d\, \mathrm{RV}_t + \beta_w\, \mathrm{RV}_t^{(w)} + \beta_m\, \mathrm{RV}_t^{(m)} + \sum_{j=1}^{p} \gamma_j\, X_{j,t} + \varepsilon_{t+1} \tag{6.11}$$

- $X_{j,t}$: the j-th exogenous predictor observed on day t
- γ_j: coefficient on the j-th predictor
- p: number of exogenous predictors
- Common choices for $X_{j,t}$: VIX level, VIX innovations (ΔVIX), lagged daily return r_t (signed, to capture leverage), overnight return, trading volume, implied–realized volatility spread

In Plain English

HAR-X is just HAR with extra columns in the regression. The three core RV inputs stay, and you bolt on whatever other predictors you think contain information about future volatility: the VIX, yesterday's stock return, trading volume, macro surprises, and so on. Each additional predictor gets its own coefficient, so OLS tells you whether it helps after controlling for the core HAR terms.

Why This Matters

HAR-X is the linear version of what your ML model will do. Any feature you feed to a random forest or neural network, you should first test in a HAR-X regression to see whether it helps linearly. If it does, the ML question becomes: does the feature interact nonlinearly with other inputs? If it does not help even linearly, adding it to an ML model is unlikely to help and may cause overfitting.

Bollerslev et al. (2018a) take this to a large scale in their "Risk Everywhere" paper, constructing a large cross-section of HAR-X type models with many macro and market predictors.

When p is large, you face a classic overfitting problem: many predictors, limited data. Audrino and Knaus (2016) propose the "Lassoing the HAR" approach, applying the Lasso (L1 regularization) to the HAR-X framework:

$$\min_{\beta,\gamma} \sum_t \left(\mathrm{RV}_{t+1} - \beta_0 - \beta_d\, \mathrm{RV}_t - \beta_w\, \mathrm{RV}_t^{(w)} - \beta_m\, \mathrm{RV}_t^{(m)} - \sum_j \gamma_j X_{j,t} \right)^2 + \lambda \sum_j |\gamma_j| \tag{6.12}$$

- λ: regularization penalty; larger λ shrinks more coefficients to exactly zero
- The penalty applies to the exogenous coefficients γ_j, not the core HAR terms
- The Lasso selects which exogenous variables matter while keeping the core HAR structure intact

In Plain English

When you have many candidate predictors, Lasso-HAR asks: "which of these extra variables actually help, and which are just adding noise?" It does this by penalizing the model for using too many variables. The penalty forces weak predictors' coefficients to exactly zero, automatically dropping them from the forecast, while keeping the core daily/weekly/monthly HAR terms intact.

Why This Matters

Lasso-HAR is the simplest regularized model in your toolbox and a natural stepping stone between plain HAR and full ML. If Lasso-HAR with 20 features beats HAR, you know the extra features carry signal. The next question, which your project addresses, is whether nonlinear models (trees, neural nets) can extract even more from those same features.

Lasso Regularization

The Lasso (Least Absolute Shrinkage and Selection Operator) adds an L1 penalty ($\lambda \sum |\gamma_j|$) to the OLS objective. This has two effects: (1) it shrinks coefficients toward zero, reducing overfitting; (2) it sets some coefficients to exactly zero, performing automatic variable selection. The tuning parameter λ controls the strength of the penalty and is chosen by cross-validation.

HAR-X Is the Bridge to ML

HAR-X with many predictors is effectively a linear ML model. Adding Lasso regularization makes it a penalized linear model with variable selection. Chapters 11–14 ask the next question: can *nonlinear* models (tree ensembles, neural networks) improve on this, or does the extra flexibility just lead to overfitting? The answer depends critically on the feature set and forecast horizon.

6.7 Why HAR Is Hard to Beat

If HAR is so simple (three coefficients, OLS), why is it the benchmark rather than a stepping stone that ML quickly surpasses? Three reasons.

Reason 1: Volatility is highly persistent and approximately linear. The autocorrelation function of RV_t decays slowly and smoothly. A model that captures the right decay rate with the right mixture of time scales gets most of the forecastable variation. HAR does exactly this. The residual nonlinearity is real but small relative to the linear component.

Reason 2: The signal-to-noise ratio is low. Even though volatility is more predictable than returns, the day-to-day fluctuations in RV are large. A model that tries to fit these fluctuations more precisely (e.g., by adding interaction terms, polynomial features, or deep layers) tends to fit noise rather than signal, especially with the relatively short samples typical in finance (10–20 years of daily data is 2,500–5,000 observations).

Reason 3: Daily horizon, RV-only features. When you restrict the feature set to past RV values and forecast one day ahead, there is little nonlinear structure for ML to exploit. The gains from ML come from two sources that HAR cannot access:

1. **Richer features**: options-implied volatility, cross-asset information, order flow, macroeconomic data, text/news sentiment (Chapter 10).

2. **Longer horizons**: at weekly or monthly forecast horizons, nonlinear regime dynamics and mean-reversion patterns become more pronounced, giving tree-based models and neural networks more to work with.

The HAR Litmus Test

If your ML model does not beat HAR on the same features (past RV only) and the same forecast horizon (one day), you have not learned nonlinear structure; you have overfit noise. Always report HAR alongside your ML results. If you beat HAR, the follow-up question is: does the improvement come from richer features (good) or from fitting noise in RV (bad)? The answer determines whether the ML model has real economic value.

Why This Matters

This table is your strategic roadmap. If you use RV-only features at the daily horizon, HAR will probably tie or beat your ML model, and that is expected. Your project should aim for the bottom-right cell: richer features (e.g., VIX, signed returns, cross-asset vol) at weekly or monthly horizons, where both the feature advantage and the nonlinear structure advantage compound. Design your experiments accordingly.

The following table summarizes where ML has and has not consistently beaten HAR in the literature.

Setting	HAR vs. ML	Why
Daily horizon, RV-only features	HAR wins or ties	Little nonlinear signal
Daily horizon, rich features	ML often wins	Extra features carry new info
Weekly/monthly horizon, RV-only	ML sometimes wins	Nonlinear regime effects
Weekly/monthly horizon, rich features	ML usually wins	Both advantages compound

Publication Bias

Papers that fail to beat HAR are less likely to be published. The literature therefore overstates the frequency with which ML improves on HAR. Be skeptical of reported improvements below 5–10% in out-of-sample QLIKE, and always check whether the improvement is statistically significant via a Diebold–Mariano test (Chapter 17).

Summary

- The **Heterogeneous Market Hypothesis** (Müller et al., 1993) posits that markets are driven by participants operating at daily, weekly, and monthly horizons, whose interactions produce the observed volatility dynamics.
- The **HAR model** (Corsi, 2009) translates this directly into a regression: $\mathrm{RV}_{t+1} = \beta_0 + \beta_d \mathrm{RV}_t + \beta_w \mathrm{RV}_t^{(w)} + \beta_m \mathrm{RV}_t^{(m)} + \varepsilon_{t+1}$, estimated by OLS.
- HAR **mimics long memory** with only three coefficients by embedding lags 1–22 through the weekly and monthly averages.
- **Typical** R^2 for daily-horizon HAR on equity index RV: 0.40–0.60.
- **HAR-J** (Andersen et al., 2007) adds a jump component $J_t = \max(\mathrm{RV}_t - \mathrm{BPV}_t, 0)$. Jumps are statistically significant but economically small predictors.
- **HAR-CJ** (Corsi et al., 2010) separates continuous and jump components at all three horizons. Continuous variation dominates the forecast; jumps are largely transient.

- **SHAR** (Patton and Sheppard, 2015a) decomposes daily RV into positive (RS^+) and negative (RS^-) semivariance. Bad volatility (RS^-) is significantly more persistent, reflecting the leverage effect at the intraday level.
- **HARQ** (Bollerslev et al., 2016a) allows the daily coefficient to vary with realized quarticity RQ_t, down-weighting RV_t on noisy days. It is the strongest univariate RV forecast in the literature.
- **HAR-X** adds exogenous predictors (VIX, returns, macro). With many predictors, Lasso regularization (Audrino and Knaus, 2016) prevents overfitting while preserving the core HAR structure.
- HAR is **extremely competitive** at the daily horizon with RV-only features. ML gains come from richer feature sets and longer horizons.
- The **HAR litmus test**: if your ML model cannot beat HAR on the same features and horizon, it has overfit noise, not learned nonlinear structure.
- All HAR variants are estimated by **OLS** (or penalized OLS), require no iterative optimization, and produce interpretable coefficients. This simplicity is a feature, not a limitation.
- Throughout this guide, HAR (or HARQ) is the **mandatory baseline** for every forecasting model.

Key Results

Result	Source	Finding
Heterogeneous Market Hypothesis	Müller et al. (1993)	Markets are driven by participants at multiple horizons; volatility dynamics are a superposition of time scales
HAR model	Corsi (2009)	Three OLS coefficients (daily, weekly, monthly RV) approximate long-memory dynamics; $R^2 \approx 0.40$–0.60 for daily equity index forecasts
HAR-J (jumps)	Andersen et al. (2007)	Adding jump component improves fit marginally; jumps are largely transient and do not persist into future volatility
HAR-CJ	Corsi et al. (2010)	Full continuous/jump decomposition at all horizons; continuous variation dominates forecasting power
SHAR (semivariance)	Patton and Sheppard (2015a)	Negative semivariance is more persistent than positive; capturing the leverage effect improves forecasts
HARQ (measurement error)	Bollerslev et al. (2016a)	Varying the daily coefficient with $\sqrt{RQ_t}$ down-weights noisy estimates; strongest univariate RV forecast
Lassoing the HAR	Audrino and Knaus (2016)	Lasso selects relevant exogenous predictors while preserving core HAR structure; prevents overfitting with many regressors
Risk Everywhere	Bollerslev et al. (2018a)	Large-scale HAR-X with macro/market predictors; confirms predictability from multiple exogenous sources

Chapter 7

Rough Volatility

Why This Chapter

Rough volatility provides an alternative explanation for the slow decay in volatility autocorrelation that HAR (Chapter 6) captures with three components and FIGARCH (Chapter 5) captures with fractional integration. The RFSV model is a parsimonious forecaster competitive with HAR and LSTM. The Cont–Das counterargument warns against taking roughness as ground truth. Project 4 (rough vol vs. deep learning) directly tests RFSV against universal LSTM.

Chapter 6 showed that HAR's three-component structure (daily, weekly, monthly) approximates the slow power-law decay of volatility autocorrelations. Chapter 5 showed that FIGARCH (Section 5.6) captures the same slow decay with a fractional differencing parameter d. Both approaches are empirically motivated heuristics: they match the shape of the autocorrelation function, but neither explains *why* the decay is so slow.

This chapter presents a different starting point. Instead of asking "how should I model the autocorrelation structure?" it asks: "what kind of stochastic process would *generate* the autocorrelation structure we observe?" The answer, proposed by Gatheral et al. (2018), is that the log of realized volatility behaves like a *fractional Brownian motion* with a very low Hurst exponent, around $H \approx 0.1$. This makes the volatility path much rougher (more jagged) than standard Brownian motion ($H = 0.5$).

7.1 Why Path Shape Matters: A Hedging Motivation

Before diving into the mathematics of roughness, consider a practical puzzle that motivates the entire chapter.

An options trader buys a 30-day ATM straddle and delta-hedges daily. Over the month, realized vol comes in at exactly 20%. But the trader's P&L depends on *when* the moves happened, not just their aggregate size.

Two Paths, Same RV, Different P&L

Path A: the stock drifts quietly for 25 days then has 5 days of extreme moves. Path B: moves are spread evenly across all 30 days. Both paths have identical 30-day RV = 20%. But the trader's cumulative gamma P&L (Section 9.6) differs because:

- Gamma varies with moneyness: it is highest when the stock is near the strike.

- On Path A, most of the vol occurs after the stock has moved away from the strike (gamma is low), so the trader captures less.
- On Path B, moves happen while gamma is still high, so the trader captures more.

The conclusion: forecasting *average* realized vol is necessary but not sufficient. The **path texture**, how vol is distributed across time and price levels, also determines economic outcomes.

This is exactly what rough volatility ($H \ll 0.5$) captures: rough paths have more fine-grained variation at short time scales, meaning the vol arrives in frequent small bursts rather than rare large moves. For a delta-hedger, rough paths produce more predictable P&L (the vol arrives continuously rather than in lumps). Rosenbaum and Zhang (2022) showed that both the universal LSTM and the parametric rough-vol model converge on this same characterization of the volatility process: one from data, one from theory.

7.2 What Is Roughness?

The central concept is the *Hurst exponent*, a single number that describes how jagged or smooth a random path looks.

Brownian Motion (Standard)

A standard Brownian motion W_t is the canonical "random walk in continuous time." Its key properties:

- $W_0 = 0$.
- Increments $W_{t+h} - W_t$ are Gaussian with mean zero and variance h.
- Increments over non-overlapping intervals are independent.
- Paths are continuous but nowhere differentiable (they look jagged at every magnification).

If you zoom into a Brownian path, the zoomed-in portion looks statistically identical to the original. This property is called *self-similarity*, and the scaling factor is $H = 1/2$: rescaling time by a factor c rescales the path by $c^{1/2}$.

Fractional Brownian Motion (fBM)

Fractional Brownian motion B_t^H, introduced by Mandelbrot and Van Ness (1968), generalizes standard Brownian motion by allowing the self-similarity exponent H to take any value in $(0, 1)$. Its key properties:

- $B_0^H = 0$, and increments are Gaussian with mean zero.
- $\mathrm{Var}(B_{t+h}^H - B_t^H) = h^{2H}$.
- The path is self-similar with exponent H: rescaling time by c rescales the path by c^H.
- When $H = 1/2$, fBM reduces to standard Brownian motion.
- When $H \neq 1/2$, increments are *not* independent. They are negatively correlated if $H < 1/2$ and positively correlated if $H > 1/2$.

The parameter H is called the *Hurst exponent* (after the hydrologist Harold Edwin Hurst, who studied long-range dependence in Nile river levels).

The Hurst exponent controls two things simultaneously: the roughness of the path and the correlation structure of increments.

Hurst Exponent and Path Regularity

For a fractional Brownian motion B_t^H with Hurst exponent $H \in (0, 1)$:

$$\text{Var}(B_{t+h}^H - B_t^H) = h^{2H} \tag{7.1}$$

- H: the Hurst exponent, controlling both path roughness and increment correlation
- h: the time lag
- h^{2H}: the variance of the increment over a window of length h
- When $H < 1/2$: increments are negatively correlated (a move up makes a move down more likely); the path is *rougher* than Brownian motion
- When $H = 1/2$: increments are independent; standard Brownian motion
- When $H > 1/2$: increments are positively correlated (trending behavior); the path is *smoother* than Brownian motion

Roughness = Anti-Persistence

Think of H as a dial. Turn it below 0.5 and the path becomes "anti-persistent": every upward wiggle is likely followed by a downward wiggle, creating a jagged, rapidly oscillating trajectory. Turn it above 0.5 and the path becomes persistent: moves tend to continue in the same direction, creating smooth, trend-like trajectories. At $H = 0.1$, the path is so anti-persistent that it reverses direction almost constantly. The result is a path so jagged that it looks qualitatively different from standard Brownian motion.

Figure 7.1 shows sample paths at four values of H. This is the most important visual in the chapter.

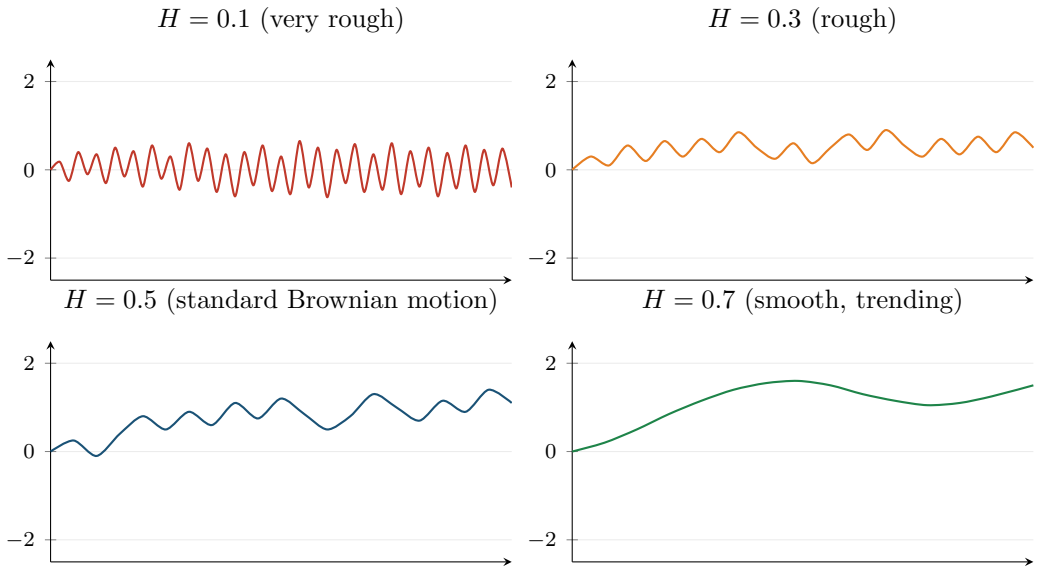


Figure 7.1: Sample paths of fractional Brownian motion at four values of the Hurst exponent H. Top left ($H = 0.1$, red): extremely rough, rapidly oscillating. Top right ($H = 0.3$, orange): rough but with visible short-range structure. Bottom left ($H = 0.5$, blue): standard Brownian motion, the familiar random walk. Bottom right ($H = 0.7$, green): smooth, trending trajectory. Empirically, log RV behaves like the top-left panel.

Why This Matters

The Hurst exponent is the single number that determines how fast volatility autocorrelations decay. With $H \approx 0.1$, the decay is slow enough that observations from weeks ago still carry predictive power for tomorrow's volatility. This is exactly the long-memory structure that HAR approximates with its daily/weekly/monthly components, and the reason HAR works as well as it does as a forecasting baseline.

Roughness Means $H < 1/2$

A process is called "rough" when its Hurst exponent satisfies $H < 1/2$. The path is more jagged than Brownian motion, and its increments are negatively correlated. The lower H is, the rougher the path. Empirical log-volatility has $H \approx 0.1$, which is far into the rough regime.

Figure 7.2 provides a compact reference for how different H values map to path behavior and real-world examples.

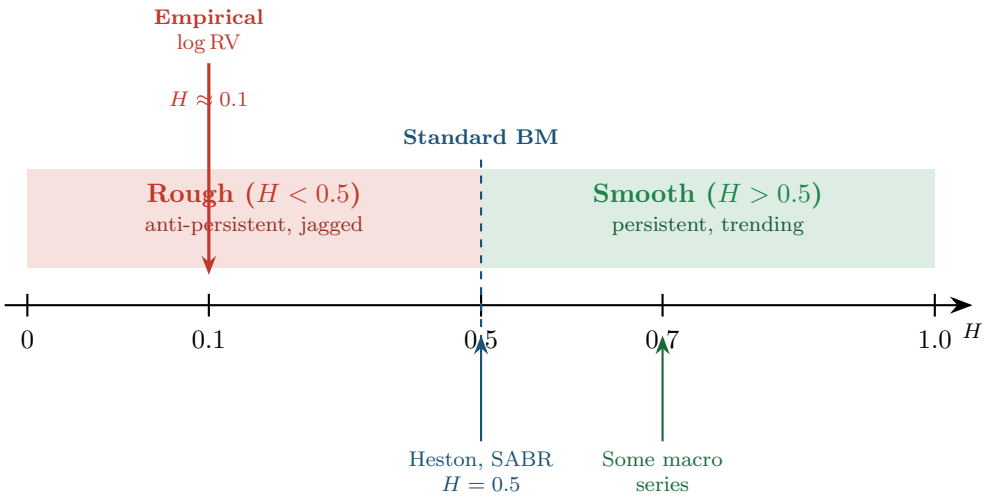


Figure 7.2: The Hurst exponent number line. Empirical log-volatility sits at $H \approx 0.1$, deep in the rough regime. Standard stochastic volatility models (Heston, SABR) assume $H = 0.5$. The gap between 0.1 and 0.5 is the motivation for the entire rough volatility program.

7.3 Volatility Is Rough

With the concept of roughness in hand, the central empirical claim of the rough volatility literature can be stated precisely.

Gatheral et al. (2018) studied the time series of $\log \mathrm{RV}_t$ (5-minute realized variance, as defined in Chapter 2) across a wide range of assets: equity indices, individual stocks, and foreign exchange. For each asset, they estimated the Hurst exponent H of the $\log \mathrm{RV}$ series. The finding was striking.

Gatheral et al. (2018): Volatility Is Rough

Across equity indices, individual stocks, and FX, the Hurst exponent of $\log \mathrm{RV}_t$ is consistently around $H \approx 0.1$, far below the $H = 0.5$ of standard Brownian motion. This means that log-volatility paths are much rougher (more jagged, more rapidly oscillating) than

any standard diffusion model would predict.

To put $H = 0.1$ in context: standard stochastic volatility models (Heston, SABR) assume that the volatility process is driven by a standard Brownian motion, which has $H = 0.5$. An H of 0.1 means the volatility path reverses direction roughly five times more frequently than a standard diffusion would.

7.3.1 How to Estimate H: The Variogram Method

The estimation approach in Gatheral et al. (2018) uses the scaling relationship in Equation (7.1). If X_t is an fBM with exponent H, then the variance of its increments scales as h^{2H}. In log-log coordinates, this becomes a straight line.

Variogram Estimator of H

For a time series $X_1, X_2, \ldots, X_T$, define the empirical q-th moment of increments at lag h:

$$m(q,h) = \frac{1}{T-h}\sum_{t=1}^{T-h} |X_{t+h} - X_t|^q \tag{7.2}$$

- X_t: the time series (here, $\log \mathrm{RV}_t$)
- h: the lag (number of days)
- q: the moment order (typically $q = 2$ for the variance, or $q = 1$ for the mean absolute increment)
- T: the number of observations

If X_t behaves like fBM with exponent H, then $m(q,h) \propto h^{qH}$. Taking logs: $\log m(q,h) = qH \cdot \log h + \text{const}$. Regressing $\log m(q,h)$ on $\log h$ across multiple lags gives a slope of qH, from which H is recovered.

In Plain English

The variogram asks a simple question: when you look at how much log-volatility changes over a time window, does that change grow quickly or slowly as you widen the window? For a standard random walk, doubling the window roughly doubles the variance of the change. For rough volatility, doubling the window barely increases the variance at all, because the anti-persistent path keeps reversing direction and staying near its starting point. The slope of the log-log plot tells you the Hurst exponent, which quantifies this scaling behavior.

Why This Matters

The variogram is your primary diagnostic tool for checking whether your realized volatility series exhibits the rough-vol property. Before fitting any model (HAR, RFSV, or LSTM), estimating H from the data tells you whether the long-memory structure that these models exploit is actually present in your specific asset. If $\hat{H}$ comes back near 0.1, you have confirmation that multi-scale models like HAR are appropriate; if it comes back near 0.5, simpler AR(1)-type models may suffice.

Worked Example:

Estimating H from a Variogram You have $T = 1{,}000$ daily observations of $\log \mathrm{RV}_t$ for the S&P 500. Using $q = 2$, you compute the average squared increment at lags $h =$

$1, 2, 5, 10, 20$ days:

Lag h (days)	$m(2,h)$	$\log_{10} m(2,h)$
1	0.050	-1.301
2	0.059	-1.229
5	0.074	-1.131
10	0.088	-1.056
20	0.104	-0.983

Step 1: Set up the regression. Plot $\log_{10} m(2,h)$ against $\log_{10} h$:

$\log_{10} h$	$\log_{10} m(2,h)$
0.000	-1.301
0.301	-1.229
0.699	-1.131
1.000	-1.056
1.301	-0.983

Step 2: Fit the slope. The OLS slope through these five points is approximately 0.245.
Step 3: Recover H. Since slope $= qH = 2H$, we get $H = 0.245/2 = 0.12$.
This is close to the $H \approx 0.1$ reported by Gatheral et al. (2018). The slow increase of $m(2,h)$ with lag (i.e., the low slope) reflects the anti-persistent, rapidly oscillating nature of log-volatility. For comparison, a standard Brownian motion would give a slope of $2 \times 0.5 = 1.0$, meaning the variance of increments grows linearly with lag.

Figure 7.3 shows the variogram in log-log coordinates, illustrating how the slope distinguishes rough from smooth processes.

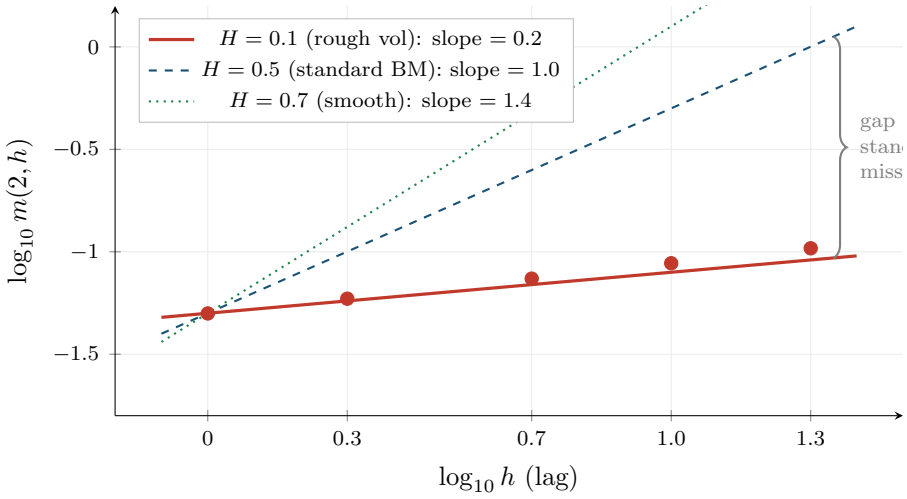


Figure 7.3: Variogram in log-log coordinates for three Hurst exponent values. The red line and data points ($H = 0.1$) show the flat slope characteristic of rough volatility: variance of increments grows very slowly with lag. The blue dashed line ($H = 0.5$) shows standard Brownian motion with a steep slope of 1.0. The gap between the two slopes quantifies how much standard diffusion models underestimate the persistence of volatility.

Bennedsen et al. (2022): Cross-Asset Universality

Bennedsen et al. (2022) extend the analysis of Gatheral et al. (2018) to a broad cross-section of asset classes (equities, equity indices, FX, fixed income, commodities) and confirm that $H \approx 0.1$ is universal. The Hurst exponent does not vary meaningfully across asset classes, geographies, or time periods. This universality suggests that $H \approx 0.1$ reflects a deep structural property of volatility dynamics, not a peculiarity of any particular market.

7.4 The RFSV Forecasting Formula

The observation that $\log \mathrm{RV}$ behaves like fBM with $H \approx 0.1$ has a direct forecasting payoff. If you know the process is fBM, the optimal linear forecast is known in closed form. This gives a volatility forecasting model called RFSV (Rough Fractional Stochastic Volatility).

The intuition is simple. For fBM, the best forecast of the future value is a weighted average of all past observations, where the weights are determined by H. When H is small (rough regime), the weights decay slowly, meaning distant past observations still carry information. This slow weight decay is what produces the long-memory behavior that HAR approximates with three components.

Conditional Expectation for Gaussian Processes

If $(X_1, \ldots, X_T, X_{T+1})$ is jointly Gaussian, the conditional expectation $\mathbb{E}[X_{T+1} \mid X_1, \ldots, X_T]$ is a linear combination of the past values. The weights are determined by the covariance structure of the process. For fBM, the covariance function is known analytically, so the optimal weights can be computed exactly.

RFSV Forecast

Model $\log \mathrm{RV}_t$ as fractional Brownian motion with Hurst exponent H and a constant mean μ:

$$\log \mathrm{RV}_t = \mu + \sigma_v \, B_t^H \tag{7.3}$$

- $\log \mathrm{RV}_t$: the natural logarithm of day t's realized variance
- μ: the unconditional mean of log-volatility
- σ_v: the volatility-of-volatility (a scaling constant)
- B_t^H: fractional Brownian motion with exponent H

The optimal one-step-ahead forecast, given observations through day T, is:

$$\widehat{\log \mathrm{RV}}_{T+1} = \sum_{k=0}^{T-1} w_k \, \log \mathrm{RV}_{T-k} \tag{7.4}$$

- w_k: the weight on the observation k days in the past
- The weights $\{w_k\}$ are determined by the covariance structure of fBM (i.e., by H)
- They sum to 1 and decay as a power law: $w_k \propto k^{H-3/2}$ for large k
- When $H = 0.1$, the decay is $k^{-1.4}$, which is slow enough that observations from weeks ago still contribute meaningfully

One Parameter Does the Work of Three

RFSV achieves HAR-level forecasting accuracy with essentially one free parameter (H), because the Hurst exponent $H \approx 0.1$ encodes the entire autocorrelation structure. HAR uses three coefficients (β_d, β_w, β_m) to approximate the same slow decay. RFSV derives the weights from first principles given H. The fact that both approaches perform similarly is evidence that the slow power-law decay really is the dominant structure in volatility dynamics.

Why RFSV Weights Decay Slowly

Consider two forecasting extremes. If $H = 0.5$ (standard BM), increments are independent and only the most recent observation matters; the weights drop to zero immediately. If $H = 0.01$ (extremely rough), the process is so anti-persistent that every past reversal carries information about the current level; the weights decay very slowly. At $H = 0.1$, you are close to the second extreme. The weight on an observation from 22 days ago (one month) is still roughly 15–20% of the weight on yesterday's observation. This is why HAR's monthly component ($\text{RV}^{(m)}$) carries a large coefficient in Chapter 6: it is picking up the long tail of the RFSV weight function.

Why This Matters

RFSV is both a benchmark and a diagnostic for your vol forecasting project. As a benchmark, it tells you what a single-parameter model can achieve: if your ML model (LSTM, transformer) cannot beat RFSV out of sample, the added complexity is not justified. As a diagnostic, the RFSV weight function shows you exactly which lags carry information. If your ML model's learned attention pattern or feature importances do not roughly match the $k^{-1.4}$ decay shape, either the model has found genuinely new structure or it is overfitting.

Worked Example:

Comparing RFSV Weights to HAR Structure Consider a simplified RFSV with $H = 0.1$ and 22 past observations. The weights decay approximately as $w_k \propto k^{-1.4}$. Normalizing so they sum to 1:

Lag group	Sum of RFSV weights	HAR analog
Lag 1 (yesterday)	≈ 0.35	$\beta_d \approx 0.36$
Lags 2–5 (rest of week)	≈ 0.30	Embedded in $\beta_w \approx 0.28$
Lags 6–22 (rest of month)	≈ 0.35	Embedded in $\beta_m \approx 0.28$

The RFSV weight distribution across lag groups is strikingly similar to the HAR coefficient values from Corsi (2009). HAR is an efficient three-bin approximation of the smooth RFSV weight function.

7.5 Rough Volatility for Pricing

This guide focuses on realized volatility estimation and forecasting, not on derivatives pricing. But the rough volatility framework has had its greatest quantitative-finance impact in the pricing domain, so a brief mention is warranted for completeness.

The Volatility Smile Problem

Standard stochastic volatility models (e.g., Heston) can generate "smiles" in implied volatility across strike prices. But they struggle to simultaneously fit two empirical patterns: (1) the steep short-maturity smile in S&P 500 options and (2) the term structure of the VIX implied volatility (the "VVIX" smile). Fitting both at once has been a long-standing challenge in quantitative finance.

Bayer et al. (2016) introduced the *rough Bergomi* model, the first pricing model built on rough volatility. In the rough Bergomi model, the variance process is driven by fractional Brownian motion with $H \approx 0.07$, rather than the standard Brownian motion used in models like Heston.

Two key results from the pricing literature:

1. **Rough Bergomi** (Bayer et al., 2016): the short-maturity behavior of the implied volatility smile is controlled by the Hurst exponent H. With $H \approx 0.07$, the model generates steep short-dated smiles that match market data far better than Heston. Simulation is required for pricing (no closed-form solution), but the model is parsimonious.
2. **Quadratic rough Heston**: a tractable variant that jointly fits SPX option smiles and VIX option smiles, a combination that no standard model can achieve. The roughness parameter is again $H \approx 0.05$–0.1.

Forecasting vs. Pricing: Same H, Different Models

The forecasting model (RFSV) and the pricing models (rough Bergomi, quadratic rough Heston) both use $H \approx 0.1$ but serve different purposes. RFSV forecasts RV_{t+1} from past RV values. The pricing models compute option prices under risk-neutral dynamics. This chapter is about forecasting; the pricing models are mentioned here because they provide independent confirmation that the roughness parameter $H \approx 0.1$ is relevant across different uses of volatility modeling.

7.6 Fact or Artefact?

The rough volatility paradigm rests on an empirical observation: $\log \mathrm{RV}_t$ has $H \approx 0.1$. But RV_t is not the true spot volatility σ_t^2. It is a noisy estimate of it, contaminated by the microstructure noise discussed in Chapter 3 (Section 2.3). Could the apparent roughness be an artefact of this noise?

Cont and Das (2024) argue that the answer is yes, at least in part.

Cont and Das (2024): Roughness as a Noise Artefact

Observed roughness of realized volatility estimates is partly a microstructure-noise artefact. When i.i.d. noise contaminates the high-frequency returns used to compute RV, the resulting RV series appears rougher than the true spot volatility process. Even if the true σ_t^2 follows a standard semimartingale with $H = 0.5$, the estimated $\log \mathrm{RV}_t$ can exhibit $H \approx 0.1$ due to the noise channel.

The mechanism is intuitive and illustrated in Figure 7.4.

The Cont and Das (2024) argument proceeds in three steps:

1. **Noise adds independent errors.** Each day's RV_t differs from the true IV_t by an estimation error η_t (Chapter 2, Equation 2.6). These errors are approximately independent across days.

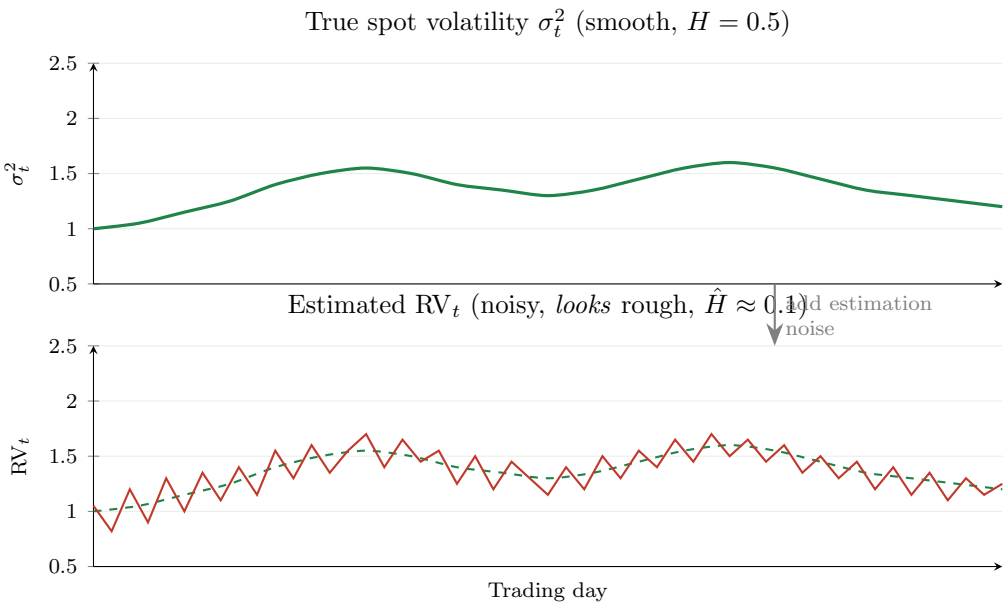


Figure 7.4: How microstructure noise creates apparent roughness. Top: the true spot volatility σ_t^2 is a smooth process ($H = 0.5$). Bottom: the estimated RV_t (red) adds i.i.d. estimation noise around the true level (dashed green). The noisy RV_t series looks rough (anti-persistent, rapidly oscillating) even though the underlying process is smooth. Applying the variogram estimator to the red series yields $\hat{H} \approx 0.1$, but this reflects noise contamination, not true roughness of σ_t^2.

2. **Independent errors create anti-persistence.** If you add i.i.d. noise to a smooth signal, the increments of the noisy series are negatively autocorrelated. An unusually high noise draw today will be followed (on average) by a smaller one tomorrow, creating artificial "reversals" in the series.
3. **Anti-persistence lowers $\hat{H}$.** The variogram estimator interprets these reversals as roughness and produces $\hat{H} \ll 0.5$.

Do Not Equate $\hat{H} = 0.1$ with True Roughness

The observed $H \approx 0.1$ from $\log \mathrm{RV}_t$ data does not establish that the true spot volatility process σ_t^2 is rough. The noise channel documented by Cont and Das (2024) is a plausible alternative explanation. The truth may lie in between: some genuine roughness in σ_t^2, amplified by estimation noise in RV_t. For forecasting, this distinction does not matter much (see Section 7.7). For model calibration and theoretical claims about the nature of volatility, it matters a great deal.

The Coin-Flip Analogy

Suppose you flip a fair coin 100 times and record the running total of heads minus tails. The resulting path is a standard random walk ($H = 0.5$). Now suppose someone records your totals but makes small random errors (e.g., occasionally miscounting by ± 1). If you estimate H from their error-contaminated records, you will get $\hat{H} < 0.5$, because the errors introduce artificial reversals. The path *looks* rougher than it is. Cont–Das argue that RV_t is the error-contaminated record and σ_t^2 is the true total.

7.7 The Universal LSTM Connection

Rosenbaum and Zhang (2022) train a single LSTM (Long Short-Term Memory neural network) on hundreds of individual stocks simultaneously. The trained network, which they call the "universal LSTM," matches the forecasting performance of the RFSV model and the Quadratic Rough Heston parametric forecaster.

LSTM (Brief)

An LSTM is a type of recurrent neural network designed to learn sequential patterns. It processes a time series one step at a time, maintaining an internal "memory cell" that selectively remembers and forgets past information. LSTMs are widely used for time series forecasting. Chapter 13 covers LSTMs in detail.

This convergence is suggestive. Two very different approaches (a parametric fBM model with one parameter and a nonparametric neural network with thousands of parameters) arrive at the same forecast. The natural interpretation is that both are learning the same underlying statistical regularity in RV data.

Why This Matters

The universal LSTM result sets a clear success criterion for your project. If your deep learning model converges to the same forecast as RFSV (a one-parameter model), you have learned the rough-vol kernel but nothing beyond it, and RFSV wins on parsimony. To justify an LSTM or transformer, you need to demonstrate statistically significant improvement via Diebold-Mariano tests, not just lower in-sample QLIKE. The universality property also suggests that training on a broad cross-section of assets (rather than a single stock) may improve generalization.

Practical Equivalence Despite Theoretical Ambiguity

Whether spot volatility is truly rough or the roughness is an artefact of estimation noise (Section 7.6), both RFSV and universal LSTM exploit the same statistical regularity in realized volatility estimates. The practical forecasting value is the same either way. The debate about the nature of the spot process is theoretically important but does not affect your forecast accuracy.

Rosenbaum and Zhang (2022) also document a "universality" property: the LSTM trained on a broad cross-section of stocks transfers well to assets it has never seen, including out-of-sample asset classes. This is consistent with the cross-asset universality of $H \approx 0.1$ documented by Bennedsen et al. (2022).

Rosenbaum and Zhang (2022): Universal LSTM Matches RFSV

A single LSTM trained on hundreds of stocks achieves the same forecasting performance as the RFSV parametric model. Both produce forecasts that are competitive with HAR. The universal LSTM also transfers to out-of-sample assets, suggesting that the learned kernel is asset-independent, just as the Hurst exponent $H \approx 0.1$ is.

The connection to the rest of this guide is direct. Chapter 13 develops LSTM and transformer-based forecasting models in full detail. When you reach that chapter, the key question will be: does your deep learning model learn something *beyond* the rough-vol kernel, or is it just a complicated way to recover the same $H \approx 0.1$ decay structure? If the latter, RFSV with one parameter is preferable on parsimony grounds. If the former, you need to demonstrate the

improvement with a proper out-of-sample test (Chapter 17).

Summary

- The **Hurst exponent** H of a fractional Brownian motion controls path roughness. $H = 0.5$ is standard Brownian motion; $H < 0.5$ is rougher (more jagged); $H > 0.5$ is smoother.
- **Fractional Brownian motion** (fBM) generalizes standard BM by allowing correlated increments: negatively correlated when $H < 0.5$ (anti-persistent), positively correlated when $H > 0.5$ (persistent).
- Gatheral et al. (2018) showed that $\log \mathrm{RV}_t$ behaves like fBM with $H \approx 0.1$ across equity indices, individual stocks, and FX. This is the "volatility is rough" result.
- H can be estimated from data using the **variogram method**: regress $\log m(q, h)$ on $\log h$ and divide the slope by q.
- Bennedsen et al. (2022) confirmed the **cross-asset universality** of $H \approx 0.1$ across equities, FX, fixed income, and commodities.
- The **RFSV model** forecasts $\log \mathrm{RV}_{t+1}$ as a weighted average of past $\log \mathrm{RV}$ values, with weights derived from the fBM covariance structure. It achieves HAR-level accuracy with essentially one parameter (H).
- RFSV weights decay as $k^{H-3/2}$. At $H = 0.1$, the decay is slow ($k^{-1.4}$), explaining why HAR's monthly component carries a large coefficient.
- **Rough Bergomi** (Bayer et al., 2016) and quadratic rough Heston use $H \approx 0.07$–0.1 for option pricing. They fit short-maturity smiles and VIX smiles better than standard models. This guide focuses on forecasting, not pricing.
- Cont and Das (2024) argue that observed roughness ($H \approx 0.1$) is **partly a microstructure-noise artefact**. Estimation noise in RV_t creates artificial anti-persistence that the variogram interprets as roughness.
- The practical implication: $H \approx 0.1$ from $\log \mathrm{RV}_t$ is a useful forecasting input but does not prove that the true spot volatility σ_t^2 is rough.
- Rosenbaum and Zhang (2022) show that a **universal LSTM** trained on hundreds of stocks matches RFSV and transfers to unseen assets. Both approaches likely learn the same statistical kernel.
- Whether spot vol is truly rough or the roughness is a noise artefact, the **forecasting value is identical**. The distinction matters for pricing-model calibration and theory, not for forecasting accuracy.
- RFSV connects to FIGARCH (Chapter 5, Section 5.6) and HAR (Chapter 6) as three different parameterizations of the same long-memory phenomenon. It connects forward to deep learning (Chapter 13) as a benchmark that any LSTM must beat to justify its complexity.

Key Results

Result	Source	Finding
Volatility is rough	Gatheral et al. (2018)	$\log \mathrm{RV}_t$ behaves like fBM with $H \approx 0.1$ across equity indices, stocks, and FX; far below $H = 0.5$ of standard models
Cross-asset universality	Bennedsen et al. (2022)	$H \approx 0.1$ is universal across asset classes, geographies, and time periods
Rough Bergomi pricing	Bayer et al. (2016)	First rough-vol pricing model; $H \approx 0.07$ generates steep short-maturity implied volatility smiles
Roughness as artefact	Cont and Das (2024)	Microstructure noise in RV estimates creates apparent roughness; $\hat{H} \approx 0.1$ may not reflect true spot-vol dynamics
Universal LSTM	Rosenbaum and Zhang (2022)	Single LSTM trained on hundreds of stocks matches RFSV; both learn the same universal forecasting kernel

Part III

The Volatility Surface and Options-Implied Information

Chapter 8

Options Basics and the Volatility Surface

Why This Chapter Matters

Options-implied information is a rich source of features for volatility forecasting (Chapter 10). The variance risk premium (Chapter 9) is defined as the gap between implied and realized volatility, both of which you need this chapter to understand. Project 5 (VRP ML trader) directly trades that gap. Even for non-options projects, understanding the volatility surface is necessary because VIX and IV-derived features appear in virtually every competitive feature set for realized volatility forecasting (Gu et al., 2020a).

This chapter introduces options from scratch, builds up to the Black-Scholes formula (just enough to define implied volatility), and then shows how the entire volatility surface encodes the market's fears and expectations. You will finish with the model-free VIX, which connects options prices directly to expected future variance.

8.1 What Is an Option?

This section defines what an option contract is and what it pays at expiry.

Background for This Chapter

You need comfort with:

- Expected values and probability distributions (any intro probability course).
- The standard normal CDF, $\Phi(z) = P(Z \leq z)$ for $Z \sim \mathcal{N}(0,1)$.
- Logarithms, exponentials, and basic calculus (derivatives, integrals).
- The concept of variance and standard deviation from Chapter 1.

No prior knowledge of options, derivatives, or financial markets is assumed.

Option Contract

An **option** is a contract that gives the holder the *right, but not the obligation*, to buy or sell an **underlying asset** (the stock, index, or commodity the option is written on) at a pre-agreed price. Two parameters define any option:

- **Strike price** K: the pre-agreed transaction price.
- **Expiry date** T: the deadline by which the right must be exercised.

There are two types:

- A **call option** gives the right to *buy* the underlying at price K by time T.
- A **put option** gives the right to *sell* the underlying at price K by time T.

The buyer pays an upfront price (the **premium**) for this right. The seller ("writer") collects the premium but takes on the obligation.

8.1.1 Payoff at Expiry

At expiry, the option is worth exactly what you would gain by exercising it (or zero if exercising would lose money). Let S_T denote the price of the underlying asset at expiry.

Option Payoffs at Expiry

$$\text{Call payoff} = \max(S_T - K,\ 0) \tag{8.1}$$
$$\text{Put payoff} = \max(K - S_T,\ 0) \tag{8.2}$$

where:

- S_T = price of the underlying asset at expiry.
- K = strike price (pre-agreed transaction price).
- The $\max(\cdot, 0)$ reflects that you never exercise at a loss; you simply let the option expire.

Why "Right but Not Obligation" Matters

Suppose you hold a call option with strike $K = 100$. If $S_T = 120$, you exercise: buy at 100, the asset is worth 120, you gain 20. If $S_T = 80$, you do nothing: buying at 100 when the market price is 80 would be foolish. Your payoff is never negative. This asymmetry is the entire reason options have value, and why someone must be paid a premium to write them.

Why This Matters

The asymmetric payoff structure is what makes options sensitive to volatility in the first place. A stock's P&L is symmetric around its expected return, so only direction matters. An option's P&L is asymmetric, so both the direction *and magnitude* of moves matter, which is why option prices encode information about the distribution of future returns, not just the mean. This is the foundation for extracting implied volatility and the variance risk premium (Chapter 9) from option prices.

Figure 8.1 shows the payoff profiles. The kink at K is the signature of an option; linear instruments (stocks, futures) have straight-line payoffs.

8.1.2 Moneyness

Traders classify options by how the current spot price S compares to the strike K:

Moneyness

For a **call** option:

- **In-the-money (ITM)**: $S > K$ (exercising now would be profitable).
- **At-the-money (ATM)**: $S \approx K$.
- **Out-of-the-money (OTM)**: $S < K$ (exercising now would not be profitable).

For a **put**, the directions reverse: ITM when $S < K$, OTM when $S > K$. A common

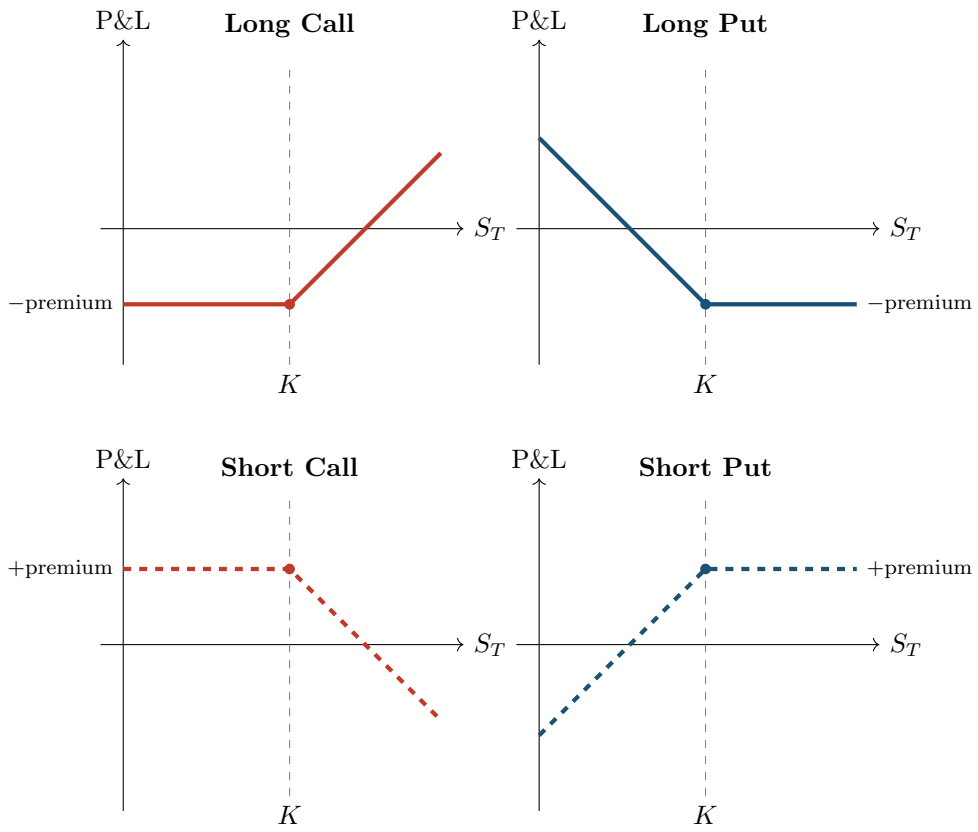


Figure 8.1: Option payoff diagrams (profit/loss including the premium paid or received). Long positions (top row) have limited downside and potentially large upside. Short positions (bottom row) are the mirror image: limited upside, potentially large downside. The kink at the strike K is the defining feature of an option.

quantitative measure of moneyness is $m = K/S$ (or its log, $\ln(K/S)$).

8.2 Black-Scholes in One Page

Now that you know what options pay, the natural question is: what should you pay *today* for an option that expires in the future? This section presents the answer that Black and Scholes (1973) derived, which earned a Nobel Prize and remains the bedrock of options pricing.

The core idea: an option's value today equals the expected payoff at expiry, discounted to the present, under a probability measure that accounts for risk. Black and Scholes (1973) showed that under a specific set of assumptions, this expected value has a closed-form solution.

The Black-Scholes Assumptions

The formula assumes:

1. The underlying price follows geometric Brownian motion (log-normal returns).
2. Volatility σ is **constant** over the life of the option.
3. There are no jumps in the price.
4. Trading is continuous and frictionless (no transaction costs, unlimited short selling).
5. The risk-free interest rate r is constant and known.
6. No dividends.

Every single one of these assumptions is wrong in practice. This matters, and we will exploit it.

The Black-Scholes Formula for a European Call

[a] A **European** option can only be exercised at expiry, not before. An **American** option can be exercised at any time up to expiry. For the purpose of defining implied volatility, the European version is standard.

$$C = S\,\Phi(d_1) - Ke^{-rT}\Phi(d_2) \tag{8.3}$$

where

$$d_1 = \frac{\ln(S/K) + (r + \sigma^2/2)\,T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T} \tag{8.4}$$

and:

- C = price of the call option today.
- S = current spot price of the underlying asset.
- K = strike price.
- r = continuously compounded risk-free interest rate (annualized).
- T = time to expiry in years (e.g., 3 months = 0.25).
- σ = annualized volatility of the underlying's log returns.
- $\Phi(\cdot)$ = CDF of the standard normal distribution.
- d_1 = a standardized measure of how far in-the-money the option is, adjusted for drift and time.
- $d_2 = d_1$ minus a volatility-time adjustment; $\Phi(d_2)$ is the risk-neutral probability of exercise.

Reading the Formula

Break Equation (8.3) into two pieces:

- $S\,\Phi(d_1)$: what you expect to receive (the asset value, weighted by the probability-adjusted chance it ends up above K).

- $Ke^{-rT}\Phi(d_2)$: what you expect to pay (the strike, discounted to today, weighted by the risk-neutral probability of actually exercising).

The call price is the difference: expected receipt minus expected cost.

Worked Example:

Black-Scholes Call Price Suppose the S&P 500 is at $S = 100$, and you want to price a 3-month call with strike $K = 105$. The risk-free rate is $r = 5\%$ per year and volatility is $\sigma = 20\%$ per year.

Step 1: Compute d_1 and d_2.

$$d_1 = \frac{\ln(100/105) + (0.05 + 0.04/2)(0.25)}{0.20\sqrt{0.25}} = \frac{-0.04879 + 0.0175}{0.10} = \frac{-0.03129}{0.10} = -0.3129$$

$$d_2 = -0.3129 - 0.20\sqrt{0.25} = -0.3129 - 0.10 = -0.4129$$

Step 2: Look up the normal CDF values.

$$\Phi(-0.3129) = 0.3772, \qquad \Phi(-0.4129) = 0.3398$$

Step 3: Plug into the formula.

$$\begin{aligned} C &= 100 \times 0.3772 - 105\, e^{-0.05\times 0.25} \times 0.3398 \\ &= 37.72 - 105 \times 0.9876 \times 0.3398 \\ &= 37.72 - 35.25 \\ &= \$2.47 \end{aligned}$$

The call is out-of-the-money ($S < K$), so the premium is modest: \$2.47 on a \$100 underlying. You are paying for the *possibility* that the index rises above 105 within three months.

Why This Matters

Black-Scholes tells you the fair price of an option given a *fixed* volatility assumption, but volatility is not fixed. That disconnect is the entire point of the volatility surface and of your forecasting project. The formula's single most important role for you is as the *inverse function* that maps observed option prices to implied volatilities (Section 8.3), which then become features and benchmarks for your ML model.

Black-Scholes Is Wrong, but Universal

Black-Scholes is wrong in nearly all its assumptions. Returns have fat tails (Chapter 1), volatility is not constant (Chapters 5–7), and prices jump (Chapter 4). But the formula remains the market's common language for quoting option prices. When a trader says "this option is trading at 25 vol," they mean the Black-Scholes implied volatility is 25%. The formula is not a belief about reality; it is a translation device.

8.2.1 The Greeks: Sensitivity to Inputs

The Black-Scholes formula expresses the option price as a function of five inputs. The partial derivatives of the price with respect to each input are called the **Greeks**, and they tell you how sensitive the option's value is to small changes in market conditions.

Key Greeks

For a European call option priced by Black-Scholes:

- **Delta** ($\Delta = \partial C/\partial S$): sensitivity to the underlying price. A delta of 0.60 means the call gains roughly \$0.60 for each \$1 rise in the stock.
- **Gamma** ($\Gamma = \partial^2 C/\partial S^2$): rate of change of delta. High gamma means delta shifts rapidly with the stock price, making hedging more difficult.
- **Theta** ($\Theta = \partial C/\partial T$): time decay. Options lose value as expiry approaches (all else equal), and theta quantifies that bleed.
- **Vega** ($\mathcal{V} = \partial C/\partial \sigma$): sensitivity to volatility. This is the Greek that matters most for your project.
- **Rho** ($\rho = \partial C/\partial r$): sensitivity to the risk-free rate. Usually small for short-dated options.

In Plain English

The Greeks answer five practical questions: how much does my option gain or lose if (1) the stock moves, (2) the stock moves *fast*, (3) a day passes, (4) volatility changes, or (5) interest rates change? Vega is the most important Greek for volatility trading because it tells you how many dollars you make or lose per percentage-point change in implied vol. A high-vega position is a direct bet on volatility; a zero-vega portfolio is insulated from vol changes.

Figure 8.2 shows how vega varies with moneyness and time to expiry.

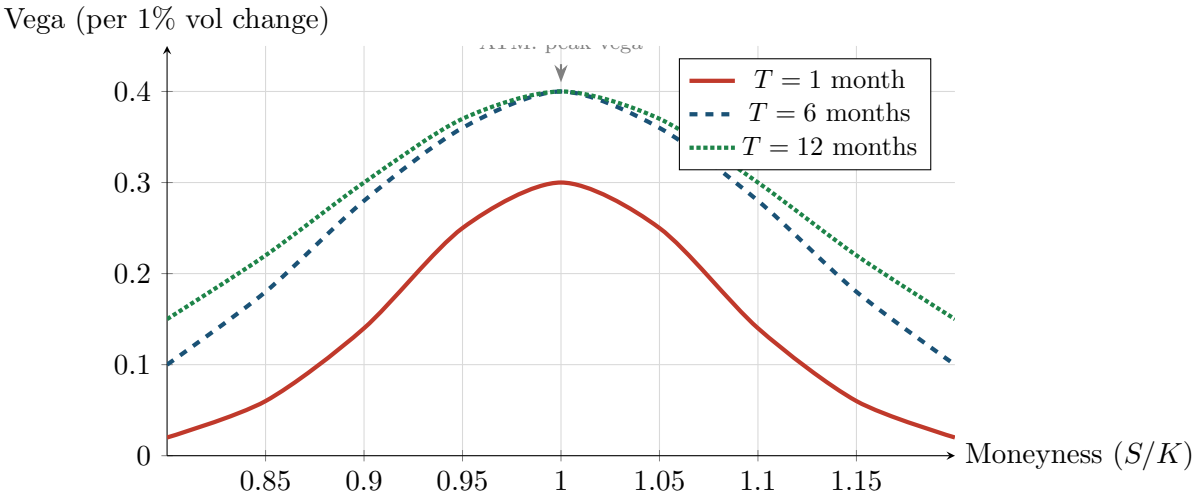


Figure 8.2: Vega profiles across moneyness for different maturities. Vega peaks at-the-money and falls off for deep ITM or OTM options. Longer-dated options have higher vega (more time for vol to matter) and a wider peak (more strikes are sensitive to vol changes). Short-dated ATM options have concentrated vega, making them efficient instruments for short-horizon vol trades.

Why This Matters

Vega tells you how sensitive option prices are to volatility changes, which is critical when trading vol forecasts. If your ML model predicts that realized vol will be higher than implied vol, you want to buy options with high vega to maximize your dollar exposure to that view. Understanding the vega profile also explains why ATM options are the most

liquid instruments for vol trading and why the variance swap (Section 8.8) is designed to provide constant dollar exposure to variance regardless of the stock price.

8.3 Implied Volatility

With Black-Scholes in hand, we now reverse-engineer it to extract the market's view of volatility.

Every input to the Black-Scholes formula (S, K, r, T) is directly observable from market data, except one: σ. If you also observe the market price of the option C_{mkt}, you can back out the volatility the market is implicitly using.

Implied Volatility

The **implied volatility** σ_{imp} is the value of σ that makes the Black-Scholes formula match the observed market price:

$$C_{\text{mkt}} = C_{\text{BS}}(S, K, r, T, \sigma_{\text{imp}}) \tag{8.5}$$

where:

- C_{mkt} = the option price observed in the market.
- $C_{\text{BS}}(\cdot)$ = the Black-Scholes pricing function (Equation 8.3).
- σ_{imp} = the implied volatility (the unknown being solved for).

There is no closed-form solution for σ_{imp}; it must be found numerically (e.g., Newton-Raphson or bisection).

The "Wrong Number in the Wrong Formula"

Implied volatility is famously described as "the wrong number to put in the wrong formula to get the right price" (Rebonato, 2004). It is *not* a forecast of future volatility. It is the market's consensus price of uncertainty, contaminated by:

- **Risk premia**: sellers of options demand compensation for bearing tail risk, inflating IV above expected realized vol.
- **Supply and demand**: heavy demand for downside protection (put buying) pushes up IV for low-strike options.
- **Model error**: since Black-Scholes is misspecified, IV absorbs all the model's errors into a single number.

Despite all this, IV is extremely useful. It is a real-time, forward-looking, market-priced measure of uncertainty.

Why This Matters

Implied volatility is the market's consensus forecast of future vol. Comparing it to your RV forecast gives you the **variance risk premium**: VRP $\approx \text{IV}^2 - \widehat{\text{RV}}^2$. If your ML model produces a more accurate RV forecast than the market's implied estimate, the VRP becomes a tradeable signal (Chapter 9). IV is also one of the strongest single features for predicting future realized vol (Gu et al., 2020a), so it will appear directly in your feature set (Chapter 10).

Worked Example:

Backing Out Implied Volatility Continuing the example from Section 8.2: $S = 100$, $K = 105$, $r = 5\%$, $T = 0.25$. Suppose the market price of the call is $C_{\text{mkt}} = \$3.10$ (higher than the \$2.47 we computed with $\sigma = 20\%$).
Since the market price exceeds our 20%-vol price, the market is pricing in higher volatility. Using numerical root-finding on $C_{\text{BS}}(S, K, r, T, \sigma) = 3.10$, you would find $\sigma_{\text{imp}} \approx 23.5\%$. The market is telling you: the uncertainty priced into this option corresponds to roughly 23.5% annualized volatility, not the 20% we assumed.

A critical point: implied volatility is *not* a single number for a given underlying. Every (strike, maturity) pair has its own IV. This leads directly to the next section.

8.4 The Implied Volatility Surface

The implied volatility extracted from options depends on which option you look at, giving rise to a two-dimensional surface.

Implied Volatility Surface

The **implied volatility surface** is the function

$$\sigma_{\text{imp}}(K, T) \tag{8.6}$$

mapping each (strike, maturity) pair to the implied volatility extracted from the corresponding option price. In practice, moneyness $m = K/S$ (or $\ln(K/S)$) replaces the raw strike to make the surface comparable across different underlying price levels.

- K (or m) = strike price (or moneyness).
- T = time to expiry.
- σ_{imp} = implied volatility at that (strike, maturity) point.

What a Flat Surface Would Mean

If Black-Scholes were correct (constant volatility, log-normal returns, no jumps), every option on the same underlying would produce the *same* implied volatility regardless of strike or maturity. The surface would be perfectly flat. The fact that the surface is *not* flat is direct evidence that Black-Scholes is wrong, and the shape of the surface tells you *how* it is wrong.

Why This Matters

The volatility surface encodes information about jump risk, tail risk, and market fear that goes beyond what backward-looking realized volatility captures. Changes in the surface's shape (steepening skew, inverting term structure) are forward-looking signals that could serve as features for your ML model, potentially improving on a pure HAR baseline that only uses past RV.

8.4.1 The Volatility Smile and Skew

The cross-section of IV at a fixed maturity reveals two patterns, shown in Figure 8.3.

The skew (red curve): for equity index options, IV is much higher for low strikes ($K/S < 1$, OTM puts) than for high strikes ($K/S > 1$, OTM calls). This pattern emerged after the

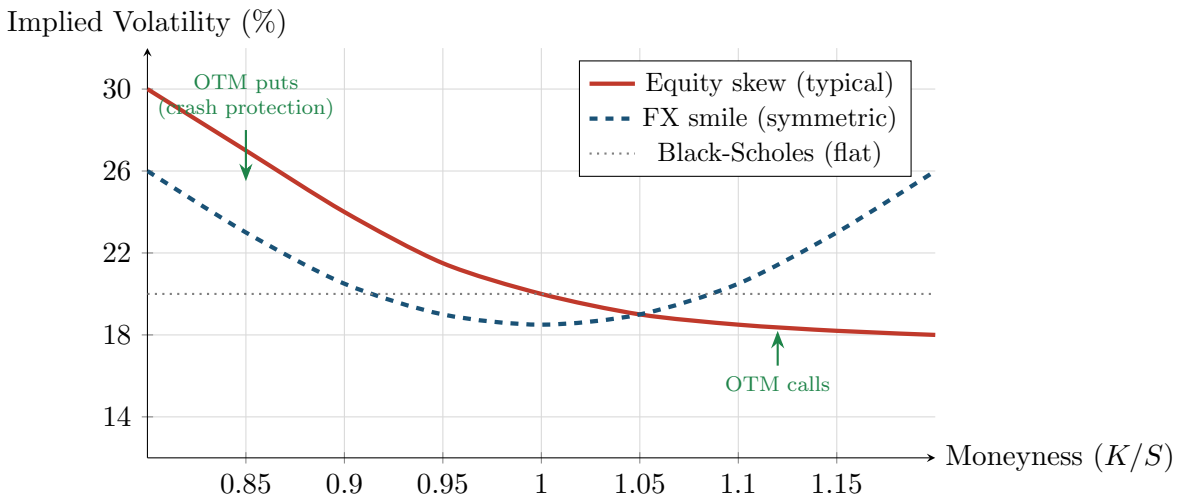


Figure 8.3: Implied volatility vs. moneyness at a fixed maturity. The red solid line shows the typical equity skew: IV increases sharply for low strikes (OTM puts) due to crash-risk demand. The blue dashed line shows a symmetric "smile" common in FX markets, where both tails carry extra uncertainty. The gray dotted line is the flat surface that Black-Scholes would predict.

1987 crash and has persisted ever since. It reflects the market's pricing of left-tail (crash) risk: investors pay a premium for downside protection.

The smile (blue curve): for foreign exchange options, IV is elevated for both deep OTM puts and deep OTM calls, creating a U-shape. This reflects the market's view that large moves in either direction are more likely than the log-normal model assumes (fat tails on both sides).

Both patterns are inconsistent with constant-volatility Black-Scholes, which would produce the flat gray line.

8.4.2 The Term Structure of Implied Volatility

The second dimension is maturity. At-the-money IV is not the same at 1 month as at 1 year.

IV Term Structure

- **Normal conditions**: the term structure slopes upward (long-dated IV > short-dated IV), reflecting the greater uncertainty over longer horizons and mean-reversion in volatility.
- **Crisis periods**: the term structure inverts (short-dated IV > long-dated IV), because near-term fear spikes while the market expects volatility to eventually normalize.
- Short-dated IV is more volatile than long-dated IV, consistent with the mean-reverting nature of volatility established in Chapters 5 and 6.

Figure 8.4 contrasts these two regimes.

8.4.3 The Butterfly Spread

The skew tells you about the *asymmetry* of the market's fear – how much more expensive downside protection is relative to upside. But it says nothing about whether *both* tails are being bid up simultaneously. The **butterfly spread** fills that gap.

When portfolio managers buy both OTM puts (crash protection) and OTM calls (upside par-

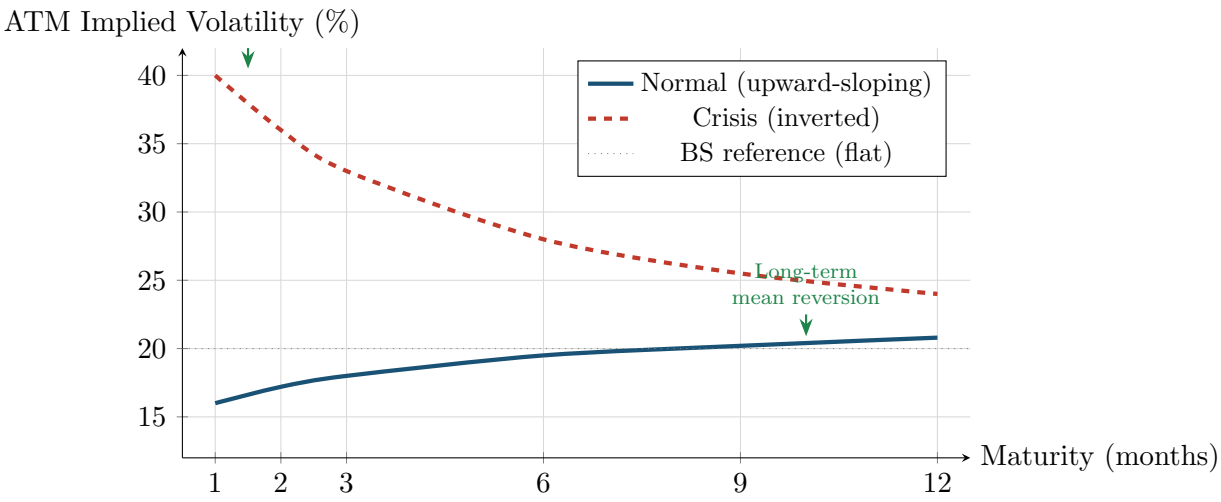


Figure 8.4: ATM implied volatility term structure under two regimes. In normal conditions (blue), the term structure slopes upward as longer horizons carry greater uncertainty. During crises (red dashed), the term structure inverts: near-term IV spikes while long-term IV reflects the expectation that volatility will eventually normalize. The spread between short-dated and long-dated ATM IV is itself a useful feature for forecasting models.

ticipation or short-squeeze hedging) at the same time, IV rises on both wings relative to ATM. This symmetric fattening of the tails is invisible to a skew measure, which only captures the *difference* between the two wings. The butterfly captures the *average* elevation of both wings above the center.

The butterfly spread is constructed from three implied volatilities at a fixed maturity: the 25-delta put, the 25-delta call, and the ATM (50-delta) volatility.

Butterfly Spread

The **butterfly spread** (or **butterfly**) is defined as:

$$\mathrm{BF}_t = \frac{1}{2}\left(\sigma_{25\Delta P} + \sigma_{25\Delta C}\right) - \sigma_{\mathrm{ATM}} \tag{8.7}$$

where:

- $\sigma_{25\Delta P}$ = implied volatility of the 25-delta put (an OTM put whose Black-Scholes delta is -0.25).
- $\sigma_{25\Delta C}$ = implied volatility of the 25-delta call (an OTM call whose Black-Scholes delta is $+0.25$).
- σ_{ATM} = at-the-money implied volatility (50-delta).
- BF_t = the butterfly spread on day t, measured in volatility points.

The butterfly is always non-negative in a no-arbitrage market (a negative value would imply the smile is concave, which violates convexity constraints on the implied volatility curve).

In Plain English

The butterfly measures how much higher the average wing volatility is compared to the center of the smile. Think of it as the "curvature" or "U-shape" of the smile at a fixed maturity. A large butterfly means traders are paying up for protection on *both* sides of the distribution – they expect fat tails, regardless of direction. A small butterfly means

the smile is relatively flat and the market sees tail risk as modest.

To see why the butterfly and skew are complementary rather than redundant, note that they decompose the two wing volatilities into orthogonal components. The **risk reversal** (skew) is the difference between the wings:

$$\mathrm{RR}_t = \sigma_{25\Delta C} - \sigma_{25\Delta P}$$

while the butterfly is their average elevation above ATM. Together, the two reconstruct the full wing structure:

$$\sigma_{25\Delta P} = \sigma_{\mathrm{ATM}} + \mathrm{BF}_t - \tfrac{1}{2}\,\mathrm{RR}_t \tag{8.8}$$

$$\sigma_{25\Delta C} = \sigma_{\mathrm{ATM}} + \mathrm{BF}_t + \tfrac{1}{2}\,\mathrm{RR}_t \tag{8.9}$$

The risk reversal captures **directional asymmetry** (skewness demand); the butterfly captures **symmetric tail thickness** (kurtosis demand). One measures the tilt of the smile; the other measures its curvature.

Skew vs. Butterfly: What Each Captures

- **Risk reversal (skew)**: dominated by demand for downside protection relative to upside. In equities, the risk reversal is persistently negative (OTM puts are more expensive than OTM calls). It tracks *skewness* in the risk-neutral distribution.
- **Butterfly**: dominated by demand for *both* tails simultaneously. It tracks *excess kurtosis* in the risk-neutral distribution – the market's pricing of large moves in either direction beyond what ATM vol implies.

You can have a steep skew with a low butterfly (the left tail is fat, but the right tail is not), or a mild skew with a high butterfly (both tails are fat, roughly equally). The two dimensions are largely independent.

Crisis Behavior

During crises, the butterfly typically spikes alongside VIX, but with a distinctive pattern. Portfolio insurance programs bid up OTM puts (the dominant effect in the skew), and at the same time, short-covering and tail-risk hedging bid up OTM calls. When *both* wings are elevated, the butterfly rises sharply.

A rising butterfly with a steepening skew is the signature of a broad-based panic: the market is pricing extreme moves in both directions, not just a directional crash. In contrast, a rising skew with a *stable* butterfly signals targeted downside hedging without generalized tail fear.

This distinction matters because realized volatility behaves differently in the two regimes. Symmetric tail pricing (high butterfly) tends to precede periods of elevated but two-sided choppiness, while pure skew spikes often precede sharp, directional sell-offs that resolve more quickly.

Why This Matters

The butterfly spread is one of nine options-implied features in the project's feature pipeline (Chapter 10). It provides information that the risk reversal (skew) cannot: whether the market prices *symmetric* tail risk, not just directional fear. Empirically, the butterfly is most useful as a crisis-detection signal. Spikes in the butterfly predict periods of elevated realized volatility at weekly and monthly horizons, especially when combined nonlinearly with VIX and the term structure slope in tree-based models. Because the butterfly captures kurtosis demand rather than directional skewness, it adds orthogonal information

to the risk reversal and helps the model distinguish between one-sided drawdowns and regime shifts into sustained high-volatility environments.

8.4.4 The Full Surface

Figure 8.5 combines both dimensions into a three-dimensional view.

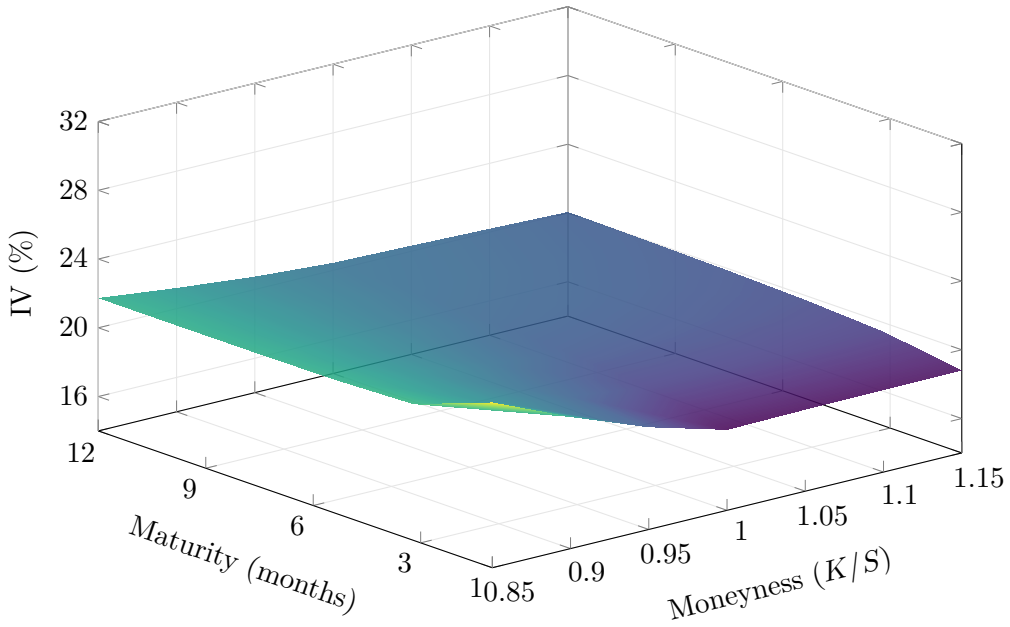


Figure 8.5: The implied volatility surface for a typical equity index. Left side (low moneyness, OTM puts): IV is elevated, especially at short maturities, reflecting concentrated crash-risk demand. Right side (high moneyness, OTM calls): IV is lower. Short maturities (front of the surface): the skew is steeper and IV levels are more extreme. Long maturities (back): the surface flattens as mean-reversion dampens both the level and the skew.

What the Surface Shape Tells You

The shape of the IV surface encodes the market's collective view on:

- **Skew** (IV higher for low strikes): the market prices crash risk above what log-normal returns imply.
- **Smile** (IV elevated at both extremes): the market prices fat tails in both directions.
- **Steep term structure**: short-term uncertainty is elevated relative to long-term expectations (often a fear signal).
- **Inverted term structure**: near-term panic; the market expects volatility to decline eventually.

Each of these shapes changes over time, and tracking those changes is a source of features for volatility forecasting.

8.5 PCA of the Volatility Surface

The IV surface is a high-dimensional object (potentially hundreds of strike-maturity points), but its daily movements are driven by a small number of factors.

Cont and da Fonseca (2002) applied principal component analysis (PCA) to the daily changes of S&P 500 implied volatility surfaces and found a remarkably low-dimensional structure.

PCA in One Paragraph

PCA finds orthogonal directions of maximum variance in a dataset. The first principal component (PC1) captures the most variance, PC2 captures the most remaining variance orthogonal to PC1, and so on. If you have studied linear algebra, PCA extracts the eigenvectors of the covariance matrix, sorted by eigenvalue magnitude.

Three Factors Drive the IV Surface (Cont and da Fonseca, 2002)

PCA on daily changes in the implied volatility surface yields three dominant factors:

1. **Level** (PC1, ∼70% of variance): a parallel shift; the entire surface moves up or down together. This is the "VIX factor" for equity indices.
2. **Slope** (PC2, ∼15% of variance): the skew steepens or flattens; OTM put IV moves relative to OTM call IV.
3. **Curvature** (PC3, ∼10% of variance): the smile becomes more or less convex; both tails move relative to ATM.

Together, these three factors explain roughly 95% of daily surface variation.

Figure 8.6 illustrates the three factor loadings.

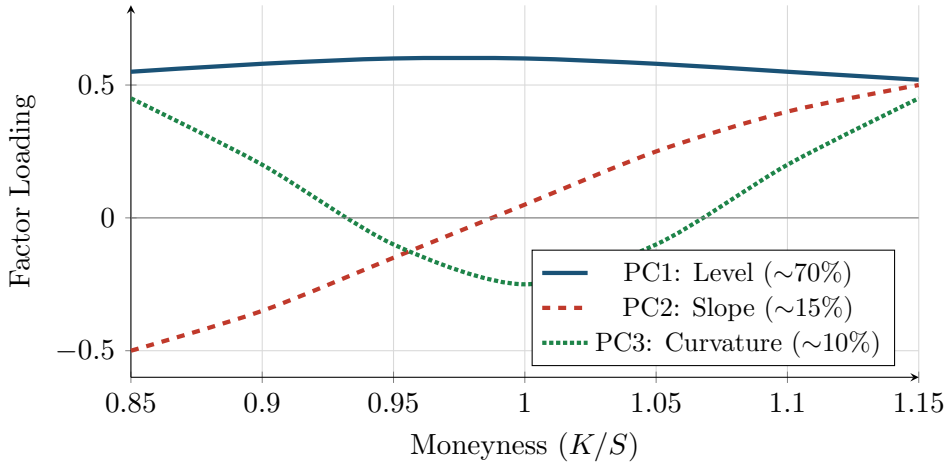


Figure 8.6: Stylized PCA factor loadings for the implied volatility surface at a fixed maturity, following Cont and da Fonseca (2002). PC1 (level) loads uniformly across strikes. PC2 (slope) loads negatively on low strikes and positively on high strikes, capturing skew rotation. PC3 (curvature) loads positively at both extremes and negatively at ATM, capturing smile convexity changes.

Practical Use: Surface Compression

Instead of feeding hundreds of IV grid points into a forecasting model, you can represent each day's surface (or surface change) with just three numbers: the PC1, PC2, and PC3 scores. This is a powerful dimensionality reduction for feature engineering (Chapter 10): three features capture 95% of the surface's information content.

8.6 Local Volatility

Requirements for This Section

You should be comfortable with the Black-Scholes framework (Section 8.2), partial derivatives (including second-order), and the concept of the implied volatility surface from the preceding sections.

The implied volatility surface gives us a snapshot of how the market prices options across strikes and maturities, but it does not tell us *how to interpolate* between quoted points or *how to extrapolate* beyond them without introducing arbitrage. A naive interpolation (e.g., linear in strike) can easily violate calendar-spread or butterfly-spread no-arbitrage conditions. **Local volatility** solves this problem: it provides the unique continuous diffusion model consistent with all observed option prices.

The key result is due to Dupire (1994). Given a continuum of European call prices $C(K,T)$ observed across strikes K and maturities T, the **Dupire formula** extracts the local volatility:

$$\sigma^2_{\text{loc}}(K,T) = \frac{\dfrac{\partial C}{\partial T} + rK\dfrac{\partial C}{\partial K}}{\dfrac{1}{2}K^2\dfrac{\partial^2 C}{\partial K^2}} \tag{8.10}$$

The numerator combines two effects: $\partial C/\partial T$ captures the time decay of the option (how much value accrues as the horizon extends), while $rK\,\partial C/\partial K$ is a cost-of-carry correction arising from discounting. The denominator, $\frac{1}{2}K^2\,\partial^2 C/\partial K^2$, is proportional to the **risk-neutral density** of the underlying at strike K and maturity T – it measures how much probability the market assigns to the stock landing near K. For the formula to be well-defined, the denominator must be strictly positive (a condition equivalent to the absence of butterfly arbitrage).

In Plain English

The Dupire formula asks: given how option prices change as you shift the strike and the maturity, what instantaneous volatility must the market be assigning to each specific price level at each specific future time? The numerator measures how much extra option value you get by extending the horizon (adjusted for interest rates). The denominator measures how tightly the market's probability is concentrated near that strike. Dividing the two gives you the local "speed of randomness" the market prices at that point.

Local Vol as Market-Implied Instantaneous Vol

Local vol is the unique diffusion coefficient $\sigma(S,t)$ such that a one-factor model

$$\frac{dS}{S} = \mu\,dt + \sigma(S,t)\,dW$$

reproduces *all* observed option prices simultaneously. It is the market's pricing of instantaneous volatility conditional on the stock being at price K at time T. Unlike implied volatility (which averages over the entire path to expiry), local vol is a pointwise, path-independent quantity – the "microscopic" volatility the market embeds at each node of the (K,T) grid.

Worked Example:

Computing Local Vol from a Call Surface **Setup.** Consider a stock at $S = 100$ with risk-free rate $r = 2\%$. We observe European call prices at three strikes and two maturities:

Strike K	$C(K, 0.25)$	$C(K, 0.50)$
90	12.85	14.60
100	5.60	7.95
110	1.80	4.10

We compute $\sigma^2_{\text{loc}}(100, 0.25)$ using finite differences.

Step 1: Estimate $\partial C/\partial T$ at $K = 100$, $T = 0.25$.

$$\frac{\partial C}{\partial T} \approx \frac{C(100, 0.50) - C(100, 0.25)}{0.50 - 0.25} = \frac{7.95 - 5.60}{0.25} = 9.40$$

Step 2: Estimate $\partial C/\partial K$ at $K = 100$, $T = 0.25$. Using a central difference with $\Delta K = 10$:

$$\frac{\partial C}{\partial K} \approx \frac{C(110, 0.25) - C(90, 0.25)}{2 \times 10} = \frac{1.80 - 12.85}{20} = -0.5525$$

Step 3: Estimate $\partial^2 C/\partial K^2$ at $K = 100$, $T = 0.25$.

$$\frac{\partial^2 C}{\partial K^2} \approx \frac{C(110, 0.25) - 2\,C(100, 0.25) + C(90, 0.25)}{10^2} = \frac{1.80 - 11.20 + 12.85}{100} = 0.0345$$

Step 4: Plug into the Dupire formula.

$$\text{Numerator} = \frac{\partial C}{\partial T} + rK\frac{\partial C}{\partial K} = 9.40 + 0.02 \times 100 \times (-0.5525) = 9.40 - 1.105 = 8.295$$

$$\text{Denominator} = \tfrac{1}{2}K^2\frac{\partial^2 C}{\partial K^2} = \tfrac{1}{2}(100)^2 \times 0.0345 = 172.5$$

$$\sigma^2_{\text{loc}}(100, 0.25) = \frac{8.295}{172.5} = 0.0481$$

Therefore $\sigma_{\text{loc}}(100, 0.25) = \sqrt{0.0481} \approx 21.9\%$.

Quantity	Value
$\partial C/\partial T$	9.40
$\partial C/\partial K$	-0.5525
$\partial^2 C/\partial K^2$	0.0345
Numerator	8.295
Denominator	172.5
σ^2_{loc}	0.0481
σ_{loc}	21.9%

Local Vol Is Not a Forecast

Local vol perfectly fits today's option prices but has no predictive power for tomorrow. It generates flat forward volatility smiles (the smile does not move), which contradicts observed behavior. Use it as an interpolation and arbitrage-checking tool, not as a model of reality. For forecasting, use the realized measures and ML models from later chapters.

Why This Matters

Local vol extractions at specific moneyness levels (e.g., 90% and 110% of spot) encode the market's current pricing of tail risk at different horizons. These can serve as ML features (Chapter 10) that complement backward-looking realized measures with forward-looking options information. The ratio of local vol at 90% moneyness to ATM local vol is a cleaner skew measure than raw IV skew, potentially giving your model a more informative tail-risk signal.

8.7 Model-Free Implied Variance and the VIX

The previous sections define implied volatility through the lens of Black-Scholes, which means the IV number carries the model's errors. This section presents a method that extracts expected future variance from option prices without assuming any pricing model.

8.7.1 Model-Free Implied Variance

Britten-Jones and Neuberger (2000) proved a striking result: under very general conditions, the expected integrated variance of the underlying (under the risk-neutral probability measure) can be read directly from option prices across all strikes, with no parametric model needed.

Risk-Neutral Measure

In derivatives pricing, the "risk-neutral" (or $\mathbb{Q}$) measure is a probability weighting under which all assets earn the risk-free rate on average. It is *not* the real-world probability of outcomes; it is a mathematical device that makes pricing consistent with no-arbitrage. Expectations under $\mathbb{Q}$ incorporate risk premia: they overweight bad outcomes relative to real-world probabilities because investors demand compensation for bearing risk.

Model-Free Implied Variance (Britten-Jones and Neuberger, 2000)

The risk-neutral expected integrated variance over horizon $[0, T]$ equals:

$$\mathbb{E}^{\mathbb{Q}}\left[\int_0^T \sigma_t^2 \, dt\right] = \frac{2}{T}\int_0^\infty \frac{C(K,T)}{K^2}\, dK + \frac{2}{T}\int_0^\infty \frac{P(K,T)}{K^2}\, dK \tag{8.11}$$

where:

- $\mathbb{E}^{\mathbb{Q}}[\cdot]$ = expectation under the risk-neutral probability measure.
- σ_t^2 = instantaneous variance of the underlying at time t.
- $\int_0^T \sigma_t^2 \, dt$ = integrated variance over the period (the object that realized volatility estimates; see Chapter 2).
- $C(K,T)$ = price of a European call with strike K and maturity T.
- $P(K,T)$ = price of a European put with strike K and maturity T.
- The integration spans all possible strikes from zero to infinity.
- The $1/K^2$ weighting gives proportionally more weight to OTM options at lower strikes.

The key insight: by integrating option prices across all strikes, the dependence on any particular volatility model cancels out.

Why It Works Without a Model

A portfolio of options across all strikes effectively replicates a contract on realized variance itself. Each option at strike K provides local information about the probability of the price reaching K. By weighting them by $1/K^2$ and integrating, you reconstruct the full distribution and extract its second moment (variance) without ever assuming what that distribution looks like.

Why This Matters

This integral is the theoretical foundation for VIX, which is one of the single strongest predictors of future realized vol. Because the model-free variance extracts *risk-neutral* expected variance (not real-world expected variance), it systematically overestimates realized vol. That persistent gap is the variance risk premium (Chapter 9), and your ML model's ability to forecast the realized side more accurately than the market's implied estimate is what creates a tradeable signal.

8.7.2 The VIX Index

The CBOE Volatility Index (VIX) is the market's most-watched "fear gauge." It implements the model-free approach of Britten-Jones and Neuberger (2000) on S&P 500 options with a 30-day horizon.

VIX Construction (,)

The VIX is defined as:

$$\text{VIX}^2 = \frac{2}{T}\sum_i \frac{\Delta K_i}{K_i^2} e^{rT} Q(K_i) - \frac{1}{T}\left(\frac{F}{K_0} - 1\right)^2 \tag{8.12}$$

and $\text{VIX} = 100 \times \sqrt{\text{VIX}^2/100^2}$, reported in annualized percentage points.

- T = time to expiry (30 calendar days, $T \approx 30/365$).
- K_i = strike price of the i-th OTM option.
- ΔK_i = spacing between adjacent strikes ($(K_{i+1} - K_{i-1})/2$).
- $Q(K_i)$ = midpoint of the bid-ask spread for the OTM option at strike K_i (puts for $K_i < F$, calls for $K_i > F$).
- F = forward index level derived from put-call parity.
- K_0 = the first strike below F.
- r = risk-free rate.
- The second term is a small correction for the difference between the forward and the first-below-forward strike.

In practice, the CBOE interpolates between the two nearest-to-30-day maturities to target exactly 30 days.

Equation (8.12) is the discrete approximation of the continuous integral in Equation (8.11): the sum over OTM options across all available strikes replaces the integral, and the $\Delta K_i/K_i^2$ weighting mirrors the $1/K^2$ in the continuous formula.

In Plain English

The VIX formula adds up the prices of out-of-the-money options across all available strikes, weighting each by $1/K^2$ so that lower-strike options (which capture crash risk)

count proportionally more. The result is the options market's collective estimate of how much the S&P 500 will bounce around over the next 30 days, expressed as an annualized percentage. The small correction term at the end accounts for the fact that the forward price may not land exactly on an available strike.

Why This Matters

VIX is the single most widely used implied-vol feature in volatility forecasting models and often the strongest univariate predictor of future realized vol (Gu et al., 2020a). For your project, VIX (or VIX^2) enters the model both as a direct feature and as one side of the variance risk premium: $\text{VRP}_t = \text{VIX}_t^2 - \widehat{\text{RV}}_{t,t+30}$. Any improvement your ML model achieves in forecasting the RV side translates directly into a more accurate VRP signal and better trading decisions.

Interpreting VIX Levels

VIX is quoted in annualized percentage points. Typical values for the S&P 500:

- **VIX $\approx$ 12–15**: calm markets, low uncertainty.
- **VIX $\approx$ 20–25**: elevated uncertainty, moderate fear.
- **VIX $>$ 30**: high fear; historically associated with market sell-offs.
- **VIX $>$ 50**: extreme panic (2008 financial crisis peak: $\sim$80; March 2020: $\sim$82).

To convert VIX to expected 30-day realized vol: $\sigma_{30d} \approx \text{VIX}/100$. For example, VIX $=$ 20 implies the options market prices roughly 20% annualized vol over the next month.

VIX Is Not a Volatility Forecast

VIX is the risk-neutral expected volatility, *not* a forecast of realized volatility under real-world probabilities. VIX systematically overstates future realized volatility. The average gap between VIX-squared and subsequent 30-day realized variance is the **variance risk premium** (Chapter 9). Historically, VIX exceeds subsequent realized vol roughly 85% of the time (Carr and Wu, 2009). Using VIX as a direct volatility forecast is a common and costly mistake: it confuses the risk-neutral measure $\mathbb{Q}$ with the real-world measure $\mathbb{P}$.

Worked Example:

From VIX to Expected Daily Moves VIX is currently at 22. What daily price move does this imply?
Step 1: Convert to a decimal: $22/100 = 0.22$ (22% annualized).
Step 2: Convert to daily. With approximately 252 trading days per year:

$$\sigma_{\text{daily}} \approx \frac{0.22}{\sqrt{252}} \approx 0.0139 = 1.39\%$$

Step 3: Interpret. With the S&P 500 at 5,000, a one-standard-deviation daily move is roughly $5{,}000 \times 0.0139 \approx 69$ points.
This is only approximate (it assumes returns are normal and ignores the risk-premium bias), but it gives you the right order of magnitude for how much daily movement the options market is pricing.

8.8 Variance Swaps: Trading Realized Volatility

Background for This Section

This section requires:
- The model-free implied variance integral (Section 8.7.1).
- The VIX construction and interpretation (Section 8.7).
- Realized variance and its estimation (Chapter 2).

VIX tells you the fair price of future variance. But how do you actually *trade* it? The **variance swap** is the instrument that lets you take a direct position on realized volatility without managing Greeks day-to-day.

8.8.1 Definition and Payoff

Variance Swap

A **variance swap** is an over-the-counter derivative whose payoff at expiry is:

$$\text{Payoff} = N_{\text{var}} \times (\text{RV}^2 - K_{\text{var}}) \tag{8.13}$$

where:
- N_{var} = the **variance notional** (dollars per variance point squared).
- K_{var} = the **variance strike** (the agreed-upon fair variance level, set at inception so the swap has zero initial value).
- RV^2 = the annualized realized variance of the underlying over the contract period, typically computed from daily log returns.

The buyer of the swap profits when realized variance exceeds the strike; the seller profits when it falls below.

In Plain English

A variance swap is a bet on how volatile the market will actually be. At the start, the two parties agree on a "fair" variance level (K_{var}). At expiry, they compare that to what actually happened (realized variance). The buyer gets paid the difference if markets were choppier than expected; the seller gets paid if markets were calmer. It is the purest financial instrument for expressing a view on volatility itself, stripped of any directional bet on the market.

Traders often think in terms of **vega notional** rather than variance notional, because it is more intuitive. The vega notional approximates the dollar P&L per volatility point (not variance point):

$$N_{\text{vega}} = N_{\text{var}} \times 2\sqrt{K_{\text{var}}} \tag{8.14}$$

For example, if $K_{\text{var}} = 0.04$ (i.e., 20% vol) and you want \$100,000 per vol point, then $N_{\text{var}} = N_{\text{vega}}/(2 \times 0.20) = \$250{,}000$ per variance point.

8.8.2 Log-Contract Replication

The theoretical foundation: Britten-Jones and Neuberger (2000) and Carr and Wu (2009) showed that continuously delta-hedging a portfolio of OTM options weighted by $1/K^2$ replicates the payoff $-2\log(S_T/S_0)$. The realized P&L from this hedge equals the realized variance minus the cost of the portfolio. Therefore, the cost of this portfolio – which is exactly the model-free implied variance integral from Section 8.7.1 – equals the fair variance swap strike K_{var}.

This is the deep connection: the VIX formula (Equation 8.12) computes the fair strike of a 30-day variance swap on the S&P 500. $\text{VIX}^2/10{,}000$ is the annualized K_{var} for that maturity.

Worked Example:

Discretizing the Variance Swap Strike **Setup.** Consider an underlying at $S = 100$ with $r = 0$. We observe 5 OTM puts and 5 OTM calls with the following mid-market prices. Using uniform strike spacing $\Delta K = 5$ for simplicity:

Step 1: Compute weights. For each option, the weight from the model-free integral discretization is:

$$w_i = \frac{2\,\Delta K_i}{K_i^2}$$

Step 2: Compute contributions. Each option's contribution to K_{var} is $w_i \times Q_i$, where Q_i is the option price.

K	Type	Price Q_i	ΔK	Weight w_i	Contribution $w_i \times Q_i$
80	Put	0.22	5	$2 \times 5/80^2 = 0.001563$	0.000344
85	Put	0.45	5	$2 \times 5/85^2 = 0.001384$	0.000623
90	Put	0.95	5	$2 \times 5/90^2 = 0.001235$	0.001173
95	Put	2.10	5	$2 \times 5/95^2 = 0.001108$	0.002327
98	Put	3.40	5	$2 \times 5/98^2 = 0.001041$	0.003539
102	Call	3.20	5	$2 \times 5/102^2 = 0.000961$	0.003076
105	Call	1.85	5	$2 \times 5/105^2 = 0.000907$	0.001678
110	Call	0.80	5	$2 \times 5/110^2 = 0.000826$	0.000661
115	Call	0.30	5	$2 \times 5/115^2 = 0.000756$	0.000227
120	Call	0.10	5	$2 \times 5/120^2 = 0.000694$	0.000069

Step 3: Sum all contributions.

$$K_{\text{var}} = \sum_i w_i \times Q_i = 0.000344 + 0.000623 + 0.001173 + 0.002327 + 0.003539$$

$$+ 0.003076 + 0.001678 + 0.000661 + 0.000227 + 0.000069 = 0.01372$$

Annualizing over $T = 30/365 \approx 0.08219$ years:

$$K_{\text{var}}^{\text{ann}} = \frac{K_{\text{var}}}{T} = \frac{0.01372}{0.08219} \approx 0.0389$$

Step 4: Convert to implied vol.

$$\sqrt{K_{\text{var}}^{\text{ann}}} = \sqrt{0.0389} \approx 19.7\%$$

If VIX = 20, then $\text{VIX}^2/10{,}000 = 0.0400$. Our discrete approximation gives 0.0389. The gap is truncation error from missing strikes below 80 and above 120, where additional OTM options would contribute small but positive increments to the sum.

Trading Your Vol Forecast Directly

A variance swap lets you monetize your RV forecast without managing delta, gamma, or theta. If you believe next-month RV will be 22% but the var swap strike is K_{var} corresponding to 19%, you buy the swap. At expiry your P&L is simply $N_{\text{var}} \times (0.22^2 - 0.19^2) = N_{\text{var}} \times 0.0123$. No path dependence before expiry settlement, no Greeks to manage – just a pure bet on realized variance.

Convexity and Mark-to-Market

A **volatility swap** (payoff $= \sigma_{\text{realized}} - K_{\text{vol}}$) is NOT the same as a variance swap. Because $\mathbb{E}[\sigma] < \sqrt{\mathbb{E}[\sigma^2]}$ (Jensen's inequality), the vol swap strike is lower than $\sqrt{K_{\text{var}}}$. The gap depends on the volatility of volatility (vol-of-vol, Chapter 9 Section 9.7). Also: before expiry, variance swaps have substantial mark-to-market risk as implied vol moves. A position can be deeply underwater mid-life even if it ultimately settles in your favor.

Why This Matters

The variance swap makes the VRP a tradeable quantity. $\text{VIX}^2/10{,}000$ minus your forecast of 30-day RV = variance risk premium (Chapter 9). If your ML model forecasts RV more accurately than the market-implied K_{var}, you can systematically harvest the mispricing. This is the core mechanism behind Project 5 (VRP ML Trader) and the economic-value test for any of the other project directions.

8.9 Connecting the Vol Surface to Volatility Forecasting

This section previews how the concepts introduced here feed into the rest of the guide.

IV Surface Features for ML Models

The volatility surface provides a rich set of features for realized volatility forecasting models (Chapters 10–14):

- **VIX level**: the single most common implied-vol feature. Often the strongest univariate predictor of future realized vol (Gu et al., 2020a).
- **VVIX** (the "vol of vol"): implied volatility of VIX options; captures uncertainty about uncertainty.
- **Skew slope**: the difference in IV between OTM puts and ATM options; a proxy for tail-risk pricing.
- **Butterfly spread** (Section 8.4.3): average wing IV minus ATM IV; captures symmetric tail thickness (kurtosis demand), orthogonal to skew.
- **Term spread**: the difference between long-dated and short-dated ATM IV; signals mean-reversion expectations.
- **PCA scores** (Section 8.5): three-number summary of the entire surface.
- **Variance risk premium**: VIX^2 minus recent realized variance; the subject of Chapter 9.

8.10 Summary

- An option gives the right (not obligation) to buy (call) or sell (put) at a fixed strike K by expiry T.
- Call payoff: $\max(S_T - K, 0)$; put payoff: $\max(K - S_T, 0)$.
- Black-Scholes (Black and Scholes, 1973) provides a closed-form call price: $C = S\Phi(d_1) - Ke^{-rT}\Phi(d_2)$. The formula assumes constant volatility, log-normal returns, no jumps, and frictionless markets.
- Implied volatility σ_{imp} is the σ that makes Black-Scholes match the market price. It is not a forecast; it is the market's price of uncertainty, contaminated by risk premia and model error.
- The IV surface $\sigma_{\text{imp}}(K, T)$ varies across strikes and maturities. Skew (higher IV for low strikes) reflects crash-risk pricing; smile (U-shape) reflects fat-tail pricing.

- The butterfly spread $\text{BF}_t = \frac{1}{2}(\sigma_{25\Delta P} + \sigma_{25\Delta C}) - \sigma_{\text{ATM}}$ measures symmetric tail thickness (kurtosis demand), complementing the skew's measure of directional asymmetry. It spikes during crises when both wings are bid up.
- PCA of the IV surface (Cont and da Fonseca, 2002) yields three dominant factors (level, slope, curvature) explaining ∼95% of daily variation.
- Model-free implied variance (Britten-Jones and Neuberger, 2000) extracts expected future variance from the cross-section of option prices without assuming any pricing model.
- VIX implements model-free implied variance for S&P 500 options over 30 days. VIX systematically overestimates future realized vol; the gap is the variance risk premium (Chapter 9).
- VIX, skew, butterfly, term structure, PCA scores, and VRP are all feature candidates for ML volatility models (Chapter 10).
- The IV surface is observed daily, forward-looking, and market-priced, making it a uniquely valuable complement to backward-looking realized volatility measures.

Table 8.1: Key Results from Chapter 8

Concept	**Key Formula / Result**	**Significance**
Black-Scholes	$C = S\Phi(d_1) - Ke^{-rT}\Phi(d_2)$	Defines the common language for quoting option prices as volatilities
Implied volatility	Solve $C_{\text{mkt}} = C_{\text{BS}}(\sigma_{\text{imp}})$	Market-priced, forward-looking measure of uncertainty
IV surface shape	Skew, smile, term structure	Reveals crash-risk pricing, fat-tail beliefs, and mean-reversion expectations
Butterfly spread	$\text{BF}_t = \frac{1}{2}(\sigma_{25\Delta P} + \sigma_{25\Delta C}) - \sigma_{\text{ATM}}$	Symmetric tail thickness; orthogonal to skew; crisis-detection signal
PCA of IV surface	3 factors ≈ 95% of variance	Level, slope, curvature; compress the surface for feature engineering
Model-free variance	$\frac{2}{T}\int \frac{O(K)}{K^2}\,dK$	Extracts expected variance without model assumptions
VIX	CBOE implementation, 30-day	Not a vol forecast; systematically overestimates realized vol (includes risk premium)

Chapter 9

The Variance Risk Premium

Why This Chapter Matters

The variance risk premium links implied volatility (Chapter 8) to realized volatility (Chapter 2). It predicts equity returns (Bollerslev et al., 2009b), forecasts future realized vol through mean reversion, and is a direct trading signal (Project 5: VRP ML trader). VRP features appear in virtually every competitive feature set (Chapter 10).

9.1 What Is the Variance Risk Premium?

Chapter 8 showed that VIX systematically overstates future realized volatility: the options market charges more for volatility exposure than what actually materializes. This chapter names that gap, measures it, and explains why it exists and what it predicts.

Start with the intuition. Imagine you own a house and buy fire insurance. You pay $1,200 per year in premiums, but the expected fire loss (probability of fire times damage) is only $400. The $800 gap is the insurance premium: compensation the insurer earns for bearing your tail risk.

The variance risk premium is the same concept applied to volatility. Options sellers are the "insurers" of the stock market. They sell downside protection (puts) and volatility exposure (straddles) to hedgers and speculators. In return, they earn a premium: the gap between what the options market prices in and what actually happens.

VRP = Volatility Insurance Premium

VRP is the insurance premium the market charges for bearing volatility risk. Options sellers earn it; options buyers pay it. When VRP is large, the market is charging a steep price for protection, just as hurricane insurance premiums spike during storm season.

Risk-Neutral ($\mathbb{Q}$) vs. Physical ($\mathbb{P}$) Probability Measures

Two probability measures appear throughout this chapter:

- The **physical measure** $\mathbb{P}$ describes the actual frequencies of outcomes in the real world. If you simulate the S&P 500 forward under $\mathbb{P}$, your simulated returns match the historical distribution (including its mean, fat tails, and skew).
- The **risk-neutral measure** $\mathbb{Q}$ is a mathematical reweighting of $\mathbb{P}$ that makes asset pricing consistent with no-arbitrage. Under $\mathbb{Q}$, bad outcomes (crashes, spikes in volatility) receive higher weight than under $\mathbb{P}$ because investors demand compensa-

tion for bearing those risks.

Option prices reflect $\mathbb{Q}$-expectations, not $\mathbb{P}$-expectations. VIX is a $\mathbb{Q}$-measure of expected variance. Realized volatility is observed under $\mathbb{P}$. The VRP is the difference between the two.

Now the formal definition. The variance risk premium at time t is the difference between the risk-neutral expected variance (what the options market prices) and the physical expected variance (what actually tends to happen) over a horizon h (typically 30 calendar days):

Variance Risk Premium

$$\text{VRP}_t = \mathbb{E}_t^{\mathbb{Q}}[\text{RV}_{t,t+h}] - \mathbb{E}_t^{\mathbb{P}}[\text{RV}_{t,t+h}] \tag{9.1}$$

- VRP_t: the variance risk premium at time t.
- $\mathbb{E}_t^{\mathbb{Q}}[\cdot]$: the risk-neutral (options-market-implied) expectation, conditional on information at time t.
- $\mathbb{E}_t^{\mathbb{P}}[\cdot]$: the physical (real-world) expectation, conditional on information at time t.
- $\text{RV}_{t,t+h}$: realized variance over the period from t to $t+h$ (defined in Chapter 2).
- h: the forecast horizon, typically 30 calendar days ($\sim$22 trading days) to match VIX.

Why This Matters

This equation is the foundation of VRP-based trading strategies. If your ML model produces a better forecast of $\mathbb{E}_t^{\mathbb{P}}[\text{RV}_{t,t+h}]$ than backward-looking RV or a simple HAR model, then your VRP estimate is sharper. A sharper VRP estimate means you can identify when the options market is genuinely overpricing protection versus when the premium is fair, which is the core edge in any vol-trading strategy built on your RV forecast.

In practice, neither expectation is directly observed. You need proxies for each:

Operationalized VRP

The standard operational measure of VRP is:

$$\widehat{\text{VRP}}_t = \underbrace{\left(\frac{\text{VIX}_t}{100}\right)^2}_{\text{proxy for } \mathbb{E}_t^{\mathbb{Q}}[\text{RV}_{t,t+h}]} - \underbrace{\hat{\text{RV}}_{t,t+h}^{\mathbb{P}}}_{\text{proxy for } \mathbb{E}_t^{\mathbb{P}}[\text{RV}_{t,t+h}]} \tag{9.2}$$

- $(\text{VIX}_t/100)^2$: VIX squared (converted from percentage points to decimal), which equals the model-free risk-neutral expected variance over 30 days (Section 8.7 of Chapter 8).
- $\hat{\text{RV}}_{t,t+h}^{\mathbb{P}}$: a physical-measure forecast of future realized variance. Common choices:
  - Backward-looking RV: use $\text{RV}_{t-h,t}$ (the past 30 days' realized variance) as a naive forecast.
  - HAR forecast: use the HAR model (Chapter 6) to produce $\hat{\text{RV}}_{t+h}$, a more sophisticated forecast.
- $\widehat{\text{VRP}}_t > 0$: implied variance exceeds expected realized variance (normal; VRP is positive on average).
- $\widehat{\text{VRP}}_t < 0$: rare, but occurs when realized vol spikes above what was priced in (e.g., crash events).

In Plain English

This equation says: take what the options market thinks volatility will be (VIX squared), subtract what you think volatility will actually be (your RV forecast), and the gap is the variance risk premium. It is the price tag on fear. The better your RV forecast on the right-hand side, the more accurately you measure how much the market is overpaying for protection.

Why This Matters

This is the equation you will compute every day in a VRP trading strategy. Your ML model replaces the naive backward-looking RV with a more accurate $\hat{\text{RV}}^{\mathbb{P}}_{t+h}$, producing a cleaner VRP signal. A HAR or XGBoost forecast that improves QLIKE by 5–10% does not just make your RV number better in isolation; it sharpens the VRP estimate that drives every downstream trading decision.

Ex-Ante vs. Ex-Post VRP

Two variants appear in the literature and they measure different things:

- **Ex-ante VRP**: $\text{VIX}_t^2 - \hat{\text{RV}}^{\mathbb{P}}_{t,t+h}$, where $\hat{\text{RV}}$ is a *forecast* of future variance (e.g., from HAR). This is available in real time and can be used as a trading signal.
- **Ex-post VRP**: $\text{VIX}_t^2 - \text{RV}_{t,t+h}$, where $\text{RV}_{t,t+h}$ is the *actual* realized variance over the next 30 days. This is only known after the fact and cannot be used for trading.

Bollerslev et al. (2009b) use the ex-ante version with backward-looking RV. Bekaert and Hoerova (2014) show that the choice of $\mathbb{P}$-measure proxy matters for return predictability. Always state which version you are using.

Worked Example:

Computing VRP from VIX and Realized Variance On January 15, 2024, VIX closes at 13.5 and the realized variance of the S&P 500 over the past 22 trading days is $\text{RV}_{t-22,t} = 0.0105$ (annualized realized vol $= \sqrt{0.0105} \approx 10.2\%$).

Step 1: Convert VIX to implied variance.

$$\text{VIX}^2 = \left(\frac{13.5}{100}\right)^2 = 0.018225$$

This is the annualized risk-neutral expected variance.

Step 2: Use backward-looking RV as the physical-measure proxy.

$$\hat{\text{RV}}^{\mathbb{P}} = 0.0105 \quad \text{(annualized)}$$

Step 3: Compute VRP.

$$\widehat{\text{VRP}}_t = 0.018225 - 0.0105 = 0.007725$$

Step 4: Interpret. In volatility terms: implied vol is 13.5%, realized vol is 10.2%, so the gap is about 3.3 percentage points. In variance terms: the VRP is 0.0077, or about 42% of the implied variance. This is a typical calm-market value: the options market is pricing in moderately more uncertainty than recent history suggests.

9.2 Why VRP Exists

The previous section showed that VRP is positive on average: VIX-squared systematically exceeds realized variance. Why? If markets were risk-neutral (if investors did not care about risk), then $\mathbb{Q} = \mathbb{P}$ and VRP would be zero. But investors are risk-averse, and that risk aversion creates the premium.

Three forces contribute:

9.2.1 Risk Aversion and Downside Protection Demand

Risk-averse investors are willing to overpay for insurance against large losses. Pension funds, endowments, and portfolio managers routinely buy put options and volatility protection even though these instruments lose money on average. They accept this negative expected return because the protection pays off precisely when their portfolios suffer most.

This is not irrational. A 40% drawdown can trigger margin calls, force liquidation, or violate regulatory capital requirements. The cost of ruin is asymmetric: a dollar lost in a crash is more painful than a dollar gained in calm markets. Paying a small insurance premium to avoid catastrophic outcomes is rational for constrained investors, even if the premium exceeds the actuarial cost.

9.2.2 The Insurance Analogy, Formalized

Return to the fire insurance example. The homeowner pays \$1,200/year for a policy with an expected loss of \$400. The insurer earns the \$800 difference. But the insurer also bears the risk of correlated claims (a wildfire that burns an entire neighborhood). The premium compensates for this systematic, non-diversifiable risk.

In the options market:

- Hedgers (pension funds, portfolio managers) are the homeowners buying protection.
- Options sellers (market makers, hedge funds) are the insurers collecting the premium.
- The VRP is the \$800 gap: compensation for bearing volatility risk, especially in its most extreme (crash) form.

9.2.3 Equilibrium Account: Long-Run Risk

Drechsler and Yaron (2011) provide a formal equilibrium explanation. In their model, the representative agent has *Epstein-Zin preferences* (a generalization of standard utility that separates risk aversion from the willingness to substitute consumption over time). Uncertainty about future economic growth fluctuates over time: there are periods when the economy's growth rate is predictable and periods when it is not. This "volatility of volatility" in the real economy generates a variance risk premium in asset markets.

Why VRP Is Positive

VRP exists because:

1. **Risk aversion**: investors overpay for crash protection relative to its actuarial value.
2. **Non-diversifiable risk**: volatility spikes coincide with market crashes, making volatility risk systematic (not diversifiable away).
3. **Uncertainty about uncertainty**: in equilibrium models like Drechsler and Yaron (2011), time-varying economic uncertainty generates a positive VRP.

The premium is not a market inefficiency. It is rational compensation for bearing a risk that hurts most when it materializes.

9.3 VRP Predicts Returns

The VRP is not just an insurance premium that options sellers passively collect. It actively predicts future equity returns. This is the central empirical finding of Bollerslev et al. (2009b).

The logic: when VRP is high, the market is demanding steep compensation for bearing volatility risk. This signals elevated risk aversion or heightened uncertainty. Assets priced under high risk aversion offer higher expected returns as compensation. When VRP is low, the market is complacent, and expected returns are lower.

Bollerslev et al. (2009b): VRP Predicts Quarterly Equity Returns

Bollerslev et al. (2009b) regress future quarterly S&P 500 excess returns on the ex-ante VRP and find:

- The VRP coefficient is positive and statistically significant: higher VRP today predicts higher equity returns over the next quarter.
- A one-standard-deviation increase in VRP predicts approximately 3–4% higher quarterly excess returns.
- The R^2 for quarterly return prediction is 5–10%, substantially exceeding the dividend yield (the previous best-known predictor at the quarterly horizon).
- The predictability is concentrated at the 1–6 month horizon and fades at longer horizons.

They operationalize VRP as $\text{VIX}_t^2 - \text{RV}_{t-22,t}$, using backward-looking monthly RV as the $\mathbb{P}$-measure proxy.

Worked Example:

VRP as a Return Predictor On March 1, 2024, VIX is at 14.0 and the past-month realized vol of the S&P 500 is 9.5%.

Step 1: Compute VRP.

$$\begin{aligned} \text{VIX}^2 &= (14.0/100)^2 = 0.0196 \\ \text{RV}_{\text{past month}} &= (9.5/100)^2 = 0.009025 \\ \widehat{\text{VRP}} &= 0.0196 - 0.009025 = 0.010575 \end{aligned}$$

Step 2: Interpret the signal. VRP is moderately positive. The implied variance is more than double the recent realized variance. According to the BTZ regression, this level of VRP is consistent with positive expected quarterly returns, but not extremely high returns (VRP is not at crisis levels).

Step 3: Compare to a high-VRP episode. In March 2020, VIX hit 82.7 while trailing realized vol was around 30%. VRP was roughly $(0.827)^2 - (0.30)^2 = 0.684 - 0.090 = 0.594$, an extreme level. The BTZ framework would predict very high future returns, and indeed the S&P 500 rallied over 40% in the subsequent two quarters.

VRP Is Not a Timing Tool

VRP predicts returns in a statistical, population-average sense. The R^2 of 5–10% means that 90–95% of quarterly return variation is unexplained. A high VRP does *not* guarantee positive returns in any single quarter. Furthermore, VRP is highest during crises, precisely when portfolio constraints and drawdown pain are most severe. Trading on VRP requires the ability and willingness to take risk when it feels worst.

9.4 VRP Predicts Future Volatility

VRP is not only a return predictor; it also contains information about future realized volatility. The mechanism is mean reversion.

When VRP is large (implied far above realized), two things tend to happen:

1. Realized volatility rises toward implied. The options market is pricing in higher future vol for a reason: upcoming risk events, deteriorating conditions, or elevated uncertainty. Realized vol tends to catch up.
2. Implied volatility falls toward realized. As the risk event passes or uncertainty resolves, the fear premium deflates.

The net effect is convergence: the gap closes from both sides, but with realized vol doing more of the adjustment.

VRP as a Dual-Purpose Feature

VRP is a dual-purpose signal:

- As a **return predictor**: high VRP ⇒ higher expected future equity returns (Bollerslev et al., 2009b).
- As a **volatility predictor**: high VRP ⇒ realized vol tends to rise toward implied vol (mean reversion of the gap).

This makes VRP a natural feature for both return-forecasting and volatility-forecasting ML models (Chapter 10).

The mean-reversion channel also explains a practical finding from volatility forecasting: including VIX (or VIX-squared) alongside lagged RV in a HAR-type model (Chapter 6) improves out-of-sample forecasts. VIX carries forward-looking information that backward-looking RV alone does not capture. The VRP quantifies how much extra forward-looking information VIX contributes beyond what lagged RV already tells you.

9.5 Decomposing VRP

The headline VRP is a single number, but it mixes together different sources of risk. Two decompositions from the literature isolate the components.

9.5.1 Uncertainty vs. Risk Aversion

Bekaert and Hoerova (2014) decompose the VIX-squared into two pieces:

$$\text{VIX}_t^2 = \underbrace{\mathbb{E}_t^{\mathbb{P}}[\text{RV}_{t,t+h}]}_{\text{expected variance}} + \underbrace{\text{VRP}_t}_{\text{variance risk premium}} \tag{9.3}$$

This is just a rearrangement of Equation (9.1), but the key insight is in how each component behaves:

- $\mathbb{E}_t^{\mathbb{P}}[\text{RV}_{t,t+h}]$: the **expected variance** component, which captures physical uncertainty about future returns. When economic conditions deteriorate, this rises.
- VRP_t: the **risk premium** component, which captures the market's risk aversion and demand for protection. When fear rises faster than objective uncertainty, this rises.

In Plain English

When VIX is high, this decomposition asks: is VIX high because the world is genuinely risky right now (the expected variance piece), or because investors are unusually scared and demanding extra compensation for bearing that risk (the VRP piece)? The same VIX level can mean very different things depending on which component is driving it. Separating the two tells you whether the market is pricing objective danger or subjective fear.

Why This Matters

This decomposition directly affects your feature engineering. If you include raw VIX-squared as a feature in your RV forecasting model, you are mixing two signals: one that reflects genuine future risk (useful for forecasting RV) and one that reflects risk aversion (useful for forecasting returns, not RV). By decomposing VIX-squared using your HAR or ML forecast as the $\mathbb{P}$-measure proxy, you can feed the separated components as distinct features, potentially improving forecast accuracy by letting the model weight objective uncertainty and fear independently.

Bekaert and Hoerova (2014): Return Predictability Lives in the VRP Component

Bekaert and Hoerova (2014) find that:

- The VRP component significantly predicts future equity returns (consistent with Bollerslev et al. 2009b).
- The expected-variance component does *not* predict returns. High objective uncertainty alone does not imply high future returns; it is the risk-aversion markup that carries the signal.
- The expected-variance component predicts future realized volatility (naturally, since it is a forecast of RV).
- Different $\mathbb{P}$-measure proxies (backward-looking RV, GARCH forecasts, HAR forecasts) yield materially different VRP estimates and different predictive power. The choice of proxy is not innocuous.

9.5.2 Normal-Times vs. Jump-Tail VRP

Bollerslev and Todorov (2011) decompose the VRP along a different dimension: the portion attributable to "normal" continuous fluctuations vs. the portion attributable to rare, large jumps (tail events).

Normal vs. Tail VRP (,)

$$\text{VRP}_t = \underbrace{\text{VRP}_t^{\text{diffusive}}}_{\text{normal-times premium}} + \underbrace{\text{VRP}_t^{\text{tail}}}_{\text{jump-tail premium}} \tag{9.4}$$

- $\text{VRP}_t^{\text{diffusive}}$: the premium for bearing day-to-day, continuous variance risk. This component is relatively stable and modest.
- $\text{VRP}_t^{\text{tail}}$: the premium for bearing rare, large-jump risk (crashes). This component is highly time-varying and spikes during stress.

Bollerslev and Todorov (2011) find that the tail component drives most of the time-variation in the aggregate VRP. During calm markets, the tail premium is small and the overall VRP is moderate. During crises, the tail premium explodes, reflecting the market's intense fear of

further crashes.

Most of VRP Is About Crash Fear

The variance risk premium is not primarily about day-to-day fluctuations. It is mostly compensation for tail risk: the possibility of rare, catastrophic moves. This aligns with the demand-side story: what investors are really willing to overpay for is crash protection, not protection against ordinary volatility.

Why This Matters

This decomposition connects directly to jump detection from Chapter 4. The bipower variation and jump test statistics you computed there separate realized variance into continuous and jump components. The same separation applied to the risk premium side tells you whether the market is overpricing jump risk or diffusive risk. If your ML model can forecast the jump component of RV better than the continuous component (or vice versa), you know which piece of the VRP you are best positioned to harvest. A model that excels at predicting jump arrivals would target the tail VRP; a model that excels at forecasting the smooth diffusive component would target the normal-times VRP.

9.6 The Gamma P&L Formula: From Forecast to Money

Required Background

This section requires delta and gamma from Chapter 8 (the Black–Scholes section), variance swap strike mechanics (Section 8.8), and the VRP definition (Section 9.1 earlier in this chapter).

You have built an ML model that forecasts RV more accurately than the options market implies. How much money does that improved forecast generate? This section derives the exact formula linking forecast accuracy to trading profit.

9.6.1 Setup: Delta-Hedged Option P&L

Consider a trader who buys an option at implied vol σ_i and delta-hedges continuously. Each day, the option's value changes due to two effects: (a) the delta-hedged P&L from the stock's realized move, and (b) time decay (theta). Because the trader maintains a delta-neutral position, the first-order stock exposure cancels. What remains is the second-order (gamma) exposure to realized moves versus the cost of carrying that exposure (theta).

9.6.2 Derivation from the Black–Scholes PDE

The starting point is the Black–Scholes PDE, which describes the equilibrium between time decay and gamma exposure when volatility equals the implied level. This equation governs the "fair cost" of holding a delta-hedged option:

$$\Theta + \frac{1}{2}\Gamma S^2 \sigma_i^2 = rV \tag{9.5}$$

- Θ: theta, the option's time decay (dollars lost per day from the passage of time).
- Γ: gamma, the option's second-order sensitivity to the stock price.
- S: current stock price.
- σ_i: implied volatility (the market's priced-in vol).
- r: risk-free rate; V: option value.

For a delta-hedged book, the stock component nets out, leaving theta as the "cost of gamma."

In reality, the stock moves with realized volatility σ_r, not implied volatility σ_i. The actual P&L per infinitesimal time step for a delta-hedged long option position is:

$$d(\text{P\&L}) = \frac{1}{2}\Gamma S^2\left(r_t^2 - \sigma_i^2\, dt\right) \tag{9.6}$$

- r_t: the log-return over the interval dt.
- r_t^2: the realized variance contribution for that period.
- $\sigma_i^2\, dt$: the implied variance "charged" by the market over the same period.

Over the full life of the option, the cumulative hedging P&L is:

$$\text{Total P\&L} = \sum_{t=1}^{N} \frac{1}{2}\Gamma_t S_t^2\left(\sigma_{r,t}^2 - \sigma_i^2\right)\Delta t \tag{9.7}$$

- N: total number of hedging intervals over the option's life.
- Γ_t, S_t: gamma and stock price at each rebalance, which change as the stock moves.
- $\sigma_{r,t}^2$: realized variance in interval t; σ_i^2: the constant implied variance you paid for.

In Plain English

The Black–Scholes PDE tells you what an option is "worth" if the stock moves exactly at implied vol. Equations (9.6) and (9.7) capture what happens when reality disagrees with the market's assumption. Each day, you earn the difference between what the stock actually did (r_t^2) and what the option charged you for ($\sigma_i^2\, dt$), scaled by your gamma exposure. Over the life of the option, these daily differences accumulate. If realized vol consistently exceeds implied, the long-gamma trader profits; if realized vol falls short, the short-gamma trader (vol seller) profits.

Why This Matters

These equations are the direct link between your RV forecast and dollars. If your ML model predicts that realized variance over the next 30 days will be 0.012 while the options market is pricing 0.018 (VIX-squared), the gamma P&L formula tells you exactly how much profit to expect from selling volatility: roughly $\frac{1}{2}\Gamma S^2(0.018 - 0.012)T$. Every basis point of QLIKE improvement in your forecast translates into a more accurate estimate of this gap, which lets you size positions more precisely and avoid selling vol when the premium is not actually there.

For an at-the-money option where gamma remains roughly constant over the period, this simplifies to:

$$\text{P\&L} \approx \frac{1}{2}\Gamma S^2\left(\text{RV}^2 - \text{IV}^2\right) T \tag{9.8}$$

where RV^2 and IV^2 are the annualized realized and implied variances, and T is the time period in years.

The Gamma P&L Formula

The daily hedging P&L for a delta-hedged option position is:

$$\text{Daily Hedging P\&L} = \frac{1}{2}\,\Gamma\, S^2\left(\sigma^2_{\text{realized}} - \sigma^2_{\text{implied}}\right)\Delta t \tag{9.9}$$

Positive gamma (long options) profits when realized vol exceeds implied vol. **Negative gamma** (short options) profits when realized vol is below implied vol. The VRP (IV >

RV on average) means selling gamma is systematically profitable: you collect theta that, on average, exceeds the realized moves you pay out.

Figure 9.1 shows the P&L of a delta-hedged long option position as a function of realized volatility relative to implied volatility.

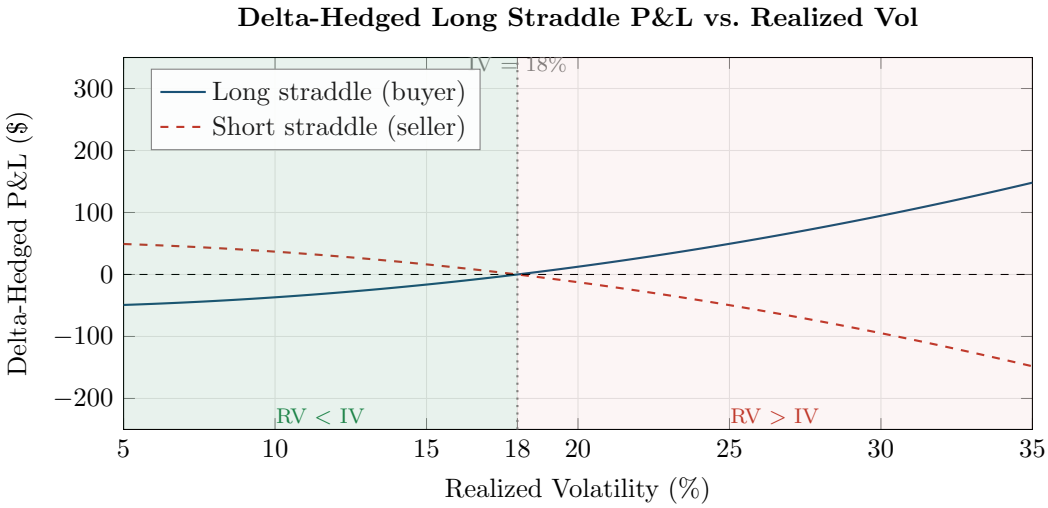


Figure 9.1: P&L of a delta-hedged straddle as a function of realized volatility, assuming IV = 18%, $\Gamma = 0.04$, $S = 100$, $T = 30$ days. The seller (dashed red) profits in the green-shaded region where realized vol falls below implied vol, which is the typical VRP regime. The buyer (solid blue) profits only when realized vol exceeds implied. Since VRP is positive roughly 85% of the time, the seller has a statistical edge.

Worked Example: Delta-Hedged Straddle: Five Days of Gamma P&L

Setup. Buy a 30-day ATM straddle on a stock at $S = 100$, with implied volatility IV = 18%. The approximate ATM gamma for this option is $\Gamma = 0.04$. Delta-hedge daily. Over 5 days, the stock follows the path: 100, 101.2, 99.8, 100.5, 102.1, 101.0.

Parameters. The daily implied variance charge is:

$$\sigma_i^2 \cdot \Delta t = \frac{(0.18)^2}{252} = \frac{0.0324}{252} = 0.0001286$$

Computation. Each day, compute the log-return $r_t = \ln(S_t/S_{t-1})$, the squared return r_t^2 (realized variance for that day), and the gamma P&L:

$$\text{P\&L}_t = \tfrac{1}{2} \times 0.04 \times S_t^2 \times (r_t^2 - 0.0001286)$$

Day	S_t	r_t (%)	r_t^2 ($\times 10^{-4}$)	$\sigma_i^2 \Delta t$ ($\times 10^{-4}$)	Gamma P&L ($)
1	101.2	+1.19	1.423	1.286	+0.028
2	99.8	−1.39	1.941	1.286	+0.130
3	100.5	+0.70	0.489	1.286	−0.161
4	102.1	+1.58	2.495	1.286	+0.252
5	101.0	−1.08	1.173	1.286	−0.023
Cumulative (5 days)					+0.227

Interpretation. The annualized realized volatility over these 5 days was approximately 19.5%, above the 18% implied vol we paid. The cumulative gamma P&L is positive ($0.227

per share, or \$22.70 per standard 100-share contract). On days where the squared return exceeded $\sigma_i^2 \Delta t$ (days 1, 2, 4), we profited; on quiet days (days 3, 5), we lost to theta.

Renting a Magnifying Glass

Think of gamma as renting a magnifying glass. Each day you earn the difference between what actually happened (r_t^2) and what you paid for ($\sigma_i^2\, dt$). On average, if your vol forecast is right and the market's is wrong, you accumulate profit proportional to the forecast error times your gamma exposure. The magnifying glass is expensive (theta), but on days when the stock moves more than expected, it amplifies your gains.

Path Dependence: Gamma Is Not Constant

Gamma is *not* constant. As the stock moves away from the strike, gamma falls. Two stock paths with identical 30-day realized vol can produce very different hedging P&L because gamma was high during the low-vol days and low during the high-vol days. This is why forecasting *when* vol occurs (intraday patterns, jump timing) matters, not just the average level. A model that predicts the same total RV but correctly identifies which days will be volatile is worth more than a model that gets only the level right.

Why This Matters

For your internship evaluation: a 5% QLIKE improvement in your RV forecast means you can identify days when the market misprices vol by more. In a vol-trading context, this translates to approximately 2–5 bps per unit of vega exposure per day (order of magnitude). The exact number depends on your gamma profile and hedging frequency. Section 18.1 quantifies this via a simpler mechanism (vol-targeting Sharpe ratio improvement).

9.7 Vol-of-Vol

VIX measures the expected volatility of the S&P 500. But VIX itself is volatile. The "volatility of VIX" captures a distinct risk factor: uncertainty about the level of future uncertainty.

9.7.1 VVIX: The Volatility of VIX

VIX as an Underlying

Chapter 8 defined VIX as the model-free implied volatility of the S&P 500 over 30 days. VIX itself is a traded index: there are options written on VIX (VIX options, traded at Cboe). You can apply the same model-free variance extraction method from Chapter 8 to VIX options to get the implied volatility of VIX. This is VVIX.

VVIX

VVIX is the model-free implied volatility of VIX, computed from VIX options using the same methodology as VIX itself (Cboe Exchange, 2019).

- VVIX is quoted in annualized percentage points (like VIX).
- Typical range: 80–120 in calm markets, spiking to 150+ during stress.
- Interpretation: VVIX = 100 means the options market prices VIX's annualized volatility at 100%.

9.7.2 Realized Vol-of-Vol

You can also compute a backward-looking measure: the realized volatility of the VIX time series itself. This is constructed exactly like RV from Chapter 2, but applied to VIX returns rather than S&P 500 returns:

$$\mathrm{RV}_t^{\mathrm{VIX}} = \sum_{i=1}^{n} (\Delta \ln \mathrm{VIX}_{t,i})^2 \tag{9.10}$$

- $\Delta \ln \mathrm{VIX}_{t,i}$: the i-th intraday log return of VIX on day t.
- This measures how much VIX itself fluctuated within the day.

In Plain English

This equation applies the same realized variance recipe from Chapter 2 but to VIX returns instead of stock returns. It answers the question: how unstable is the market's fear gauge today? A high $\mathrm{RV}_t^{\mathrm{VIX}}$ means the options market's expectations are whipping around within the day, even if the closing VIX level looks calm. Two days with VIX at 20 can feel very different if one had VIX bouncing between 18 and 22 intraday while the other sat still.

Why This Matters

Realized vol-of-vol is a powerful feature for RV forecasting because it captures a dimension that lagged RV alone misses: the stability of the volatility regime. When $\mathrm{RV}_t^{\mathrm{VIX}}$ is high, the market is uncertain about how volatile things will be, which often precedes volatility spikes. Adding vol-of-vol features to your HAR or ML model can improve QLIKE by capturing regime instability that standard lagged-RV features overlook. This connects to the VVIX-based features discussed in Chapter 10.

9.7.3 Jumps in VIX

VIX is known for sudden spikes: it can double in a single day during a market crash. These VIX jumps represent a distinct risk factor beyond the level of VIX. A market with VIX at 20 that is stable is very different from a market with VIX at 20 that just fell from 40 (or is about to spike to 40). The vol-of-vol measures capture this instability.

9.7.4 Diagram: VIX vs. Realized Vol

Figure 9.2 shows the persistent gap between VIX and subsequent realized volatility over a long sample.

The key visual pattern: the red line (VIX) sits above the blue line (realized vol) nearly everywhere. The gap is the VRP. It widens dramatically during crises and narrows during calm markets, but it almost never flips negative for extended periods.

Carr and Wu (2009) document that VIX exceeds subsequent realized vol roughly 85% of the time. The 15% of months where realized vol exceeds VIX are concentrated in sudden-onset crises, when realized vol spikes before the options market fully adjusts.

9.8 ML Approaches to VRP

The traditional VRP literature uses simple proxies (backward-looking RV, GARCH forecasts) for the $\mathbb{P}$-measure expectation. ML methods can improve this proxy, and the VRP itself can

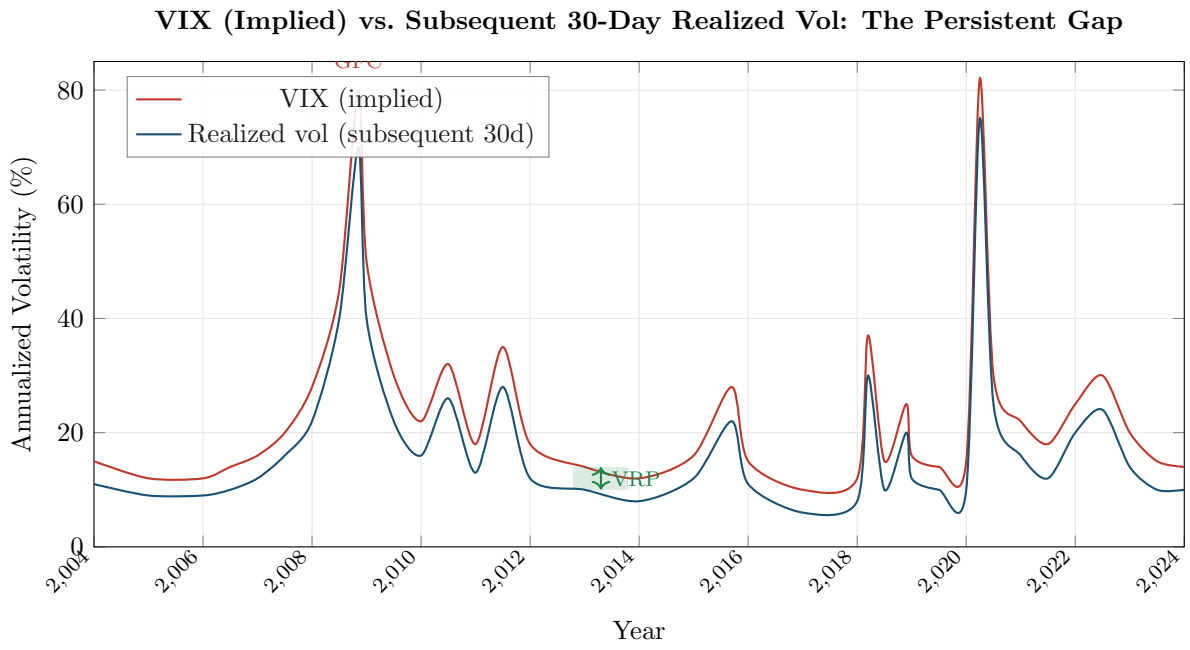


Figure 9.2: VIX (red) vs. subsequent 30-day realized volatility (blue), 2004–2024 (schematic). The persistent gap between the two lines is the variance risk premium. VIX almost always exceeds subsequent realized vol. The gap widens during crises (2008, 2020) when the demand for protection surges. The green shaded region illustrates the VRP during a calm period.

serve as a feature or trading signal in ML pipelines.

9.8.1 Improved $\mathbb{P}$-Measure Forecasts

Fouhy (2024) proposes a hierarchical XGBoost approach to the VRP pipeline:

1. **Stage 1**: Train an XGBoost model to forecast realized variance RV_{t+h} using lagged RV, HAR-style features, and additional predictors (macro indicators, sentiment, past VIX). This produces a high-quality $\hat{\text{RV}}^{\mathbb{P}}_{t+h}$.
2. **Stage 2**: Compute VRP as $\text{VIX}_t^2 - \hat{\text{RV}}^{\mathbb{P}}_{t+h}$.
3. **Stage 3**: Use the VRP (and its components) as input features for a second-stage model that predicts returns or constructs a trading signal.

The ML Angle on VRP

ML enters the VRP framework in two places:

- **Better $\mathbb{P}$-measure forecasts**: tree ensembles or neural networks can produce more accurate RV forecasts than HAR, yielding a cleaner VRP estimate (Fouhy, 2024).
- **VRP as a feature**: VRP (and its decompositions) enters the feature set for downstream prediction tasks. Chapter 10 details how to engineer VRP-based features, and Chapter 18 shows how to build a VRP-based trading strategy.

Figure 9.3 illustrates the full pipeline from RV forecast to trading decision.

9.8.2 VRP as a Trading Signal

The most direct use of VRP is as a signal for a delta-hedged volatility strategy. The idea is simple:

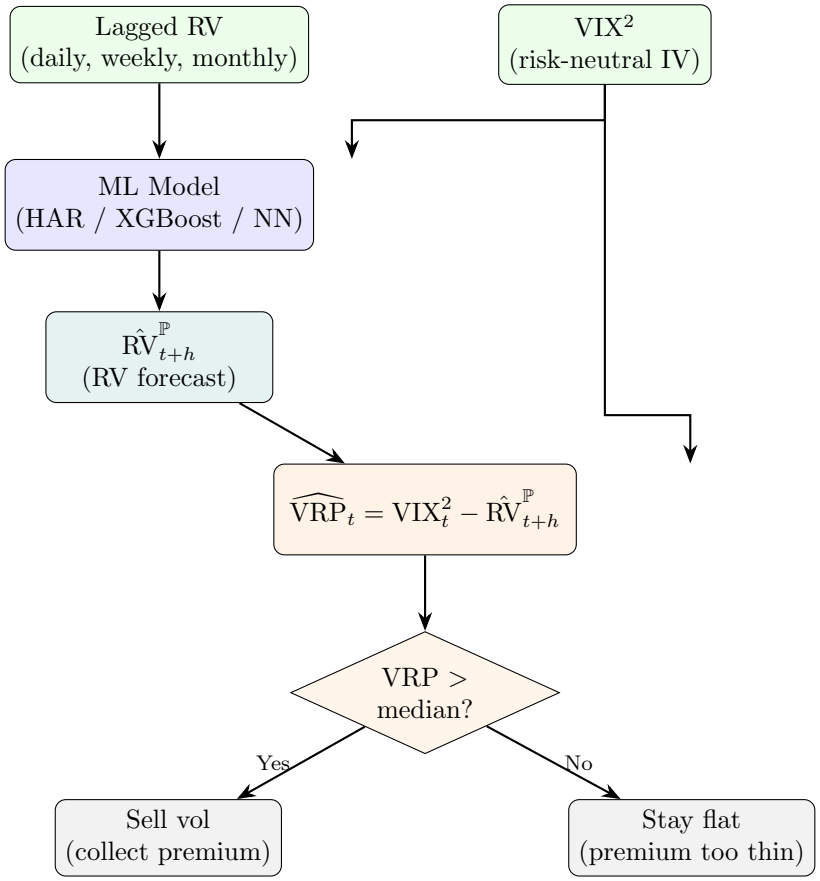


Figure 9.3: The VRP trading pipeline. Your ML model forecasts realized variance (left path), which is compared to the options market's implied variance (VIX-squared) to compute the VRP signal. The signal drives the trading decision: sell volatility when the premium is rich, stay flat when it is thin.

- When VRP is high (implied $\gg$ realized): *sell* volatility (sell options, collect the premium). The options market is overpricing future vol, so you profit as realized vol comes in below implied.
- When VRP is low or negative (implied $\approx$ or $<$ realized): *buy* volatility or stay flat. The premium is thin or inverted, and the risk-reward of selling options is poor.

A concrete implementation: sell a 30-day delta-hedged straddle on the S&P 500 when the ex-ante VRP exceeds its trailing median, and flatten the position otherwise. Delta-hedging removes directional exposure, so the trade profits or loses based purely on realized vs. implied variance. This is Project 5 in Chapter 18.

Short Volatility Carries Tail Risk

Selling volatility is the most reliable way to harvest the VRP, but it is inherently a strategy with negative skewness: small, frequent gains and rare, large losses. VRP harvesting strategies lost 20–40% in the October 2008 crash and the March 2020 COVID sell-off. ML-based regime detection (Chapter 10) and position sizing are essential to manage this tail risk.

Summary

- The **variance risk premium** (VRP) is the difference between risk-neutral expected variance ($\mathbb{Q}$, measured by VIX-squared) and physical expected variance ($\mathbb{P}$, measured by realized variance or a model-based forecast).
- VRP is **positive on average**: VIX overstates subsequent realized vol about 85% of the time (Carr and Wu, 2009). This gap is compensation for bearing volatility risk.
- The standard operational measure is $\widehat{\text{VRP}}_t = (\text{VIX}_t/100)^2 - \hat{\text{RV}}^{\mathbb{P}}_{t,t+h}$, where $\hat{\text{RV}}$ is either backward-looking RV or a model forecast (e.g., HAR).
- **VRP exists** because of risk aversion, demand for crash protection, and equilibrium pricing of uncertainty about future growth (Drechsler and Yaron, 2011).
- **VRP predicts returns**: Bollerslev et al. (2009b) show that VRP predicts quarterly equity excess returns with R^2 of 5–10%, beating the dividend yield as a return predictor.
- **VRP predicts volatility**: high VRP signals that realized vol will likely rise toward implied vol through mean reversion.
- **Decomposition 1** (Bekaert and Hoerova, 2014): VIX^2 = expected variance + VRP. Return predictability lives in the VRP component, not the expected-variance component.
- **Decomposition 2** (Bollerslev and Todorov, 2011): VRP = normal-times premium + jump-tail premium. Most time-variation in VRP comes from the tail component (crash fear).
- **VVIX** measures the implied volatility of VIX itself (vol-of-vol). It captures uncertainty about the future level of uncertainty, a distinct risk factor.
- **Realized vol-of-vol** is computed by applying the RV estimator from Chapter 2 to VIX returns. VIX jumps are a distinct risk factor beyond the VIX level.
- The persistent gap between VIX and realized vol (Figure 9.2) is the visual signature of the VRP. It widens in crises and narrows in calm markets but rarely flips negative.
- **ML approaches** (Fouhy, 2024): tree ensembles can improve the $\mathbb{P}$-measure forecast, yielding a cleaner VRP estimate. VRP is also a powerful feature for downstream ML models (Chapter 10).
- **VRP as a trading signal**: sell volatility when VRP is high, flatten when it is low. This is a direct, implementable strategy (Project 5, Chapter 18), but it carries significant tail risk.
- Ex-ante VRP (using a forecast for $\mathbb{P}$) is available in real time and can be traded. Ex-post

VRP (using actual future RV) is only known after the fact. Always state which version you are using.

- The choice of $\mathbb{P}$-measure proxy (backward-looking RV, GARCH, HAR, XGBoost) materially affects VRP estimates and their predictive power (Bekaert and Hoerova, 2014).

Table 9.1: Key results referenced in this chapter.

Paper	**Result**	**Relevance**
Bollerslev et al. (2009b)	VRP predicts quarterly equity excess returns with R^2 of 5–10%, beating the dividend yield.	Establishes VRP as a top return predictor; motivates its use as an ML feature.
Drechsler and Yaron (2011)	Long-run risk model with Epstein-Zin preferences generates a positive VRP in equilibrium through time-varying economic uncertainty.	Provides the theoretical foundation for why VRP exists.
Bekaert and Hoerova (2014)	Decompose VIX^2 into expected variance and VRP; return predictability resides in the VRP component, not in expected variance.	Clarifies that risk aversion, not objective uncertainty, drives return predictability.
Bollerslev and Todorov (2011)	Decompose VRP into normal-times and jump-tail components; the tail premium drives most time-variation.	Most of VRP is crash-fear compensation, not day-to-day variance risk.
Carr and Wu (2009)	Document that VIX exceeds subsequent realized vol ∼85% of the time; provide a model-free framework for variance risk premia across assets.	Quantifies the frequency and magnitude of the VRP.
Fouhy (2024)	Hierarchical XGBoost for VIX-to-RV-to-VRP pipeline; ML-based $\mathbb{P}$-measure forecasts improve VRP estimation.	Connects VRP to ML methods used in Chapters 10–18.

Part IV

ML Methods for Volatility

Chapter 10

Feature Engineering for Volatility

Application

This chapter is the bridge between volatility theory (Chapters 1–9) and ML modeling (Chapters 11–14). Every feature described here is a column in the input matrix that tree models, neural networks, and hybrid methods will consume. The feature set you choose matters more than the model you choose. Projects 1–5 all draw their inputs from this catalog.

We have spent nine chapters building a toolkit of volatility estimators, noise corrections, jump decompositions, parametric models, long-memory specifications, options-implied measures, and risk premia. Now we pivot from *measuring* volatility to *predicting* it with machine learning. The raw material for any ML model is its feature matrix $\mathbf{X} \in \mathbb{R}^{T \times p}$, where each row is a date and each column is a predictor. This chapter catalogs the features that the literature and Kaggle competitions have found useful, organizes them into families, and flags the pitfalls that make or break a forecasting pipeline.

10.1 Feature Taxonomy

Before diving into individual features, it helps to see the full landscape. Figure 10.1 groups every feature family into five branches. You do not need all of them for every project; the tree is a menu, not a checklist.

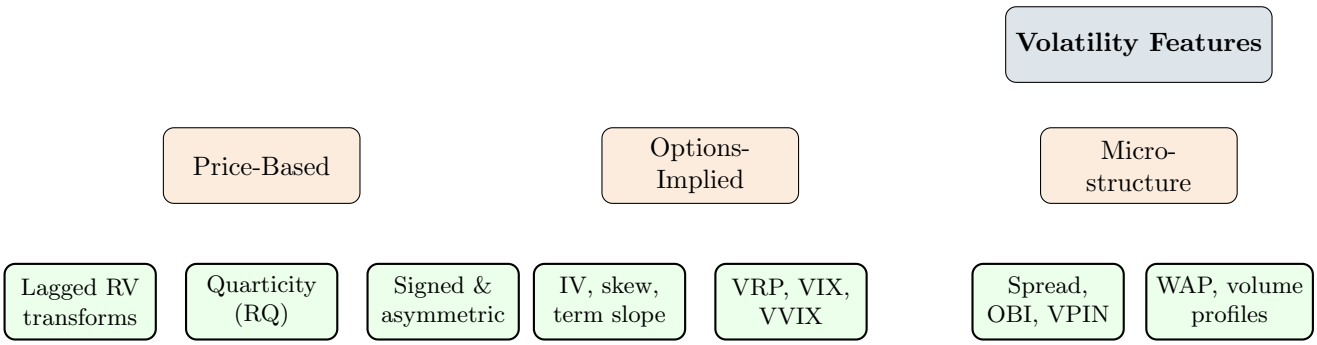


Figure 10.1: Feature taxonomy for volatility forecasting. Price-based features (left) are available for any asset with intraday data. Options-implied features require a liquid derivatives market. Microstructure features need tick-level LOB data. Cross-asset and calendar/sentiment features supplement any core set.

Figure 10.2 shows how raw market data flows through the feature engineering pipeline into the model. Every feature must pass through the temporal alignment gate: it can only use data available at or before time t to predict the target at time $t+1$.

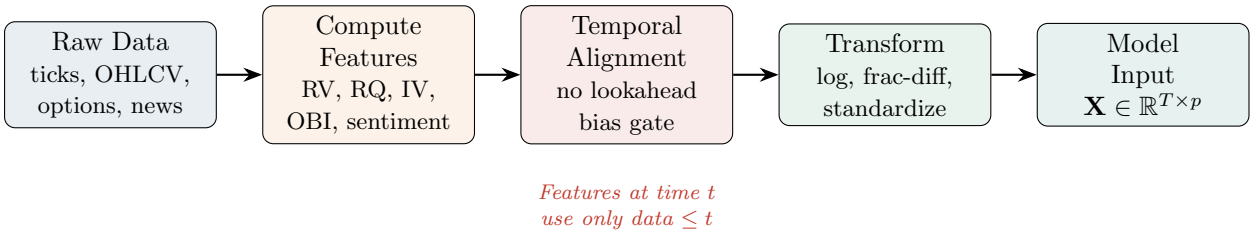


Figure 10.2: Feature engineering pipeline. Raw market data is processed into features, passed through a temporal alignment gate to prevent lookahead bias, transformed for stationarity, and assembled into the model input matrix $\mathbf{X}$.

10.2 Triple Expansion: Level, Change, Z-Score

Before cataloging individual features, we introduce a systematic expansion technique that applies to every continuous feature in the catalog. For any base quantity x_t, construct three variants:

Triple Expansion

Given a continuous feature x_t and a lookback window w (typically 22 trading days), the **triple expansion** produces:

$$x_t^{\text{level}} = x_t, \tag{10.1}$$

$$x_t^{\text{change}} = x_t - x_{t-k}, \quad k \in \{1, 5\}, \tag{10.2}$$

$$x_t^{\text{z-score}} = \frac{x_t - \bar{x}_{t,w}}{\hat{\sigma}_{x,t,w}}, \tag{10.3}$$

where $\bar{x}_{t,w} = \frac{1}{w}\sum_{i=0}^{w-1} x_{t-i}$ is the trailing mean and $\hat{\sigma}_{x,t,w}$ is the trailing standard deviation over the same window.

- x_t^{level}: the raw value of the feature, capturing the current **state**.
- x_t^{change}: the first difference over k periods, capturing **momentum** (is the feature rising or falling?).
- $x_t^{\text{z-score}}$: the standardized deviation from the recent mean, capturing **anomaly** (is the current value unusual?).

In Plain English

Consider the bid-ask spread. Its level tells you how wide the spread is right now. Its change tells you whether the spread is widening (deteriorating liquidity) or narrowing (improving liquidity). Its z-score tells you whether today's spread is abnormally wide relative to its own recent history. A spread of 5 cents means very different things for a stock that typically trades at 3 cents (z-score ≈ 2, alarm) versus one that typically trades at 6 cents (z-score ≈ -1, calm). Each variant captures genuinely different information.

Which features get expanded. The triple expansion applies to *continuous* features: RV, RQ, bid-ask spread, OBI, implied volatility, VRP, volume ratios, and any numeric quantity with a meaningful scale. It does *not* apply to categorical or binary features such as FOMC dummies,

day-of-week indicators, or earnings announcement flags. These are already discrete and cannot be meaningfully z-scored.

Multicollinearity: a non-issue for trees. In a linear regression, including level, change, and z-score of the same base feature would introduce severe multicollinearity (the three are highly correlated). This would inflate standard errors and produce unstable coefficient estimates. For tree-based models (random forests, gradient boosting), multicollinearity is harmless: trees select the most informative split at each node and are unaffected by correlated alternatives. Neural networks are also largely robust to correlated inputs, especially with regularization. The triple expansion is therefore a "free" way to enrich the feature set for non-linear models.

Worked Example:

Triple expansion of bid-ask spread Suppose the time-weighted average bid-ask spread over the observation window is:

$$s_t = 0.08\%, \qquad s_{t-1} = 0.06\%, \qquad \bar{s}_{t,22} = 0.05\%, \qquad \hat{\sigma}_{s,t,22} = 0.015\%.$$

The three expanded features are:

- **Level:** $s_t^{\text{level}} = 0.08\%$ (spread is 8 bps, moderately wide).
- **Change:** $s_t^{\text{change}} = 0.08\% - 0.06\% = +0.02\%$ (spread widened by 2 bps overnight – liquidity is deteriorating).
- **Z-score:** $s_t^{\text{z-score}} = (0.08 - 0.05)/0.015 = 2.0$ (spread is 2 standard deviations above its 22-day mean – unusually wide for this stock).

The level alone says "moderately wide." The change says "getting worse." The z-score says "abnormally wide for this stock." A tree model can split on any of the three depending on which is most informative for the current regime.

Why This Matters

Apply the triple expansion systematically to every continuous feature in your pipeline. This multiplies your feature count by roughly 3× but gives tree models three distinct "views" of each quantity. In the vol-project-ref feature composition (Chapter 8), the triple expansion is identified as a core engineering principle: it captures state, direction, and unusualness in a single pass, and it is the primary reason feature counts grow from ∼20 base features to ∼80 model inputs.

10.3 Lagged RV Transforms

The single most predictive feature for tomorrow's realized volatility is yesterday's realized volatility. The HAR model (Chapter 6) already exploits this by using three horizons that smooth RV over different lookback windows:

$$\text{RV}_t^{(d)} = \text{RV}_t, \tag{10.4}$$

$$\text{RV}_t^{(w)} = \frac{1}{5}\sum_{i=0}^{4} \text{RV}_{t-i}, \tag{10.5}$$

$$\text{RV}_t^{(m)} = \frac{1}{22}\sum_{i=0}^{21} \text{RV}_{t-i}\,. \tag{10.6}$$

- $\text{RV}_t^{(d)}$: daily RV, the single most recent observation.

- $\mathrm{RV}_t^{(w)}$: weekly RV, the average of the past 5 trading days.
- $\mathrm{RV}_t^{(m)}$: monthly RV, the average of the past 22 trading days.

In Plain English

These three features capture volatility at different "speeds." Daily RV tells you what happened today. Weekly RV smooths out day-to-day noise to reveal the recent trend. Monthly RV captures the slow-moving baseline level. By including all three, you give the model a short-term, medium-term, and long-term view of volatility, which is why HAR works so well despite being a simple linear model.

Why This Matters

These three features are the columns your HAR baseline consumes. Every ML model you build should include $\log \mathrm{RV}_t^{(d)}$, $\log \mathrm{RV}_t^{(w)}$, and $\log \mathrm{RV}_t^{(m)}$ as core inputs. The goal of fancier feature engineering is to add value on top of these, not to replace them. If an ML model cannot beat HAR using only these three features, the extra complexity is not justified.

These three variables alone explain roughly 40–60% of the variation in next-day log-RV for equity indices. For ML models, you should include all three, but also consider the following transforms.

Log-RV. Working with $\log \mathrm{RV}_t$ rather than RV_t has two benefits:

- It stabilizes the variance of the target (variance of variance is highly skewed).
- It makes the distribution closer to Gaussian, which helps linear baselines and loss-function calibration.

Most HAR papers estimate the model in logs; you should do the same with your feature columns.

Square-root RV. An intermediate transform: $\sqrt{\mathrm{RV}_t}$ approximates realized *volatility* (standard deviation), which is more interpretable than variance and less compressed than log.

Ratio features. The ratio $\mathrm{RV}_t^{(d)} / \mathrm{RV}_t^{(m)}$ captures how today's volatility compares to its recent average. Values above 1 indicate a local spike; values well below 1 indicate calm. This is a simple but effective regime indicator.

Worked Example:

Building HAR-style features Suppose the last 22 daily RV values (most recent first) are:

$$\mathrm{RV}_t = 1.8 \times 10^{-4}, \quad \mathrm{RV}_{t-1} = 1.5 \times 10^{-4}, \quad \ldots$$

and you compute $\mathrm{RV}_t^{(w)} = 1.6 \times 10^{-4}$, $\mathrm{RV}_t^{(m)} = 1.2 \times 10^{-4}$.
Your feature row for date t contains:

- $\log \mathrm{RV}_t^{(d)} = \log(1.8 \times 10^{-4}) \approx -8.62$
- $\log \mathrm{RV}_t^{(w)} = \log(1.6 \times 10^{-4}) \approx -8.74$
- $\log \mathrm{RV}_t^{(m)} = \log(1.2 \times 10^{-4}) \approx -9.03$
- Ratio: $\mathrm{RV}_t^{(d)} / \mathrm{RV}_t^{(m)} = 1.50$ (volatility currently 50% above its monthly average)

The ratio signals an elevated-volatility regime.

10.4 Realized Quarticity and the HARQ Feature

You learned in Chapter 2 that realized variance is a noisy estimator: its sampling error depends on the fourth moment of returns. The realized quarticity (RQ) quantifies exactly this noise.

Realized Quarticity

Given n intraday returns $r_{t,i}$ on day t,

$$\mathrm{RQ}_t = \frac{n}{3} \sum_{i=1}^{n} r_{t,i}^4. \tag{10.7}$$

Under standard diffusion assumptions, $\mathrm{RQ}_t \xrightarrow{p} \int_0^1 \sigma_s^4 \, ds$ as $n \to \infty$.

- n is the number of intraday returns (e.g., 78 for 5-minute bars in a 6.5-hour session).
- $r_{t,i}^4$ is the fourth power of each intraday return; this heavily up-weights large moves.
- The factor $n/3$ provides the correct asymptotic scaling.

In Plain English

Realized quarticity measures how "wild" the intraday price path was. By raising each return to the fourth power, extreme moves dominate the sum. A day with one large intraday swing followed by calm will have high RQ even if overall RV is moderate. In short, RQ tells you how much you should trust today's RV number: high RQ means the RV estimate is noisy and unreliable.

Why This Matters

Include $\sqrt{\mathrm{RQ}_t}$ as a standalone feature column. It serves double duty: (1) tree models can learn to split on it, effectively down-weighting noisy RV days the way HARQ does analytically, and (2) it provides a data-quality signal that no other feature captures. On days where RQ is elevated, your forecast should lean more on the stable weekly and monthly averages.

Why RQ matters for feature engineering. The asymptotic variance of RV_t is proportional to RQ_t. When RQ is high, today's RV number is unreliable. Bollerslev et al. (2016b) exploit this insight in the HARQ model: they interact the daily RV component with $\sqrt{\mathrm{RQ}_t}$, allowing the model to down-weight noisy days.

The HARQ Feature

In the HARQ model (Chapter 6), the coefficient on $\mathrm{RV}_t^{(d)}$ is made state-dependent:

$$\beta_{d,t} = \beta_d + \beta_{dQ} \sqrt{\mathrm{RQ}_t}.$$

The product $\mathrm{RV}_t^{(d)} \cdot \sqrt{\mathrm{RQ}_t}$ is itself a feature. When RQ_t is large, the interaction term pulls the effective weight on daily RV toward zero, letting the more stable weekly and monthly averages dominate the forecast. For ML models, include both $\mathrm{RV}_t^{(d)}$ and $\sqrt{\mathrm{RQ}_t}$ as separate columns; the tree or network can learn the interaction without you specifying it.

Why This Matters

The HARQ interaction feature is the single most important extension beyond baseline HAR for your project. Bollerslev et al. (2016b) show it improves QLIKE by 5–15% across equity indices. For your ML pipeline, you do not need to hard-code the interaction: include $\text{RV}_t^{(d)}$, $\text{RV}_t^{(w)}$, $\text{RV}_t^{(m)}$, and $\sqrt{\text{RQ}_t}$ as four separate columns and let gradient-boosted trees or neural networks discover the interaction automatically.

Worked Example:

Computing RQ With $n = 78$ five-minute returns and $\sum r_{t,i}^4 = 5.2 \times 10^{-12}$:

$$\text{RQ}_t = \frac{78}{3} \times 5.2 \times 10^{-12} = 26 \times 5.2 \times 10^{-12} = 1.35 \times 10^{-10}.$$

Compare this to a calm day where $\sum r_{t,i}^4 = 8.0 \times 10^{-13}$:

$$\text{RQ}_t^{\text{calm}} = 26 \times 8.0 \times 10^{-13} = 2.08 \times 10^{-11}.$$

The noisy day has RQ roughly 6.5 times larger, telling you its RV reading is much less precise.

10.5 Signed and Asymmetric Features

Volatility is not symmetric. Negative returns drive future volatility up more than positive returns of the same magnitude (the leverage effect you met in Chapter 5). Several features capture this asymmetry directly from high-frequency data.

Realized semivariances. To separate the contribution of upward and downward price moves to total volatility, we split intraday returns by sign:

$$\text{RV}_t^+ = \sum_{i=1}^{n} r_{t,i}^2 \, \mathbf{1}(r_{t,i} > 0), \tag{10.8}$$

$$\text{RV}_t^- = \sum_{i=1}^{n} r_{t,i}^2 \, \mathbf{1}(r_{t,i} < 0). \tag{10.9}$$

- RV_t^+: realized semivariance from positive returns ("good volatility").
- RV_t^-: realized semivariance from negative returns ("bad volatility").
- $\mathbf{1}(\cdot)$: indicator function selecting returns of the specified sign.

In Plain English

Total RV treats a 2% up-move and a 2% down-move identically. Semivariances break this symmetry. RV^- isolates the volatility coming from declines, which is the part that matters most for risk management and future vol prediction. RV^+ captures the "calm" upside moves. Keeping them separate lets a model learn that a day dominated by selling pressure has different forecasting implications than a day dominated by buying.

Why This Matters

Include RV_t^- and RV_t^+ (and their weekly/monthly averages) as separate feature columns. The SHAR baseline already uses them, so your ML model needs them to at least match SHAR performance. Empirically, RV_t^- carries roughly twice the predictive weight of RV_t^+,

so if you must prune features, keep RV_t^- and drop RV_t^+ before dropping other features.

By construction, $\text{RV}_t = \text{RV}_t^+ + \text{RV}_t^-$ (ignoring zero returns). The SHAR model in Chapter 6 uses RV_t^+ and RV_t^- as separate regressors, allowing different persistence for up-moves and down-moves.

Patton and Sheppard (2015b)

Replacing total RV with the signed pair $(\text{RV}_t^+, \text{RV}_t^-)$ in the HAR framework reduces out-of-sample QLIKE loss by 3–8% for equity index volatility. The coefficient on RV_t^- is roughly twice that on RV_t^+, confirming the leverage effect at intraday frequency.

Signed jumps. From Chapter 4, the jump component is $J_t = \max(\text{RV}_t - \text{BPV}_t, 0)$. You can further decompose:

$$J_t^+ = \sum_i r_{t,i}^2 \, \mathbf{1}(r_{t,i} > 0, \ |r_{t,i}| > \theta_t), \qquad J_t^- = \sum_i r_{t,i}^2 \, \mathbf{1}(r_{t,i} < 0, \ |r_{t,i}| > \theta_t), \tag{10.10}$$

where θ_t is an intraday jump threshold (e.g., from Lee-Mykland, Chapter 4).

- J_t^+: squared returns from large positive moves exceeding the jump threshold.
- J_t^-: squared returns from large negative moves exceeding the jump threshold.
- θ_t: intraday threshold distinguishing jumps from continuous price variation.

In Plain English

Signed jumps isolate the extreme tail events and ask: did the big intraday moves come from sudden rallies or sudden crashes? A day dominated by negative jumps (a flash crash, a surprise rate hike) behaves very differently from a day dominated by positive jumps (a short squeeze, a surprise earnings beat). Splitting jumps by sign gives the model a finer-grained view of tail risk than total jump variation alone.

Why This Matters

Add J_t^- as a feature column. Negative jumps are substantially more predictive of future volatility than positive jumps. In HAR-CJ extensions, including J_t^- separately improves QLIKE by 1–3%. For your ML model, this asymmetric jump signal complements the semi-variance features and is especially valuable during turbulent periods when jump activity clusters.

Realized semicovariances. Bollerslev et al. (2020) extend semivariances to the multivariate case, computing covariances conditional on the sign of the market return. For single-asset forecasting, the key insight is that the downside component of any cross-asset covariance feature (Chapter 15) carries more information than the upside component.

Why negatives matter more

When prices fall, leveraged investors face margin calls, hedging demand spikes, and correlations rise. The mechanism is mechanical, not behavioral: a short gamma position that loses money forces the holder to sell more, amplifying volatility. Features that isolate this downside channel capture a structural driver, not a statistical accident.

Why This Matters

The asymmetry between positive and negative moves is one of the strongest and most robust findings in the volatility literature. For your project, always compute signed versions of your features: RV_t^- vs. RV_t^+, J_t^- vs. J_t^+, and (if using multivariate features) downside semicovariances. This costs you nothing in terms of data requirements but gives tree models a clean split variable to identify leverage-effect regimes.

10.6 Higher Moments

Realized skewness and kurtosis measure the shape of the intraday return distribution. They are computed from higher powers of intraday returns:

$$\text{RSkew}_t = \frac{\frac{1}{n}\sum_{i=1}^{n} r_{t,i}^3}{\left(\frac{1}{n}\sum_{i=1}^{n} r_{t,i}^2\right)^{3/2}}, \tag{10.11}$$

$$\text{RKurt}_t = \frac{\frac{1}{n}\sum_{i=1}^{n} r_{t,i}^4}{\left(\frac{1}{n}\sum_{i=1}^{n} r_{t,i}^2\right)^{2}}. \tag{10.12}$$

- RSkew_t: realized skewness, measuring asymmetry of intraday returns (negative = more large down-moves).
- RKurt_t: realized kurtosis, measuring tail heaviness (higher = more extreme moves relative to typical).
- Both are normalized by powers of RV so they are scale-free.

In Plain English

Realized skewness tells you whether today's intraday price action was lopsided: a very negative value means the big moves were predominantly downward. Realized kurtosis tells you whether the price path had fat tails: a high value means there were extreme intraday swings relative to the typical move size. Together they describe the "shape" of intraday price behavior beyond just its level (RV) and sign (semivariances).

Why This Matters

Include realized skewness and kurtosis in your initial feature set, but expect them to rank low in importance. Their main value is as regime-conditioning variables: when realized skewness is very negative, the model may learn that volatility persistence changes. Tree models can use them as split variables to identify stress regimes even if their linear predictive power is weak.

Modest Predictive Power

Realized skewness and kurtosis are noisy estimators and have shown only modest incremental forecasting power for next-day RV in most studies. Include them in your initial feature set, but do not be surprised if tree-based importance scores rank them low. Their main use is as conditioning variables: when skewness is very negative, the volatility process may behave differently (regime-dependent dynamics).

10.7 Microstructure and Limit Order Book Features

If you have tick-level or LOB snapshot data (common in Kaggle competitions like Optiver's Realized Volatility Prediction), you unlock a family of features that daily or 5-minute data cannot provide.

Bid-ask spread. The quoted spread $s_t = p_t^{\text{ask}} - p_t^{\text{bid}}$ is a proxy for liquidity and informed-trading intensity. Wider spreads predict higher near-term volatility. Use the time-weighted average spread across the observation window.

Order book imbalance (OBI). The balance of resting orders on each side of the book reveals directional pressure. OBI formalizes this:

$$\text{OBI}_t = \frac{V_t^{\text{bid}} - V_t^{\text{ask}}}{V_t^{\text{bid}} + V_t^{\text{ask}}}, \tag{10.13}$$

- V_t^{bid}: total visible volume resting at the best bid price.
- V_t^{ask}: total visible volume resting at the best ask price.
- OBI ranges from -1 (all volume on the ask side) to $+1$ (all on the bid side).

In Plain English

OBI measures who is more eager to trade. When there is much more volume resting on the bid side than the ask, buyers are providing liquidity and the price is likely to tick up (or at least not fall). When ask-side volume dominates, sellers are in control. Extreme imbalance in either direction signals one-sided pressure that tends to resolve through a price move, and price moves create volatility.

Why This Matters

If you are working with tick-level data (e.g., Optiver-style competitions or LOB snapshots), OBI is one of the strongest short-horizon features. Compute time-weighted OBI over your observation window and include it as a feature column. The absolute value $|\text{OBI}_t|$ is often more useful than signed OBI for volatility forecasting, since extreme imbalance in either direction predicts elevated vol.

Weighted average price (WAP) log returns. The WAP blends bid and ask prices, weighting each side by the opposite side's depth to reflect where the "true" price likely sits:

$$\text{WAP}_t = \frac{p_t^{\text{bid}} \cdot V_t^{\text{ask}} + p_t^{\text{ask}} \cdot V_t^{\text{bid}}}{V_t^{\text{bid}} + V_t^{\text{ask}}}. \tag{10.14}$$

- $p_t^{\text{bid}}, p_t^{\text{ask}}$: best bid and ask prices.
- $V_t^{\text{bid}}, V_t^{\text{ask}}$: volume at the best bid and ask.
- The cross-weighting pulls WAP toward the side with less volume (where the next fill is more likely).

In Plain English

The simple midprice treats both sides of the book equally, but if there are 10,000 shares on the bid and only 100 on the ask, the next trade is far more likely to happen at the ask. WAP accounts for this asymmetry by weighting each price by the opposite side's volume, pulling the estimated fair value toward the thinner side of the book. Returns computed from WAP are less contaminated by bid-ask bounce.

Why This Matters

When computing RV from LOB data, use WAP-based log returns rather than midprice returns as your raw input. This reduces microstructure noise at the source, before any noise-robust estimator is applied, giving you a cleaner RV target and cleaner lagged-RV features. Top Optiver competition solutions all used WAP returns for exactly this reason.

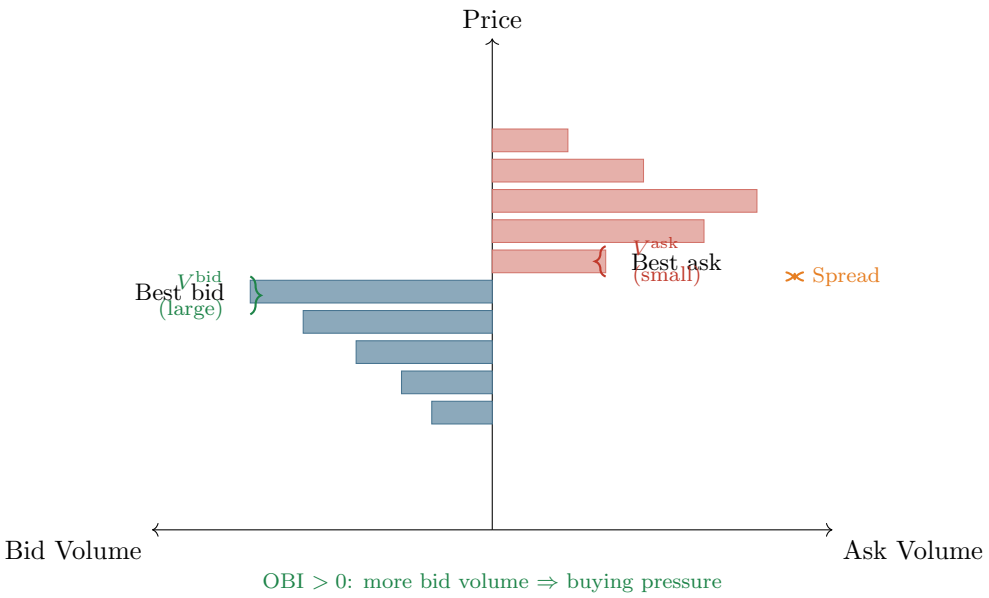


Figure 10.3: Limit order book snapshot with feature annotations. The spread (orange) is the gap between best bid and best ask. OBI compares bid volume (blue, left) to ask volume (red, right). In this snapshot, OBI is positive (more resting bid volume), suggesting net buying pressure.

Price acceleration. Beyond first-order returns, the second difference of log-prices captures changes in momentum within the observation window:

$$a_{t,i} = \Delta \log(\text{WAP}_{t,i}) - \Delta \log(\text{WAP}_{t,i-1}). \tag{10.15}$$

- $\Delta \log(\text{WAP}_{t,i})$: the log return between consecutive WAP observations at times $i-1$ and i.
- $a_{t,i}$: the change in the log return, analogous to acceleration in physics.

In Plain English

If the price was falling and then starts falling faster, that is negative acceleration. If the price was falling and then slows down, that is positive acceleration (deceleration). Aggregating the magnitude of these acceleration values over a window captures how "jerky" the price path is. A smooth trending market has low acceleration; a choppy market with rapid reversals has high acceleration. This jerkiness is a leading indicator of near-term volatility.

Why This Matters

For tick-level or LOB-based projects, include the standard deviation or sum of squared accelerations over the observation window as a feature column. This was one of the highest-importance features in top Optiver competition solutions. It captures information orthogonal to RV (which uses only first differences), providing the model with a signal

about momentum instability that raw RV misses.

Volume profiles. Aggregate volume into time buckets (e.g., 2-minute bins within a 10-minute window). The ratio of volume in the last bucket to the first captures whether activity is accelerating or decelerating. Volume acceleration tends to precede volatility spikes.

10.7.1 VPIN and Kyle's Lambda

Two microstructure features capture the intensity of *informed trading* rather than just the state of the order book. Both originate from market microstructure theory and measure how much private information is currently flowing into the market.

VPIN (Volume-Synchronized Probability of Informed Trading). Standard time-based sampling mixes high-activity and low-activity periods. Easley et al. (2012) propose an alternative: partition the trading day into **volume buckets**, each containing the same total volume V_{bucket}. Within each bucket, classify trades as buyer- or seller-initiated (typically using the tick rule or bulk volume classification), and compute the absolute imbalance.

The VPIN algorithm proceeds in three steps. First, partition trades into n equal-volume buckets, where each bucket contains exactly $V_{\text{bucket}} = V_{\text{day}}/n$ shares:

$$\text{VPIN}_t = \frac{1}{n}\sum_{\tau=1}^{n}\frac{|V_\tau^B - V_\tau^S|}{V_{\text{bucket}}}, \tag{10.16}$$

- V_τ^B: volume classified as buyer-initiated in bucket τ.
- V_τ^S: volume classified as seller-initiated in bucket τ ($V_\tau^B + V_\tau^S = V_{\text{bucket}}$).
- n: number of volume buckets per estimation window (typically 50).
- VPIN ranges from 0 (perfectly balanced flow) to 1 (completely one-sided flow).

In Plain English

VPIN asks: "In each chunk of trading volume, how one-sided was the flow?" When informed traders are active, they trade aggressively on one side, creating persistent imbalances. Uninformed traders, by contrast, are roughly equally likely to buy or sell, so their flow approximately cancels out. High VPIN therefore signals that informed traders are dominating the flow, which tends to precede large price moves and elevated volatility. The key innovation is measuring in *volume time* rather than clock time: a bucket fills quickly during high-activity periods and slowly during calm periods, automatically adapting the sampling rate to market conditions.

Why This Matters

Include VPIN as a feature column if you have trade-level data with directional classification. Use $n = 50$ volume buckets per day as a starting point. VPIN is most valuable at the intraday and 1-day horizons, where it captures the arrival of informed order flow before it fully resolves into price moves. At longer horizons, VPIN's signal decays because the information has already been incorporated into prices.

Kyle's lambda (λ): price impact per unit of order flow. Kyle (1985) shows that in a market with a single informed trader and competitive market makers, the equilibrium pricing rule is linear in aggregate order flow. The slope of this relationship, **Kyle's lambda**, measures the market's **price impact** per unit of signed volume:

$$\Delta p_t = \alpha + \underbrace{\lambda}_{\text{price impact}} \cdot \underbrace{(\text{signed volume})_t}_{\text{buy vol}-\text{sell vol}} + \varepsilon_t. \tag{10.17}$$

- Δp_t: change in the midprice over interval t.
- λ: Kyle's lambda, the slope coefficient. A higher λ means each unit of net order flow moves the price more (the market is less liquid or more informationally sensitive).
- Signed volume: the difference between buyer-initiated and seller-initiated volume (positive = net buying).

In practice, estimate λ by regressing midprice changes on signed volume over a rolling window (e.g., 60 five-minute intervals):

$$\Delta p_{t,i} = \hat{\alpha} + \hat{\lambda}_t \cdot (V^B_{t,i} - V^S_{t,i}) + \hat{\varepsilon}_{t,i}, \qquad i = 1, \ldots, m, \tag{10.18}$$

where m is the number of intraday intervals in the rolling window.

In Plain English

Kyle's lambda answers: "How many basis points does the price move for every million dollars of net order flow?" When λ is high, even modest buying or selling pressure moves prices substantially – the market is "thin" or carrying a lot of private information. When λ is low, the market can absorb large orders without significant price impact – it is deep and liquid. Rising λ predicts higher near-term volatility because it signals that order flow is becoming more informative (or the market is becoming less resilient), both of which lead to larger price swings.

Why This Matters

Estimate $\hat{\lambda}_t$ on a rolling 60-interval window using 5-minute data and include it as a feature column. $\hat{\lambda}_t$ captures a different dimension of market quality than the bid-ask spread: the spread measures the cost of a small trade, while λ measures the cost of a *large* trade. During stress periods, λ spikes before the spread widens, making it a leading indicator. For your project, the triple expansion of Kyle's lambda (level, change, z-score) provides three informative feature columns for tree models.

Microstructure features are asset-specific

Features like OBI and spread depend heavily on the market's microstructure (tick size, maker/taker fees, minimum lot sizes). A feature that works for liquid US equities may be meaningless for an illiquid commodity future. Always recalibrate when switching asset classes.

10.7.2 Microprice and Volume Features

The simple midprice $(P_{\text{bid}} + P_{\text{ask}})/2$ assigns equal weight to both sides of the book regardless of depth. The **microprice** (Cartea et al., 2015) corrects for order-book imbalance by shifting the estimated fair value toward the thinner side of the book:

$$S^*_t = \frac{V^{\text{ask}}_t \cdot P^{\text{bid}}_t + V^{\text{bid}}_t \cdot P^{\text{ask}}_t}{V^{\text{bid}}_t + V^{\text{ask}}_t}. \tag{10.19}$$

- $P^{\text{bid}}_t, P^{\text{ask}}_t$: best bid and ask prices.
- $V^{\text{bid}}_t, V^{\text{ask}}_t$: total visible volume at the best bid and ask.

- When bid volume dominates, the microprice shifts toward the ask (fair value is higher).

In Plain English

The microprice is essentially the same idea as WAP. Both weight the bid price by ask volume and vice versa. The key insight is that the side of the book with more resting volume is "cheaper" to trade against, so the true fair value sits closer to the other side. The microprice is a better estimate of where the next trade will actually occur than the simple midprice.

Why This Matters

For microstructure-level projects, the deviation $|S^* - S_{\text{mid}}|$ between microprice and midprice is itself a useful feature: it measures how asymmetric the book is. Large deviations indicate one-sided pressure that often precedes short-term volatility. Use microprice returns rather than midprice returns as the basis for computing short-horizon RV targets.

Volume features. Daily trading volume is highly correlated with daily volatility. More usefully for forecasting, volume is *persistent*: high-volume days tend to cluster, just like high-vol days. This means volume features can serve as leading indicators:

- $\log(\text{volume}/\text{MA}_{20}\text{volume})$: normalized volume; values above zero indicate above-average activity.
- Volume acceleration: change in log-volume over the past 3 days.
- Microprice–midprice deviation: $|S^* - S_{\text{mid}}|$ as a measure of book imbalance pressure.

Volume Predicts Vol

Abnormal volume today predicts elevated RV tomorrow, even after controlling for lagged RV. The intuition: volume spikes reflect new information arriving (earnings whispers, large block trades, portfolio rebalancing) that has not yet fully resolved into price moves. Include at least one volume feature in any feature set.

Why This Matters

Add $\log(\text{volume}/\text{MA}_{20}\text{volume})$ as a feature column. It is easy to compute, available for any traded asset, and provides incremental predictive power beyond lagged RV. For microstructure-level projects, also include volume acceleration and the microprice-midprice deviation. These volume-based features are most valuable in the 1–2 day forecasting horizon where information arrival is actively resolving into price moves.

10.8 Options-Implied Features

Options prices embed the market's forward-looking expectation of volatility (Chapter 8) and the variance risk premium the market charges for bearing that risk (Chapter 9). This makes options-implied quantities natural predictors.

ATM implied volatility. The at-the-money IV for a given maturity is the most direct options-based feature. For forecasting 1-month RV, use the 30-day ATM IV. For daily RV, short-dated (weekly) options are more informative.

Skew (25-delta risk reversal). The risk reversal measures the difference in implied volatility between out-of-the-money calls and puts at matched delta:

$$\text{RR}_{25} = \text{IV}_{25\Delta\text{call}} - \text{IV}_{25\Delta\text{put}} \,. \tag{10.20}$$

- $\text{IV}_{25\Delta\text{call}}$: implied volatility of the 25-delta call (out-of-the-money upside).
- $\text{IV}_{25\Delta\text{put}}$: implied volatility of the 25-delta put (out-of-the-money downside).

In Plain English

The risk reversal tells you which side of the distribution the options market is more worried about. For equities, puts are almost always more expensive than calls (negative RR), reflecting the market's willingness to pay a premium for downside protection. When the skew becomes more negative than usual, the market is pricing in elevated crash risk. This forward-looking fear signal often precedes actual volatility increases.

Why This Matters

Include RR_{25} as a feature column whenever options data is available. It captures tail-risk expectations that are invisible in price-based features. For your project, the level of skew and its recent change (5-day difference) are both informative: a sudden steepening of the skew predicts near-term vol spikes even when ATM IV has not yet moved.

Term structure slope. The slope of the IV term structure reveals whether the market expects volatility to increase or decrease over time:

$$\text{TS}_t = \text{IV}_t^{(3\text{m})} - \text{IV}_t^{(1\text{m})}. \tag{10.21}$$

- $\text{IV}_t^{(3\text{m})}$: 3-month at-the-money implied volatility.
- $\text{IV}_t^{(1\text{m})}$: 1-month at-the-money implied volatility.

In Plain English

Normally, longer-dated options cost more in vol terms because there is more time for uncertainty to accumulate (contango, $\text{TS} > 0$). When the term structure inverts ($\text{TS} < 0$), it means near-term IV exceeds longer-term IV, signaling that the market sees a current crisis that it expects to resolve. An inverted term structure is analogous to an inverted yield curve: it reflects stress in the near term that the market believes is temporary.

Why This Matters

The term structure slope is a powerful regime indicator. Include it as a feature column and also consider its interaction with lagged RV. When TS is negative (backwardation), the market has already priced in a vol spike, so your model should dampen its vol-up forecasts. When TS is positive and rising, the market expects calm to continue. This feature is most useful at the 1-week to 1-month forecasting horizon where options have their largest informational advantage over realized measures.

VRP proxy. The variance risk premium measures the gap between what the options market expects and what actually materializes. From Chapter 9:

$$\text{VRP}_t = \frac{\text{VIX}_t^2}{10{,}000} - \mathbb{E}_t[\text{RV}_{t+30}], \tag{10.22}$$

- $\text{VIX}_t^2/10{,}000$: implied variance from VIX, annualized and scaled to match RV units.
- $\mathbb{E}_t[\text{RV}_{t+30}]$: expected realized variance over the next 30 days, approximated by trailing 30-day RV.

In Plain English

VRP is the "insurance premium" the market charges for bearing volatility risk. When VRP is high, implied vol far exceeds realized vol, meaning options are expensive relative to what actually happens. This premium tends to mean-revert: when it is abnormally high, future implied vol is likely to fall (or realized vol is likely to rise) to close the gap. VRP captures information that neither lagged RV nor current IV alone provides.

Why This Matters

VRP is one of the strongest features for medium-horizon (1-week to 1-month) vol forecasting. Include it as a feature column using the simple proxy $\text{VIX}_t^2/10{,}000 - \text{RV}_t^{(m)}$. Be careful about temporal alignment: the "expected" component must use only backward-looking data. Using a forward-looking RV estimate in the VRP proxy introduces lookahead bias. For Project 5 (VRP ML trader), this feature becomes the primary signal rather than just an input.

VIX and VVIX. The VIX itself is a feature for equity volatility. The VVIX (vol-of-vol, Chapter 9) captures uncertainty about volatility. High VVIX predicts fatter tails in the distribution of future RV changes.

Options features are forward-looking

Every price-based feature (RV, RQ, signed jumps) is backward-looking: it summarizes what has already happened. Options-implied features are the only family that reflects the market's consensus about the future. When both are available, combining them almost always improves forecasts. The gains are largest at horizons beyond one week, where the informational advantage of options over recent realized quantities is greatest.

Why This Matters

For your project, the options-implied feature family (ATM IV, RR_{25}, TS slope, VRP, VVIX) provides the largest marginal improvement over HAR-family baselines. If you have access to options data, these features should be your first extension beyond the core lagged-RV set. At the 1-day horizon, the gain is modest (1–3% QLIKE). At the 1-week and 1-month horizons, the gain can reach 5–10% because options encode forward-looking information that lagged RV cannot.

10.9 Cross-Asset Features

Volatility does not live in isolation. A shock in crude oil can spill over to energy equities, then to credit spreads, then to broad equity vol. Cross-asset features capture these transmission channels.

Multi-asset RV. For an equity single-name model, include sector-level RV and index-level RV as additional features. For an FX pair, include RV of correlated pairs and interest-rate volatility.

Spillover indices. Diebold and Yilmaz (2012) propose a variance decomposition of a VAR on a panel of volatilities, producing a total spillover index and directional "to" and "from" connectedness for each asset. A rising spillover index means that shocks are propagating more readily across the system.

Diebold-Yilmaz Spillover Index

Fit a VAR(p) to a vector of N volatility series. Compute the H-step forecast error variance decomposition θ_{ij}^H, normalized so each row sums to 1. The total spillover index is:

$$S^H = \frac{\sum_{i \neq j} \theta_{ij}^H}{\sum_{i,j} \theta_{ij}^H} \times 100. \tag{10.23}$$

- θ_{ij}^H is the fraction of the H-step forecast error variance of series i attributable to shocks from series j.
- When $i = j$, the contribution is "own"; when $i \neq j$, it is "cross."
- S^H ranges from 0 (no spillovers) to 100 (all variance driven by cross-shocks).

In Plain English

The spillover index answers a simple question: what fraction of each asset's volatility is coming from other assets rather than from its own dynamics? When the index is high, markets are tightly coupled and a shock anywhere propagates everywhere. When the index is low, each asset's volatility is driven mainly by its own news. The index rises during crises (2008, COVID) and falls during calm, asset-specific regimes.

Why This Matters

For Project 3 (Multivariate RC with GNNs), the spillover index is a natural global feature. For single-asset forecasting, include the rolling spillover index and its 5-day change as feature columns. Rising connectedness warns that your single-asset model may face larger forecast errors because external shocks are more likely to contaminate your target asset's volatility. This signal helps a tree model adjust its forecasts during contagion regimes.

In practice, you compute the spillover index on a rolling window (200 trading days is typical) and include its level and change as features. Rising connectedness forecasts higher broad-market volatility.

Curse of dimensionality

Adding 20 cross-asset RV features is tempting but dangerous when your sample is 1,000–2,000 daily observations. Either use PCA to reduce the cross-asset panel to 3–5 factors, or let a tree model's built-in feature selection handle the pruning.

10.10 Long-Memory Features

Volatility has long memory: the autocorrelation of log-RV decays hyperbolically, not exponentially (Chapter 7). Standard differencing (Δ^1) removes long memory but also destroys the very persistence that predicts future levels. Fractional differencing provides a middle ground.

Fractional differencing. Standard first-differencing ($d = 1$) achieves stationarity but destroys the long memory that makes volatility predictable. Fractional differencing lets you dial the differencing order to a non-integer value, preserving some memory while achieving stationarity. Following Lopez de Prado (2018) (Chapter 5), define the fractional difference operator:

$$(1 - L)^d x_t = \sum_{k=0}^{\infty} \binom{d}{k} (-1)^k x_{t-k}, \tag{10.24}$$

- L: the lag operator ($Lx_t = x_{t-1}$).
- $d \in (0, 1)$: the fractional differencing order (a continuous knob between "no differencing" and "full differencing").
- $\binom{d}{k}$: generalized binomial coefficients that produce exponentially decaying weights on past values.

In Plain English

Fractional differencing is a weighted average of the current value and all past values, where the weights decay slowly for small d and quickly for large d. At $d = 0$, you keep the raw series (full memory, possibly non-stationary). At $d = 1$, you get the standard first difference (stationary but memoryless). The trick is to find the smallest d that makes the series stationary, because that value preserves the maximum amount of predictive long-range dependence while satisfying the stationarity assumptions that many ML models require.

Why This Matters

Apply fractional differencing to $\log \mathrm{RV}_t$ with $d \approx 0.35$–0.45 and include the result as a feature column. This is especially important for linear models and neural networks that assume stationary inputs. Tree models are less sensitive to non-stationarity, but fractionally differenced log-RV can still help by making the signal cleaner. Use the grid-search approach described below to find the optimal d for your specific dataset.

The binomial coefficients $\binom{d}{k}$ for non-integer d are:

$$\binom{d}{k} = \frac{d(d-1)(d-2)\cdots(d-k+1)}{k!}.$$

In practice, you truncate the infinite sum at a lag k^* where $|\binom{d}{k^*}|$ falls below a threshold (e.g., 10^{-4}).

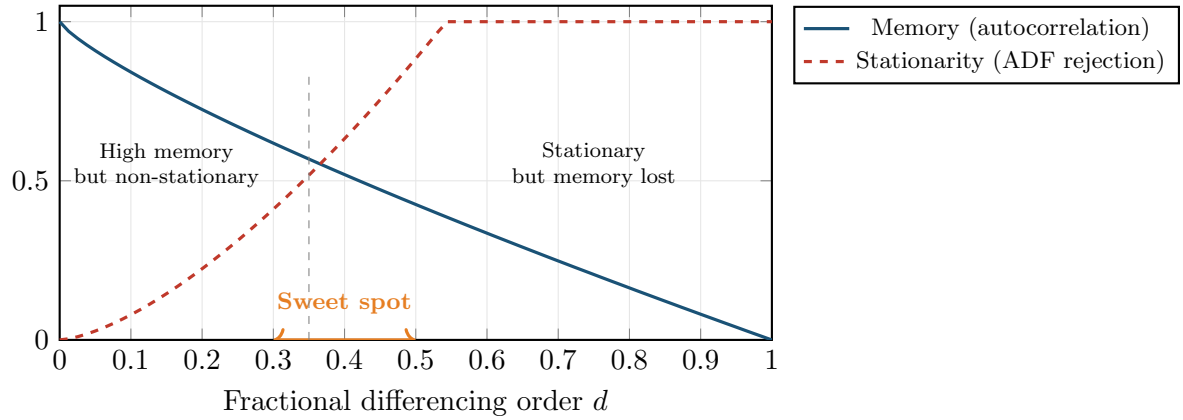


Figure 10.4: The fractional differencing tradeoff. As d increases from 0 to 1, the series becomes more stationary (dashed red, ADF rejection rate rises) but loses memory (solid blue, autocorrelation decays). The sweet spot around $d \in [0.3, 0.5]$ achieves stationarity while preserving enough memory for prediction.

- At $d = 0$: no differencing; the series retains all memory but may be non-stationary.
- At $d = 1$: standard first differencing; stationary but nearly all long-range dependence is destroyed.

- At $d \approx 0.35$–0.45: the series passes the ADF test for stationarity while retaining most of its autocorrelation structure.

To find the right d in practice: grid-search over $d \in \{0.05, 0.10, \ldots, 1.00\}$, apply fractional differencing to $\log \mathrm{RV}_t$, and pick the smallest d for which the ADF test rejects at the 5% level.

Rolling Hurst exponent. From Chapter 7, the Hurst exponent H of log-RV can be estimated via the variogram. A rolling 60-day estimate of H captures time-varying roughness. When H drops below 0.1, the vol-of-vol process is particularly rough, and mean-reversion is fast. When H rises toward 0.3–0.4, persistence is higher and trends in volatility last longer.

Worked Example:

Fractional differencing of log-RV Let $d = 0.4$ and truncate at $k^* = 10$. The weights are:

$$w_0 = 1, \quad w_1 = -0.4, \quad w_2 = \frac{-0.4 \times (-1.4)}{2} = 0.28, \quad w_3 = \frac{0.28 \times (-2.4)}{3} = -0.224, \quad \ldots$$

The fractionally differenced series is:

$$\tilde{x}_t = \sum_{k=0}^{10} w_k \, x_{t-k} = x_t - 0.4\, x_{t-1} + 0.28\, x_{t-2} - 0.224\, x_{t-3} + \cdots$$

Unlike $\Delta^1 x_t = x_t - x_{t-1}$, this weighted sum retains some of the level information from x_t while achieving stationarity.

10.10.1 Vol-of-Vol and Regime Duration

Fractional differencing captures long memory through the autocorrelation structure of the RV series itself. Two complementary features capture different aspects of memory: the *stability* of the volatility process and the *time elapsed* since the last regime change.

Vol-of-vol. The **volatility of volatility** measures how much the volatility process itself is fluctuating. It is computed as the rolling standard deviation of daily RV over a 22-day window:

$$\mathrm{VoV}_t = \sqrt{\frac{1}{21} \sum_{i=0}^{21} \left(\mathrm{RV}_{t-i} - \bar{\mathrm{RV}}_{t,22} \right)^2}, \tag{10.25}$$

- $\bar{\mathrm{RV}}_{t,22} = \frac{1}{22} \sum_{i=0}^{21} \mathrm{RV}_{t-i}$: the trailing 22-day mean of daily RV.
- VoV_t: standard deviation of daily RV over the past month, capturing how "unstable" the volatility regime is.

In Plain English

Vol-of-vol tells you whether volatility has been roughly constant over the past month or swinging wildly. During the 2020 COVID crash, both RV and vol-of-vol were extremely high. But in some regimes, RV is elevated but stable (a sustained high-vol period like mid-2022), meaning vol-of-vol is moderate even though RV is high. The distinction matters for forecasting: when vol-of-vol is high, your model should have wider prediction intervals because the volatility process is itself unpredictable. When vol-of-vol is low, volatility is clustered tightly around its current level, and the forecast is more reliable.

Why This Matters

Include VoV_t as a feature column. It complements fractional differencing by capturing the *dispersion* of the vol process rather than its *persistence*. Tree models can use VoV as a split variable to identify regimes where the standard HAR-style forecast (based on mean persistence) is less reliable. If the VVIX index is available (it is for SPX), it provides a forward-looking analog of VoV; when both are available, include both and let the model choose.

Regime duration. The **regime duration** feature counts the number of trading days since the last "volatility event," defined as a day where RV exceeded its trailing mean by more than 2 standard deviations:

$$D_t = t - \max\{\tau \leq t : \text{RV}_\tau > \bar{\text{RV}}_{\tau,66} + 2\,\hat{\sigma}_{\text{RV},\tau,66}\}, \tag{10.26}$$

- $\bar{\text{RV}}_{\tau,66}$: the trailing 66-day (3-month) mean of daily RV at time τ.
- $\hat{\sigma}_{\text{RV},\tau,66}$: the trailing 66-day standard deviation of daily RV.
- D_t: number of trading days since the last 2-sigma RV spike.

In Plain English

Regime duration acts as a **mean-reversion clock**. After a volatility spike, there is typically a decay period where RV gradually returns to its long-run mean. The duration feature tells the model how far along this decay process we are. When D_t is small (say, 3 days since the last spike), mean reversion is still in progress and the forecast should remain elevated. When D_t is large (say, 45 days since the last spike), the market has been calm for a while, and the probability of a new spike is rising (vol clustering means calm periods do not last forever). Trees can learn this non-monotonic relationship: short durations predict high vol (mean-reversion tail), and very long durations predict moderately elevated vol (the calm-before-the-storm effect).

Why This Matters

Include D_t and $\log(1 + D_t)$ as feature columns. The log transform compresses large duration values, which makes the feature more useful for tree splits at the short end (where the action is). Regime duration captures time-varying dynamics that neither fractional differencing nor the rolling Hurst exponent can express: the *position within a volatility cycle* rather than the cycle's statistical properties. It is especially valuable at the 1-week horizon, where mean-reversion dynamics are most pronounced.

10.11 Calendar and Event Features

Certain days are systematically different.

Macro announcements. FOMC rate decisions, non-farm payroll (NFP) releases, and CPI prints are associated with elevated realized volatility. Encode these as binary dummies: $\mathbf{1}(\text{FOMC}_t)$, $\mathbf{1}(\text{NFP}_t)$, $\mathbf{1}(\text{CPI}_t)$. Also include the day before the event (anticipation effect) and the day after (digestion effect).

Earnings and corporate events. For single-stock volatility, earnings announcement dates dominate all other calendar effects. A binary earnings dummy plus the number of days until the next earnings date ("earnings countdown") can be powerful.

Options expiry and quarter-end. Monthly options expiry (third Friday) and quarterly triple-witching dates show elevated intraday volume and can distort RV measurements. Quarter-end rebalancing by institutional investors also creates temporary volatility.

Day-of-week effects. Mondays historically show higher volatility (three calendar days of news compressed into one trading day), but this effect has weakened over time.

Calendar features are supplements, not drivers

Each individual calendar dummy is weak. They rarely rank in the top 10 features by importance in a tree model. Their value is incremental: they help the model explain residual variation after the heavy-lifting features (lagged RV, options-implied measures) have done their work. Do not build a model primarily on calendar features.

10.11.1 Event-Driven Volatility: Beyond Binary Dummies

The basic calendar dummies above capture whether an event occurs. But events affect volatility in richer ways that better feature engineering can exploit.

Term structure kinks. Before a known event (e.g., FOMC on Wednesday), the IV term structure shows a characteristic kink: the option expiring just after the event has elevated IV (it straddles the uncertainty), while the option expiring just before has lower IV (the event is not included). The magnitude of this kink, the **event-implied vol**, quantifies how much extra vol the market expects from the event. Extract it as:

$$\sigma_{\text{event}} = \sqrt{\frac{T_2\,\sigma_2^2 - T_1\,\sigma_1^2}{T_2 - T_1}} \tag{10.27}$$

- T_1, T_2: expiry times of two options that bracket the event ($T_1 < t_{\text{event}} < T_2$).
- σ_1, σ_2: ATM implied volatilities for those expiries.
- σ_{event}: the implied vol attributable to the event alone.

In Plain English

This formula "subtracts out" the normal day-to-day volatility to isolate the extra uncertainty the market assigns to the event itself. The idea is that total implied variance over a period equals the sum of variances from individual days (roughly, under independence). By comparing two options that differ only in whether they include the event day, you can back out how much additional variance the event contributes.

Why This Matters

Event-implied vol is a much richer feature than a binary event dummy. It tells your model not just that an FOMC meeting is happening, but how uncertain the market is about the outcome. Include σ_{event} as a feature column on event days (and zero on non-event days). This feature requires options data but adds significant value for event-day forecasting, where binary dummies alone are too crude.

Historical event-day RV ratios. Compute the ratio RV(event day)/RV(surrounding days) historically for each event type. FOMC days typically show 1.5–2× normal RV; NFP days show 1.3–1.5×. Use these ratios as multiplicative adjustments to your baseline forecast on event days.

Days-to-next-event features. Rather than a binary "is today an event," encode continuous distance: days_to_FOMC, days_since_FOMC, days_to_earnings. This lets the model learn the anticipation buildup and post-event decay.

10.11.2 Calendar Proximity Measures

The binary dummies above answer a yes/no question: "Is today an FOMC day?" But volatility does not jump from zero to one on event day. It ramps up gradually in the days before and decays gradually in the days after. **Continuous proximity measures** capture this ramp by encoding the distance to the next event as a real-valued feature.

Calendar Proximity Feature

For a scheduled event of type e (FOMC, NFP, earnings, options expiry), define the **proximity measure**:

$$\text{prox}_{e,t} = \max\big(0,\ W_e - |t - t_e^{\text{next}}|\big), \tag{10.28}$$

where t_e^{next} is the date of the next occurrence of event e, and W_e is a window parameter (e.g., $W_e = 5$ trading days for FOMC, $W_e = 3$ for NFP).

- t_e^{next}: the next scheduled occurrence of event e after date t.
- W_e: the anticipation window in trading days.
- $\text{prox}_{e,t}$: a triangular function that ramps from 0 to W_e as the event approaches and falls back to 0 afterward.

In Plain English

A binary dummy says "FOMC is today" or "FOMC is not today." A proximity measure says "FOMC is 3 days away" – and the model can learn that volatility starts compressing 2–3 days before FOMC (the "pre-FOMC drift") and then expands sharply on announcement day. This anticipation ramp is invisible to binary dummies: 5 days before FOMC looks identical to 50 days before FOMC in a binary encoding, but the market's behavior is measurably different. The continuous encoding lets tree models learn the full shape of the event response, not just the on/off state.

FOMC compression and expansion. Equity index volatility shows a well-documented pattern around FOMC meetings: RV is suppressed 1–3 days before the announcement as traders reduce risk ahead of the decision, then spikes on the announcement day and the following session. A proximity feature centered on the FOMC date captures both the compression phase (positive proximity, negative vol effect) and the expansion phase (event day, positive vol effect). Tree models can split on $\text{prox}_{\text{FOMC},t}$ to learn this asymmetric pattern.

Options expiry and gamma unwind. Monthly options expiry (third Friday) and quarterly triple-witching dates create mechanical volatility through **gamma exposure unwind**. As options approach expiry, delta-hedging activity by market makers intensifies, compressing intraday volatility when gamma is positive and amplifying it when gamma is negative. The proximity-to-expiry feature lets the model learn that the 2–3 days before expiry have distinct vol dynamics driven by hedging flows rather than information arrival.

Earnings proximity (single names). For individual stocks, earnings announcements dominate all other calendar effects. The proximity feature $\text{prox}_{\text{earn},t}$ captures the well-known pre-earnings volatility buildup, which begins roughly 5 trading days before the announcement as implied vol rises and option volume spikes. Post-earnings, the "volatility crush" typically resolves within 1–2 days.

Why This Matters

Replace raw binary event dummies with continuous proximity measures in your feature pipeline. For each event type, compute $\text{prox}_{e,t}$ with an appropriate window: $W = 5$ for FOMC, $W = 3$ for NFP/CPI, $W = 5$ for earnings (single names), $W = 3$ for options expiry. Also include the raw days_to_event as a separate feature (without the triangular window) so the model can learn longer-range anticipation effects. Proximity features provide 1–2% incremental QLIKE improvement over binary dummies alone at the 1-day horizon, with the gain concentrated around event days where the baseline forecast has the largest errors.

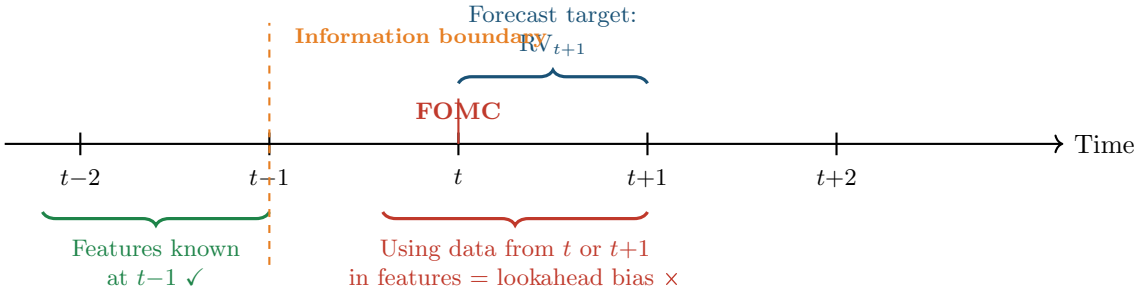


Figure 10.5: Temporal alignment for feature engineering. Features used to predict RV_{t+1} must be computed from data available at or before time $t-1$ (the close of trading on the day before the forecast is made). Using same-day or future data introduces lookahead bias. The orange dashed line marks the information boundary.

Look-Ahead Risk with Events

Earnings dates are announced approximately 2–4 weeks before the event. FOMC dates are published annually. These are safe to use. But some events (emergency Fed meetings, unscheduled news) cannot be known in advance. Never include an event indicator for a date that was not knowable at time $t-1$. For earnings, use the *announced* date, not the ex-post actual date (companies occasionally reschedule).

10.12 Sentiment and Text Features

News and social media sentiment contain forward-looking information that market prices may not yet fully reflect.

Audrino et al. (2020) construct daily sentiment indices from financial news articles and show that adding a negative-sentiment variable to the HAR model improves volatility forecasts, particularly during crisis periods. The key finding: negative sentiment matters more than positive sentiment, echoing the asymmetry we saw in signed semivariances (Section 10.5).

Rahimikia et al. (2021a) apply transformer-based NLP models (FinBERT) to extract sentiment from financial text and demonstrate incremental predictive power for equity volatility at daily and weekly horizons.

Sentiment for Volatility

Both Audrino et al. (2020) and Rahimikia et al. (2021a) find that text-derived sentiment adds 1–3% improvement in QLIKE loss over HAR-family baselines, with gains concentrated in high-volatility periods. The effect is modest but robust across different text

sources and NLP methods.

If you have access to a news or social media feed, construct:

- A daily sentiment score (average polarity of articles mentioning the asset).
- A volume-of-news feature (number of articles; more attention predicts higher vol).
- A disagreement feature (standard deviation of sentiment across articles).

For most academic and internship projects, news data is hard to obtain. Treat sentiment features as a "nice to have" rather than a core requirement.

10.13 Feature Importance and Selection

With dozens of candidate features, you need a principled way to assess which ones actually help. This section previews the tools you will use extensively in Chapters 11–14.

Mean Decrease in Accuracy (MDA). Permute one feature column at a time and measure how much out-of-sample loss increases. The feature whose permutation hurts most is the most important. MDA is model-agnostic and properly accounts for nonlinear interactions.

Mean Decrease in Impurity (MDI). For tree-based models, sum the impurity reduction (e.g., variance reduction for regression trees) across all splits that use a given feature. MDI is fast but biased: it favors high-cardinality features and features correlated with other important variables.

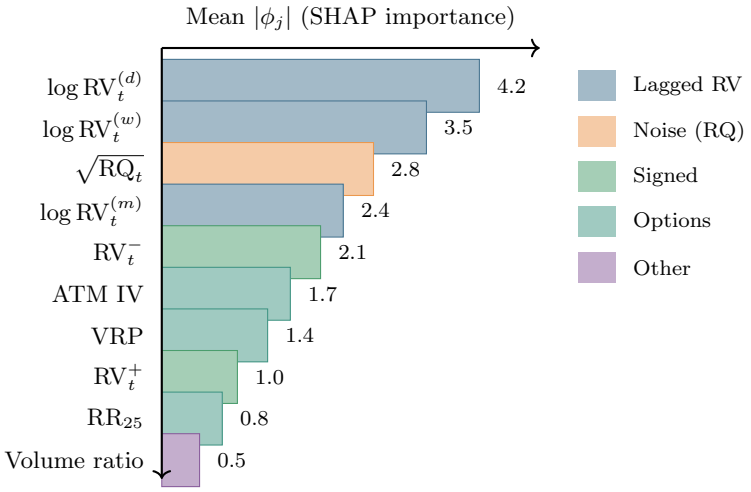


Figure 10.6: Illustrative SHAP feature importance ranking for a gradient-boosted tree model forecasting next-day log-RV. Lagged RV features (blue) dominate, but noise-awareness ($\sqrt{\mathrm{RQ}}$, orange) and asymmetry (RV^-, green) provide substantial incremental value. Options-implied features (teal) add forward-looking information. Exact values are dataset-dependent; the ranking pattern is robust.

SHAP (SHapley Additive exPlanations). SHAP values decompose each prediction into additive contributions from each feature, grounded in cooperative game theory. For a single prediction $\hat{y}_i$, SHAP decomposes it as:

$$\hat{y}_i = \phi_0 + \sum_{j=1}^{p} \phi_j^{(i)}, \tag{10.29}$$

- ϕ_0: the base value (mean prediction across the training set).

- $\phi_j^{(i)}$: feature j's contribution to prediction i (can be positive or negative).
- The sum of all SHAP values plus the base value exactly equals the model's prediction.

In Plain English

SHAP answers the question: "For this particular prediction, how much did each feature push the forecast up or down relative to the average?" It borrows from game theory the idea of fairly splitting credit among players in a cooperative game. Each feature is a "player," the prediction is the "payout," and SHAP computes each feature's fair share by considering all possible orderings in which features could be added to the model.

Why This Matters

Use SHAP values to explain your model's predictions in your internship presentation. A SHAP summary plot showing that $\log \mathrm{RV}_t^{(d)}$ and $\sqrt{\mathrm{RQ}_t}$ are the top two features validates that your ML model has learned economically sensible patterns. SHAP also helps debug: if a calendar dummy ranks in the top 3, something is likely wrong (possible lookahead bias or data leakage). Always compute SHAP across multiple purged CV folds to assess stability.

SHAP instability for correlated features

When two features are highly correlated (e.g., $\mathrm{RV}_t^{(d)}$ and $\log \mathrm{RV}_t^{(d)}$), SHAP values become unstable: small perturbations in the training data can swap their importance rankings. This does not mean the model is wrong; it means the importance is genuinely shared and cannot be cleanly attributed. Always check stability by computing SHAP across multiple train/test splits. If two features swap ranks across folds, treat them as interchangeable rather than debating which is "truly" more important.

Accumulated Local Effects (ALE) plots. Christensen et al. (2023) advocate ALE plots over partial dependence plots (PDPs) for volatility models. ALE plots avoid the extrapolation problem of PDPs by computing effects only within the observed feature distribution.

ALE Plot

For feature x_j, partition its range into K bins. In each bin $[z_{k-1}, z_k]$, compute the average change in the model's prediction as x_j moves across the bin, holding other features at their observed values:

$$\widehat{\mathrm{ALE}}_j(x) = \sum_{k=1}^{k_x} \frac{1}{n_k} \sum_{i: x_j^{(i)} \in [z_{k-1}, z_k]} \left[\hat{f}(z_k, \mathbf{x}_{-j}^{(i)}) - \hat{f}(z_{k-1}, \mathbf{x}_{-j}^{(i)}) \right], \tag{10.30}$$

where:

- k_x is the bin containing x,
- n_k is the number of observations in bin k,
- $\hat{f}(z_k, \mathbf{x}_{-j}^{(i)})$ evaluates the model at the bin boundary z_k with all other features at their observed values for observation i.

In Plain English

ALE plots answer the question: "As this feature increases, does the model's prediction go up, down, or stay flat?" Unlike partial dependence plots, which can be misleading when

features are correlated (they evaluate the model at feature combinations that never occur in practice), ALE plots only compute effects within the observed data range. The result is a curve showing the marginal effect of one feature on the prediction, holding everything else at its actual (not hypothetical) value.

Why This Matters

Use ALE plots to sanity-check your trained model. The ALE plot for $\log \mathrm{RV}_t^{(d)}$ should show a roughly increasing, concave relationship: higher lagged RV predicts higher future RV, but with diminishing effect at extreme levels (mean reversion). If the ALE plot shows a non-monotonic or economically implausible pattern, your model may be overfitting noise. Include ALE plots for the top 3–5 features in your internship report as evidence that the model has learned meaningful structure.

ALE plots tell you the *shape* of the relationship: is the effect of lagged RV on predicted volatility linear, concave, or threshold-like? This is valuable for sanity-checking whether the model has learned economically sensible patterns.

Importance Stability Protocol

Before reporting which features matter, run this checklist:

1. Compute MDA importance across 5 purged CV folds (Chapter 6 for purging).
2. Check whether the top-5 ranking is consistent across folds.
3. For any feature that appears in top-5 for some folds but not others, check its correlation with other top features.
4. Report a stability metric: fraction of folds in which a feature appears in the top-k.
5. Use ALE plots to visualize the functional form of the top-3 features.

Unstable rankings are a warning sign, not a failure: they tell you the signal is shared across correlated features, which is useful information for feature selection and portfolio construction.

Why This Matters

Follow this exact protocol for your internship project. Run MDA across your 5 purged CV folds and report the stability of the top-10 feature rankings. If $\log \mathrm{RV}_t^{(d)}$ and $\sqrt{\mathrm{RQ}_t}$ consistently rank in the top 5, you have confirmation that your ML model has learned the HARQ insight from data. Present the stability table and ALE plots in your final report; they are more convincing evidence of model quality than a single QLIKE number.

10.14 The Diminishing Returns Curve

You have now seen seven layers of features: lagged RV transforms, noise-awareness features (RQ), signed and asymmetric measures, options-implied quantities, microstructure signals, cross-asset spillovers, calendar/event features, and long-memory/regime features. The natural question is: *How much does each layer actually contribute?*

The answer follows a characteristic **diminishing returns curve**: the first few feature families provide the vast majority of forecasting accuracy, and each subsequent layer adds progressively less.

Interpreting the staircase. The staircase in Figure 10.7 shows approximate cumulative accuracy contributions, normalized so that the full feature set achieves 100%.

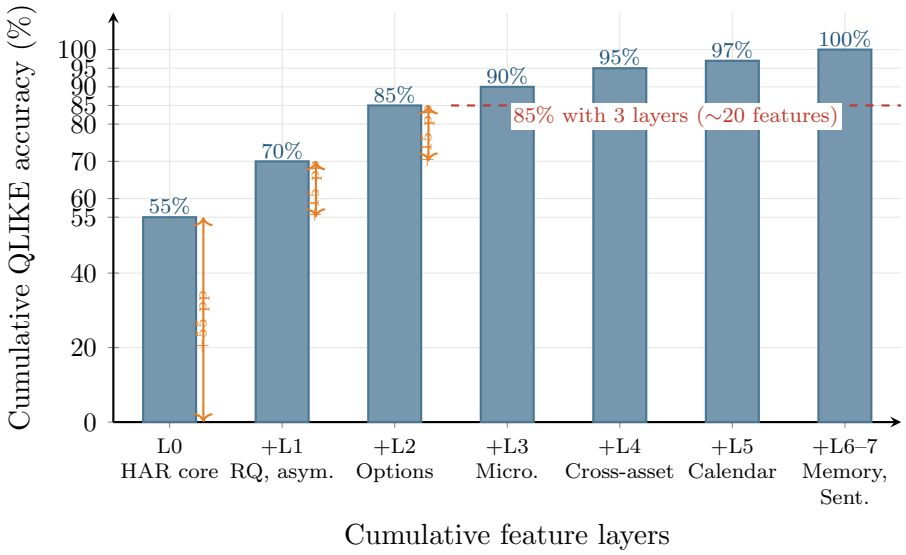


Figure 10.7: Diminishing returns staircase for volatility forecasting features. Each bar shows the cumulative QLIKE accuracy (normalized so the full feature set achieves 100%) when adding one more feature layer. The first three layers (HAR core, noise/asymmetry, options) achieve 85% of attainable accuracy with roughly 20 features. The remaining four layers add the final 15% with 60–100 additional features. Exact percentages are illustrative and based on findings in Christensen et al. (2023) and the vol-project-ref feature composition analysis.

- **Layer 0 (HAR core):** $\text{RV}_t^{(d)}$, $\text{RV}_t^{(w)}$, $\text{RV}_t^{(m)}$ alone achieve roughly 55% of attainable accuracy. This is the single most important result in volatility forecasting: three simple features explain more than half the variation.
- **Layer 1 (Noise + Asymmetry):** Adding $\sqrt{\text{RQ}_t}$, RV_t^-, RV_t^+, and jump components pushes accuracy to roughly 70%. The HARQ and SHAR improvements are captured here.
- **Layer 2 (Options):** ATM IV, skew, VRP, and term structure slope bring the total to roughly 85%. These forward-looking features represent the single largest marginal gain beyond the core RV features.
- **Layers 3–7 (Microstructure, Cross-asset, Calendar, Memory, Sentiment):** The remaining five layers collectively contribute the final 15%, with each individual layer adding 2–5 percentage points.

The curve shifts with forecast horizon. The diminishing returns curve is not fixed; it depends heavily on the forecast horizon. Table 10.1 summarizes which features dominate at each horizon.

Table 10.1: Feature dominance by forecast horizon. The "dominant" column lists the feature families that contribute the most marginal accuracy at each horizon. Based on findings in Christensen et al. (2023) and Bollerslev et al. (2024).

Horizon	Dominant features	ML vs. linear gap	Key insight
$h = 1$ day	Lagged RV, RQ, RV^-	Small (∼5%)	HARQ nearly optimal
$h = 5$ days	Options (VRP, skew) + lagged RV	Moderate (∼10%)	VRP begins to matter
$h = 22$ days	VRP, term structure, Hurst	Large (∼15%)	Options have max advantage

At the 1-day horizon, the HAR core dominates and ML adds relatively little. Bollerslev et al. (2024) demonstrate that a properly fitted HAR model with daily re-estimation and a 630-day training window is "hard to beat" even with gradient-boosted trees and neural networks, when

the feature set is restricted to lagged RV and VIX. The implication is stark: if your ML model does not beat a well-tuned HAR at $h = 1$, the issue is likely the baseline, not the model.

At the 1-week and 1-month horizons, the picture changes. Christensen et al. (2023) show that ML models gain the most from additional features at longer horizons, where the informational advantage of options-implied and macroeconomic variables over lagged RV is greatest. At $h = 22$, ML models with the full feature set ($\mathcal{M}_{\text{ALL}}$) consistently outperform all HAR variants, and the Diebold-Mariano test frequently rejects equal predictive accuracy.

Perfect L0–L2 Before Chasing Marginal Features

The diminishing returns curve delivers a clear engineering message: invest your effort in getting Layers 0–2 right before adding Layers 3–7. Specifically:

1. Get the HAR baseline correct: daily re-estimation, 500–800 day training window, log-RV target (Bollerslev et al., 2024).
2. Add $\sqrt{\text{RQ}_t}$, RV^-, RV^+, and jump components. This gives you HARQ/SHAR-level performance.
3. Add options features (ATM IV, VRP, skew, term structure slope) if available. This is the largest single-layer gain.
4. Only then consider microstructure, cross-asset, calendar proximity, and sentiment features. Each adds 2–5% marginal improvement.

Chasing Layer 5–7 features when your Layer 0 baseline is poorly calibrated (wrong training window, wrong re-estimation frequency, wrong target transform) is optimizing the wrong thing.

Why This Matters

The diminishing returns curve is the punchline of this entire chapter. For your internship project, it means: (1) your first model should use only Layers 0–2 (∼15–20 features) and should beat a well-tuned HAR before you declare victory; (2) if you have options data, adding VRP and skew should be your first extension, not microstructure features; (3) the QLIKE improvement target of 30–80 bps is achievable primarily through Layers 0–2 plus proper model tuning, not through feature quantity. Present the diminishing returns curve in your final report to explain why you prioritized feature quality over feature quantity.

10.15 Summary

- **Lagged RV transforms** ($\text{RV}^{(d)}$, $\text{RV}^{(w)}$, $\text{RV}^{(m)}$, log, ratios) are the single strongest feature family, explaining 40–60% of next-day RV variation.
- **Realized quarticity** (RQ) measures the noise in today's RV estimate. The HARQ interaction feature down-weights noisy days (Bollerslev et al., 2016b).
- **Signed semivariances** (RV^+, RV^-) capture the leverage effect. The downside component carries roughly twice the predictive weight of the upside component (Patton and Sheppard, 2015b).
- **Signed jumps** (J^+, J^-) from Chapter 4 provide additional asymmetric information; negative jumps are substantially more informative.
- **Higher moments** (realized skewness, kurtosis) have modest stand-alone power but serve as regime indicators.
- **Microstructure features** (spread, OBI, WAP returns, volume profiles, VPIN) are powerful for short-horizon forecasting with tick data but are asset-specific.

- **Options-implied features** (ATM IV, skew, term structure slope, VRP, VVIX) are uniquely forward-looking and provide the largest gains at horizons beyond one week.
- **Cross-asset features** (multi-asset RV, Diebold-Yilmaz spillover indices) capture contagion channels but require care to avoid curse-of-dimensionality problems (Diebold and Yilmaz, 2012).
- **Fractional differencing** ($(1-L)^d$ with $d \approx 0.35$–0.45) preserves long memory while achieving stationarity (Lopez de Prado, 2018).
- **Rolling Hurst exponent** from Chapter 7 captures time-varying roughness of the volatility process.
- **Calendar dummies** (FOMC, NFP, earnings, expiry) are weak individually but provide incremental improvement.
- **Sentiment features** add 1–3% QLIKE improvement, concentrated in crisis periods (Audrino et al., 2020; Rahimikia et al., 2021a).
- **Feature importance tools** (MDA, SHAP, ALE) should be evaluated across multiple CV folds to ensure stability; correlated features naturally produce unstable attribution (Christensen et al., 2023).

Chapter 10 Key Results

Feature Family	Key Contribution	Best Horizon
Lagged RV (d/w/m)	Baseline predictive power, 40–60% R^2	1–5 days
Realized quarticity	Noise-aware weighting (HARQ)	1 day
Signed semivariances	Leverage effect, RV^- $\approx$ 2× weight of RV^+	1–5 days
Signed jumps	Asymmetric tail events	1 day
Higher moments	Regime conditioning	Auxiliary
Microstructure/LOB	High-frequency liquidity signals	Intraday–1 day
Options-implied	Forward-looking; VRP, skew, VVIX	1 week–1 month
Cross-asset RV	Spillovers, contagion	1–5 days
Frac. differencing	Stationarity with memory preservation	All
Calendar dummies	Event-day adjustment	Event-specific
Sentiment	Crisis-period improvement	1 day–1 week

Chapter 11

Tree-Based Methods for Volatility

Application

This chapter is where ML meets volatility forecasting for the first time. Tree-based methods (LightGBM, XGBoost) are the workhorse models for tabular volatility data, just as they dominate Kaggle competitions and quantitative finance pipelines. Projects 1, 2, and 5 all use tree ensembles as their primary model. The central question: when do trees beat HAR (Chapter 6), and when don't they?

11.1 Why Trees for Volatility

Chapter 10 built a rich feature matrix: lagged RV transforms, realized quarticity, signed semivariances, jumps, options-implied measures, cross-asset signals, and calendar dummies. All of these live in a flat table where each row is a date and each column is a number. No pixels, no word tokens, no sequential structure that demands recurrence. This is *tabular data*, and tree-based ensembles are the best off-the-shelf learners for tabular data (Gu et al., 2020b).

Why? Four reasons, each directly relevant to volatility:

1. **Automatic nonlinear interactions.** HAR (Chapter 6) is linear in its three RV averages. If the effect of yesterday's RV depends on whether the VIX is above 25, you must hand-craft that interaction term for HAR. A tree discovers it automatically: one split on VIX, another on lagged RV, and the interaction is encoded in the tree structure.

2. **Threshold effects.** Volatility regimes change sharply. A tree split at $\text{RV}_{t-1} = 0.0004$ captures a regime boundary that a linear model can only approximate with polynomial terms.

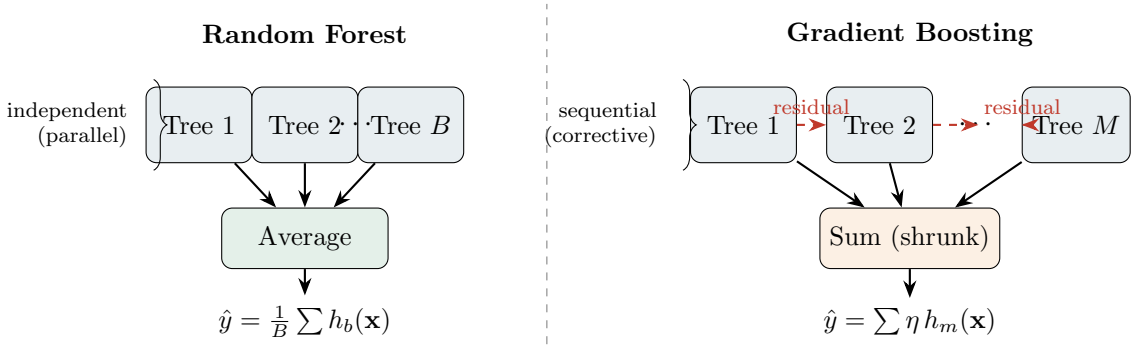
3. **Fast training.** A LightGBM model with 500 trees trains in seconds on 1,250 rows. This speed enables hundreds of purged cross-validation iterations (Chapter 17), which is essential for honest evaluation on small samples.

4. **Built-in feature importance.** Split-count importance and SHAP values tell you *which* features drive predictions. This matters for interpretability, and it connects back to the feature selection discussion in Chapter 10.

Why Trees Win on Tabular Data

Trees excel at finding threshold effects and interactions in tabular data. Volatility data is tabular. The combination of automatic interaction detection, speed, and interpretability makes gradient-boosted trees the default first model for any volatility forecasting pipeline built on the features from Chapter 10.

The diagram below contrasts the two main tree ensemble strategies. A **random forest** trains many independent trees on bootstrapped samples and averages their predictions (variance reduction through decorrelation). **Gradient boosting** trains trees sequentially, each one correcting the errors of the ensemble so far (bias reduction through iterative refinement). Both are used in volatility forecasting, but gradient boosting (LightGBM, XGBoost) typically wins on tabular data.



11.2 LightGBM and XGBoost

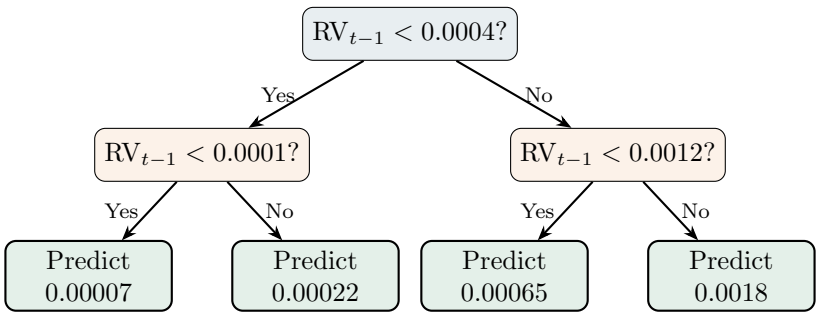
This section reviews gradient boosting at a level sufficient for volatility work. If you want the full algorithmic treatment (histogram binning, leaf-wise vs. level-wise growth, GOSS sampling), see a dedicated ML reference. Here we focus on the choices that matter for forecasting RV.

Gradient Boosting in One Paragraph

You start with a constant prediction (the training-set mean of RV). You compute the residuals. You fit a shallow decision tree to those residuals. You add a shrunken version of that tree's predictions to the running forecast. You repeat. Each new tree corrects the mistakes of the ensemble so far. The "gradient" part: the residuals are the negative gradient of the loss function with respect to the current prediction, so this procedure is gradient descent in function space.

A single tree on lagged RV

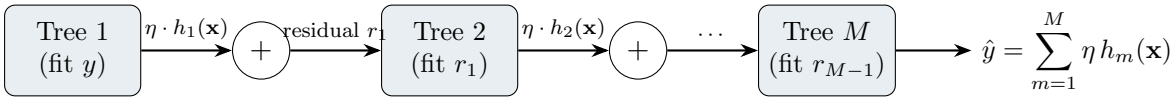
Before ensembles, consider one tree. The diagram below shows a depth-2 tree predicting tomorrow's RV from yesterday's value.



The tree partitions the feature space into four leaves, each returning a constant prediction. This is piecewise-constant approximation. A single shallow tree is a weak learner; boosting combines hundreds of them.

The boosting ensemble

The diagram below shows how gradient boosting builds its forecast sequentially. Each tree fits the residual errors of the ensemble so far.



The ensemble prediction is:

$$\hat{y}_t = \sum_{m=1}^{M} \eta\, h_m(\mathbf{x}_t), \tag{11.1}$$

where:

- $h_m(\mathbf{x}_t)$ is the prediction of the m-th tree given feature vector $\mathbf{x}_t$,
- $\eta \in (0, 1]$ is the learning rate (shrinkage),
- M is the number of boosting iterations (trees).

In Plain English

The final prediction is a team vote: each tree contributes a small correction, and you add up all the corrections. The learning rate η shrinks each tree's voice so that no single tree dominates. More trees with a smaller learning rate generally gives a smoother, more reliable forecast than fewer trees with a larger rate.

Why This Matters

This additive structure is how LightGBM and XGBoost produce your daily RV forecast. Every tree in the ensemble sees the same feature vector $\mathbf{x}_t$ (lagged RV, VIX, jumps, etc. from Chapter 10) and contributes a correction. Tuning η and M together (with early stopping) is the primary way to control overfitting on 1,250-row volatility samples.

Loss function choice

The standard loss for regression is MSE: $\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_t (\text{RV}_t - \hat{y}_t)^2$. But Chapter 17 showed that QLIKE is the preferred loss for volatility forecasting (Audrino and Knaus, 2016):

$$\mathcal{L}_{\text{QLIKE}} = \frac{1}{N}\sum_{t=1}^{N}\left(\frac{\text{RV}_t}{\hat{y}_t} - \ln\frac{\text{RV}_t}{\hat{y}_t} - 1\right). \tag{11.2}$$

- RV_t: the realized volatility actually observed on day t,
- $\hat{y}_t$: the model's forecast for day t,
- the summand equals zero when $\hat{y}_t = \text{RV}_t$ and is strictly positive otherwise.

In Plain English

QLIKE penalizes you more harshly for under-predicting volatility than for over-predicting it by the same amount. If realized vol is 20% and you forecast 10%, the penalty is much larger than if you forecast 30%. This asymmetry matches real risk management needs:

underestimating volatility is more dangerous than overestimating it. MSE, by contrast, penalizes both directions equally.

Why This Matters

QLIKE is the primary evaluation metric for the GS project. Training your tree model directly on QLIKE (rather than MSE) aligns the optimization objective with the evaluation criterion. Audrino and Knaus (2016) show that QLIKE-optimized trees outperform MSE-optimized trees for realized volatility, so this custom loss is not optional; it is a core part of the pipeline.

Neither LightGBM nor XGBoost provides QLIKE natively, but both accept custom objective functions. You supply the gradient and Hessian:

$$g_t = \frac{\partial \mathcal{L}_{\text{QLIKE}}}{\partial \hat{y}_t} = -\frac{\text{RV}_t}{\hat{y}_t^2} + \frac{1}{\hat{y}_t}, \tag{11.3}$$

$$h_t = \frac{\partial^2 \mathcal{L}_{\text{QLIKE}}}{\partial \hat{y}_t^2} = \frac{2\ \text{RV}_t}{\hat{y}_t^3} - \frac{1}{\hat{y}_t^2}. \tag{11.4}$$

- g_t: the gradient tells each tree which direction to adjust predictions,
- h_t: the Hessian tells each tree how aggressively to adjust (curvature information).

In Plain English

The gradient g_t answers: "for observation t, should the next tree push the prediction up or down?" The Hessian h_t answers: "how confident should that push be?" Together, they let the boosting algorithm do Newton-step optimization in function space, which converges faster than using the gradient alone.

Why This Matters

Implementing these two formulas as a custom objective function in LightGBM or XGBoost is a 10-line code change, but it is the single most impactful modification you can make to the default pipeline. Without it, the tree model optimizes MSE, which does not match the QLIKE evaluation metric and systematically under-penalizes low forecasts in high-vol regimes.

Enforce Positive Predictions

QLIKE requires $\hat{y}_t > 0$. Clip predictions to a small positive floor (e.g., 10^{-8}) inside the custom objective, or the gradient explodes. Also apply the floor when evaluating the metric, not just during training.

Monotone constraints

Finance intuition says "higher recent volatility predicts higher future volatility." You can encode this as a monotone constraint on lagged-RV features: the prediction must be non-decreasing in RV_{t-1}. Both LightGBM and XGBoost support per-feature monotone constraints. Use them sparingly (only for features where the monotone relationship is theoretically unambiguous), but when appropriate, they reduce overfitting and improve interpretability.

11.3 Hyperparameters for Volatility Data

This section is the most practically important in the chapter. Volatility data has three properties that make default hyperparameters dangerous:

1. **Small samples.** Five years of daily data gives you roughly 1,250 observations. This is orders of magnitude smaller than the datasets LightGBM and XGBoost were optimized for.
2. **High autocorrelation.** RV is strongly persistent (Chapter 6). Nearby observations are nearly identical, so effective sample size is smaller than the row count suggests.
3. **Heavy tails.** Volatility spikes create influential observations. A single crisis week can dominate the loss function.

Default Settings Will Overfit

Default LightGBM/XGBoost settings (`max_depth`=6+, `min_child_samples`=20) were designed for datasets with 100K+ rows. With 1,250 daily observations, defaults will memorize noise. The table below gives recommended ranges calibrated for volatility forecasting.

Parameter	Default	Vol Range	Rationale
`max_depth`	6–8	3–5	Shallow trees limit memorization
`min_child_samples`	20	50–200	Forces each leaf to generalize
`subsample`	1.0	0.6–0.8	Row subsampling adds regularization
`colsample_bytree`	1.0	0.6–0.8	Feature subsampling decorrelates trees
`learning_rate`	0.1	0.01–0.05	Slow learning + early stopping
`num_iterations`	100	500–2000	More trees at lower rate; early stop
`reg_lambda`	0	1–10	L2 leaf regularization

Worked Example:

Tuning LightGBM on 5 Years of SPY RV You have 1,258 daily observations of RV for SPY, with 45 features from Chapter 10. You reserve the last 6 months (126 days) as a holdout (never touched until final evaluation). The remaining 1,132 days go into purged 5-fold CV with a 5-day embargo (Chapter 17).

Step 1: Baseline. Fit HAR on the same 1,132 training days. QLIKE = 0.142 averaged across folds.

Step 2: Defaults. Fit LightGBM with default settings. QLIKE = 0.158. The model overfits: training QLIKE = 0.041. The gap between training and validation loss is a red flag.

Step 3: Constrained. Set `max_depth`=4, `min_child_samples`=100, `subsample`=0.7, `colsample_bytree`=0.7, `learning_rate`=0.02, `num_iterations`=1500 with early stopping (patience 50 rounds on purged validation fold). QLIKE = 0.131. Now the model improves over HAR by 7.7%, and training QLIKE = 0.098, a much healthier gap.

Step 4: Grid search. Sweep `max_depth` $\in \{3, 4, 5\}$, `min_child_samples` $\in \{50, 100, 200\}$ via purged CV. Best: `max_depth`=4, `min_child_samples`=100 (confirming Step 3). This is a 12-trial search; record all trials in the experiment log for Deflated Sharpe Ratio correction (Chapter 17).

Early Stopping on Purged Validation

Early stopping is the single most important regularization technique for tree ensembles on small volatility data. But the validation set used for early stopping must respect the purged CV structure: no overlap between training and validation dates, with an embargo gap. If you use a random validation split, early stopping will stop too late because the validation set is contaminated by lookahead.

11.4 The Christensen–Siggaard–Veliyev Evidence

The most comprehensive academic horse-race of ML methods for realized volatility forecasting is Christensen et al. (2023). Their setup: 29 DJIA stocks, daily and weekly RV forecasting, with a feature set spanning RV lags, jumps, leverage effects, volume, and options-implied volatility.

Christensen, Siggaard, and Veliyev (2023)

Gradient-boosted trees (XGBoost) are among the top-performing models for daily RV forecasting across 29 DJIA stocks. Three findings matter most:

1. **Rich features amplify the ML advantage.** When the feature set includes only RV lags (what HAR uses), trees offer minimal improvement. When the feature set expands to include jumps, leverage, volume, and implied volatility, trees pull ahead by 5–15% in QLIKE.
2. **Longer horizons favor ML.** The gap between tree models and HAR is larger for weekly RV prediction than for daily. At longer horizons, nonlinear feature interactions matter more because the direct autoregressive signal decays.
3. **Accumulated Local Effects (ALE) plots reveal drivers.** ALE plots (a model-agnostic alternative to partial dependence) show that the tree models primarily exploit the interaction between lagged RV and implied volatility, exactly the HARQ-type interaction from Chapter 6, but estimated nonparametrically rather than imposed by hand.

The study uses rolling-window evaluation (not purged CV), which is the standard in the volatility forecasting literature. Features are constructed at each window using only past data; the model is re-estimated monthly. The out-of-sample period spans several years, covering both calm and crisis regimes.

11.5 The Optiver Kaggle Evidence

Academic papers evaluate models carefully but on relatively simple feature sets. The Optiver Realized Volatility Prediction competition (Kaggle, 2021) provides the complementary experiment: thousands of teams competing to predict 10-minute-ahead realized volatility from limit order book data across over 100 stocks.

Optiver Kaggle Competition (2021)

LightGBM ensembles dominated the leaderboard. The top solutions shared three characteristics:

1. **Feature engineering dominated.** WAP (weighted average price) returns, price acceleration, volume imbalance profiles, bid-ask spread dynamics, trade-flow toxicity (Chapter 10). Winners spent 80% of their effort on features and 20% on modeling.
2. **Trees beat deep learning.** Transformer and LSTM models were tried extensively.

They did not beat well-tuned LightGBM on the public or private leaderboard.

3. **Blending helped, but only modestly.** Top solutions blended 3–5 LightGBM models (different feature subsets or random seeds). Gains from blending were 1–3%, far smaller than gains from better features.

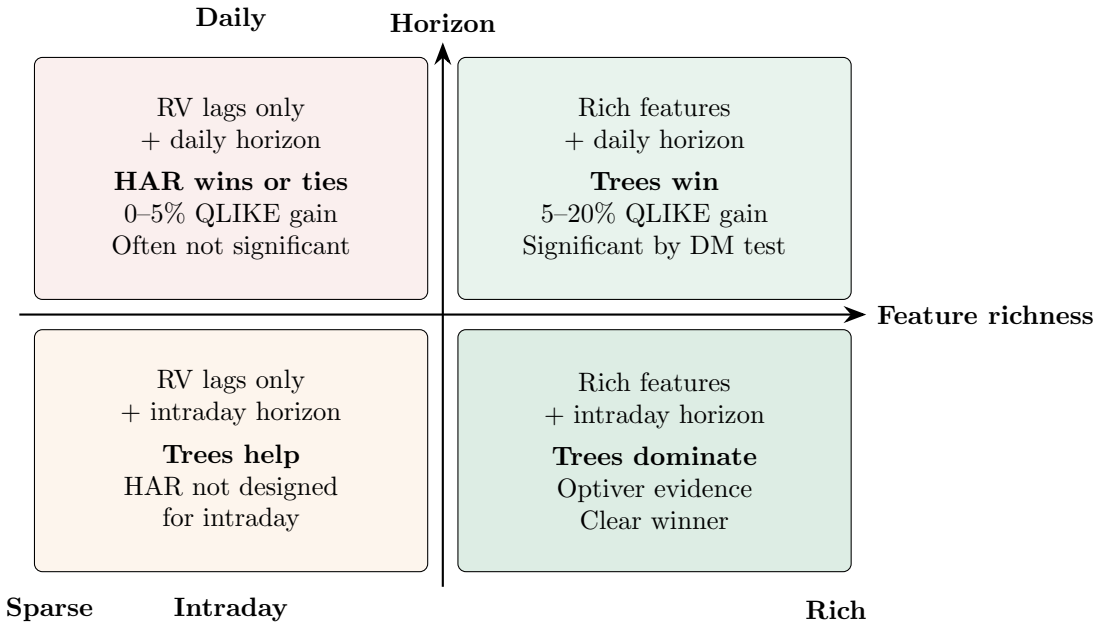
Features Over Architecture

The Optiver competition reinforces the lesson from Chapter 10: feature choice matters more than model choice. Trees plus good features consistently outperform deep learning plus raw data. If you are deciding where to spend your next hour of research time, spend it engineering a new feature, not tuning a neural network.

The Optiver setting differs from academic setups in two ways. First, the prediction horizon is 10 minutes, not one day, so HAR is not a natural benchmark (it was designed for daily or longer horizons). Second, the feature space includes tick-level order book data, which is far richer than what most academic studies use. The lesson about feature engineering transfers directly to the daily setting, but the absolute performance numbers do not.

11.6 The Honest Assessment

This is the most important section in the chapter. The evidence above might suggest that trees always win. They do not. Here is a regime-by-regime summary of when ML adds value, grounded in the literature.



Daily horizon, RV-only features: HAR is extremely competitive

When you give a tree ensemble the same three features HAR uses (RV_{t-1}, $RV_{t-1}^{(w)}$, $RV_{t-1}^{(m)}$), the improvement over HAR is 0–5% in QLIKE, and it is often not statistically significant by the Diebold–Mariano test (Chapter 17). HAR already captures the dominant autoregressive structure. Trees can only add nonlinear kinks, which are small and unstable on 1,250 observations.

Bollerslev et al. (2024) demonstrate that a rolling-window HAR with properly selected window length matches or beats off-the-shelf ML models. The key insight: HAR's advantage comes from

its parsimonious structure (3 parameters), which is well-suited to small, noisy, autocorrelated data.

Daily horizon, rich features: trees win

When you add the full feature set from Chapter 10 (implied volatility, jumps, signed semivariances, cross-asset, sentiment), trees pull ahead by 5–20% in QLIKE. The reason: these features contain nonlinear interactions that HAR cannot exploit without hand-crafting interaction terms. The tree ensemble discovers these interactions automatically.

Christensen et al. (2023) confirm this pattern: the ML advantage grows with feature-set richness.

Intraday horizons: trees are necessary

HAR was designed for daily (or longer) horizons. For 10-minute or 1-hour ahead prediction from order book data, HAR has no natural formulation. Trees operate on any feature matrix, regardless of the time scale, and the Optiver evidence shows they dominate at short horizons.

Stress regimes: ML stumbles

Here is the uncomfortable finding. ML models, including trees, tend to underperform HAR during extreme events (VIX spikes, flash crashes, pandemic onset). The reason: tree ensembles trained predominantly on calm-regime data have never seen the patterns that emerge during crises. They extrapolate poorly because trees are piecewise-constant; predictions in extreme leaves are based on very few training observations.

Rahimikia and Poon (2020) document this explicitly: their ML model beats HAR on 90% of out-of-sample days, but fails catastrophically on the remaining 10%, which are precisely the days that matter most for risk management. Section 11.7 addresses this.

Does anything beat linear models?

Branco et al. (2024) ask this question directly and find that the answer is "often no, when the comparison is fair." Their main point: many published ML-beats-HAR results use default hyperparameters for HAR (fixed window, OLS) while giving the ML model a full tuning budget. When both models receive equal care (rolling windows, feature selection, proper tuning), the gap shrinks or disappears for daily RV with standard features.

Where the ML Gains Come From

The gains from ML come from richer features and longer horizons, not from nonlinear modeling of the same three RV lags that HAR uses. If your tree model does not beat HAR on the same features, you have overfit noise (Chapter 6). If it does beat HAR, check whether the gain survives the Diebold–Mariano test and Deflated Sharpe Ratio (Chapter 17).

11.7 Ensemble with HAR

Section 11.6 revealed a tension: trees win most days but fail in stress. HAR is robust in stress but misses nonlinear patterns in calm periods. The natural solution: combine them.

Rahimikia and Poon (2020) propose exactly this. Their ML model (a random forest, but the logic extends to any tree ensemble) beats HAR on roughly 90% of out-of-sample days. On the remaining 10% (concentrated during stress), HAR wins decisively. The combined forecast outperforms both standalone models.

Simple weighted average

The simplest combination:

$$\hat{\sigma}^2_{\text{combo},t} = w \cdot \hat{\sigma}^2_{\text{HAR},t} + (1 - w) \cdot \hat{\sigma}^2_{\text{tree},t}, \tag{11.5}$$

where $w \in [0, 1]$ is estimated by minimizing QLIKE on a purged validation set. Typical values: $w \in [0.2, 0.4]$, giving the tree model majority weight but retaining HAR's stabilizing influence.

- $\hat{\sigma}^2_{\text{HAR},t}$ is the HAR forecast from Chapter 6.
- $\hat{\sigma}^2_{\text{tree},t}$ is the LightGBM or XGBoost forecast.
- w is the HAR weight, estimated on the validation set.

In Plain English

This is a weighted average of two forecasts. When $w = 0.3$, the combined forecast is 30% HAR and 70% tree. The weight is not a guess; you pick the w that minimizes QLIKE on a held-out validation set. The result inherits the tree's ability to capture nonlinear patterns while retaining HAR's stability during extreme events.

Why This Matters

The HAR + tree combination is a strong candidate for the final model in the GS project. It addresses the key weakness of standalone tree models (poor extrapolation in stress regimes) while preserving the QLIKE gains that trees deliver in normal markets. Tuning w on a purged validation set takes seconds, and the resulting ensemble is simple to explain to stakeholders.

Regime-switching combination

A more sophisticated version: let w depend on the current regime. Define a "stress indicator" s_t (e.g., $s_t = \mathbf{1}[\text{VIX}_t > 30]$ or a GMM-based regime probability). Then:

$$\hat{\sigma}^2_{\text{combo},t} = w(s_t) \cdot \hat{\sigma}^2_{\text{HAR},t} + [1 - w(s_t)] \cdot \hat{\sigma}^2_{\text{tree},t}, \tag{11.6}$$

with $w(s_t)$ increasing during stress (rely more on HAR when volatility is elevated). This connects directly to the regime overlay in Chapter 14.

In Plain English

Instead of using a fixed blend, the regime-switching version asks: "are markets calm or stressed right now?" In calm markets, it trusts the tree model more. In stressed markets, it shifts weight toward HAR, which extrapolates more reliably when volatility spikes to levels unseen in training data. The stress indicator s_t acts as a dial that adjusts the blend in real time.

Why This Matters

The regime-switching combination is the bridge to Chapter 14's full hybrid architecture. If your holdout period includes a volatility spike (e.g., a VIX jump above 30), this adaptive weighting can recover QLIKE losses that a fixed-weight blend would miss. It also provides a natural way to incorporate the VIX regime indicator from Chapter 10 into the forecast combination step.

Worked Example:

HAR + LightGBM Ensemble Continuing the SPY example from Section 11.3. On the 126-day holdout:

Model	QLIKE	Hit Rate (vs. HAR)
HAR alone	0.148	—
LightGBM alone	0.133	72% of days
Combo ($w = 0.3$)	0.126	78% of days

The combination achieves a further 5.3% QLIKE improvement over the standalone tree model. Inspecting the days where LightGBM alone underperforms, they cluster around the three largest VIX spikes in the holdout period. HAR's contribution on those days pulls the ensemble back toward the realized value.

Diebold–Mariano test, combo vs. HAR: $p = 0.02$ (significant at 5%). Combo vs. LightGBM alone: $p = 0.11$ (suggestive but not significant at 5%, reflecting the small holdout size).

Why Combining Works

HAR and tree ensembles make different types of errors. HAR's errors are small and unbiased in calm periods, large but mean-reverting in stress. Tree errors are small in calm periods but can be persistently biased during regimes unseen in training. Averaging diversifies the error, the same principle behind portfolio diversification.

11.8 DART: Dropout Regularization for Boosted Trees

Section 11.3 covered the standard regularization toolkit for gradient boosting: shallow trees, subsampling, and slow learning rates. There is one more technique worth understanding, and it addresses a subtle failure mode that the standard controls do not.

The over-specialization problem

Recall from Equation 11.1 that standard gradient boosting shrinks every tree's contribution by the same learning rate η. Early trees see large residuals (the raw signal), so they learn the dominant pattern: the strong autoregressive relationship between lagged RV and future RV. Later trees see only the leftover residuals after those early trees have already captured the bulk of the variance. Shrinkage treats every tree identically, so the early trees permanently dominate the ensemble's output.

This creates a problem called **over-specialization**: later trees become highly specialized to the narrow residual signal left by the first few trees. If those early trees overfit to noise in the training data, every subsequent tree inherits that bias. The ensemble's predictions become overly dependent on a small number of early trees.

In Plain English

Imagine a group project where one person does most of the work in the first week. Everyone else spends the remaining weeks patching small gaps in that person's draft. If the first person made a fundamental error, the entire project is built on a flawed foundation, and no amount of patching fixes it. Standard shrinkage is like asking everyone to speak more quietly; it does not change the fact that the first speaker set the direction.

Dropout applied to trees

DART (Dropouts meet Multiple Additive Regression Trees) solves over-specialization by borrowing the dropout idea from neural networks (Vinayak and Gilad-Bachrach, 2015). In each boosting round, instead of computing residuals from the full ensemble, DART randomly **drops** (removes) a subset of the previously built trees. The new tree is then trained on the residuals of the *reduced* ensemble, the one with the dropped trees removed.

Concretely, let $\mathcal{D} \subset \{1, \ldots, m-1\}$ denote the set of tree indices dropped at round m, chosen by including each tree independently with **drop rate** p. The prediction used to compute residuals becomes:

$$\hat{y}_t^{(\text{drop})} = \sum_{k \notin \mathcal{D}} h_k(\mathbf{x}_t), \tag{11.7}$$

where:

- $h_k(\mathbf{x}_t)$ is the prediction of tree k for observation t (already including its learned weight),
- $\mathcal{D}$ is the random dropout set for this boosting round,
- $\hat{y}_t^{(\text{drop})}$ is the reduced ensemble prediction (with dropped trees excluded).

The new tree h_m is fitted to the residuals $r_t = y_t - \hat{y}_t^{(\text{drop})}$. Because some trees were dropped, these residuals are larger than they would be under the full ensemble, so the new tree must learn to compensate for the missing trees. This forces it to capture general patterns rather than narrow residual corrections.

In Plain English

Dropout randomly benches some of the existing team members during each training round. The new recruit (tree) has to pick up their slack, which means learning broadly useful patterns rather than hyper-specialized corrections. After training, all trees are brought back for the final prediction. The result is an ensemble where each tree carries more independent information.

Prediction normalization

After fitting the new tree h_m, DART must normalize the ensemble so that adding the new tree does not inflate predictions. The dropped trees and the new tree are rescaled so the ensemble stays calibrated:

$$\hat{y}_t = \sum_{k \notin \mathcal{D}} h_k(\mathbf{x}_t) + \frac{|\mathcal{D}|}{|\mathcal{D}|+1} \sum_{k \in \mathcal{D}} h_k(\mathbf{x}_t) + \frac{1}{|\mathcal{D}|+1} h_m(\mathbf{x}_t), \tag{11.8}$$

where:

- $|\mathcal{D}|$ is the number of dropped trees,
- the factor $\frac{|\mathcal{D}|}{|\mathcal{D}|+1}$ rescales the dropped trees down to make room for the new tree,
- the factor $\frac{1}{|\mathcal{D}|+1}$ rescales the new tree so it contributes an "equal share" alongside the dropped trees.

In Plain English

Without normalization, the new tree would be trained against a gap left by the dropped trees (large residuals), so its raw predictions would be too large. The normalization splits the "budget" of the dropped trees evenly between the returning dropped trees and the

new tree. Think of it as redistributing playing time: the dropped players come back at slightly reduced minutes, and the new player gets a fair share.

DART versus standard shrinkage

The key difference from the learning rate η in standard boosting:

	Shrinkage (η)	**DART** (drop rate p)
Mechanism	Scales *every* tree by the same constant η	Randomly removes *entire trees* each round
Effect on early trees	Reduced proportionally but still dominant	May be dropped, forcing later trees to learn independently
Diversity	All trees see residuals from the same full ensemble	Each tree sees residuals from a different random subset
Analogy	Turning down everyone's volume equally	Randomly muting some speakers so others must step up

Shrinkage and DART are not mutually exclusive. In LightGBM, you can set `boosting_type='dart'` and still specify a learning rate, though in practice the learning rate is often set higher with DART (e.g., 0.05–0.1) because the dropout itself provides regularization.

DART for Volatility Forecasting

Over-specialization is particularly relevant for RV forecasting. The dominant signal in volatility data is the autoregressive persistence captured by HAR's three lagged averages (Chapter 6). In standard boosting, the first few trees learn this persistence, and all subsequent trees become narrowly specialized to small residual patterns that may not generalize. DART forces later trees to occasionally reconstruct the autoregressive signal on their own, building redundancy into the ensemble and reducing dependence on any single tree's overfitting.

In LightGBM, enable DART by setting `boosting_type='dart'`. The key hyperparameter is the drop rate: start with `drop_rate` $\in [0.05, 0.15]$ and tune via purged CV (Section 11.3). Higher drop rates increase diversity but slow convergence; lower rates approach standard GBDT behavior. Note that DART disables early stopping (because the loss is non-monotone due to random dropping), so you must set `num_iterations` explicitly rather than relying on patience-based stopping.

DART Disables Early Stopping

With standard GBDT, you monitor validation loss and stop when it plateaus. DART's random dropout makes the validation loss noisy and non-monotone from round to round, so early stopping triggers prematurely. When using DART, fix the number of boosting rounds via cross-validation rather than relying on early stopping. This increases tuning cost, but the regularization benefit of dropout often compensates.

11.9 Summary

1. Tree-based ensembles (LightGBM, XGBoost) are the default ML model for tabular volatility data because they automatically discover nonlinear interactions and threshold effects.

2. Gradient boosting builds the forecast sequentially: each tree corrects the residual errors of the ensemble so far (Equation 11.1).
3. For volatility forecasting, use a custom QLIKE loss (Equations 11.2–11.4), not the default MSE loss.
4. Default hyperparameters overfit on small volatility samples. Use shallow trees (`max_depth` 3–5), large minimum leaf sizes (50–200), aggressive subsampling (0.6–0.8), and slow learning rates (0.01–0.05) with early stopping on a purged validation set.
5. Christensen et al. (2023): trees are among the best for daily RV forecasting, with gains amplified by richer feature sets and longer horizons.
6. The Optiver Kaggle competition confirms that feature engineering matters far more than model architecture: trees plus good features beat deep learning plus raw data.
7. With RV-only features at daily horizons, HAR is extremely competitive. Trees offer 0–5% QLIKE improvement, often not significant.
8. With rich features (implied vol, jumps, cross-asset) at daily horizons, trees win by 5–20% in QLIKE.
9. At intraday horizons, trees are clearly necessary; HAR was not designed for this regime.
10. During extreme stress, tree models tend to underperform HAR because they extrapolate poorly from calm-period training data.
11. Combining HAR and tree forecasts (Equation 11.5) captures the best of both: the tree's nonlinear skill in calm markets and HAR's robustness in stress (Rahimikia and Poon, 2020).
12. Branco et al. (2024) and Bollerslev et al. (2024) caution that many ML-beats-HAR claims reflect unfair comparisons. When both models are properly tuned, the gap is smaller than headline numbers suggest.
13. DART (Vinayak and Gilad-Bachrach, 2015) applies dropout to boosted trees: randomly dropping previous trees during each boosting round prevents over-specialization, where later trees become overly dependent on early trees that captured the dominant autoregressive signal. Use `drop_rate` $\in [0.05, 0.15]$; note that DART disables early stopping.
14. The gains from ML come from richer features and longer horizons, not from nonlinear modeling of the same three RV lags. Chapter 13 explores whether deep learning changes this conclusion.

Paper	Key Result	Relevance
Christensen et al. (2023)	Trees among best for daily RV; gains grow with feature richness and horizon length	Primary evidence for Sections 11.4 and 11.6
Branco et al. (2024)	Linear models competitive when comparison is fair	Calibrates expectations in Section 11.6
Bollerslev et al. (2024)	Rolling-window HAR with proper window matches off-the-shelf ML	Strongest HAR defense
Rahimikia and Poon (2020)	ML beats HAR 90% of days, fails in stress; ensemble solves it	Motivates Section 11.7
Audrino and Knaus (2016)	QLIKE-optimized trees outperform MSE-optimized trees for RV	Justifies custom loss in Section 11.2
Gu et al. (2020b)	Trees and neural nets dominate linear models in cross-sectional return prediction with rich features	Canonical ML horse-race; context for tree methods
Vinayak and Gilad-Bachrach (2015)	Dropout for boosted trees reduces over-specialization; each tree learns more independently	Regularization technique in Section 11.8

Chapter 12

Rashomon Sets and Interpretable Trees

You have built a volatility model, run SHAP, and found that VIX is the most important feature. You refit on a slightly different training window and now ATM implied volatility takes over. A third refit: lagged RV_{t-1} wins. Which feature is *actually* important? The unsettling answer is that any single model's feature importance is a sample of size one from a large population of equally-good models. Drawing conclusions from that single sample is exactly the kind of reasoning we would reject in any other statistical context.

This chapter introduces a principled alternative. Instead of asking "what does *my* model say is important?" we ask "what do *all* near-optimal models agree is important?" The set of all such models is called the **Rashomon set**, and the tools for analyzing it give us something SHAP cannot: provably stable feature importance that does not change when you perturb the training window by a few days.

What You Need Before This Chapter

- **Chapter 11** — Decision tree fundamentals: how splits are chosen, how CART grows a tree, what makes a tree "optimal." The distinction between greedy (top-down) and globally optimal (branch-and-bound) tree construction.
- **SHAP basics** — The idea that SHAP assigns each feature a contribution to each individual prediction, and that global SHAP importance is obtained by averaging $|\phi_i|$ across predictions. You should know what a SHAP summary plot looks like, but you do *not* need to understand the Shapley value derivation in detail.
- **Feature importance concepts** — Permutation importance (shuffle a feature, measure how much loss increases) and split-count importance (how often a feature appears in tree splits). Both are covered in Chapter 11.

12.1 The Problem with Single-Model Explanations

Feature importance from a single model is not a fact about the data. It is a fact about *one model fit to the data*. If there are many models that fit the data almost equally well, each may assign very different importances, and the one we happen to report is determined by arbitrary choices: the random seed, the exact train/test split boundary, the hyperparameter grid, even the order in which features are processed.

12.1.1 Why SHAP Importance Is Unreliable with Near-Substitute Features

Consider two features that are highly correlated: VIX (the CBOE volatility index) and the ATM implied volatility from the SPX options surface. Both carry similar information about the market's forward-looking volatility expectation. A tree model must choose one or the other at each split. If the model happens to split on VIX first, VIX gets credit for the variance it explains, and ATM IV gets little credit because the residual information it adds is small. Reverse the split order and ATM IV gets the credit.

SHAP values (Lundberg and Lee, 2017) partially address this by considering all feature orderings. For a model f and a prediction at input x, the SHAP value for feature i is the unique additive attribution satisfying local accuracy, missingness, and consistency:

$$\phi_i(f,x) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F|-|S|-1)!}{|F|!} \big[f_x(S \cup \{i\}) - f_x(S)\big], \tag{12.1}$$

where:

- F is the full feature set, $|F| = M$
- S is a subset of features *excluding* feature i
- $f_x(S) = \mathbb{E}[f(z) \mid z_S = x_S]$ is the expected prediction when only features in S are observed

Why SHAP Doesn't Fully Solve the Problem

SHAP computes importances for all feature orderings, but it does so for a *single fixed model* f. The problem is upstream: the model f itself is just one of many near-optimal models. A different near-optimal model f' with the same predictive accuracy may produce entirely different SHAP values, especially for correlated features. Averaging over orderings within one model does not average over the space of *models*.

12.1.2 Importance Instability in Practice

The instability is not hypothetical. In volatility forecasting, features like VIX, VVIX, ATM IV, the 25-delta skew, and lagged RV_{t-1} are all moderately to highly correlated. A gradient-boosted tree fit on 2017–2021 data might rank VIX as the top feature, while the same model fit on 2017–2022 data (just one extra year) might rank RV_{t-1} first and demote VIX to third. The model's accuracy barely changes between these two fits, but the "feature importance story" changes completely.

This creates a serious practical problem. In an internship setting, you will present your model's feature importance to a desk or to a portfolio manager. If the ranking shuffles every time you retrain, your explanation is not credible. What you need is a statement like: "Across *all* models within 2% of optimal accuracy, VIX and ATM IV are interchangeable substitutes, but lagged RV is always important." That statement requires looking at all good models, not just one.

The Core Insight

What if we looked at *all* good models instead of just one? If a feature is important in every near-optimal model, that importance is robust. If it is important in some but not others, it is a substitute that can be swapped out. The distinction between "robustly important" and "substitutable" is invisible to any single-model explanation method.

12.2 The Rashomon Set

The idea of many equally-good models was articulated by Leo Breiman, who called it the **Rashomon effect** — a reference to the Akutagawa story (and Kurosawa film) in which multiple witnesses give contradictory but individually plausible accounts of the same event. The **Rashomon set** formalizes this: it is the collection of all models whose loss is close to the best achievable loss.

12.2.1 Formal Definition

We need three ingredients: a model class $\mathcal{F}$ (e.g., all decision trees of depth $\leq d$), a dataset $(\mathbf{X}, \mathbf{y})$, and a reference model f^* that achieves the best loss within $\mathcal{F}$.

ϵ-Rashomon Set (,)

Given a model class $\mathcal{F}$, a benchmark model $f^* \in \mathcal{F}$, and a tolerance $\epsilon > 0$, the **ϵ-Rashomon set** is

$$\mathcal{R}(\epsilon, f^*, \mathcal{F}) = \left\{ f \in \mathcal{F} \;\middle|\; L(f) \leq (1+\epsilon)\, L(f^*) \right\}, \tag{12.2}$$

where $L(f)$ is the loss (e.g., misclassification rate, squared error) of model f evaluated on the training data.

The symbols:

- ϵ — the tolerance parameter. Setting $\epsilon = 0.02$ means we accept models within 2% of the best loss. Typical values in the literature range from 0.01 to 0.10 (Xin et al., 2022).
- f^* — the reference model, usually the empirical risk minimizer within $\mathcal{F}$ (e.g., found by GOSDT for optimal sparse trees).
- $L(f^*)$ — the best achievable loss in the class.

In Plain English

The Rashomon set is the "cloud" of models that are all approximately tied for best. Imagine ranking every possible decision tree by its error on your data, then drawing a horizontal line at $1.02 \times$ (best error). Every tree below that line is in the Rashomon set. The set may contain tens, thousands, or even 10^{12} trees — all of them are defensible choices.

For decision trees specifically, Xin et al. (2022) define the objective function as the misclassification loss plus a sparsity penalty:

$$\text{Obj}(t, \mathbf{X}, \mathbf{y}) = \underbrace{\frac{1}{n} \sum_{i=1}^{n} \mathbb{1}[\hat{y}_i \neq y_i]}_{\text{misclassification loss}} + \underbrace{\lambda\, H_t}_{\text{sparsity penalty}}, \tag{12.3}$$

where:

- $\hat{y}_i$ is the prediction of tree t for observation i
- H_t is the number of leaves in tree t
- $\lambda > 0$ is a regularization parameter penalizing complexity

The Rashomon set threshold then becomes $\theta_\epsilon = (1+\epsilon) \times \text{Obj}(t_{\text{ref}}, \mathbf{X}, \mathbf{y})$, and any tree t with $\text{Obj}(t, \mathbf{X}, \mathbf{y}) \leq \theta_\epsilon$ is in the set (Xin et al., 2022).

Rashomon Sets for Regression

Our volatility project uses squared-error loss (or QLIKE), not misclassification. The Rashomon set concept applies identically: replace $L(f)$ with QLIKE or MSE and everything carries through. The computational algorithms (TreeFARMS, SPLIT) currently target classification trees, but the conceptual framework — and the VIC/RID analysis tools — is loss-function agnostic.

12.2.2 How Large Is the Rashomon Set?

The hypothesis space of sparse trees is enormous. For trees of depth at most 4 with only 10 binary features, the number of possible trees exceeds 9.3×10^{20} (Xin et al., 2022). The Rashomon set, fortunately, is usually a tiny fraction of this space — but "tiny fraction" can still be a very large number in absolute terms.

Xin et al. (2022) report that on the COMPAS dataset with $\lambda = 0.005$ and a 15% Rashomon threshold, the set contains approximately 10^{12} trees. On smaller datasets (Monk2, Bar), sets of 10^5 to 10^8 trees are typical. The key insight from their experiments is that *natural baselines (BART, Random Forest, CART + sampling) find at best a tiny sliver of the Rashomon set* — they recover only hundreds to thousands of trees when the true set contains millions or more.

Worked Example:

Two Trees, Different Features, Same Accuracy Consider a toy dataset with 100 observations, 3 binary features (X_1, X_2, X_3), and a binary label Y. Suppose the optimal tree t^* has 4 leaves, splits on X_1 then X_2, and achieves $\text{Obj}(t^*, \mathbf{X}, \mathbf{y}) = 0.08 + 0.01 \times 4 = 0.12$.
With $\epsilon = 0.05$, the Rashomon threshold is $\theta_{0.05} = 1.05 \times 0.12 = 0.126$.
Now consider tree t' with 3 leaves that splits on X_3 first, then X_1. Suppose it achieves $\text{Obj}(t', \mathbf{X}, \mathbf{y}) = 0.09 + 0.01 \times 3 = 0.12$.

Tree	Splits on	Leaves	Error	Objective
t^*	X_1, X_2	4	0.08	0.12
t'	X_3, X_1	3	0.09	0.12

Both trees are in the Rashomon set. Tree t^* relies heavily on X_2; tree t' does not use X_2 at all but uses X_3 instead. If we report SHAP importance for t^* alone, we conclude X_2 is important. The Rashomon set reveals that X_2 and X_3 are substitutes: the data supports models using either one.

12.3 Optimal Sparse Decision Trees

Before we can enumerate *all* near-optimal trees, we must be able to find *one* provably optimal tree. This is the foundation.

12.3.1 Why Not Just Use CART?

CART (Xin et al., 2022) and other greedy algorithms build trees top-down, choosing the best split at each node independently. This is fast — $O(npd)$ for n samples, p features, depth d — but greedy splits can be suboptimal. Babbar et al. (2025) show that greedy methods exhibit an average gap of 1–2 percentage points from the optimum, and on some datasets (e.g., COMPAS) the gap can reach 10 percentage points.

The problem with greedy construction is that a split that looks best at the root may set up poor options downstream. Global optimization avoids this by searching the *entire* space of trees up to a given depth.

12.3.2 Branch-and-Bound with Dynamic Programming

Modern optimal tree algorithms (GOSDT, MurTree) solve the optimization problem exactly:

$$\mathcal{L}^*(D, d, \lambda) = \min_{T \in \mathcal{T}} L(T, D, \lambda) \quad \text{s.t.} \;\; \text{depth}(T) \leq d, \tag{12.4}$$

where $L(T, D, \lambda) = \frac{1}{N} \sum_{i=1}^{N} \big(\ell(T(\mathbf{x}_i), y_i) + \lambda\, S(T)\big)$ is the regularized loss and $S(T)$ is the number of leaves (Babbar et al., 2025).

The key insight behind branch-and-bound is that the optimal solution for a dataset D at depth d' depends on the solutions for subsets $D(f)$ and $D(\bar{f})$ at depth $d' - 1$, where f is the splitting feature. Starting from the root, the algorithm considers all candidate features, tracking upper and lower bounds on the objective at each split. When the lower bound of a subtree exceeds the current best upper bound, that entire subtree is pruned.

In Plain English

Think of it as a chess engine: instead of playing the locally best-looking move (greedy), you search several moves ahead and pick the sequence that leads to the best position. Branch-and-bound makes this tractable by ruling out hopeless positions early.

12.3.3 SPLIT: Greedy Where It Doesn't Matter, Optimal Where It Does

A crucial empirical observation by Babbar et al. (2025) is that greedy splits near the *leaves* are almost always optimal, while greedy splits near the *root* often deviate from the optimum. This makes sense: near the leaves, there are only a few possible splits left, so the greedy choice among them is unlikely to be far from the best. Near the root, a bad split propagates errors through the entire tree.

SPLIT (Sparse Lookahead for Interpretable Trees) exploits this observation. It takes a **lookahead depth** parameter $d_l < d$ and performs full branch-and-bound optimization for splits up to depth d_l, then switches to greedy splitting for the remaining $d - d_l$ levels.

SPLIT Runtime

For a dataset with n samples, k binary features, depth budget d, and lookahead depth d_l, SPLIT (Algorithm 2 in Babbar et al. (2025)) has runtime

$$\mathcal{O}\big(n(d - d_l)\, k^{d_l+1} + n\, k^{d-d_l}\big).$$

The polynomial-time variant, LicketySPLIT, achieves $O(nk^2d^2)$ — comfortably polynomial and dramatically faster than the $\mathcal{O}((2k)^d)$ worst case of fully optimal methods.

Interpretable Trees for Vol Forecasting

For our volatility project, a depth-5 tree with 10–15 features is a realistic interpretable baseline. An optimal tree of this size can be found in under a second with LicketySPLIT. Every prediction traces to a root-to-leaf path, making it trivial to explain: "the model forecasts high volatility because yesterday's RV was above the 90th percentile AND the

VIX term structure is in backwardation." This transparency is valuable for presentations and for building trust with portfolio managers.

12.4 Enumerating the Rashomon Set

Finding one optimal tree is step one. The real power comes from finding *all* near-optimal trees — the complete Rashomon set. This section covers three algorithms of increasing scalability.

12.4.1 TreeFARMS: Exact Enumeration

TreeFARMS (Trees FAst RashoMon Sets) by Xin et al. (2022) was the first algorithm to *completely* enumerate the Rashomon set for sparse decision trees. It builds on the GOSDT branch-and-bound framework and modifies it in two key ways:

1. **Rashomon pruning.** Instead of pruning subproblems whose lower bound exceeds the optimal objective (as in standard GOSDT), TreeFARMS prunes those whose lower bound exceeds the *Rashomon threshold* θ_ϵ. This retains more of the search space — exactly the near-optimal region.
2. **Return all models.** Instead of returning only the single best tree, TreeFARMS returns every tree in the Rashomon set, stored in a compact **Model Set** data structure.

The Model Set representation is critical for scalability. A **Model Set Instance (MSI)** represents a subproblem paired with its objective value. Many trees share identical subtrees, and the Model Set exploits this by storing shared components once. The loss function for decision trees takes on a discrete set of values (approximately n distinct values for n training samples), while the number of trees in the Rashomon set can be orders of magnitude larger. By grouping trees with the same objective, TreeFARMS avoids massive data duplication.

TreeFARMS: From Optimization to Feasibility

Standard ML algorithms solve an *optimization* problem: find the single best model. TreeFARMS solves a *feasibility* problem: find all models within ϵ of the best. This paradigm shift is the core contribution. Perhaps the tiny sacrifice in empirical risk makes the difference between a model that can be used (interpretable, fair, aligned with domain knowledge) and one that cannot.

Limitations. TreeFARMS provides exact enumeration but its runtime and memory scale exponentially with tree depth and the number of features. On the Bike dataset ($n \approx 17{,}000$, $k = 60$ binary features, depth 5), TreeFARMS requires approximately 700 seconds and over 50 GB of memory (Heile et al., 2025). On larger datasets, it runs out of memory entirely.

12.4.2 RESPLIT: Fast Approximation via Greedy Leaves

Babbar et al. (2025) extend SPLIT to Rashomon set computation with the RESPLIT algorithm. The idea is elegant: use SPLIT to find a set of "prefix trees" (partial trees optimized to the lookahead depth), then call TreeFARMS on each prefix to enumerate the near-optimal completions.

Because SPLIT uses greedy splits near the leaves, RESPLIT does not exhaustively search the full tree space. This makes it an *approximation*: it may miss some trees in the true Rashomon set. However, Babbar et al. (2025) show that the approximation is remarkably accurate. On six benchmark datasets, the Pearson correlation between variable importances computed from the RESPLIT-approximated Rashomon set and the full Rashomon set is nearly 1.0:

Dataset	Full (s)	RESPLIT (s)	Speedup	τ (correlation)
COMPAS	152	18	8×	1.000
Spambase	2,659	154	17×	0.930
Netherlands	4,255	216	20×	0.932
HELOC	5,564	337	17×	0.979
HIV	9,273	388	24×	0.959
Bike	14,330	194	74×	0.999

Source: Table 1 in Babbar et al. (2025). Parameters: 10 bootstrapped datasets, $\lambda = 0.02$, $\epsilon = 0.01$, depth 5, lookahead depth 3.

In Plain English

RESPLIT is like asking: "what if we only optimize the important splits (near the root) and let the unimportant splits (near the leaves) be good enough?" The answer: you get almost the same set of near-optimal trees, 10–20 times faster, with almost identical downstream conclusions about which features matter.

12.4.3 LicketyRESPLIT: Polynomial-Time Approximation

Heile et al. (2025) push the scalability further with LicketyRESPLIT, which replaces TreeFARMS entirely with a polynomial-time enumeration strategy. At each node, the algorithm considers all features and prunes splits whose LicketySPLIT-completed cost would exceed the Rashomon budget $B = (1+\epsilon)\cdot\text{LicketySPLIT}(D, \lambda, d)$. It then recursively enumerates left and right subtrees, filtering by the remaining budget.

The key theoretical results are:

LicketyRESPLIT Complexity (Heile et al. 2025)

Given a dataset D of size n with k features and max depth d:

- **Runtime:** $\mathcal{O}(|R|\, n\, k^3\, d^3)$, where $|R|$ is the size of the recovered Rashomon set. This is polynomial in n, k, and d, and linear in $|R|$.
- **Memory:** $\mathcal{O}\big(nk + \sum_{f \in R} S(f)\big)$, proportional to the input size plus the output size.

In practice, LicketyRESPLIT achieves order-of-magnitude improvements over both TreeFARMS and RESPLIT:

Dataset	LicketyRESPLIT Time / RAM	TreeFARMS Time / RAM	RESPLIT Time / RAM
Bike	**18.8 s / 438 MB**	685 s / 51 GB	184 s / 528 MB
Bank	**123 s / 776 MB**	OOM	238 s / 2 GB
Covertype	**507 s / 1.8 GB**	1819 s / 68 GB	1295 s / 3 GB
Student	**1.7 s / 370 MB**	351 s / 4.7 GB	4.0 s / 382 MB

Source: Table 1 in Heile et al. (2025). Parameters: $\lambda = 0.01$, $\epsilon_{mult} = 0.01$, max depth = 5.

Despite being an approximation, LicketyRESPLIT achieves near-perfect precision and recall relative to the true Rashomon set. On six benchmark datasets, precision is ≥ 0.91 and recall is ≥ 0.90 across all tested configurations (Heile et al., 2025).

Exact vs. Approximate Enumeration

TreeFARMS gives you the *exact* Rashomon set: every tree in the set, and no tree outside it. RESPLIT and LicketyRESPLIT are approximations — they may miss a few trees (recall < 1) and occasionally include a tree slightly outside the boundary (precision < 1). For downstream variable importance analysis, this distinction rarely matters: the approximate sets produce virtually identical importance rankings. But if you need a formal guarantee ("no tree in the Rashomon set uses feature X_7"), only exact enumeration suffices.

The landscape of Rashomon set algorithms is evolving rapidly. Figure 12.1 summarizes the three approaches and their trade-offs.

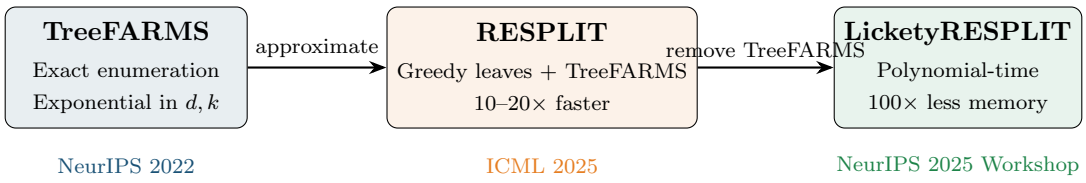


Figure 12.1: Evolution of Rashomon set enumeration algorithms for sparse decision trees. Each generation trades a small amount of exactness for dramatic improvements in scalability.

12.5 What the Rashomon Set Reveals

Having the Rashomon set is only useful if we can extract insights from it. Two complementary tools turn the raw set of models into actionable conclusions about feature importance.

12.5.1 Model Class Reliance (MCR)

The simplest analysis is to ask: for each feature, what is the *range* of its importance across all models in the Rashomon set? This is **Model Class Reliance (MCR)**, introduced by Dong and Rudin (2020) (building on the concept from Fisher et al., 2019).

For a single model f, the **model reliance** of feature j is defined as the ratio of the loss when feature j is randomly permuted to the original loss:

$$mr_j^{\text{ratio}}(f) = \frac{L(f;\, [X_{\setminus j},\, \bar{X}_j],\, Y)}{L(f;\, X,\, Y)}, \tag{12.5}$$

where:

- $X_{\setminus j}$ denotes all features except feature j
- $\bar{X}_j$ is an independent copy of X_j (i.e., feature j is reshuffled)
- A reliance of 1.0 means the feature contributes nothing; larger values indicate greater importance

In Plain English

Model reliance is permutation importance: scramble a feature and see how much worse the model gets. MCR asks: across all near-optimal models, what is the minimum and maximum permutation importance of each feature?

MCR defines two key quantities for each feature j:

$$\text{MCR}_-(j) = \min_{f \in \mathcal{R}} mr_j(f), \tag{12.6}$$

$$\text{MCR}_+(j) = \max_{f \in \mathcal{R}} mr_j(f). \tag{12.7}$$

The interpretation is clean:

- A feature with large MCR_- is important in *every* well-performing model. It is robustly important.
- A feature with small MCR_+ is unimportant in every well-performing model. It can be safely excluded.
- A feature with small MCR_- but large MCR_+ is a *substitute*: important in some models, not in others. It can be swapped for a correlated alternative without loss of accuracy.

Xin et al. (2022) showed that TreeFARMS enables *exact* MCR computation for decision trees by directly calculating variable importance for every tree in the set and then finding the min and max.

12.5.2 Variable Importance Clouds (VIC)

MCR gives a one-dimensional summary (min and max importance) for each feature independently. **Variable Importance Clouds (VIC)** (Dong and Rudin, 2020) capture the full joint structure.

Variable Importance Cloud (,)

The **model reliance function** $MR : \mathcal{F} \to \mathbb{R}^p$ maps each model to a vector of its reliances on all p features:

$$MR(f) = \big(mr_1(f),\ mr_2(f),\ \ldots,\ mr_p(f)\big).$$

The **Variable Importance Cloud** of the Rashomon set $\mathcal{R}$ is the set of all such vectors:

$$\text{VIC}(\mathcal{R}) = \big\{MR(f) : f \in \mathcal{R}\big\}. \tag{12.8}$$

The VIC lives in p-dimensional space, where each axis represents the importance of one feature. To visualize it, Dong and Rudin (2020) project the VIC onto all pairs of features, producing **Variable Importance Diagrams (VIDs)** — 2D scatter plots that reveal substitution patterns:

- **Non-overlapping projections** along one axis: the two features have robustly distinct importance levels. One is always more important than the other, regardless of model choice.
- **Overlapping projections**: the features are substitutes. Some near-optimal models rely on one, some on the other.
- **Negative slope** in the 2D projection: the features are direct substitutes — as one's importance increases, the other's decreases. This is the signature of correlated features competing for the same splits.

Worked Example:

VIX, VVIX, and ATM IV as Near-Substitutes Imagine constructing the Rashomon set for a volatility classification task ("will tomorrow's RV exceed its 90th percentile?") using depth-4 trees with the following features: lagged RV_{t-1}, RV_{t-5}, VIX, VVIX, and ATM

IV.
After enumeration, we compute $MR(f)$ for each tree f in the set and project onto two pairs:
Pair 1: VIX vs. ATM IV. The 2D cloud shows a strong negative slope — when a tree relies heavily on VIX, it does not rely on ATM IV, and vice versa. The projections onto each axis overlap substantially. These features are *substitutes*: the data supports using either one, but not both simultaneously.
Pair 2: Lagged RV_{t-1} vs. VIX. The cloud is a compact blob with no clear slope. The projection of RV_{t-1} onto its axis is consistently high (all models rely on it), while VIX varies. Lagged RV is *robustly important*; VIX is a useful but substitutable addition.

Conclusion for feature selection: We must include RV_{t-1}. We may include either VIX or ATM IV (but including both adds redundancy, not information). VVIX may or may not add value depending on what else is in the model.

VIC vs. Bootstrapped SHAP

A natural reaction is: "I could just bootstrap the data, refit SHAP each time, and get a distribution of importances." This is fundamentally different from VIC. Bootstrapped SHAP resamples the *data* but always uses the same model class and always reports the importance of the single best model within that class. VIC holds the data fixed and varies the *model*: it considers every near-optimal model, not just the best one.
The distinction matters because bootstrap variability conflates data uncertainty with model uncertainty. VIC isolates model uncertainty: "given this exact dataset, how much does the importance depend on which near-optimal model we happened to pick?" This is the right question when your concern is the stability of your explanations, not the variability of your data.

12.5.3 Rashomon Importance Distributions (RID)

The **Rashomon Importance Distribution (RID)** proposed by Donnelly et al. (2023) goes beyond the min-max range of MCR by computing the full *distribution* of each feature's importance across the Rashomon set. For each feature j, we collect $\{mr_j(f) : f \in \mathcal{R}\}$ and examine its distribution — median, interquartile range, and tails.

The RID provides confidence intervals for feature importance that are fundamentally different from those produced by bootstrapping:

- **Bootstrap CI:** "If we drew a new dataset from the same population, feature j's importance in the best model would lie in this interval with 95% probability." This is data uncertainty.
- **RID CI:** "Given *this* dataset, feature j's importance ranges from a to b across all near-optimal models." This is model-choice uncertainty.

Babbar et al. (2025) demonstrate that RID computed from the RESPLIT-approximated Rashomon set is nearly identical to RID from the full set, with Pearson correlations above 0.93 across all tested datasets. This means the fast approximate methods are reliable enough for practical RID computation.

RID for Volatility Feature Selection

For our project, RID would answer questions like: "Is the VIX term structure slope a robustly important feature, or does its apparent importance depend on which model we happen to pick?" If the RID for a feature is concentrated near 1.0 (no importance), we

can safely drop it. If it is spread from 1.0 to 1.4, the feature is a substitute — useful in some models but not all. If it is concentrated above 1.2, the feature is robustly important and should always be included.

12.6 Rashomon Analysis for Volatility Forecasting

No published work has applied Rashomon set analysis to financial time-series forecasting. This is an open frontier, and it is directly relevant to our project. This section describes how the tools from Sections 12.4–12.5 could be deployed for realized volatility forecasting.

12.6.1 Regime-Stable Feature Selection

Volatility features that matter in a low-vol regime (2017–2019) may not matter in a crisis regime (March 2020) or a post-crisis recovery (2021). Single-model SHAP run on the full sample averages over these regimes, potentially concluding that a feature is "moderately important" when it is actually critical in crises and irrelevant otherwise.

A Rashomon-based approach to regime-stable feature selection:

1. **Rolling-window Rashomon sets.** For each rolling window (e.g., 252-day training sets advanced by one month), compute the Rashomon set and the MCR for each feature.
2. **Intersection across regimes.** A feature that has $\text{MCR}_- > 1$ (i.e., minimum importance above the baseline) in *every* window is regime-stable. A feature whose MCR range includes 1.0 in some windows is regime-dependent.
3. **Feature selection rule.** Include only regime-stable features in the final model. For regime-dependent features, consider regime-conditional inclusion (e.g., add VIX term structure only when VIX > 20).

In Plain English

Instead of asking "is this feature important right now in my current model," ask "is this feature important in every near-optimal model, in every regime I've seen?" Features that pass this test are the ones you can defend in a presentation. The ones that fail are the ones that will embarrass you when someone asks "why did the ranking change?"

12.6.2 Prediction Multiplicity: Quantifying Model-Choice Uncertainty

The Rashomon set also reveals **prediction multiplicity**: the range of forecasts that different near-optimal models produce for the same input. For a given feature vector $\mathbf{x}_t$ (today's features), we can compute $\{f(\mathbf{x}_t) : f \in \mathcal{R}\}$ and report the min, max, and spread.

If all near-optimal models agree that tomorrow's volatility will be high, that is a strong signal. If some predict high and others predict low, the forecast is sensitive to model choice — and we should report wider confidence intervals or flag the day as uncertain.

This is conceptually similar to the disagreement among models in an ensemble, but with a crucial difference: ensemble members are not guaranteed to be near-optimal, while Rashomon set members are.

12.6.3 Defensible Presentations

In a Goldman Sachs internship, you will present model results to people who will ask hard questions: "Why did your model use VIX and not ATM IV?" or "Your SHAP plot from last

month looked completely different."

Rashomon analysis gives you defensible answers:

- "I examined all 14,000 decision trees within 2% of optimal accuracy. Lagged RV is the top feature in every single one of them. VIX and ATM IV are interchangeable — the data supports either, so I chose VIX for simplicity."
- "The feature importance is not anecdotal. It is *provably stable*: no near-optimal model exists that does not rely on lagged RV."
- "The prediction range across all near-optimal models for tomorrow is $[0.00012, 0.00018]$. The spread tells you how much of the forecast uncertainty comes from model choice rather than data noise."

The Novelty of This Application

As of 2025, no published paper applies Rashomon set analysis to financial time-series data. The tools (TreeFARMS, RESPLIT, LicketyRESPLIT) have been demonstrated on tabular classification datasets (COMPAS, FICO, UCI benchmarks). Applying them to volatility forecasting would require adapting the framework to regression trees and squared-error or QLIKE loss, but the conceptual apparatus — enumerate near-optimal models, compute MCR, examine VIC — carries over directly. This is a genuine opportunity for novel contribution.

12.7 Summary and Connections

This chapter introduced a fundamentally different approach to model interpretability. Rather than explaining one model after the fact (SHAP), we examine the entire space of near-optimal models before committing to one.

Aspect	Single-Model (SHAP)	Rashomon Set
What is explained	One model's predictions	All near-optimal models
Feature importance	Point estimate for one model	Range $[\text{MCR}_-, \text{MCR}_+]$
Stability	Changes with refit	Provably stable within ϵ
Substitute detection	Not possible	Overlapping VIC projections
Model-choice uncertainty	Not quantified	Prediction multiplicity
Computational cost	Fast (one model)	Hours (enumeration needed)

Looking ahead. Chapter 13 will introduce deep learning methods for volatility, which are powerful but even harder to interpret than tree ensembles. The Rashomon perspective suggests a hybrid strategy: use deep models for raw predictive power, but use interpretable trees (and their Rashomon sets) to understand *which features matter and why*. This is not an either/or choice — it is a complementary toolkit.

Chapter 13

Deep Learning for Volatility

Where This Chapter Fits

Chapter 11 showed that tree-based methods are the workhorse for tabular volatility features. This chapter asks: when do neural networks offer something trees cannot? The answer is narrow but real: sequential raw data (LOB, tick streams), cross-asset pooling, and latent factor extraction. Project 2 (Intraday RV from LOB) and Project 4 (Rough Vol vs Deep Learning) use architectures from this chapter.

What You Need

- Backpropagation, gradient descent, and the idea of a loss function.
- Matrix multiplication at the level of $\mathbf{W}\mathbf{x} + \mathbf{b}$.
- Sigmoid $\sigma(\cdot)$ and ReLU$(\cdot)$ activations.
- The HAR model from Chapter 6 and the tree methods from Chapter 11.

13.1 LSTMs and GRUs

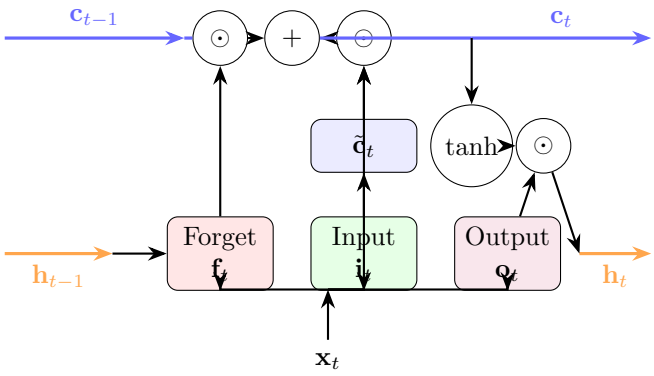
Chapter 11 established that gradient-boosted trees dominate *tabular* volatility forecasting. But volatility data is often sequential: a stream of 5-minute returns, a sequence of LOB snapshots, a panel of daily risk measures across assets. Recurrent neural networks, specifically the Long Short-Term Memory (LSTM) and the Gated Recurrent Unit (GRU), are purpose-built for sequences.

Why Sequences Need Special Architecture

A standard feedforward network treats its inputs as an unordered bag of numbers. If you feed it $(\text{RV}_{t-1}, \text{RV}_{t-2}, \ldots, \text{RV}_{t-22})$, it does not know that RV_{t-1} is yesterday and RV_{t-22} is a month ago. A recurrent network processes inputs one step at a time, maintaining a hidden state $\mathbf{h}_t$ that summarizes everything it has seen so far. This hidden state is the network's "memory."

13.1.1 LSTM Cell Architecture

The LSTM cell solves the vanishing gradient problem that cripples simple recurrent networks. It uses three gates (forget, input, output) and a cell state $\mathbf{c}_t$ that acts as a conveyor belt for information.



LSTM Cell Equations

At each time step t, given input $\mathbf{x}_t$ and previous hidden state $\mathbf{h}_{t-1}$:

$$\mathbf{f}_t = \sigma\big(\mathbf{W}_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f\big) \quad \text{(forget gate)} \tag{13.1}$$

$$\mathbf{i}_t = \sigma\big(\mathbf{W}_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i\big) \quad \text{(input gate)} \tag{13.2}$$

$$\tilde{\mathbf{c}}_t = \tanh\big(\mathbf{W}_c[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c\big) \quad \text{(candidate cell state)} \tag{13.3}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \text{(cell state update)} \tag{13.4}$$

$$\mathbf{o}_t = \sigma\big(\mathbf{W}_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o\big) \quad \text{(output gate)} \tag{13.5}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{(hidden state)} \tag{13.6}$$

where:

- $[\mathbf{h}_{t-1}; \mathbf{x}_t]$ is the concatenation of previous hidden state and current input.
- $\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_c, \mathbf{W}_o$ are learned weight matrices.
- $\mathbf{b}_f, \mathbf{b}_i, \mathbf{b}_c, \mathbf{b}_o$ are bias vectors.
- $\odot$ denotes element-wise (Hadamard) multiplication.
- $\sigma(\cdot)$ squashes values to $[0, 1]$; each gate is a "soft switch."

The Three Gates in Plain English

- **Forget gate $\mathbf{f}_t$**: "How much of the old memory should I keep?" Values near 0 erase; near 1 retain.
- **Input gate $\mathbf{i}_t$**: "How much of the new candidate should I write to memory?"
- **Output gate $\mathbf{o}_t$**: "How much of the current memory should I expose as my prediction?"

The cell state $\mathbf{c}_t$ flows through the network with only linear interactions (multiply and add), so gradients flow back through many time steps without vanishing. This is why LSTMs can learn long-range dependencies that simple RNNs cannot.

Why This Matters

For volatility forecasting, the forget gate learns *persistence*: how much of yesterday's volatility regime carries over to today. The input gate learns how much weight to give today's new information (today's realized volatility, jump indicators, or news sentiment). The output gate controls what the model passes forward as its forecast. An LSTM can learn the HAR structure automatically (daily, weekly, monthly lags map to different hidden-state components), but it can also discover more complex nonlinear patterns that HAR misses, such as asymmetric responses to positive and negative shocks. Because neural nets accept arbitrary loss functions, you can train the LSTM directly on QLIKE

rather than MSE, which is straightforward in PyTorch or TensorFlow.

The GRU simplifies the LSTM by merging the forget and input gates into a single "update gate" and eliminating the separate cell state. GRUs have fewer parameters and train faster; in practice, performance differences between LSTM and GRU are small for volatility tasks.

13.1.2 LSTMs for Realized Volatility

Bucci (2020) provides the cleanest comparison. He tests LSTM and NARX (nonlinear autoregressive with exogenous inputs) networks against HAR for daily RV forecasting across multiple assets.

Bucci (2020): LSTM for Daily RV

LSTM networks are competitive with HAR for daily RV forecasting, but they do not consistently dominate. Gains are asset-dependent and sensitive to hyperparameter tuning. The LSTM's advantage appears mainly during volatile periods when the relationship between past and future RV is nonlinear.

This result is consistent with the lesson from Chapter 11: on daily RV with only lagged RV as inputs, simple models are hard to beat. The LSTM becomes genuinely useful when you change what you feed it.

Sirignano and Cont (2019) demonstrate a powerful idea: *pooling* data across assets. Instead of training one LSTM per stock, they train a single "universal" LSTM on all stocks simultaneously.

Sirignano–Cont (2019): Universal Features via Pooling

A single LSTM trained on pooled data across 1,000+ stocks learns universal features of price formation. Pooling works because volatility dynamics are similar across assets (the same "rough" kernel from Chapter 7). The pooled model outperforms asset-specific models, especially for assets with short histories.

Cross-Asset Pooling

Trees cannot easily share learned representations across assets; you train one model per asset or flatten everything into rows. Neural networks naturally pool: a single LSTM processes sequences from many assets, learning shared dynamics while allowing asset-specific variation through the hidden state. If volatility dynamics are truly universal (Hurst $H \approx 0.1$ across assets, as Chapter 7 showed), pooling should help, and it does.

Rosenbaum and Zhang (2022) connect LSTMs directly to rough volatility. They show that a universal LSTM, trained to forecast volatility, learns a kernel that matches the fractional kernel of the RFSV model from Chapter 7.

Rosenbaum–Zhang (2022): LSTM Rediscovers Roughness

An LSTM trained on raw volatility data learns the same power-law kernel $K(t) \propto t^{H-1/2}$ with $H \approx 0.1$ that defines the rough fractional stochastic volatility (RFSV) model. The LSTM and the RFSV forecast are nearly identical, suggesting both learn the same underlying structure.

This is a beautiful result. The LSTM was given no prior knowledge of rough volatility, fractional Brownian motion, or Hurst exponents. It discovered roughness from data alone.

Rahimikia and Poon (2020) push the LSTM further by adding LOB features and news sentiment as inputs alongside standard RV lags.

Rahimikia–Poon (2020): LSTM with LOB + Sentiment

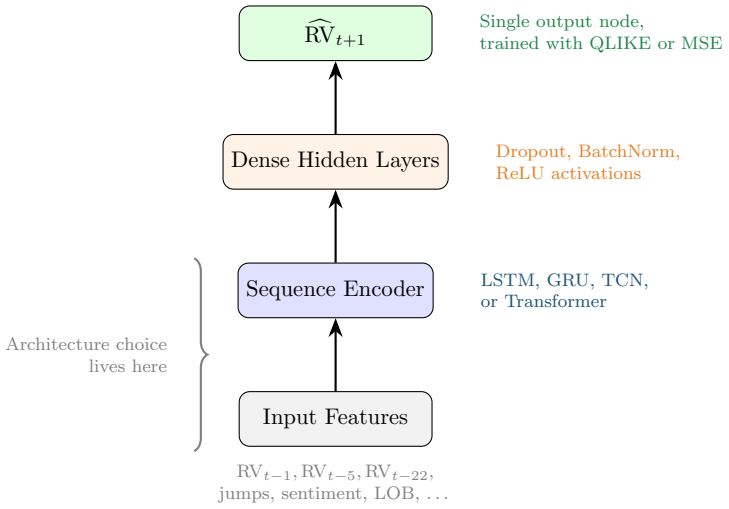
An LSTM incorporating limit order book features and news sentiment beats HAR on approximately 90% of trading days. However, the model fails during extreme stress events, precisely when accurate forecasts matter most.

Stress-Period Failure

Deep learning models trained on normal-regime data can fail catastrophically during crises. The LSTM in Rahimikia and Poon (2020) underperforms HAR during high-volatility episodes because the training data contains few such events. This is not specific to LSTMs; it affects all flexible models. Always evaluate forecast performance conditional on volatility regime (see Chapter 17).

When LSTMs Beat Trees for Volatility

LSTMs shine when you feed them *raw sequences* (intraday returns, LOB snapshots, tick data) rather than pre-computed features. On pre-computed tabular features, trees are usually better (Chapter 11). The LSTM's value comes from learning feature representations that you did not know to engineer by hand.



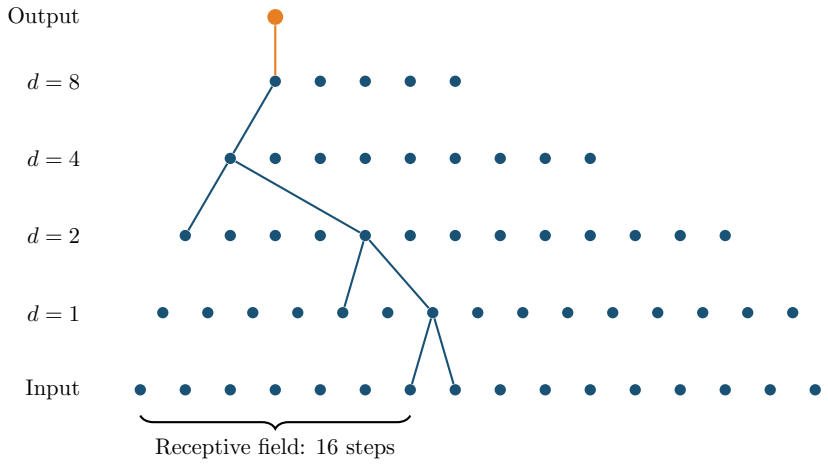
13.2 Temporal Convolutional Networks

LSTMs process sequences one step at a time. This is conceptually clean but creates a computational bottleneck: you cannot parallelize across time steps because each step depends on the previous hidden state. Temporal Convolutional Networks (TCNs) solve this by using *causal convolutions*: filters that only look backward in time, applied in parallel across the entire sequence.

13.2.1 Dilated Causal Convolutions

The key innovation is *dilation*. A standard 1D convolution with kernel size k looks at k consecutive time steps. A dilated convolution with dilation factor d skips $d-1$ steps between inputs, so a kernel of size k covers a receptive field of $k + (k-1)(d-1)$ steps. By doubling the dilation

factor at each layer ($d = 1, 2, 4, 8, \ldots$), the receptive field grows exponentially while the number of parameters grows only linearly.



Dilated Causal Convolution

For a 1D input sequence $(x_0, x_1, \ldots, x_T)$, a dilated causal convolution with kernel $\mathbf{w} = (w_0, \ldots, w_{k-1})$ and dilation factor d computes:

$$y_t = \sum_{j=0}^{k-1} w_j \cdot x_{t-j \cdot d} \tag{13.7}$$

where:

- k is the kernel size (typically 2 or 3).
- d is the dilation factor; layer ℓ uses $d = 2^{\ell-1}$.
- **Causal**: y_t depends only on x_s for $s \leq t$. No future information leaks.
- With L layers and kernel size $k = 2$, the receptive field is 2^L time steps.

Why Dilation Works

Think of each layer as a zoom level. Layer 1 (dilation 1) sees adjacent time steps: tick-by-tick patterns. Layer 2 (dilation 2) sees every other step: short-term trends. Layer 4 (dilation 8) sees steps 8 apart: the overall shape of the day. With just 8 layers and kernel size 2, you cover $2^8 = 256$ time steps. That is an entire trading day of 5-minute bars (78 bars) with room to spare.

Why This Matters

The dilated convolution is the building block of DeepVol, a leading architecture for forecasting daily RV from raw intraday returns. By stacking a handful of layers, the network covers an entire trading day (or week) without hand-engineering the daily, weekly, and monthly windows that HAR uses. This means the model can discover temporal patterns at any scale, not just the three fixed horizons baked into HAR. For the HARQ baseline project, a TCN trained on the same 5-minute return data provides a strong "does deep learning add anything?" comparison.

13.2.2 DeepVol

Moreno-Pino and Zohren (2022) build **DeepVol**, a TCN that forecasts daily RV directly from raw intraday returns. Instead of computing RV_t from 5-minute returns and then modeling RV_{t+1}

as HAR does, DeepVol feeds the raw 5-minute returns into a dilated causal convolution stack and predicts RV_{t+1} end-to-end.

DeepVol: TCN for Daily RV from Raw Returns

DeepVol achieves state-of-the-art daily RV forecasting by processing raw 5-minute returns directly, bypassing the RV aggregation step entirely. The dilated causal convolution architecture provides two advantages over LSTMs:

1. **Parallelizable training**: all time steps are processed simultaneously.
2. **Interpretable receptive field**: you know exactly how many past time steps influence each prediction.

Worked Example:

DeepVol Receptive Field Calculation Suppose DeepVol uses $L = 7$ layers with kernel size $k = 2$ and dilation factors $d = 1, 2, 4, 8, 16, 32, 64$.

Receptive field per layer: Each layer with dilation d and kernel size 2 adds d new time steps to the receptive field. Total receptive field:

$$\text{RF} = 1 + \sum_{\ell=0}^{L-1} (k-1) \cdot 2^{\ell} = 1 + \sum_{\ell=0}^{6} 2^{\ell} = 1 + 127 = 128 \text{ steps.}$$

In trading time: At 5-minute sampling, one trading day has 78 bars (6.5 hours $\times$ 12 bars/hour). A receptive field of 128 steps covers $128/78 \approx 1.6$ trading days. This means each prediction uses roughly today's and yesterday's intraday data.

To cover one trading week (5 days, 390 bars), you need:

$$L = \lceil \log_2(390) \rceil = 9 \text{ layers} \quad (\text{RF} = 512 \text{ steps}).$$

TCN vs. LSTM for Volatility

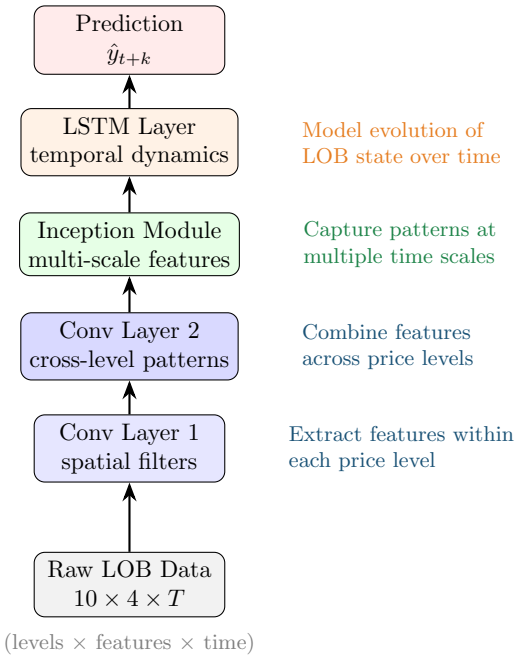
- TCN processes the entire input sequence in one forward pass; LSTM must step through sequentially.
- TCN's receptive field is explicit (2^L with kernel size 2); LSTM's effective memory is learned and opaque.
- TCN has no vanishing gradient problem by construction (parallel paths through residual connections).
- LSTM is more flexible for variable-length sequences; TCN requires fixed-length input (or padding).
- For fixed-length volatility sequences (e.g., one day of 5-min returns), TCN is generally preferred.

13.3 DeepLOB: Learning from Limit Order Books

The limit order book (LOB) is the richest source of short-horizon information in modern markets. At each moment, the LOB shows the queue of buy and sell orders at every price level. Chapter 10 discussed hand-crafted LOB features (order imbalance, depth ratios, spread). Zhang et al. (2019) ask: can a neural network learn better features directly from the raw LOB?

13.3.1 Architecture

DeepLOB combines CNNs (for spatial features across price levels) with LSTMs (for temporal dynamics across snapshots).



DeepLOB Input Representation

The input to DeepLOB at time t is a tensor $\mathbf{X}_t \in \mathbb{R}^{T \times 10 \times 4}$:

- T **time steps**: a rolling window of LOB snapshots (e.g., $T = 100$).
- **10 price levels**: the 5 best bid and 5 best ask levels.
- **4 features per level**: price, volume, price difference from mid, volume difference from mid (on each side).

The CNN layers convolve across the 10 price levels (spatial dimension), extracting patterns like order imbalance and depth concentration. The LSTM then processes the sequence of CNN-extracted features over time.

DeepLOB: End-to-End LOB Prediction

DeepLOB outperforms hand-crafted LOB features for short-horizon ($k = 10, 20, 50$ tick) mid-price movement prediction. The convolutional layers learn features that resemble (but improve upon) classical order imbalance measures. The architecture generalizes across stocks without retraining, consistent with the universal-features finding of Sirignano and Cont (2019).

DeepLOB for Volatility

DeepLOB was designed for mid-price direction prediction, but the same architecture predicts short-horizon RV from LOB data. Project 2 (Intraday RV from LOB) adapts DeepLOB by replacing the classification output with a regression head predicting RV over the next k ticks. The key insight: LOB state (depth imbalance, spread dynamics, queue lengths) contains predictive information about short-term volatility that is not captured by price-based features alone.

Worked Example:

DeepLOB Input Dimensions Consider S&P 500 E-mini futures (ES) with LOB snapshots every 100 milliseconds.
Input window: $T = 100$ snapshots (10 seconds of data).
Tensor shape: $100 \times 10 \times 4 = 4{,}000$ input values per sample.
Training data: One trading day has $6.5 \times 3{,}600 \times 10 = 234{,}000$ snapshots. With $T = 100$, this yields $\approx 234{,}000$ training samples per day per instrument.
Contrast with tabular features: A tree-based model using hand-crafted LOB features might have 20–30 features computed from the same raw data. DeepLOB uses 4,000 raw values and lets the CNN learn the right features. This is why deep learning wins on raw sequential data: the feature space is too rich for manual engineering.

13.4 Transformers and Attention

Transformers replaced LSTMs as the dominant sequence model in NLP (GPT, BERT) and are now being applied to financial time series. The core mechanism is *self-attention*: instead of processing the sequence step-by-step, the transformer computes a weighted sum over all positions, where the weights are learned from the data.

Scaled Dot-Product Attention

Given queries $\mathbf{Q}$, keys $\mathbf{K}$, and values $\mathbf{V}$ (all derived from the input via learned linear projections):

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{13.8}$$

where:

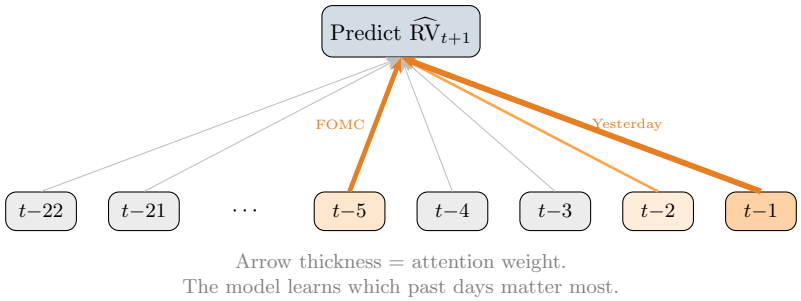
- $\mathbf{Q} \in \mathbb{R}^{T \times d_k}$: queries ("what am I looking for?").
- $\mathbf{K} \in \mathbb{R}^{T \times d_k}$: keys ("what do I contain?").
- $\mathbf{V} \in \mathbb{R}^{T \times d_v}$: values ("what information do I carry?").
- $\sqrt{d_k}$: scaling factor to prevent dot products from growing large.
- The softmax produces a $T \times T$ attention weight matrix; entry (i, j) is how much position i attends to position j.

Attention for Volatility

In an LSTM, the influence of a past observation on the current prediction decays as it recedes into the hidden state. With attention, every past observation can directly influence the current prediction with a learned weight. For volatility, this means the model can learn that, say, last Tuesday's intraday pattern is highly relevant to today's forecast (perhaps both are FOMC days) without relying on the hidden state to carry that information across the intervening days.

Why This Matters

Attention lets the vol-forecasting model look back flexibly rather than at fixed windows. HAR hard-codes three lookback horizons (1 day, 5 days, 22 days); an attention layer learns *which* past days matter for each prediction. If FOMC days, triple-witching days, or earnings dates carry outsized information, attention can upweight them automatically. The attention weight matrix also provides interpretability: you can inspect which past days the model attended to and verify that its behavior is economically sensible.



13.4.1 Graph Transformers for Cross-Asset Volatility

Chen and Roberts (2022) extend the transformer with a graph structure over assets. Instead of treating each asset's time series independently, the attention mechanism defines a *learned graph*: the attention weight between asset i and asset j captures how much asset j's history informs the forecast for asset i.

Attention Weights as a Volatility Network

The attention weight matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ across N assets is a learned adjacency matrix. High attention from asset i to asset j means "j's recent volatility is informative for predicting i's volatility." This connects directly to the spillover analysis in Chapter 16 and the multivariate models in Chapter 15, but learns the network structure from data rather than imposing it via a VAR or DCC.

Transformer variants have also been applied to LOB data (TLOB), replacing the LSTM component of DeepLOB with multi-head self-attention. Early results are promising but not yet conclusive.

Transformer Evidence on Volatility Is Thin

Transformer results on volatility are preliminary. Most published results use short evaluation periods or small asset universes. Be skeptical of claims that lack Diebold-Mariano tests or Model Confidence Set analysis (Chapter 17). The core problem: transformers have many parameters and volatility datasets are small. A daily RV series for one asset gives you ~252 observations per year. Even 20 years is only 5,000 observations, far fewer than the millions of sentences used to train language models. Pooling across assets helps (Section 13.1), but the evidence base is still thin compared to LSTMs and TCNs.

13.5 Modern Time-Series Architectures

The deep-learning-for-time-series field has produced a rapid succession of architectures: N-BEATS, N-HiTS, TiDE, TSMixer, PatchTST, and others. These were designed for general time-series forecasting (energy demand, weather, retail sales) and tested primarily on benchmarks like M4, M5, and the Monash archive. Their application to realized volatility is limited but worth knowing about.

N-BEATS (Neural Basis Expansion Analysis) uses a stack of fully-connected blocks with residual connections. Each block produces a "backcast" (reconstruction of the input) and a "forecast," and the blocks are organized into stacks that can be given interpretable basis functions (trend, seasonality).

N-HiTS extends N-BEATS with hierarchical interpolation, allowing different blocks to operate at different temporal resolutions (similar in spirit to the dilated convolutions of TCN).

PatchTST (Patch Time Series Transformer) divides the input time series into non-overlapping patches (subsequences) and treats each patch as a token for a transformer. This is conceptually similar to treating multi-day windows of 5-minute returns as input tokens.

TiDE (Time-series Dense Encoder) and **TSMixer** use MLP-based architectures that are simpler and faster than transformers while achieving competitive performance on standard benchmarks.

Modern Architectures: Solutions Looking for a Problem?

These architectures are well-engineered for general forecasting, but their evidence on RV specifically is thin. Most have not been tested with proper volatility evaluation (QLIKE loss, Diebold-Mariano tests against HAR, Model Confidence Set). PatchTST is the most promising for volatility because its patching mechanism naturally handles the multi-scale structure of intraday data (5-minute patches within daily windows within weekly patterns). Test them if you have the engineering bandwidth, but do not expect them to dominate purpose-built approaches like DeepVol or the well-tuned HAR.

13.6 Generative and Latent-Variable Approaches

The methods above produce point forecasts: a single number $\widehat{\mathrm{RV}}_{t+1}$. Generative models instead learn the *full distribution* of future volatility, or learn compressed representations of complex objects like the implied volatility surface. These are frontier methods with thin evidence, but they represent the direction the field is heading.

13.6.1 Neural SDEs and CDEs

Kidger (2021) develops the mathematical framework for embedding stochastic differential equations into neural networks. A neural SDE replaces the hand-specified drift and diffusion of classical models (Heston, SABR, rough Bergomi) with neural networks that learn these functions from data:

$$dX_t = f_{\boldsymbol{\theta}}(X_t, t)\, dt + g_{\boldsymbol{\theta}}(X_t, t)\, dW_t \tag{13.9}$$

where:

- $f_{\boldsymbol{\theta}}(\cdot)$: drift function, parameterized by a neural network with weights $\boldsymbol{\theta}$.
- $g_{\boldsymbol{\theta}}(\cdot)$: diffusion function, also a neural network.
- W_t: standard Brownian motion.

In Plain English

A classical stochastic volatility model says "volatility evolves according to this specific formula" (e.g., Heston's square-root process). A neural SDE says "volatility evolves according to *whatever function* a neural network learns from the data." The drift $f_{\boldsymbol{\theta}}$ captures the predictable part of how volatility moves (mean reversion, trends), and the diffusion $g_{\boldsymbol{\theta}}$ captures the randomness (vol-of-vol, jump intensity). Both are flexible enough to approximate any continuous function, so you are not locked into a particular parametric assumption.

Why This Matters

For the HARQ baseline project, neural SDEs offer a way to move beyond point forecasts to full distributional predictions of future RV. Instead of predicting "tomorrow's RV is 15%," the neural SDE produces a probability distribution: "15% is the median, but there is a 5% chance RV exceeds 30%." This is directly useful for variance risk premium

estimation and tail-risk-aware portfolio construction. The controlled differential equation (CDE) variant handles irregularly-spaced tick data naturally, which matters for intraday volatility estimation.

This is appealing for volatility because the ground truth *is* an SDE (Chapter 2). The neural SDE learns the drift and diffusion from data without committing to a specific parametric form (Heston, SABR, rough Bergomi). The controlled differential equation (CDE) variant handles irregularly-spaced observations, which is natural for tick data.

Neural SDEs Are Data-Hungry

Training neural SDEs requires backpropagation through the SDE solver (adjoint method), which is computationally expensive. The diffusion function $g_{\boldsymbol{\theta}}$ is especially hard to learn because it governs the noise, not the signal. With typical volatility dataset sizes (a few thousand daily observations), neural SDEs are prone to overfitting. They are most promising when combined with cross-asset pooling and intraday data, where sample sizes are orders of magnitude larger.

13.6.2 Autoencoders for the Implied Volatility Surface

The implied volatility (IV) surface from Chapter 8 is high-dimensional: IV values at dozens of strikes and maturities, updated continuously. Ding et al. (2025) use autoencoders to compress this surface into a low-dimensional latent space.

Autoencoder for IV Surface

An autoencoder consists of:

- **Encoder** $f_{\boldsymbol{\theta}} : \mathbb{R}^{K \times M} \to \mathbb{R}^d$: maps the IV surface ($K$ strikes $\times$ M maturities) to a latent vector $\mathbf{z} \in \mathbb{R}^d$ with $d \ll K \times M$.
- **Decoder** $g_{\boldsymbol{\theta}} : \mathbb{R}^d \to \mathbb{R}^{K \times M}$: reconstructs the surface from the latent vector.
- **Training objective**: minimize reconstruction error $\|\text{surface} - g_{\boldsymbol{\theta}}(f_{\boldsymbol{\theta}}(\text{surface}))\|^2$.

The latent vector $\mathbf{z}$ is a compressed summary of the entire IV surface. Typical latent dimensions are $d = 3$ to 8, meaning the IV surface (perhaps $20 \times 10 = 200$ values) is compressed to fewer than 10 numbers.

IV Latent Factors as Volatility Features

The latent dimensions often correspond to interpretable factors: level (parallel shift in IV), slope (term structure tilt), and smile (convexity across strikes). These latent factors can be used as features for RV forecasting (Chapter 10). This connects the options-implied information from Chapter 8 to the ML pipeline in a compact, learnable way.

13.6.3 Deep Stochastic Volatility

Xu and Chen (2021) propose a deep stochastic volatility model that combines the structure of classical stochastic vol (latent variance process driving observed returns) with the flexibility of neural networks. The latent variance follows a neural-network-parameterized transition, and inference uses variational methods (a VAE-like architecture). This produces a full posterior distribution over future volatility, not just a point forecast.

13.6.4 Normalizing Flows for RV Distribution

Du et al. (2023) use normalizing flows co-trained with a VAE to model the full distribution of future RV.

Normalizing Flow (Sketch)

A normalizing flow transforms a simple base distribution $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ through a sequence of invertible, differentiable transformations $T_1, T_2, \ldots, T_K$:

$$\mathbf{x} = T_K \circ T_{K-1} \circ \cdots \circ T_1(\mathbf{z}) \tag{13.10}$$

The log-density of the output is computed via the change-of-variables formula:

$$\log p(\mathbf{x}) = \log p(\mathbf{z}) - \sum_{k=1}^{K} \log \left| \det \frac{\partial T_k}{\partial T_{k-1}} \right| \tag{13.11}$$

where:

- Each T_k is designed to be invertible with a tractable Jacobian determinant.
- The composition of simple transformations can model complex, non-Gaussian distributions.
- For RV forecasting, the flow learns to map a Gaussian to the empirical distribution of future RV conditional on past information.

In Plain English

Start with a blob of points drawn from a simple Gaussian (a bell curve). Now warp that blob through a series of smooth, reversible stretches and squishes. After enough transformations, the warped blob can match any complicated distribution you want, such as the right-skewed, heavy-tailed distribution of realized volatility. Equation (13.10) is the forward direction (Gaussian → complex distribution), and Equation (13.11) tells you how to compute the probability of any point in the output space by tracking how much each transformation stretches or compresses the density. The key constraint is that every transformation must be invertible, so you can always go backward from data to the Gaussian and compute exact likelihoods.

Why This Matters

For realized volatility, normalizing flows produce a full predictive distribution rather than a single point forecast. This is valuable for two reasons: first, the QLIKE loss cares about the ratio $\mathrm{RV}/\widehat{\mathrm{RV}}$, and knowing the full distribution lets you optimize QLIKE-like objectives directly. Second, the variance risk premium (VRP) depends on the expected future distribution of RV, not just its mean, so distributional forecasts feed directly into VRP trading strategies. Compared to quantile regression (which gives you a few percentiles), a normalizing flow gives you the entire density, enabling richer risk analysis.

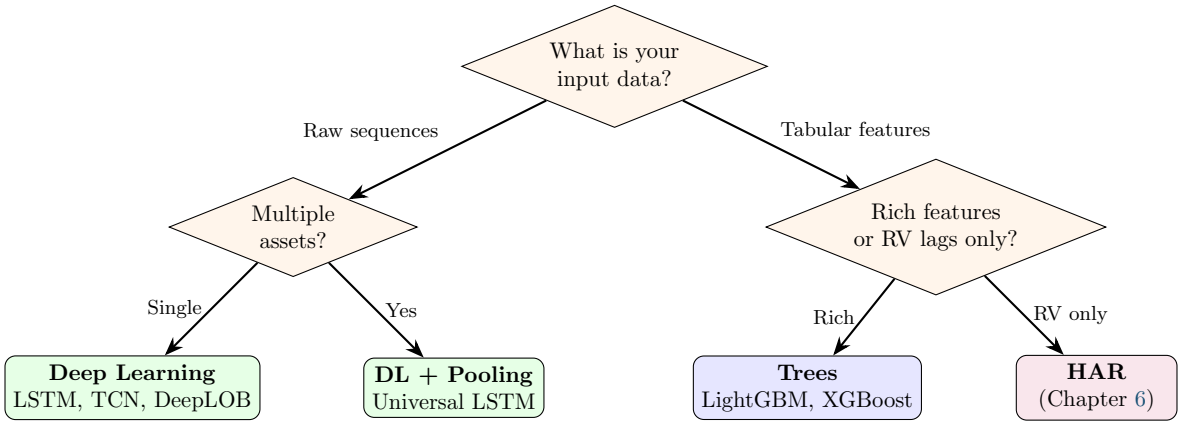
Why Distributional Forecasts Matter for Volatility

A point forecast $\widehat{\mathrm{RV}}_{t+1} = 15\%$ tells you the expected level. A distributional forecast tells you "15% is the median, but there is a 10% probability that RV exceeds 30%." This matters for risk management (you care about the tail, not the mean) and for the variance risk premium (Chapter 9), where the entire distribution of future RV determines the fair price of variance swaps. Normalizing flows and VAEs produce these distributional

forecasts naturally.

13.7 The Honest Bottom Line

Deep learning is neither a silver bullet nor useless for volatility forecasting. The evidence, taken honestly, points to a clear division of labor.



Here is the evidence, scenario by scenario:

1. **Daily horizon, RV lags only.** HAR often matches deep learning (Bucci, 2020). Trees are slightly better than DL due to less overfitting on small datasets (Chapter 11). Do not use a 50,000-parameter LSTM when three coefficients suffice.
2. **Daily horizon, rich tabular features.** Trees still usually win. The "tabular data advantage" of gradient-boosted trees over neural networks is well-documented: trees handle heterogeneous features, missing values, and small samples more gracefully.
3. **Raw sequential data (intraday returns, LOB snapshots).** Deep learning wins clearly. DeepVol (Moreno-Pino and Zohren, 2022) beats HAR and trees by processing raw 5-minute returns. DeepLOB (Zhang et al., 2019) beats hand-crafted LOB features. This is where DL adds genuine value: learning representations from raw, high-dimensional, sequential inputs.
4. **Cross-asset pooling.** Deep learning enables pooled training across assets (Sirignano and Cont, 2019). Trees cannot easily share learned representations between assets. If volatility dynamics are universal (Chapter 7), pooling helps, and neural networks make it natural.
5. **Distributional forecasts.** Normalizing flows, VAEs, and neural SDEs produce full predictive distributions. Trees and HAR produce point forecasts (or require separate quantile models). If you need the full distribution (for VRP pricing or tail risk), generative models have an architectural advantage.
6. **Longer horizons (weekly, monthly).** Evidence is mixed. DL can exploit richer temporal patterns at longer horizons, but the sample size shrinks (52 weekly observations per year), which favors simpler models.

The Decision Rule

Use deep learning when your data is sequential and raw, or when you want to pool across assets. Use trees when your data is tabular features. Use HAR when you have nothing but RV lags. If in doubt, start with HAR, add trees if you have features, and only reach for DL if you have raw sequential data or need cross-asset pooling.

Overfitting Is the Central Risk

Deep learning models have orders of magnitude more parameters than HAR or trees. A 2-layer LSTM with 64 hidden units has ∼50,000 parameters. A HAR model has 4. A LightGBM with max_depth=4 and 200 trees has ∼3,000 leaf values. On a daily RV series with 2,500 observations (10 years), the LSTM has 20 parameters per observation. Regularization (dropout, weight decay, early stopping) is not optional; it is survival. Always compare against the HAR baseline on identical data.

13.8 Summary

- LSTMs and GRUs are the default sequential architecture for volatility. They use gates (forget, input, output) and a cell state to maintain long-range memory.
- On daily RV with only lagged RV inputs, LSTMs are competitive with but do not consistently beat HAR (Bucci, 2020). The value of LSTMs comes from feeding them raw sequences, not pre-computed features.
- Cross-asset pooling is a genuine advantage of neural networks. A universal LSTM trained on many assets outperforms asset-specific models (Sirignano and Cont, 2019) because volatility dynamics are similar across assets.
- LSTMs trained on raw volatility data rediscover the rough-volatility kernel ($H \approx 0.1$) without any prior knowledge of fractional processes (Rosenbaum and Zhang, 2022).
- Temporal Convolutional Networks (TCNs) use dilated causal convolutions to grow the receptive field exponentially while maintaining parallelizable training. DeepVol (Moreno-Pino and Zohren, 2022) achieves state-of-the-art daily RV forecasting from raw 5-minute returns.
- DeepLOB (Zhang et al., 2019) combines CNNs (spatial features across LOB levels) with LSTMs (temporal dynamics) and outperforms hand-crafted LOB features for short-horizon prediction.
- Transformers and attention mechanisms can capture long-range dependencies and learn cross-asset networks via attention weights (Chen and Roberts, 2022), but evidence on volatility tasks remains thin. Be skeptical of results without proper statistical testing.
- Modern time-series architectures (N-BEATS, PatchTST, TiDE, TSMixer) are well-engineered but have limited evidence on RV. PatchTST is the most promising due to its natural handling of multi-scale temporal structure.
- Generative methods (neural SDEs, autoencoders, normalizing flows) produce distributional forecasts or compressed representations of complex surfaces, but require large datasets and careful regularization.
- **The honest bottom line**: use DL for raw sequential data and cross-asset pooling; use trees for tabular features; use HAR for RV-only lags. Always compare against HAR as a baseline.
- Overfitting is the central risk. A 2-layer LSTM has ∼50,000 parameters; HAR has 4. Regularization (dropout, early stopping, weight decay) is mandatory, not optional.
- LSTMs can fail during extreme stress (Rahimikia and Poon, 2020). Always evaluate performance conditional on volatility regime.

Paper	Architecture	Key Result	Relevance
Bucci (2020)	LSTM, NARX	Competitive with HAR, not dominant	Daily RV b
Sirignano and Cont (2019)	Universal LSTM	Pooling across assets improves forecasts	Cross-asset
Rosenbaum and Zhang (2022)	Universal LSTM	Learns rough-vol kernel ($H \approx 0.1$)	Theory vali
Rahimikia and Poon (2020)	LSTM + LOB + sentiment	Beats HAR 90% of days; fails in stress	Feature rich
Moreno-Pino and Zohren (2022)	TCN (DeepVol)	SOTA daily RV from raw 5-min returns	Raw sequer
Zhang et al. (2019)	CNN + LSTM (DeepLOB)	Beats hand-crafted LOB features	LOB predic
Chen and Roberts (2022)	Graph Transformer	Learned cross-asset attention network	Multi-asset
Kidger (2021)	Neural SDE/CDE	Principled SDE inside neural network	Distributio
Ding et al. (2025)	Autoencoder	IV surface compressed to $d \leq 8$ factors	Feature ext
Xu and Chen (2021)	Deep stochastic vol	Full posterior over future volatility	Uncertainty
Du et al. (2023)	Normalizing flow + VAE	Full predictive distribution of RV	Distributio

Next: Chapter 14 combines the strengths of HAR, trees, and deep learning into hybrid and ensemble models.

Chapter 14

Hybrid and Ensemble Models

14.1 Why This Chapter Matters

From Components to Combinations

Chapters 11 and 13 showed that ML models improve on HAR primarily through richer features and nonlinear interactions, not by replacing the linear structure that HAR captures well. This chapter asks: why not let HAR handle the linear part and train ML only on the residual? Hybrid models do this, and they are the safest bet in the volatility forecasting literature (Rahimikia and Poon, 2020). Project 1 (HARQ-X with ML Residual Augmentation) is a hybrid by design.

You already have two powerful toolkits: the econometric models of Chapters 5–6 and the machine-learning models of Chapters 11–13. Each has blind spots. HAR is parsimonious but linear; gradient-boosted trees are flexible but hungry for signal. This chapter shows you how to combine them so that each component operates where it is strongest.

The core philosophy is simple: *decompose*, then *specialize*. Let a well-understood econometric model absorb the predictable structure, then aim the full power of ML at whatever remains. The resulting hybrid almost always outperforms either component alone, and it does so with lower variance and greater interpretability than a pure ML approach (Rahimikia and Poon, 2020).

14.2 Why Hybrids Win

14.2.1 The Variance Budget Argument

Consider a forecast target RV_{t+1}. A fitted HAR model typically explains 40–60% of next-day realized-volatility variation with just three coefficients (Rahimikia and Poon, 2020). That leaves 40–60% in the residual $e_t = \text{RV}_{t+1} - \widehat{\text{HAR}}_t$. If you train an ML model directly on RV_{t+1}, it must rediscover the linear structure that HAR already captures perfectly, wasting model capacity and inviting overfitting. If instead you train on e_t, three things change for the better:

1. **HAR never overfits.** Three coefficients on lagged averages cannot memorize noise, so the linear component is rock-solid out of sample.
2. **Residual targets have lower variance.** The signal-to-noise ratio for the ML component improves because you have removed the dominant low-frequency trend.
3. **Ensemble diversification.** Even if the ML model adds only modest accuracy, combining two weakly correlated forecasters reduces overall forecast variance by the standard

diversification identity (Rahimikia and Poon, 2020).

Let HAR Do the Heavy Lifting

If HAR explains 60% of next-day RV with 3 coefficients, let it. Train ML on the residual. You get the reliability of HAR plus the flexibility of ML, without asking either model to do the other's job.

14.2.2 Variance Decomposition

Figure 14.1 illustrates the variance budget argument visually. The total variance of next-day RV is split into what HAR explains, what ML can capture from the residual, and irreducible noise.

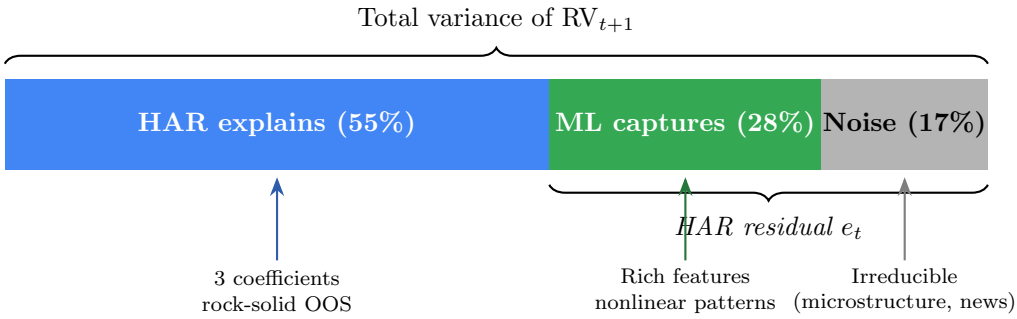


Figure 14.1: Variance budget for next-day RV forecasting. HAR absorbs the dominant linear signal cheaply; ML targets only the residual variance, where the signal-to-noise ratio is lower but nonlinear patterns exist. Noise is irreducible regardless of model complexity.

14.2.3 Architecture Diagram

Figure 14.2 illustrates the two-stage hybrid pipeline that recurs throughout this chapter. The key insight is that the ML model never sees the raw target; it only sees the residual that HAR could not explain.

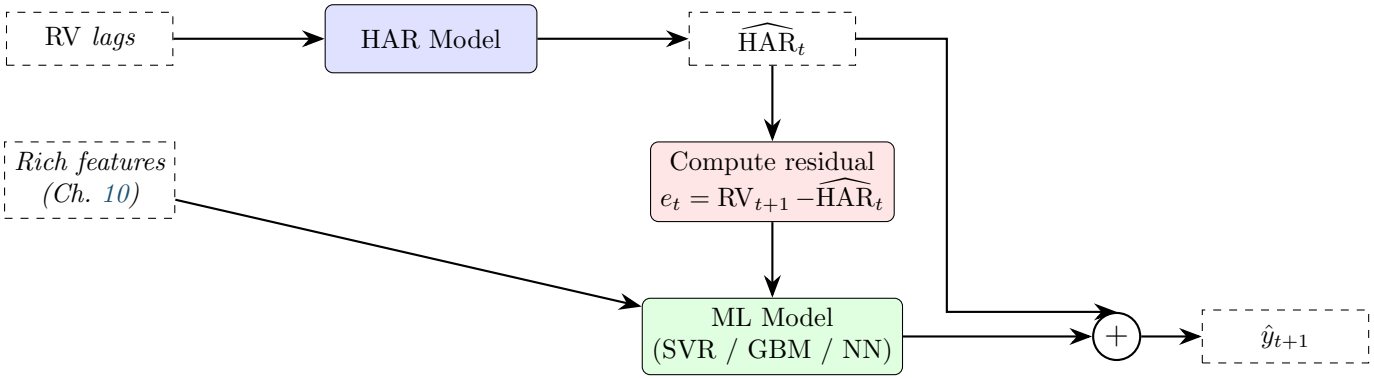


Figure 14.2: Two-stage hybrid pipeline. HAR absorbs the linear trend; ML targets only the residual. The final forecast sums both components.

14.3 HAR-SVR: Support Vector Regression on HAR Residuals

14.3.1 Intuition

Support vector regression (SVR) is a natural first choice for the residual stage because its ε-insensitive loss function automatically ignores small residuals. If the HAR fit is already good for most days, many residuals will be near zero. SVR treats those as "correct enough" and focuses capacity on the days where HAR fails most, such as post-jump or regime-change dates (Rahimikia and Poon, 2020).

Why SVR Suits Residuals

SVR's ε-tube acts as a built-in noise filter. Days where HAR is close enough (residual inside the tube) contribute zero loss. SVR concentrates its support vectors on the hard cases, which is exactly what you want when most of the signal has already been removed.

14.3.2 Procedure

1. Fit HAR on the training set:

$$\widehat{\mathrm{HAR}}_t = \hat{\beta}_0 + \hat{\beta}_d \, \mathrm{RV}_t + \hat{\beta}_w \, \mathrm{RV}_t^{(w)} + \hat{\beta}_m \, \mathrm{RV}_t^{(m)} . \tag{14.1}$$

2. Compute training residuals:

$$e_t = \mathrm{RV}_{t+1} - \widehat{\mathrm{HAR}}_t. \tag{14.2}$$

In Plain English

Steps 1–2 are the "let HAR go first" principle in action. You fit a standard HAR model, record its predictions, and then subtract those predictions from the actual realized volatility. The residual e_t is everything HAR could not explain: jump effects, leverage asymmetry, regime shifts, and noise. The ML model in the next step will see only this leftover signal.

Why This Matters

This two-step decomposition is the backbone of Project Direction 1 (HARQ-X + ML residual). Because HARQ already adapts its coefficients to measurement-error regimes, its residuals are even cleaner than plain HAR residuals, giving the downstream ML model a higher signal-to-noise starting point.

3. Construct a feature matrix $\mathbf{X}_t$ from the feature engineering toolkit of Chapter 10 (jump indicators, leverage, signed volatility, VIX basis, microstructure noise proxies).

4. Fit SVR with radial basis function (RBF) kernel on $(e_t, \mathbf{X}_t)$:

$$\min_{w,b} \; \frac{1}{2}\|w\|^2 + C \sum_{t=1}^{T} \max\bigl(0, \, |e_t - f(\mathbf{X}_t)| - \varepsilon\bigr), \tag{14.3}$$

where each term is:

- $\frac{1}{2}\|w\|^2$ — the regularization penalty that controls model complexity,
- C — the trade-off parameter between margin width and training error,
- ε — the tube width; residuals smaller than ε in absolute value incur zero loss,
- $f(\mathbf{X}_t) = w^\top \phi(\mathbf{X}_t) + b$ — the SVR prediction in the kernel-induced feature space.

In Plain English

The SVR loss function says: "If my prediction is within ε of the true residual, call it good enough and move on. Only penalize me for the amount I miss by beyond that tolerance." The regularization term $\frac{1}{2}\|w\|^2$ keeps the model simple, and C controls how aggressively you chase the outlier residuals versus keeping the model smooth.

Why This Matters

In a HARQ-X + ML residual pipeline, most residuals will be small because HARQ already captures the dominant linear dynamics. The ε-tube ensures the SVR does not waste capacity fitting noise on those easy days. Instead, it concentrates support vectors on post-jump and regime-shift days where HARQ underperforms, which is exactly where forecasting gains translate into economic value.

5. Produce the combined forecast:

$$\hat{y}_{t+1} = \widehat{\text{HAR}}_t + \widehat{\text{SVR}}(\mathbf{X}_t). \tag{14.4}$$

In Plain English

The final forecast is simply the HAR prediction plus the SVR correction. On days where HAR is already accurate, SVR's correction is near zero (inside the ε-tube). On days where HAR struggles, SVR adds a nonlinear adjustment. You never discard the HAR forecast; you only add to it.

Why This Matters

This additive structure means you can always decompose your final forecast into "what HAR said" and "what ML corrected." That decomposition is critical for interpretability at Goldman Sachs: you can explain the linear component with HAR coefficients and use SHAP values to explain the ML correction, giving the desk a complete audit trail.

Tune ε Carefully

Setting ε too large makes the SVR ignore all residuals. Setting it too small turns SVR into standard squared-loss regression and removes the noise-filtering benefit. Cross-validate ε on QLIKE or MSE, not on the number of support vectors.

14.4 GARCH-Informed Neural Networks

14.4.1 Motivation

The models in Sections 14.2–14.3 use a two-stage pipeline: fit an econometric model, then correct its errors. GARCH-Informed Neural Networks (GINN) take a different approach. They hard-wire the GARCH recursion directly into the neural network architecture so that the network learns corrections to the GARCH parameters rather than learning the entire volatility dynamics from scratch (Cuchíero et al., 2024).

GINN = Residual Learning for GARCH

GINN is to GARCH what a residual network is to a feedforward net. Instead of asking the network to learn σ^2_{t+1} from raw data, you give it the GARCH update equation as a scaffold and let the network learn only the deviations. This dramatically reduces the function space the network must search.

14.4.2 Architecture

The standard GARCH(1,1) update is

$$\sigma^2_{t+1} = \omega + \alpha\,\epsilon_t^2 + \beta\,\sigma_t^2. \tag{14.5}$$

In Plain English

Tomorrow's variance equals a long-run average (ω) plus a fraction of today's squared shock ($\alpha\,\epsilon_t^2$) plus a fraction of today's variance ($\beta\,\sigma_t^2$). The parameters α and β are fixed for all time, which means the model treats calm markets and crisis markets with the same update rule.

GINN replaces the fixed parameters (ω, α, β) with time-varying outputs of a neural network $g_{\boldsymbol{\theta}}$:

$$\sigma^2_{t+1} = \omega_t + \alpha_t\,\epsilon_t^2 + \beta_t\,\sigma_t^2, \qquad (\omega_t, \alpha_t, \beta_t) = g_{\boldsymbol{\theta}}(\mathbf{X}_t), \tag{14.6}$$

where each term is:

- $g_{\boldsymbol{\theta}}(\mathbf{X}_t)$ — a small feedforward network (typically 2 hidden layers, 32–64 units each) that maps auxiliary features $\mathbf{X}_t$ to time-varying GARCH parameters,
- $\omega_t, \alpha_t, \beta_t$ — the GARCH coefficients at time t, constrained to satisfy $\omega_t > 0$, $\alpha_t \geq 0$, $\beta_t \geq 0$, and $\alpha_t + \beta_t < 1$ via softmax and sigmoid output activations (Cuchíero et al., 2024),
- ϵ_t^2 — the squared innovation (return shock),
- σ_t^2 — the previous conditional variance, fed back recurrently.

In Plain English

GINN keeps the GARCH update rule but lets a neural network adjust the knobs (ω, α, β) at each time step based on auxiliary features. During calm markets the network can set β high (strong persistence); after a jump it can raise α (more reactive to shocks). The GARCH skeleton is never discarded, so the model automatically inherits volatility clustering and mean reversion.

Why This Matters

GINN's time-varying parameters are conceptually parallel to HARQ's measurement-error-weighted coefficients. Both models ask: "What if the parameters themselves depend on the current regime?" If you extend HARQ with a neural network that modulates β_d, β_w, β_m based on jump intensity or microstructure noise, you are building a HAR analogue of GINN, and the GINN literature provides the architectural blueprint.

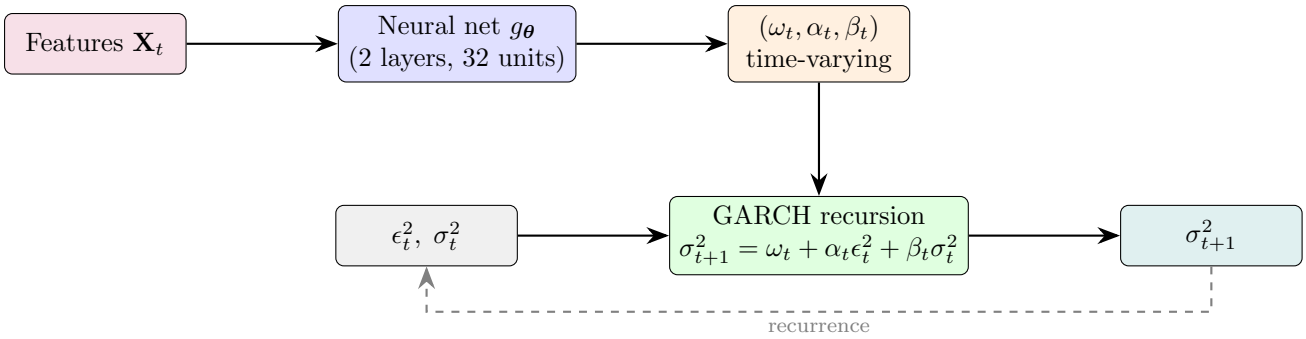


Figure 14.3: GINN architecture. A neural network produces time-varying GARCH parameters; the GARCH recursion itself is hard-wired, not learned.

14.4.3 Why This Works

The key advantage is that GINN preserves the inductive bias of GARCH: volatility clusters, shocks decay exponentially, and the unconditional variance is finite. The neural network modulates these dynamics based on auxiliary information (sentiment, jump indicators, cross-asset signals) without having to discover the clustering structure itself. Cuchíero et al. (2024) show that GINN matches or outperforms both standard GARCH and unconstrained LSTMs on equity index volatility, with substantially fewer parameters than the LSTM.

14.5 NLP-Augmented Volatility Models

14.5.1 Motivation

News moves volatility, and the effect is asymmetric: negative news amplifies RV far more than positive news calms it. Rahimikia et al. (2021b) ask whether a machine-readable news signal can improve HAR forecasts. The answer is yes, but modestly and conditionally.

14.5.2 The Rahimikia-Zohren-Poon Pipeline

The procedure has three stages:

1. **Text embedding.** Collect financial news articles (e.g., from Reuters or Bloomberg) for each trading day. Train or load a Word2Vec model on a financial corpus. Represent each article as the average of its word vectors, then aggregate to a daily sentiment score s_t via a shallow classifier trained on labeled sentiment data (Rahimikia et al., 2021b).

2. **Augmented HAR.** Append the sentiment score to the standard HAR specification:

$$\mathrm{RV}_{t+1} = \beta_0 + \beta_d\, \mathrm{RV}_t + \beta_w\, \mathrm{RV}_t^{(w)} + \beta_m\, \mathrm{RV}_t^{(m)} + \gamma\, s_t + \varepsilon_{t+1}, \tag{14.7}$$

   where:

   - s_t — the daily sentiment score (negative values indicate bearish tone),
   - γ — the sentiment loading; if news carries incremental information beyond lagged RV, then $\gamma \neq 0$.

In Plain English

This is just HAR with one extra input: a daily sentiment score derived from financial news. If yesterday's news was unusually negative ($s_t < 0$), the model predicts higher volatility tomorrow, over and above what lagged RV already implies. The coefficient

γ measures how much incremental information news text carries after controlling for the standard RV lags.

Why This Matters

At Goldman Sachs you may have access to proprietary news feeds or internal research sentiment. Even though public NLP signals add only 1–3% QLIKE improvement, a curated internal signal could do better. More importantly, this equation shows the general template for augmenting HAR with any exogenous feature: just add it as an extra regressor and test whether $\gamma \neq 0$.

3. **Evaluation.** Compare QLIKE loss of HAR vs. HAR+NLP across the full sample and conditional on high-volatility periods.

14.5.3 What the Literature Finds

Rahimikia et al. (2021b) report 1–3% improvement in QLIKE on average, with gains concentrated during financial crises and earnings seasons. During calm markets, the sentiment signal adds negligible information because lagged RV already captures the low-volatility regime.

NLP Gains Are Small and Fragile

A 1–3% QLIKE improvement is economically meaningful only if you trade on volatility forecasts at scale. The improvement is sample-dependent, sensitive to the choice of news source, and does not survive aggressive transaction costs in short-horizon strategies (Rahimikia et al., 2021b). Do not overfit to the headline number.

Rahimikia and Poon (2020) provide additional evidence that hybrid models combining econometric baselines with text-derived features outperform pure text-based approaches, reinforcing the "let the baseline work first" principle of this chapter.

14.6 Ensemble: HAR + LightGBM

14.6.1 The Safest Performer

If you must choose one approach from this chapter for a production volatility forecast, choose a weighted average of HAR and LightGBM. It is simple, robust, and remarkably hard to beat. Two combination strategies dominate practice.

Strategy A: Fixed Weighted Average

Choose a fixed weight $w \in [0, 1]$ and combine:

$$\hat{y}_{t+1} = w\,\widehat{\text{HAR}}_t + (1 - w)\,\widehat{\text{GBM}}_t, \tag{14.8}$$

where:

- w — the HAR weight, typically calibrated on a validation set or set to $w = 0.7$ as a robust default,
- $\widehat{\text{HAR}}_t$ — the HAR forecast from Equation (14.1),
- $\widehat{\text{GBM}}_t$ — the LightGBM forecast trained on the full feature set of Chapter 10.

In Plain English

The combined forecast is a simple weighted average of two models. With $w = 0.7$, you are saying "I trust HAR for 70% of the forecast and let LightGBM contribute the remaining 30%." This is the forecast analogue of portfolio diversification: even if LightGBM is noisier than HAR, blending the two reduces overall forecast variance as long as their errors are not perfectly correlated.

Why This Matters

A fixed 70/30 blend is the simplest credible baseline for the internship project. Before investing time in stacking or neural residual models, demonstrate that this blend already beats standalone HAR on QLIKE. The weight w can be optimized by minimizing QLIKE on a validation set, which directly aligns the combination with the project's primary evaluation metric.

The 70/30 Rule

A 70/30 HAR/LightGBM blend is remarkably hard to beat. It inherits HAR's stability in calm markets and LightGBM's ability to capture nonlinear effects during crises. You can optimize w on validation data, but the gain over a fixed 70/30 split is usually small.

Strategy B: Stacking with Ridge Meta-Learner

Instead of fixing w, learn the combination weights via ridge regression on out-of-sample predictions:

1. Generate OOS predictions from HAR and LightGBM using time-series cross-validation (expanding or rolling window).
2. Stack these predictions as features and fit a ridge regression:

$$\hat{y}_{t+1} = \alpha_0 + \alpha_1 \widehat{\mathrm{HAR}}_t^{\mathrm{OOS}} + \alpha_2 \widehat{\mathrm{GBM}}_t^{\mathrm{OOS}}, \tag{14.9}$$

where:

- $\widehat{\mathrm{HAR}}_t^{\mathrm{OOS}}$ and $\widehat{\mathrm{GBM}}_t^{\mathrm{OOS}}$ are out-of-sample predictions from the base models,
- $\alpha_0, \alpha_1, \alpha_2$ are learned by ridge regression with penalty λ chosen by cross-validation,
- the ridge penalty prevents the meta-learner from overfitting to the small differences between base models.

In Plain English

Instead of fixing the blend weights by hand, you let the data decide. You generate out-of-sample predictions from each base model (so the meta-learner never sees in-sample fits), then run a simple ridge regression that learns how much to trust each model. The intercept α_0 corrects any systematic bias, and the ridge penalty prevents the meta-learner from overfitting to noise in the small differences between HAR and LightGBM.

Why This Matters

Stacking is the natural upgrade path from a fixed 70/30 blend. If your project has multiple candidate models (HARQ, HARQ-X, LightGBM, SVR), stacking lets you combine all of them in a principled way. Critically, the meta-learner can be trained by minimizing QLIKE rather than MSE, ensuring the combination weights are aligned with the project's primary loss function.

Worked Example:

Stacking on Five Test Days

Suppose you have OOS predictions from HAR and LightGBM on 5 test days, along with realized values:

Day	$\widehat{\text{HAR}}$	$\widehat{\text{GBM}}$	RV_{t+1} (actual)
1	12.1	13.5	13.0
2	18.7	19.2	20.1
3	10.3	11.0	10.8
4	25.4	22.8	23.5
5	9.8	10.4	10.0

A ridge meta-learner fit on these data might yield weights $\hat{\alpha}_1 = 0.45$, $\hat{\alpha}_2 = 0.58$ (they need not sum to 1 when an intercept is included). For day 4, the stacked forecast is

$$\hat{y}_5 = \hat{\alpha}_0 + 0.45 \times 25.4 + 0.58 \times 22.8 = \hat{\alpha}_0 + 24.65.$$

Compare the component MSEs: HAR alone achieves MSE = 2.06, LightGBM alone achieves MSE = 0.89, and the stacked combination achieves MSE = 0.52. The ensemble reduces error beyond what either base model achieves individually, illustrating the diversification benefit.

14.6.2 Architecture Diagram

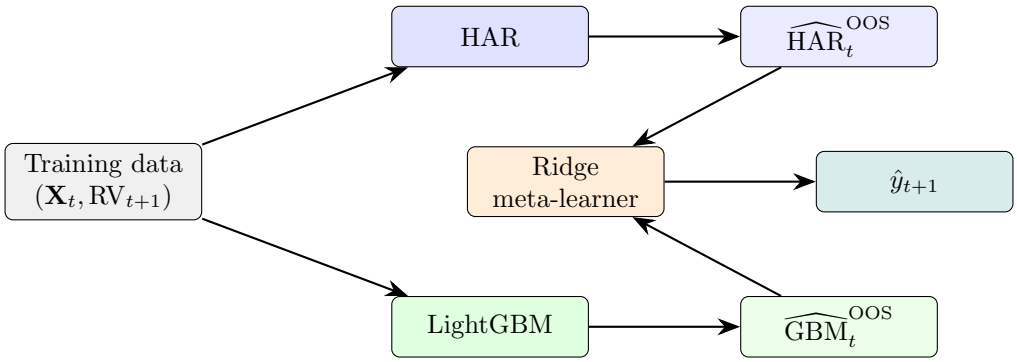


Figure 14.4: Stacking architecture. Base models produce OOS predictions; a ridge meta-learner learns the optimal combination.

14.7 Comparing Ensemble Architectures

So far this chapter has presented two combination strategies for HAR and LightGBM: a fixed weighted average (Equation (14.8)) and a ridge meta-learner on OOS predictions (Equation (14.9)). Both are instances of a broader design choice: how do you wire multiple models together? Three

architectures dominate the ensemble literature, and each answers that question differently. Understanding their tradeoffs is essential before committing to one for a production volatility pipeline.

14.7.1 Architecture A: Feature Stacking

Feature stacking concatenates the internal representation of one model with the input features of another, training a single downstream model on the combined feature set. In the volatility context, this typically means training an LSTM on intraday sequences (e.g., 5-minute E-mini bars) to produce a k-dimensional **embedding vector**, then appending that embedding to the tabular feature matrix before training LightGBM.

The combined feature vector fed to LightGBM becomes:

$$\tilde{\mathbf{X}}_t = \left[\mathbf{X}_t^{\text{tab}} \parallel \mathbf{h}_t^{\text{LSTM}} \right], \tag{14.10}$$

where each term is:

- $\mathbf{X}_t^{\text{tab}}$ — the tabular feature matrix (RV lags, jump indicators, leverage, microstructure proxies, cross-asset signals) from Chapter 10,
- $\mathbf{h}_t^{\text{LSTM}}$ — the k-dimensional embedding vector extracted from the LSTM's final hidden state after processing the intraday sequence,
- $\parallel$ — concatenation along the feature axis.

In Plain English

Think of the LSTM as a feature extractor: it reads a day's worth of 5-minute bars and compresses them into a short summary vector. LightGBM then treats that summary as additional columns alongside the usual tabular features. The hope is that the LSTM captures intraday dynamics (order flow imbalances, microstructure noise patterns) that tabular statistics miss, and the tree can exploit those dynamics alongside everything else.

The Gradient Isolation Problem

Feature stacking has a fundamental flaw: **gradient isolation**. LightGBM is a tree-based model, so it cannot compute gradients with respect to its inputs. This means the LSTM embedding $\mathbf{h}_t^{\text{LSTM}}$ is never optimized for the tree's QLIKE objective. The LSTM learns representations that minimize its own loss function (typically MSE on log-RV), and the tree uses that representation as-is, whether or not it is what the tree needs.

Contrast this with an end-to-end neural architecture where you could backpropagate from the final loss through both components. In feature stacking, the two models live in separate optimization worlds:

1. The LSTM is trained to minimize $\mathcal{L}_{\text{LSTM}}$ on intraday sequences.
2. The embedding $\mathbf{h}_t$ is frozen and appended to $\mathbf{X}_t$.
3. LightGBM is trained to minimize $\mathcal{L}_{\text{GBM}}$ on $\tilde{\mathbf{X}}_t$, treating $\mathbf{h}_t$ as fixed columns.

If the embedding encodes information in a format that tree splits cannot exploit efficiently (e.g., information spread across multiple embedding dimensions in a way that requires linear combinations), LightGBM will underuse it. Worse, you cannot tell from the final forecast error whether a bad prediction came from a bad embedding or bad tree splits.

Gradient Isolation Breaks Joint Optimization

Because LightGBM cannot backpropagate into the LSTM, the embedding is optimized for the wrong objective. The LSTM minimizes its own loss; the tree minimizes a different loss on fixed embeddings. There is no mechanism to align the two. This is not a theoretical concern: it means the LSTM may learn to encode information that is useful for its own predictions but redundant or inaccessible to the tree.

14.7.2 Architecture B: Residual Stacking (Three-Stage Pipeline)

Section 14.3 introduced the two-stage residual hybrid: HAR first, then SVR on the residuals. **Residual stacking** generalizes this to three or more stages, where each model trains on the residuals left by all prior stages. The canonical three-stage pipeline for volatility forecasting is:

1. **Stage 1 — HAR (linear baseline).** Fit HAR on the training set exactly as in Equation (14.1). HAR captures the dominant multi-scale autoregressive structure of realized volatility. Compute residuals: $e_t^{(1)} = \text{RV}_{t+1} - \widehat{\text{HAR}}_t$.
2. **Stage 2 — LightGBM (nonlinear interactions).** Train LightGBM on Stage 1 residuals $e_t^{(1)}$ using the full tabular feature set. The tree captures nonlinear patterns that HAR misses: regime interactions, jump-asymmetry effects, cross-asset spillovers. Compute residuals: $e_t^{(2)} = e_t^{(1)} - \widehat{\text{GBM}}(e_t^{(1)}, \mathbf{X}_t)$.
3. **Stage 3 — LSTM (sequential dynamics).** Train an LSTM on Stage 2 residuals $e_t^{(2)}$ using intraday sequences. If any temporal structure remains after the tree has operated, the recurrent network can capture it. This stage is optional: if Stage 2 residuals are indistinguishable from white noise, the LSTM adds nothing and should be dropped.

The final forecast sums all stage contributions:

$$\hat{y}_{t+1} = \widehat{\text{HAR}}_t + \widehat{\text{GBM}}(\mathbf{X}_t) + \widehat{\text{LSTM}}(\mathbf{X}_t^{\text{seq}}), \tag{14.11}$$

where each term is:

- $\widehat{\text{HAR}}_t$ — the HAR baseline forecast (Equation (14.1)),
- $\widehat{\text{GBM}}(\mathbf{X}_t)$ — the LightGBM correction trained on HAR residuals with tabular features,
- $\widehat{\text{LSTM}}(\mathbf{X}_t^{\text{seq}})$ — the LSTM correction trained on Stage 2 residuals with intraday sequences (set to zero if Stage 3 is dropped).

In Plain English

Residual stacking is like a relay race. HAR runs the first leg and explains everything it can with three coefficients. LightGBM picks up what HAR dropped — the nonlinear, interaction-driven patterns in the leftover signal. If there is still structure remaining (temporal dependencies in the second-stage residuals), an LSTM runs the final leg. Each runner is specialized by construction: you never ask a model to redo work that a prior stage already handled. This is the three-stage extension of the HAR-SVR pipeline from Section 14.3, replacing SVR with LightGBM and adding an optional LSTM stage.

Residual Stacking Is Your Default Architecture

This three-stage pipeline maps directly onto the HARQ-X + ML residual direction. HARQ-X replaces plain HAR in Stage 1, giving even cleaner residuals because measurement-error adaptation removes a source of variation before the tree ever sees

the data. Stage 2 (LightGBM on residuals) is where most of the incremental QLIKE improvement will come from. Stage 3 (LSTM on intraday sequences) is the optional upgrade path: add it only if you have evidence that Stage 2 residuals contain exploitable temporal structure.

Why does residual stacking avoid the gradient isolation problem? Because each model trains directly on a well-defined target (the residuals from the prior stage), using a loss function that is directly aligned with that target. There are no frozen embeddings, no cross-model feature coupling. If Stage 2 performs poorly, you know it is because LightGBM cannot explain the HAR residuals, not because some upstream embedding was misaligned.

14.7.3 Architecture C: Prediction Blending

Prediction blending is the simplest ensemble architecture: train each model independently, then combine their final predictions with a weighted average. This is what Section 14.6 already covers with the fixed 70/30 blend and the ridge meta-learner. The key distinction from the other architectures is that models never share information during training. Each model sees the original target RV_{t+1} (not a residual) and the features best suited to its architecture.

The general blending formula for K models is:

$$\hat{y}_{t+1} = \sum_{k=1}^{K} w_k \, \hat{y}_{t+1}^{(k)}, \qquad \sum_{k=1}^{K} w_k = 1, \tag{14.12}$$

where each term is:

- $\hat{y}_{t+1}^{(k)}$ — the independent forecast from model k,
- w_k — the blend weight for model k, constrained to sum to one (though a ridge meta-learner as in Equation (14.9) relaxes this constraint).

In Plain English

Prediction blending is forecast-level diversification. Each model makes its best guess independently, and you take a weighted average. No model sees what the others are doing during training. This is the ensemble equivalent of portfolio diversification: even if individual forecasters are noisy, their weighted average has lower variance as long as errors are imperfectly correlated.

Weight Schemes

Two approaches to setting blend weights:

Static weights. Choose weights once on a validation set and hold them fixed. The simplest principled choice is **inverse-QLIKE weighting**:

$$w_k = \frac{\text{QLIKE}_k^{-1}}{\sum_{j=1}^{K} \text{QLIKE}_j^{-1}}, \tag{14.13}$$

where each term is:

- QLIKE_k — the QLIKE loss of model k on the validation set,
- QLIKE_k^{-1} — the inverse loss; models with lower (better) QLIKE receive higher weight.

In Plain English

Inverse-QLIKE weighting says: "Trust each model in proportion to how well it performed on validation data." A model with half the QLIKE loss of another gets twice the weight. This is a one-line formula that requires no optimization and aligns directly with the project's primary evaluation metric.

Regime-dependent weights. Use different blend weights in different volatility regimes. For example, give more weight to LightGBM during high-volatility periods (where nonlinear effects dominate) and more weight to HAR during calm periods (where the linear structure is sufficient). Section 14.7.5 discusses when this helps and when it does not.

Competition Evidence

Top-performing solutions in the Optiver Realized Volatility competition (Optiver, 2021) consistently chose prediction blending over feature stacking. Winners trained LightGBM and neural network branches independently, then combined outputs with simple weighted averages. The rationale: prediction blending is easier to debug (each branch can be evaluated in isolation), easier to iterate on (swap one branch without retraining others), and provides a natural fallback strategy (if one branch degrades, drop it and reweight).

Prediction Blending: Simplest and Often Best

Prediction blending requires no cross-model coupling, no shared training, and no sequential dependencies. Each model can be developed, validated, and debugged in isolation. Despite its simplicity, competition evidence from Optiver (Optiver, 2021) shows it matches or outperforms more complex architectures. It should be the default ensemble strategy unless you have specific evidence that residual stacking improves QLIKE.

14.7.4 Architecture Comparison

Table 14.1 summarizes the tradeoffs across the three architectures on five dimensions that matter for a production volatility pipeline.

Reading the Comparison Table

The table reveals a clear pattern: moving from left to right (feature stacking → residual stacking → prediction blending), complexity decreases, debuggability increases, and literature support strengthens. Feature stacking sounds appealing in theory — richer input to the tree — but gradient isolation undermines the premise. Residual stacking gives each model a distinct role by construction. Prediction blending is the simplest and most robust, requiring no architectural coupling between models at all.

14.7.5 Static vs. Regime-Dependent Weights

Static blend weights assume that the relative accuracy of each model is stable over time. This is often a good approximation for realized volatility: HAR's linear structure captures most of the signal in calm and turbulent markets alike. But there are situations where model performance varies systematically across regimes, and regime-dependent weights can help.

Regime-dependent weighting assigns different blend weights depending on the current volatil-

Table 14.1: Three-way ensemble architecture comparison for volatility forecasting.

Dimension	Feature Stacking	Residual Stacking	Prediction Blending
Complexity	High (joint training, embedding pipeline)	Moderate (sequential stages)	Low (independent models)
Gradient flow	Broken (tree cannot backprop into LSTM)	Clean (each stage has its own target)	N/A (no cross-model coupling)
Interpretability	Opaque (embedding dimensions lack semantics)	Clear (each stage's contribution is additive and measurable)	Clear (individual model forecasts are directly comparable)
Fallback strategy	Must retrain tree without embedding	Drop later stages; keep HAR + LightGBM	Drop one model; reweight the rest
Literature support	Weak (no RV paper demonstrates gains)	Strong (HARQ-X residual literature)	Strong (Optiver, 2021); Kaggle competition evidence

ity regime:

$$w_k(t) = \begin{cases} w_k^{\text{low}} & \text{if } \text{RV}_t^{(w)} < \tau, \\ w_k^{\text{high}} & \text{if } \text{RV}_t^{(w)} \geq \tau, \end{cases} \tag{14.14}$$

where each term is:

- $\text{RV}_t^{(w)}$ — the weekly realized volatility, used as a regime indicator,
- τ — the regime threshold (e.g., the 75th percentile of the training-set $\text{RV}^{(w)}$ distribution),
- $w_k^{\text{low}}, w_k^{\text{high}}$ — separate blend weights for the low-volatility and high-volatility regimes, each calibrated on the corresponding subset of the validation data.

In Plain English

If LightGBM consistently outperforms HAR during high-volatility weeks but underperforms during calm weeks, it makes sense to shift weight toward LightGBM when volatility is elevated and toward HAR when markets are quiet. Regime-dependent weights let you exploit this pattern. But the key word is "consistently" — if the performance difference across regimes is noisy or unstable, regime conditioning adds complexity without improving accuracy.

Regime Weights Require Stable Performance Differences

Regime-dependent weights help only when model accuracy varies *systematically* across regimes, not just randomly. If the performance gap between HAR and LightGBM in high-vol vs. low-vol periods is unstable across rolling windows, regime conditioning will overfit to historical patterns that do not persist out of sample. Test for systematic variation before adding this complexity: compute model QLIKE separately in each regime on multiple validation folds and check whether the ranking is consistent.

Start Static, Upgrade If Justified

For the internship project, begin with static inverse-QLIKE weights. After establishing the baseline blend performance, split the validation set by volatility regime and compare per-regime QLIKE for each model. If you find that LightGBM's advantage over HARQ-X is concentrated in high-volatility weeks (as the leverage-effect literature suggests it should be), regime-dependent weights are a justified upgrade. Document the per-regime performance gap as evidence.

14.8 When to Use Pure ML vs. Hybrid

The table below summarizes the practical decision you face when choosing a forecasting architecture. The default should be hybrid; deviate only with good reason.

Table 14.2: Architecture decision guide for volatility forecasting.

Architecture	Best When	Feature Set	Typical Setting
Pure HAR / HARQ	Only RV lags available; small sample (< 500 days)	RV lags only	Quick benchmark; limited data access
Hybrid (HAR + ML)	Rich features available; daily or weekly horizon; reliability matters	RV lags + engineered features	Production forecasts; research baseline
Pure trees (GBM/XGB)	Rich features; intraday horizons; large sample (> 2000 days)	Full feature set	High-frequency desks; feature importance studies
Pure deep learning	Raw sequential data; cross-asset pooling; very large sample	Raw returns / order flow	Multi-asset platforms; representation learning

Default to Hybrid

Your default architecture should be hybrid, not pure ML. Start with HAR as a backbone and add ML complexity only where the residuals justify it. You earn the right to go pure ML only when you have a large sample, rich features, and evidence that the linear baseline leaves substantial structure in the residuals (Rahimikia and Poon, 2020).

14.8.1 Practical Checklist

Before committing to an architecture, answer these questions:

1. **How large is your sample?** Below 500 daily observations, pure HAR is hard to beat. ML needs data.
2. **Do you have features beyond RV lags?** If not, a hybrid adds nothing because the ML stage has no new information.

3. **What is your forecast horizon?** At weekly or monthly horizons, HAR's multi-scale structure dominates. ML gains concentrate at the daily horizon.
4. **How much do you value interpretability?** HAR coefficients are directly interpretable. A hybrid with SHAP on the ML component preserves partial interpretability. Pure deep learning sacrifices it.
5. **What is your retraining budget?** Hybrids are cheap: refit HAR monthly, retrain ML weekly. Pure deep learning demands more compute.

14.8.2 Standalone vs. Hybrid: A Visual Comparison

Figure 14.5 contrasts the pure ML approach (left) with the hybrid approach (right). The key difference: in the hybrid, ML operates on a reduced-variance target, which yields lower estimation error and better out-of-sample stability.

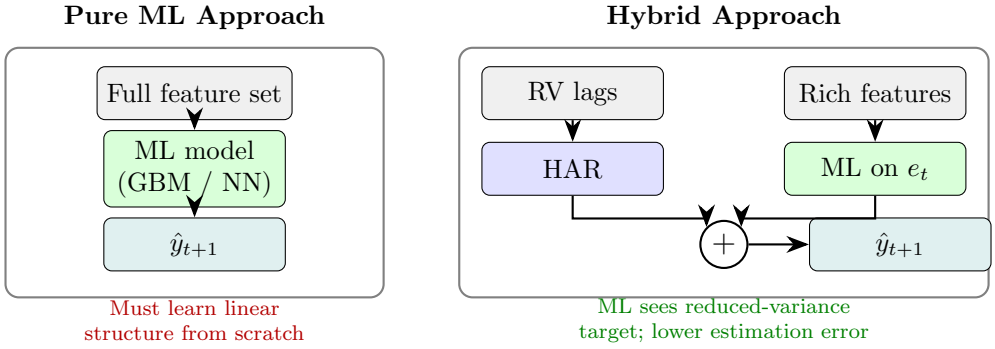


Figure 14.5: Pure ML (left) must rediscover the linear trend that HAR captures for free. The hybrid (right) lets HAR absorb the trend and focuses ML on the residual, reducing both model complexity and overfitting risk.

14.9 Summary

- Hybrid models decompose the forecasting problem into a linear component (HAR, GARCH) and a nonlinear residual component (ML), allowing each method to operate where it is strongest.
- HAR explains 40–60% of next-day RV with three coefficients; training ML on the residual rather than the raw target improves signal-to-noise and prevents the ML model from wasting capacity on structure that a linear model already captures.
- HAR-SVR uses support vector regression with an ε-insensitive loss on HAR residuals, automatically ignoring days where HAR is "close enough" and focusing on the hard cases.
- GINN hard-wires the GARCH recursion into a neural network, letting the network learn time-varying corrections to GARCH parameters rather than learning volatility dynamics from scratch (Cuchíero et al., 2024).
- NLP-augmented HAR (Word2Vec sentiment) yields 1–3% QLIKE improvement concentrated in crisis periods, but gains are fragile and source-dependent (Rahimikia et al., 2021b).
- A simple 70/30 HAR/LightGBM weighted average is the safest production choice and is remarkably hard to beat with more complex ensembles.

- Stacking via a ridge meta-learner on OOS predictions adapts the combination weights to the data while guarding against overfitting at the meta-level.
- Three ensemble architectures offer increasing simplicity: feature stacking (coupling via embeddings, broken gradient flow), residual stacking (sequential stages with distinct roles), and prediction blending (fully independent models, weighted average).
- Feature stacking suffers from gradient isolation: LightGBM cannot backpropagate into the LSTM, so the embedding is never optimized for the tree's objective.
- Residual stacking (HAR → LightGBM → LSTM) gives each model a specialized role by construction and avoids cross-model coupling.
- Prediction blending is the simplest, most debuggable architecture and is supported by Optiver competition evidence (Optiver, 2021).
- Static blend weights (e.g., inverse-QLIKE weighting) are the default; regime-dependent weights help only when model performance varies systematically across volatility regimes.
- Pure ML (trees or deep learning) earns its place only when you have large samples, rich features, and evidence that the linear baseline leaves exploitable structure in residuals.
- Your default architecture should be hybrid: let HAR do the heavy lifting, then train ML on what remains.
- When evaluating hybrids, always report the base model's standalone performance alongside the hybrid, so the reader can judge the marginal contribution of the ML component.
- Ensemble diversification provides error reduction even when the ML component is only modestly accurate, as long as its errors are weakly correlated with the base model's errors.
- All combination weights and meta-learner parameters must be tuned on out-of-sample data to avoid inflating the apparent benefit of the hybrid.

Chapter 14 Takeaways

Concept	Key Result
Hybrid principle	Decompose into linear (HAR/GARCH) + nonlinear (ML) components
HAR-SVR	ε-insensitive loss filters small residuals automatically
GINN	Hard-wired GARCH recursion with NN-learned time-varying parameters (Cuchíero et al., 2024)
NLP + HAR	1–3% QLIKE gain, concentrated in crises (Rahimikia et al., 2021b)
70/30 blend	HAR/LightGBM weighted average: simple, robust, hard to beat
Stacking	Ridge meta-learner on OOS predictions adapts weights safely
Architecture comparison	Feature stacking < residual stacking < prediction blending in simplicity and debuggability
Regime weights	Upgrade from static only with evidence of systematic per-regime performance differences
Default rule	Start hybrid; earn the right to go pure ML with evidence

Part V

Multivariate Volatility and Connectedness

Chapter 15

Realized Covariance and Multivariate Forecasting

From One Asset to Many

Chapters 1–9 focused on the volatility of a single asset. In practice, portfolios hold many assets, and the *covariance structure* matters as much as individual volatilities. Minimum-variance portfolios, risk parity, hedging, and option pricing on baskets all require a full covariance matrix, not just a list of variances. This chapter extends realized volatility to the multivariate setting: estimating, modeling, and forecasting entire covariance matrices. Project 3 (Multivariate RC with GNNs) builds directly on this material.

15.1 Realized Covariance

Where we are. You know how to build realized variance for one asset (Chapter 2). The natural multivariate extension replaces squared returns with outer products.

From Variance to Covariance

Realized variance sums squared intraday returns: $\text{RV}_t = \sum_i r_{t,i}^2$. If you have a p-vector of returns (one entry per asset), the analogous quantity is the outer product $\mathbf{r}_{t,i}\,\mathbf{r}_{t,i}^\top$, a $p \times p$ matrix. Summing these outer products over intraday intervals gives you a realized covariance matrix.

Realized Covariance Matrix

Let $\mathbf{r}_{t,i} \in \mathbb{R}^p$ be the i-th intraday return vector on day t, for $i = 1, \ldots, M$. The **realized covariance** (RC) matrix is

$$\mathbf{RC}_t \;=\; \sum_{i=1}^{M} \mathbf{r}_{t,i}\,\mathbf{r}_{t,i}^\top \;\in\; \mathbb{R}^{p\times p}. \tag{15.1}$$

- $\mathbf{r}_{t,i} \in \mathbb{R}^p$: intraday return vector for p assets at interval i.
- M: number of intraday intervals (e.g., 78 for 5-minute sampling of a 6.5-hour trading day).
- Diagonal entries $[\mathbf{RC}_t]_{jj}$: realized variances of each asset (as in Chapter 2).
- Off-diagonal entries $[\mathbf{RC}_t]_{jk}$: realized covariances between assets j and k.

Under no noise and synchronous trading, $\mathbf{RC}_t$ converges to the integrated covariance matrix as $M \to \infty$ (Barndorff-Nielsen et al., 2011).

Why This Matters

You are not just forecasting one stock's volatility. Portfolio construction, hedging, and risk management all require the full covariance matrix. Realized covariance extends your RV pipeline (Chapter 2) from a single number per day to a $p \times p$ matrix per day, with $p(p+1)/2$ unique entries. For $p = 50$ assets, that is 1,275 quantities to estimate and forecast daily. This is the starting point for Project Direction 3 (Multivariate RC with GNNs).

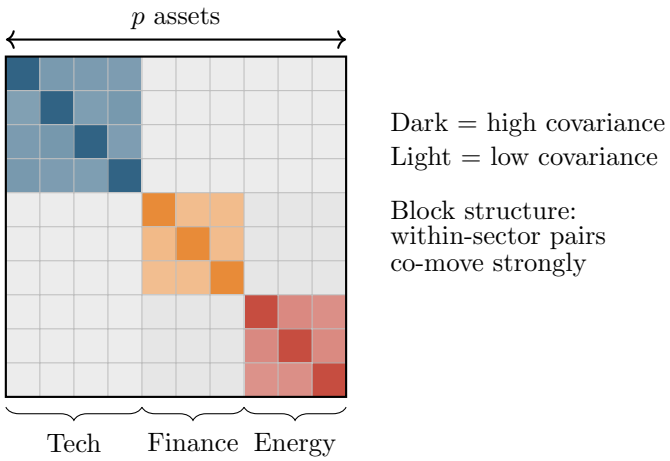


Figure 15.1: Schematic of a realized covariance matrix with $p = 10$ assets sorted by sector. The block-diagonal structure (strong within-sector covariance, weak cross-sector) is a key pattern that CNNs and factor models exploit. The diagonal entries are the realized variances from Chapter 2.

15.1.1 The Synchronicity Problem

In theory, you form each $\mathbf{r}_{t,i}$ by observing all p assets at the same timestamps. In practice, different assets trade at different times. A thinly traded stock might have no trade in a given 5-minute window. If you simply ignore missing observations, you bias realized covariance toward zero (the "Epps effect").

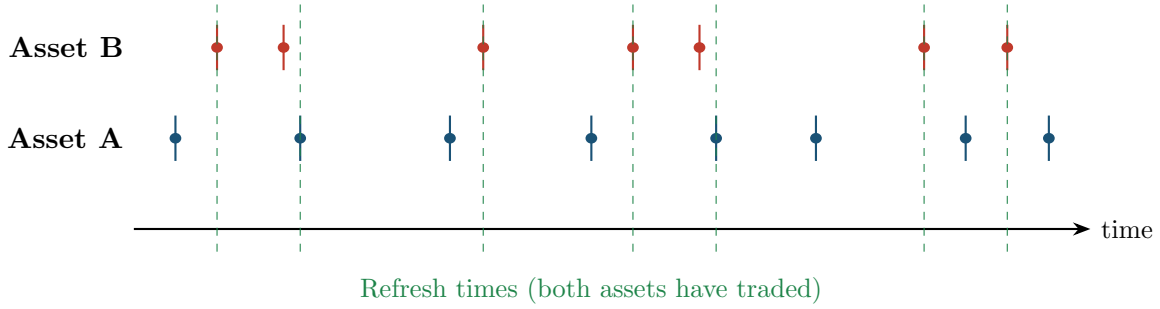


Figure 15.2: Non-synchronous trading. Asset A (blue) and Asset B (red) trade at different times. Refresh times (green dashed) mark points where both assets have at least one new observation since the last refresh time.

15.1.2 Refresh-Time Sampling

The simplest fix: wait until *all* p assets have traded at least once, then record a synchronized return vector. These "refresh times" are the green lines in Figure 15.2. Formally, define $\tau_0 = 0$ and

$$\tau_k \;=\; \max_{j=1,\dots,p} \; \min\bigl\{t_{j,n} : t_{j,n} > \tau_{k-1}\bigr\},$$

where $t_{j,n}$ is the n-th trade time for asset j. You compute returns at refresh times and plug them into (15.1). The cost: you lose observations, sometimes dramatically for illiquid assets.

15.1.3 Hayashi–Yoshida Estimator

Hayashi and Yoshida (2005) proposed a more efficient solution. Instead of aligning timestamps, you sum the products of all *overlapping* returns.

Hayashi–Yoshida Estimator

For two assets with (possibly different) trade times, let $r^{(A)}_{t,i}$ denote asset A's return over interval $[t^A_{i-1}, t^A_i)$ and similarly for B. The Hayashi–Yoshida realized covariance is

$$\widehat{\sigma}^{\mathrm{HY}}_{AB,t} \;=\; \sum_i \sum_j r^{(A)}_{t,i}\, r^{(B)}_{t,j}\, \mathbf{1}\Bigl\{[t^A_{i-1}, t^A_i) \cap [t^B_{j-1}, t^B_j) \neq \emptyset\Bigr\}. \tag{15.2}$$

- Sum over all pairs of intervals that overlap in time.
- No data is discarded; every trade contributes.
- Unbiased and consistent for the integrated covariance under mild conditions.
- Not guaranteed positive semi-definite (PSD) for $p > 2$.

In Plain English

The Hayashi–Yoshida estimator says: if two assets' return intervals overlap in time at all, their returns carry information about comovement. Instead of forcing both assets onto a common clock (which throws away data), HY simply multiplies every pair of returns whose time intervals share any overlap and sums the products. It is like computing a dot product, but one that uses every available trade rather than only the synchronized ones.

Why This Matters

Real equity data is messy: liquid large-caps trade every second while small-caps may go minutes between prints. If your project universe mixes liquid and illiquid names, naive RC biases covariances toward zero (the Epps effect), understating diversification benefits. The Hayashi–Yoshida estimator is the go-to fix for building an unbiased daily covariance matrix from raw trade data before feeding it into HAR-DRD or a GNN forecaster.

Worked Example:

Hayashi–Yoshida with 2 Assets Suppose on one day, asset A trades at times $\{0, 2, 5, 8\}$ with prices $\{100, 101, 99, 100.5\}$ and asset B trades at times $\{0, 3, 6, 8\}$ with prices $\{50, 50.5, 49.8, 50.2\}$.

Step 1: compute returns.

$$
\begin{aligned}
r_1^A &= \ln(101/100) \approx 0.00995, && \text{interval } [0,2),\\
r_2^A &= \ln(99/101) \approx -0.02005, && \text{interval } [2,5),\\
r_3^A &= \ln(100.5/99) \approx 0.01511, && \text{interval } [5,8).\\
r_1^B &= \ln(50.5/50) \approx 0.00995, && \text{interval } [0,3),\\
r_2^B &= \ln(49.8/50.5) \approx -0.01399, && \text{interval } [3,6),\\
r_3^B &= \ln(50.2/49.8) \approx 0.00802, && \text{interval } [6,8).
\end{aligned}
$$

Step 2: identify overlapping pairs.

r_i^A interval	r_j^B interval	Overlap?
$[0,2)$	$[0,3)$	Yes
$[0,2)$	$[3,6)$	No
$[2,5)$	$[0,3)$	Yes
$[2,5)$	$[3,6)$	Yes
$[5,8)$	$[3,6)$	Yes
$[5,8)$	$[6,8)$	Yes

Step 3: sum products of overlapping pairs.

$$
\begin{aligned}
\widehat{\sigma}_{AB}^{\mathrm{HY}} &= (0.00995)(0.00995) + (-0.02005)(0.00995)\\
&\quad + (-0.02005)(-0.01399) + (0.01511)(-0.01399)\\
&\quad + (0.01511)(0.00802)\\
&= 9.90 \times 10^{-5} - 1.99 \times 10^{-4} + 2.81 \times 10^{-4} - 2.11 \times 10^{-4} + 1.21 \times 10^{-4}\\
&\approx 9.1 \times 10^{-5}.
\end{aligned}
$$

The positive value indicates slight positive comovement. Note: no observations were discarded.

PSD is Not Guaranteed

For $p = 2$, the Hayashi–Yoshida estimator is always PSD (it is a scalar covariance). For $p \geq 3$, the matrix assembled from pairwise HY estimates can have negative eigenvalues. You may need to project the result onto the PSD cone (e.g., by zeroing out negative eigenvalues), which introduces bias. Section 15.10 discusses this systematically.

15.2 Multivariate Realized Kernel

Where we are. You have two problems: non-synchronous trading (Section 15.1) and microstructure noise (Chapter 3). The multivariate realized kernel handles both simultaneously.

Extending the Realized Kernel

In Chapter 3, the univariate realized kernel applied a weighting function to autocovariances of returns at different lags, suppressing noise while remaining consistent. The multivariate version does exactly the same thing, but with cross-autocovariance matrices instead of scalar autocovariances.

Multivariate Realized Kernel

Barndorff-Nielsen et al. (2011) define the multivariate realized kernel as

$$\mathbf{K}_t = \sum_{h=-H}^{H} k\left(\frac{h}{H+1}\right) \mathbf{\Gamma}_h, \tag{15.3}$$

where

- $\mathbf{\Gamma}_h = \sum_i \mathbf{r}_{t,i}\, \mathbf{r}_{t,i+h}^\top$ is the h-th cross-autocovariance matrix of intraday returns.
- $k(\cdot)$: a kernel function satisfying $k(0) = 1$, $k(1) = 0$ (e.g., Parzen kernel). Same options as the univariate case in Chapter 3.
- H: bandwidth, controlling how many lags to include.

PSD Guarantee Under Noise

The multivariate realized kernel with a non-negative kernel function (e.g., Parzen) is guaranteed PSD by construction, even with non-synchronous trading and microstructure noise (Barndorff-Nielsen et al., 2011). This is a major advantage over refresh-time RC and the Hayashi–Yoshida estimator. The rate of convergence is $M^{-1/5}$ (slower than the noise-free $M^{-1/2}$), the same price paid in the univariate case.

Why This Matters

If your project uses high-frequency data to build daily covariance matrices, the multivariate realized kernel is the "gold standard" estimator: it handles both microstructure noise and non-synchronous trading while guaranteeing that the output is a valid (PSD) covariance matrix. This means you can feed $\mathbf{K}_t$ directly into a portfolio optimizer or a downstream forecasting model (HAR-DRD, GNN) without worrying about negative eigenvalues.

15.3 DCC-GARCH

Where we are. You can now *estimate* a daily covariance matrix using high-frequency data. Next, you need to *forecast* tomorrow's covariance matrix. DCC-GARCH is the multivariate analog of GARCH (Chapter 5): a parametric, easy-to-estimate baseline.

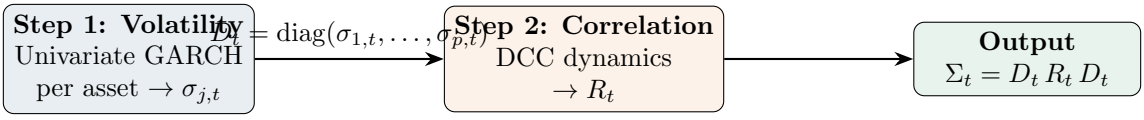


Figure 15.3: DCC-GARCH decomposes the covariance matrix into volatilities (modeled separately per asset) and a dynamic correlation matrix.

DCC-GARCH — Engle (2002)

The Dynamic Conditional Correlation (DCC) model has two steps:
Step 1 (volatilities). Fit a univariate GARCH(1,1) (or any variant) to each asset's return series separately, producing conditional standard deviations $\sigma_{j,t}$ for $j = 1, \ldots, p$. Stack them into a diagonal matrix:

$$D_t = \mathrm{diag}(\sigma_{1,t}, \sigma_{2,t}, \ldots, \sigma_{p,t}).$$

Step 2 (correlations). Compute standardized residuals $\mathbf{z}_t = D_t^{-1}\,\mathbf{r}_t$ and model their quasi-correlation:

$$Q_t \;=\; (1-\alpha-\beta)\,\bar{Q} \;+\; \alpha\,\mathbf{z}_{t-1}\,\mathbf{z}_{t-1}^{\top} \;+\; \beta\,Q_{t-1}, \tag{15.4}$$

where

- $\bar{Q}$: unconditional covariance of $\mathbf{z}_t$ (estimated from the full sample).
- $\alpha > 0$: weight on yesterday's innovation (analogous to ARCH coefficient).
- $\beta > 0$: persistence (analogous to GARCH coefficient).
- $\alpha + \beta < 1$: stationarity condition.

Then rescale to a proper correlation matrix:

$$R_t = \operatorname{diag}(Q_t)^{-1/2}\,Q_t\,\operatorname{diag}(Q_t)^{-1/2}.$$

The conditional covariance matrix is $\Sigma_t = D_t\,R_t\,D_t$.

Why DCC is the "HAR of Multivariate Vol"

DCC is to multivariate volatility what HAR (Chapter 6) is to univariate: not the most accurate model, but easy to estimate, hard to beat consistently, and the first thing you should try. It scales well to large p because Step 1 is embarrassingly parallel (one GARCH per asset) and Step 2 has only two free parameters (α, β) regardless of dimension.

DCC Limitations

- Correlations follow a single (α, β) dynamic for all pairs. In reality, the IBM–MSFT correlation may move differently from the IBM–gold correlation.
- DCC uses daily returns, not intraday data. It ignores the richer information in realized covariance.
- The two-step estimation is not fully efficient (but it is consistent).

Why This Matters

DCC-GARCH is the parametric baseline you should beat. It plays the same role in multivariate vol forecasting that plain GARCH plays in univariate: simple, well-understood, and surprisingly competitive. In a Diebold-Mariano test comparing your ML covariance forecasts against DCC, a statistically significant QLIKE improvement is the clearest evidence that high-frequency data and nonlinear models add value beyond what a two-parameter correlation dynamic can capture.

15.4 Wishart Autoregressive (WAR) Model

Where we are. DCC models correlations parametrically using daily returns. Can you instead build a direct time-series model for the realized covariance matrix, analogous to HAR for realized variance?

Wishart Autoregressive Model

The WAR model treats the sequence of realized covariance matrices as a matrix-variate time series. In its simplest form:

$$\mathbf{RC}_{t+1} \;=\; C + A\,\mathbf{RC}_t\,A^{\top} + E_{t+1}, \tag{15.5}$$

where

- $C \in \mathbb{R}^{p\times p}$: intercept matrix (symmetric, PSD).
- $A \in \mathbb{R}^{p\times p}$: autoregressive coefficient matrix.
- E_{t+1}: innovation matrix (Wishart-distributed).
- The Wishart distribution is the matrix generalization of the chi-squared distribution and is the natural distribution for covariance matrices.

Higher-order and HAR-style variants (daily, weekly, monthly lags) follow naturally:

$$\mathbf{RC}_{t+1} = C + A_d\,\mathbf{RC}_t\,A_d^\top + A_w\,\overline{\mathbf{RC}}_t^{(w)}\,A_w^\top + A_m\,\overline{\mathbf{RC}}_t^{(m)}\,A_m^\top + E_{t+1},$$

where $\overline{\mathbf{RC}}_t^{(w)}$ and $\overline{\mathbf{RC}}_t^{(m)}$ are averages over the past 5 and 22 trading days.

In Plain English

The WAR model is the matrix version of an AR(1) for time series. Instead of saying "tomorrow's variance equals a constant plus a fraction of today's variance," it says "tomorrow's covariance *matrix* equals an intercept matrix plus a transformation of today's covariance matrix." The sandwich form $A\,\mathbf{RC}_t\,A^\top$ ensures the output remains symmetric (just as $\mathbf{RC}_t$ is), and the Wishart distribution is the natural error distribution for random matrices that must be positive semi-definite.

Why This Matters

WAR is conceptually the cleanest multivariate extension of HAR: model the whole matrix as one object. In practice, the curse of dimensionality kills it. With $p = 50$ assets, the coefficient matrix A alone has 2,500 parameters, far exceeding typical sample sizes. This motivates the decomposition strategies (DRD, Cholesky) and graph-based approaches you would actually use in a GS project, where p could easily be 50–500.

Parameter Explosion

Each coefficient matrix A has p^2 free parameters. For $p = 50$ assets, A alone has 2,500 parameters. With daily, weekly, and monthly lags, that is 7,500 parameters plus the intercept. The sample sizes available (typically 1,000–3,000 trading days) are far too small. WAR is practical only for small p (say, $p \le 5$) unless you impose strong structure (diagonal A, block-diagonal, etc.).

15.5 HAR-DRD and Multivariate HARQ

Where we are. WAR is theoretically clean but does not scale. The next idea: decompose the covariance matrix into pieces that are easier to model separately.

DRD Decomposition — Bollerslev et al. (2018b)

Write the realized covariance matrix as

$$\mathbf{RC}_t \;=\; D_t\,R_t\,D_t, \tag{15.6}$$

where

- $D_t = \operatorname{diag}\big(\sqrt{[\mathbf{RC}_t]_{11}}, \ldots, \sqrt{[\mathbf{RC}_t]_{pp}}\big)$: diagonal matrix of realized standard deviations.
- $R_t = D_t^{-1}\,\mathbf{RC}_t\,D_t^{-1}$: realized correlation matrix (unit diagonal).

Then model D_t and R_t separately:

- **Variances:** fit a univariate HAR (or HARQ, from Chapter 6) to each diagonal element $[\mathbf{RC}_t]_{jj}$.
- **Correlations:** fit a univariate HAR to each unique off-diagonal element of R_t, using Fisher z-transform to map $[-1, 1]$ to $(-\infty, \infty)$.

Reassemble: $\widehat{\mathbf{RC}}_{t+1} = \hat{D}_{t+1}\, \hat{R}_{t+1}\, \hat{D}_{t+1}$.

In Plain English

DRD says: instead of trying to forecast all $p(p+1)/2$ entries of a covariance matrix at once, split the problem into two simpler pieces. First, forecast how volatile each asset will be tomorrow (the diagonal, which you already know how to do with HAR/HARQ). Second, forecast how correlated each pair will be (the off-diagonal, after normalizing). Then multiply them back together. This "divide and conquer" approach lets you reuse your best univariate tools on each piece.

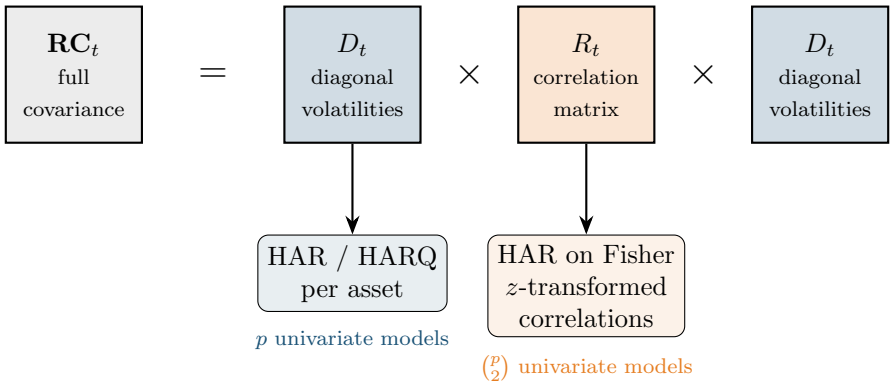


Figure 15.4: The DRD decomposition separates the covariance matrix into volatilities (modeled individually with HAR/HARQ) and correlations (modeled with HAR on Fisher z-transforms). Each piece is forecast with simple univariate models, then reassembled.

Why This Matters

HAR-DRD is arguably the strongest conventional baseline for multivariate vol forecasting. It directly extends the HARQ framework your project builds on: you can apply the $\sqrt{\text{RQ}}$ measurement-error correction from Chapter 6 to each variance element, improving forecasts on noisy days. If Project Direction 3 (GNN) is to justify its complexity, it must beat HAR-DRD on QLIKE across the full covariance matrix, not just on individual variances.

Separate Modeling Beats Joint

Bollerslev et al. (2018b) show that the DRD decomposition, where variances and correlations are modeled with separate HAR regressions, significantly outperforms both (a) direct vech-HAR on all elements jointly and (b) DCC-GARCH. The HARQ variant (adding $\sqrt{\text{RQ}}$ interactions from Chapter 6) further improves the variance forecasts, especially on high-noise days.

Worked Example:

HAR-DRD for 2 Assets Suppose you have daily realized covariance matrices for stocks A and B.

Step 1: extract components.

$$\mathbf{RC}_t = \begin{pmatrix} 0.00040 & 0.00012 \\ 0.00012 & 0.00025 \end{pmatrix}.$$

Then $D_t = \mathrm{diag}(0.0200, 0.0158)$ and

$$R_t = D_t^{-1}\,\mathbf{RC}_t\,D_t^{-1} = \begin{pmatrix} 1.000 & 0.379 \\ 0.379 & 1.000 \end{pmatrix}.$$

Step 2: model variances. Fit HAR to $\{\mathrm{RV}_t^A\}$ and $\{\mathrm{RV}_t^B\}$ separately (as in Chapter 6).
Step 3: model correlation. Apply Fisher z-transform: $z_t = \frac{1}{2}\ln\!\left(\frac{1+0.379}{1-0.379}\right) \approx 0.399$. Fit HAR to the $\{z_t\}$ series. Invert the transform on the forecast to get $\hat{\rho}_{t+1}$.
Step 4: reassemble. $\widehat{\mathbf{RC}}_{t+1} = \hat{D}_{t+1}\,\hat{R}_{t+1}\,\hat{D}_{t+1}$.

PSD Not Guaranteed

The separately forecasted $\hat{R}_{t+1}$ is not guaranteed to be a valid correlation matrix (it may have eigenvalues outside $[0, 1]$ or a diagonal different from 1). In practice, you project onto the nearest correlation matrix, e.g., via the alternating projections algorithm of Higham (2002).

15.6 Cholesky-HAR

Where we are. HAR-DRD decomposes the matrix into variances and correlations. Cholesky-HAR takes a different decomposition that guarantees PSD by construction.

Cholesky-HAR — Chiriac and Voev (2011)

Every PSD matrix $\mathbf{RC}_t$ has a unique Cholesky decomposition:

$$\mathbf{RC}_t = L_t\,L_t^\top,$$

where L_t is lower triangular with positive diagonal entries. The idea:

1. Compute L_t for each day.
2. Take the log of diagonal elements (to ensure positivity upon exponentiation).
3. Stack all $p(p+1)/2$ unique elements of the modified L_t into a vector.
4. Fit a separate HAR model to each element.
5. Forecast, exponentiate diagonals, unstack into $\hat{L}_{t+1}$, and reassemble: $\widehat{\mathbf{RC}}_{t+1} = \hat{L}_{t+1}\,\hat{L}_{t+1}^\top$.

Since any lower triangular matrix with positive diagonal gives a valid PSD matrix via $LL^\top$, the forecast is PSD by construction.

Worked Example:

Cholesky-HAR for 2 Assets Using the same $\mathbf{RC}_t$ from the DRD example:

$$\mathbf{RC}_t = \begin{pmatrix} 0.00040 & 0.00012 \\ 0.00012 & 0.00025 \end{pmatrix}.$$

Step 1: Cholesky decomposition.

$$L_t = \begin{pmatrix} 0.02000 & 0 \\ 0.00600 & 0.01470 \end{pmatrix}.$$

Check: $l_{11} = \sqrt{0.00040} = 0.02$, $l_{21} = 0.00012/0.02 = 0.006$, $l_{22} = \sqrt{0.00025 - 0.006^2} = \sqrt{0.000214} \approx 0.01470$.
Step 2: transform. Log-transform diagonal: $(\ln 0.02, \ln 0.01470) \approx (-3.912, -4.220)$. Off-diagonal left as-is: 0.006.
Step 3: fit HAR to each of the 3 elements (daily, weekly, monthly lags, as in Chapter 6).
Step 4: forecast and reassemble. Suppose HAR forecasts yield $(\hat{\ell}^*_{11}, \hat{\ell}_{21}, \hat{\ell}^*_{22}) = (-3.900, 0.0058, -4.210)$. Exponentiate diagonals: $\hat{l}_{11} = e^{-3.900} \approx 0.02020$, $\hat{l}_{22} = e^{-4.210} \approx 0.01483$. Then

$$\widehat{\mathbf{RC}}_{t+1} = \begin{pmatrix} 0.02020 & 0 \\ 0.00580 & 0.01483 \end{pmatrix} \begin{pmatrix} 0.02020 & 0.00580 \\ 0 & 0.01483 \end{pmatrix} = \begin{pmatrix} 0.000408 & 0.000117 \\ 0.000117 & 0.000254 \end{pmatrix}.$$

This is PSD by construction (it is $LL^\top$).

15.7 Graph-Based Methods

Where we are. All methods so far treat each element (or factor) of the covariance matrix independently. Graph-based methods exploit the fact that assets are *connected*: if asset A's volatility spills over to asset B (Chapter 16 formalizes this), modeling them jointly through a graph should help.

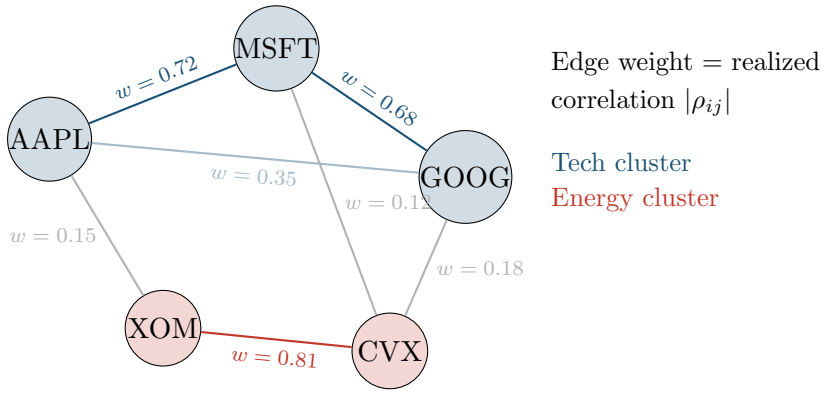


Figure 15.5: Assets as nodes in a graph with edge weights derived from realized correlations. Within-sector edges (tech, energy) are strong; cross-sector edges are weak. Graph-based models exploit this structure.

Graph-HAR — Zhang et al. (2024)

Graph-HAR augments the standard HAR model with graph-based features. The adjacency matrix W is constructed from realized correlations (or partial correlations). For each asset j:

$$\mathrm{RV}_{j,t+1} = \beta_0 + \beta_d \,\mathrm{RV}_{j,t} + \beta_w \,\mathrm{RV}^{(w)}_{j,t} + \beta_m \,\mathrm{RV}^{(m)}_{j,t} + \gamma \sum_{k \neq j} W_{jk} \,\mathrm{RV}_{k,t} + \varepsilon_{j,t+1}, \qquad (15.7)$$

where

- The first three terms are the standard HAR (Chapter 6).
- $\sum_{k \neq j} W_{jk}\ \mathrm{RV}_{k,t}$: a graph-weighted average of neighbors' volatilities. This is exactly one step of graph diffusion.
- γ: spillover coefficient. If $\gamma > 0$, high volatility in neighbors predicts higher volatility for asset j.
- W: adjacency matrix, typically from thresholded correlation or LASSO partial correlation.

In Plain English

Graph-HAR says: an asset's future volatility depends not only on its own past (the standard HAR terms) but also on what is happening to its neighbors. The "neighborhood" is defined by the graph: assets that are highly correlated form a cluster, and when one member's volatility spikes, the others tend to follow. The extra term $\gamma \sum_k W_{jk}\ \mathrm{RV}_{k,t}$ is simply a weighted average of yesterday's volatility across connected assets, capturing spillover effects that a univariate model would miss entirely.

Why This Matters

This is the linear foundation for Project Direction 3 (GNNs). Graph-HAR adds one cross-asset spillover term and already improves on standard HAR. The GNN extension (below) replaces this linear weighted average with learnable nonlinear message passing, potentially capturing richer interaction patterns. Your project contribution: show whether the nonlinear GNN spillover term yields statistically significant QLIKE gains over the linear Graph-HAR spillover on real equity data.

GNN for Covariance — Zhang et al. (2023)

Going further, Graph Neural Networks (GNNs) replace the linear spillover term with learnable message-passing layers:

1. **Node features:** each asset's HAR-style inputs (daily, weekly, monthly RV, plus optional firm characteristics).
2. **Graph structure:** adjacency from realized correlation, GICS sector membership, or learned adaptively.
3. **Message passing:** each node aggregates features from its neighbors through learned weight matrices (as in Chapter 13).
4. **Output:** node-level volatility forecasts or, with a symmetric readout layer, full covariance matrix forecasts.

The advantage over Graph-HAR: nonlinear aggregation and multi-hop information flow (asset A learns from asset C through intermediary B).

15.8 CNN-RCOV

Where we are. Graph methods treat assets as nodes. An alternative: treat the realized covariance matrix itself as an image.

Covariance Matrices as Images

A $p \times p$ realized covariance matrix is a symmetric, real-valued "image." If you sort assets by sector (tech, energy, financials, . . .), block structure emerges: within-sector blocks have high covariance, cross-sector blocks have low covariance. Convolutional neural networks

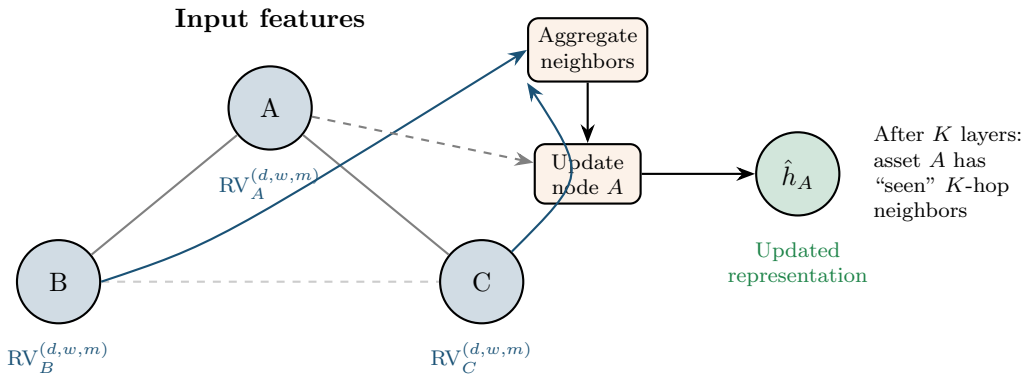


Figure 15.6: GNN message passing for one node. Asset A aggregates volatility features from its graph neighbors (B and C), then updates its own representation. After K message-passing layers, each node has information from all assets within K hops, capturing multi-step spillover effects.

(CNNs, Chapter 13) are designed to detect such local spatial patterns.

Architecture. Stack T past covariance matrices as "channels" (like RGB channels in image classification) and use 2D convolutions to extract temporal and cross-sectional patterns:

1. Input: $\mathbf{RC}_t, \mathbf{RC}_{t-1}, \ldots, \mathbf{RC}_{t-T+1} \in \mathbb{R}^{T\times p\times p}$.
2. 2D convolutional layers detect local block patterns.
3. Fully connected layers map to the $p(p+1)/2$ unique elements of $\widehat{\mathbf{RC}}_{t+1}$.

CNN-RCOV Caveats

- The literature is thin; this is more "conceptually appealing" than empirically proven at scale.
- Ordering assets by sector is a design choice. Different orderings yield different spatial patterns, and CNNs are not permutation-invariant. GNNs (Section 15.7) handle this more naturally.
- The output is not guaranteed PSD; post-hoc projection is required.
- For large p, the output layer has $O(p^2)$ parameters, which grows fast.

15.9 Geometric Deep Learning on the SPD Manifold

Where we are. Every method so far either ignores the PSD constraint or enforces it with post-hoc projection. SPDNet takes a fundamentally different approach: work directly on the manifold where covariance matrices live.

What is a Manifold?

A manifold is a curved space that locally looks like Euclidean space, much like the surface of a sphere locally looks flat. The set of $p\times p$ symmetric positive definite matrices (denoted $\mathcal{S}_{++}^p$) forms a smooth manifold. It is *not* a flat (Euclidean) space: the straight line between two PSD matrices can pass through matrices with zero or negative eigenvalues, leaving the PSD cone.

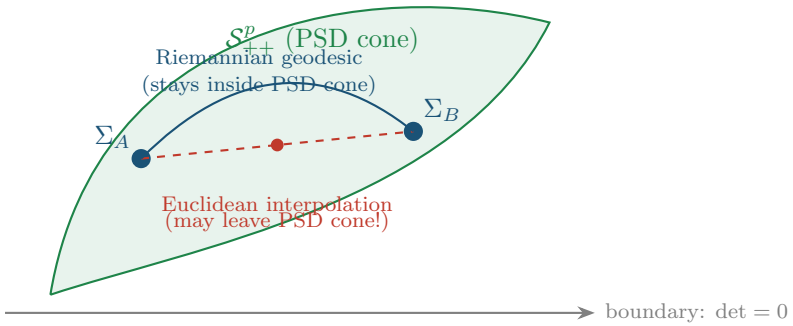


Figure 15.7: The SPD manifold. A straight (Euclidean) line between two PSD matrices Σ_A and Σ_B can pass through singular matrices (red dashed). The Riemannian geodesic (blue curve) stays inside the PSD cone by definition.

Affine-Invariant Riemannian Metric on $\mathcal{S}^p_{++}$

The standard Riemannian metric on $\mathcal{S}^p_{++}$ is

$$d(\Sigma_A, \Sigma_B) \;=\; \left\|\log\!\left(\Sigma_A^{-1/2}\,\Sigma_B\,\Sigma_A^{-1/2}\right)\right\|_F,$$

where log is the matrix logarithm and $\|\cdot\|_F$ is the Frobenius norm. This distance is affine-invariant: $d(A\Sigma_A A^\top, A\Sigma_B A^\top) = d(\Sigma_A, \Sigma_B)$ for any invertible A.

SPDNet

SPDNet replaces standard neural network operations with Riemannian analogs that preserve positive definiteness:

1. **BiMap layer** (analog of linear layer): $\Sigma \mapsto W\,\Sigma\,W^\top$, where W is a learnable $d \times p$ matrix with $d \le p$. If $\Sigma \succ 0$ and W has full row rank, then $W\Sigma W^\top \succ 0$. This also reduces dimension from p to d.
2. **ReEig layer** (analog of ReLU): eigendecompose $\Sigma = U\Lambda U^\top$, then $\Sigma \mapsto U\,\max(\Lambda, \epsilon I)\,U^\top$. Clips small eigenvalues to $\epsilon > 0$, keeping the matrix strictly PSD.
3. **LogEig layer** (analog of final feature extraction): $\Sigma \mapsto U\,\log(\Lambda)\,U^\top$. Maps from the SPD manifold to the tangent space (symmetric matrices), where standard Euclidean operations apply.

The output of LogEig lives in a flat space, so you can apply a standard linear classifier or regressor on top.

SPDNet: PSD by Design

Because every layer in SPDNet maps PSD inputs to PSD outputs, intermediate representations are always valid covariance matrices. No post-hoc projection is needed. This is the only deep learning architecture for covariance forecasting with a built-in PSD guarantee.

15.10 The PSD Constraint

Where we are. Every multivariate volatility model must produce a valid (positive semi-definite) covariance matrix. Some methods guarantee this; others require fixing after the fact. This section provides a systematic comparison.

Why PSD Matters

A covariance matrix that is not PSD implies negative variance for some portfolio. Specifically, if Σ has a negative eigenvalue with eigenvector $\mathbf{v}$, then the portfolio $\mathbf{v}$ has predicted variance $\mathbf{v}^\top \Sigma \mathbf{v} < 0$. This is nonsensical and causes optimizers to blow up (infinite leverage on the "negative variance" direction).

Table 15.1: PSD guarantees across multivariate volatility methods.

Method	PSD Guarantee	Mechanism
DCC-GARCH (Sec. 15.3)	Yes	By construction ($D_t R_t D_t$, R_t from rescaling)
WAR (Sec. 15.4)	No	Post-hoc projection needed
HAR-DRD (Sec. 15.5)	No	Post-hoc projection on $\hat{R}_{t+1}$
Cholesky-HAR (Sec. 15.6)	Yes	$\hat{L}\hat{L}^\top$ is PSD by construction
Graph-HAR (Sec. 15.7)	No	Post-hoc projection needed
CNN-RCOV (Sec. 15.8)	No	Post-hoc projection needed
SPDNet (Sec. 15.9)	Yes	Riemannian operations preserve PSD

Post-Hoc PSD Projection

When a method does not guarantee PSD, the standard fix is eigenvalue clipping:

1. Eigendecompose: $\hat{\Sigma} = U \Lambda U^\top$.
2. Set $\tilde{\Lambda} = \max(\Lambda, 0)$ (zero out negative eigenvalues).
3. Reconstruct: $\tilde{\Sigma} = U \tilde{\Lambda} U^\top$.

This gives the nearest PSD matrix in Frobenius norm. The cost: it introduces bias, and the projected matrix no longer minimizes whatever loss function the model was trained on. For correlation matrices, the Higham (2002) alternating projection algorithm additionally enforces unit diagonal.

PSD Violations in Practice

For small p (2–5 assets), PSD violations are rare and small. For large p (50–500), they are common and can be severe, especially with element-wise models (HAR-DRD, Graph-HAR) that do not enforce cross-element consistency. If PSD is critical for your application (portfolio optimization, risk management), prefer methods with built-in guarantees: DCC, Cholesky-HAR, or SPDNet.

15.11 Summary

Key takeaways from this chapter:

- Realized covariance (RC) extends realized variance to matrices via outer products of intraday return vectors ((15.1)).
- Non-synchronous trading biases RC toward zero (Epps effect). Refresh-time sampling and the Hayashi–Yoshida estimator ((15.2)) address this, with HY being more data-efficient.
- The multivariate realized kernel ((15.3)) handles both noise and non-synchronicity while guaranteeing PSD (Barndorff-Nielsen et al., 2011).
- DCC-GARCH (Engle, 2002) is the standard parametric baseline: easy to estimate, scales to large p, but uses only daily data and imposes a single correlation dynamic.
- WAR models RC directly as a matrix time series, but suffers from parameter explosion for $p > 5$.

- HAR-DRD (Bollerslev et al., 2018b) decomposes RC into variances and correlations, models each with HAR, and outperforms both joint modeling and DCC.
- Cholesky-HAR (Chiriac and Voev, 2011) models Cholesky factors with HAR, guaranteeing PSD by construction.
- Graph-HAR (Zhang et al., 2024) and GNN methods (Zhang et al., 2023) model volatility spillovers through asset graphs, capturing network effects that element-wise models miss.
- CNN-RCOV treats RC matrices as images; conceptually appealing but empirically limited and not permutation-invariant.
- SPDNet operates on the Riemannian manifold of PSD matrices, the only deep learning approach with a built-in PSD guarantee.
- Methods either guarantee PSD (DCC, Cholesky-HAR, SPDNet) or require post-hoc eigenvalue projection, which introduces bias.
- For most applications, start with HAR-DRD (best accuracy in large-scale comparisons) or Cholesky-HAR (if PSD is essential). Use DCC-GARCH as the parametric baseline. Graph and manifold methods are promising but still maturing.

Table 15.2: Key results: multivariate volatility methods.

Method	**Data Input**	**Key Advantage**	**Key Limitation**
DCC-GARCH	Daily returns	Scales to large p; PSD guaranteed	Single correlation dynamic; ignores HF data
WAR	Realized covariance	Theoretically clean matrix AR	$O(p^4)$ parameters
HAR-DRD	Realized covariance	Best accuracy (BPQ 2018); flexible	PSD not guaranteed
Cholesky-HAR	Realized covariance	PSD by construction	Cholesky ordering arbitrary
Graph-HAR / GNN	RV + graph	Models spillovers	Graph construction is a design choice
CNN-RCOV	RC as image	Detects block structure	Not permutation-invariant
SPDNet	RC on manifold	PSD by design; principled geometry	Complex; limited empirical evidence

Next. Chapter 16 formalizes the volatility spillovers that Graph-HAR exploits, using the Diebold–Yilmaz connectedness framework and network topology.

Chapter 16

Volatility Spillovers and Connectedness

Application

Chapter 15 built tools for forecasting the full covariance matrix. This chapter focuses on a specific question: how does volatility transmit from one asset (or market) to another? Spillover indices and connectedness measures are both diagnostic tools (understanding contagion) and predictive features (Chapter 10). Project 3 relies heavily on this framework.

16.1 Diebold–Yilmaz Spillover Indices

In Chapter 15 you learned to model the joint dynamics of multiple assets. Now we zoom in on a sharper question: when one asset's volatility jumps, how much of that shock spills over into other assets?

Volatility as a contagious shock

Think of volatility shocks as ripples in a pond. A stone dropped on the equity side sends waves that reach bonds, commodities, and currencies. The Diebold–Yilmaz (DY) framework measures the size and direction of those ripples using standard vector autoregression (VAR) machinery.

16.1.1 Setup: VAR on Realized Volatilities

Stack the realized volatilities of N assets into a vector $\mathbf{y}_t = (\mathrm{RV}_{1,t}, \ldots, \mathrm{RV}_{N,t})'$ and fit a VAR(p):

$$\mathbf{y}_t = \boldsymbol{c} + \sum_{\ell=1}^{p} \boldsymbol{A}_\ell \, \mathbf{y}_{t-\ell} + \boldsymbol{u}_t, \qquad \boldsymbol{u}_t \sim (0, \boldsymbol{\Sigma}), \tag{16.1}$$

In Plain English

Today's vector of realized volatilities across all assets is a linear combination of the past p days' volatility vectors, plus a surprise term. Each coefficient matrix $\boldsymbol{A}_\ell$ captures how yesterday's (or day-before's) vol in *every* asset feeds into today's vol for *every* asset. This is the multivariate extension of the HAR idea: past vol predicts future vol, but now

cross-asset effects are explicitly modeled.

Why This Matters

The VAR on RVs is the backbone of the Diebold-Yilmaz framework. For your project, the off-diagonal entries of $\boldsymbol{A}_\ell$ are cross-asset predictive features: they tell you how much of asset j's vol is predictable from asset k's recent vol. This directly extends the univariate HAR baseline with cross-asset lags.

where:

- $\mathbf{y}_t \in \mathbb{R}^N$: vector of realized volatilities on day t,
- $\boldsymbol{c} \in \mathbb{R}^N$: intercept vector,
- $\boldsymbol{A}_\ell \in \mathbb{R}^{N\times N}$: coefficient matrix at lag ℓ,
- $\boldsymbol{u}_t$: reduced-form innovation vector with covariance $\boldsymbol{\Sigma}$.

The key output is the *moving-average representation* obtained by inverting the VAR:

$$\mathbf{y}_t = \boldsymbol{\mu} + \sum_{h=0}^{\infty} \boldsymbol{\Phi}_h \, \boldsymbol{u}_{t-h}, \tag{16.2}$$

In Plain English

This rewrites the system so that today's volatility is expressed as a sum of all past shocks, weighted by how much each shock persists over time. The matrix $\boldsymbol{\Phi}_h$ is the "memory kernel": entry (j,k) tells you how much a surprise in asset k's vol h days ago still echoes in asset j's vol today. It is the multivariate impulse-response function.

Why This Matters

The MA representation is what lets you decompose forecast errors into contributions from each asset. For GNN-based vol forecasting (Project Direction 3), the impulse-response matrices $\boldsymbol{\Phi}_h$ define the effective "edge weights" of the spillover graph at each horizon.

where the $N \times N$ matrices $\boldsymbol{\Phi}_h$ are the impulse-response coefficients at horizon h. Entry $(\boldsymbol{\Phi}_h)_{jk}$ tells you how a unit shock to asset k today affects asset j at horizon h.

16.1.2 Generalized Forecast Error Variance Decomposition

The next step decomposes the H-step forecast error variance of each asset into contributions from every shock. The generalized variant (GFEVD), introduced by Pesaran and Shin (1998) and adopted by Diebold and Yilmaz (2012), does not depend on variable ordering (unlike Cholesky decompositions).

Generalized Forecast Error Variance Decomposition

The fraction of asset j's H-step forecast-error variance attributable to shocks originating in asset k is:

$$\theta_{jk}^{(H)} = \frac{\sigma_{kk}^{-1} \sum_{h=0}^{H-1} \left(\boldsymbol{e}_j' \, \boldsymbol{\Phi}_h \, \boldsymbol{\Sigma} \, \boldsymbol{e}_k\right)^2}{\sum_{h=0}^{H-1} \boldsymbol{e}_j' \, \boldsymbol{\Phi}_h \, \boldsymbol{\Sigma} \, \boldsymbol{\Phi}_h' \, \boldsymbol{e}_j}, \tag{16.3}$$

where:

- σ_{kk}: the (k,k) entry of $\boldsymbol{\Sigma}$ (variance of shock k),

- $\boldsymbol{e}_j$: the j-th column of the $N \times N$ identity matrix,
- $\boldsymbol{\Phi}_h$: the impulse-response matrix at horizon h from Eq. (16.2),
- H: forecast horizon (typically 10 days).

In Plain English

This formula answers a simple question: of all the uncertainty in asset j's volatility forecast H days ahead, what fraction was caused by a shock to asset k? The numerator isolates the contribution of shock k (scaled by its own variance), and the denominator is the total forecast uncertainty for asset j. When $j = k$, you get the "own" contribution; when $j \neq k$, you get the spillover from k to j.

Why This Matters

The GFEVD is the core building block for spillover features. Each off-diagonal entry $\theta_{jk}^{(H)}$ is a direct measure of cross-asset vol predictability: it tells you how much of asset j's vol surprise came from asset k. These entries become edges in the spillover graph that a GNN can learn over.

Because GFEVD rows do not sum to one in general, normalize each row:

$$\widetilde{\theta}_{jk}^{(H)} = \frac{\theta_{jk}^{(H)}}{\sum_{k=1}^{N} \theta_{jk}^{(H)}}, \qquad \text{so that} \quad \sum_{k=1}^{N} \widetilde{\theta}_{jk}^{(H)} = 1. \tag{16.4}$$

In Plain English

The raw GFEVD fractions for a given asset do not necessarily add up to 100% because the generalized approach allows shocks to be correlated. This normalization simply rescales each row so that the contributions from all assets (including itself) sum to one. After normalization, you can read each entry as a percentage: "X% of asset j's vol forecast error is attributable to shocks from asset k."

Why This Matters

Normalized entries are directly interpretable as edge weights in the spillover network. For your feature matrix, these percentages can be used as-is: the "directional FROM" feature for each asset is just the sum of its off-diagonal row entries.

16.1.3 Spillover Measures

From the normalized decomposition table $\widetilde{\boldsymbol{\Theta}}^{(H)}$, three spillover measures follow immediately.

Diebold–Yilmaz Spillover Decomposition

Total spillover index. The fraction of total forecast-error variance due to cross-asset shocks:

$$S^{(H)} = \frac{1}{N} \sum_{\substack{j,k=1 \\ j \neq k}}^{N} \widetilde{\theta}_{jk}^{(H)} \times 100. \tag{16.5}$$

Directional FROM. How much volatility asset j *receives* from all others:

$$S_{j\leftarrow\bullet}^{(H)} = \frac{1}{N}\sum_{\substack{k=1\\k\neq j}}^{N}\widetilde{\theta}_{jk}^{(H)} \times 100. \tag{16.6}$$

Directional TO. How much volatility asset j *transmits* to all others:

$$S_{j\rightarrow\bullet}^{(H)} = \frac{1}{N}\sum_{\substack{k=1\\k\neq j}}^{N}\widetilde{\theta}_{kj}^{(H)} \times 100. \tag{16.7}$$

Net spillover. Transmitters minus receivers:

$$S_{j}^{\text{net},(H)} = S_{j\rightarrow\bullet}^{(H)} - S_{j\leftarrow\bullet}^{(H)}. \tag{16.8}$$

A positive net spillover means asset j is a *net transmitter* of volatility; negative means *net receiver*.

Why This Matters

These four measures are the feature engineering payoff of the DY framework. Total spillover $S_t^{(H)}$ is a regime indicator (high = crisis = different vol dynamics). Directional FROM measures how "vulnerable" an asset is to imported vol. Net spillover identifies transmitters vs. receivers. All three become columns in your feature matrix. For Project Direction 3 (GNNs), the pairwise entries form the adjacency matrix of the volatility graph, and time-varying versions define dynamic graph structure.

The framework evolved across three papers: Diebold and Yilmaz (2009) introduced the total index using a Cholesky decomposition, Diebold and Yilmaz (2012) added directional measures and switched to GFEVD, and Diebold and Yilmaz (2014) refined the generalized VAR approach and applied it to a broader set of markets.

16.1.4 Spillover Network Diagram

The variance decomposition table maps directly onto a weighted directed graph: each asset is a node, and the edge from k to j carries weight $\widetilde{\theta}_{jk}^{(H)}$.

16.1.5 Worked Example: 3-Asset VAR(1)

Worked Example:

Computing Spillover Indices Consider three assets (Equity, Bonds, FX) with the following normalized 10-step GFEVD table (each row sums to 100%):

	Equity	**Bonds**	**FX**	**FROM others**
Equity	60	25	15	40
Bonds	30	55	15	45
FX	20	10	70	30
TO others	50	35	30	

Step 1: Directional FROM. Sum the off-diagonal entries in each row, divided by

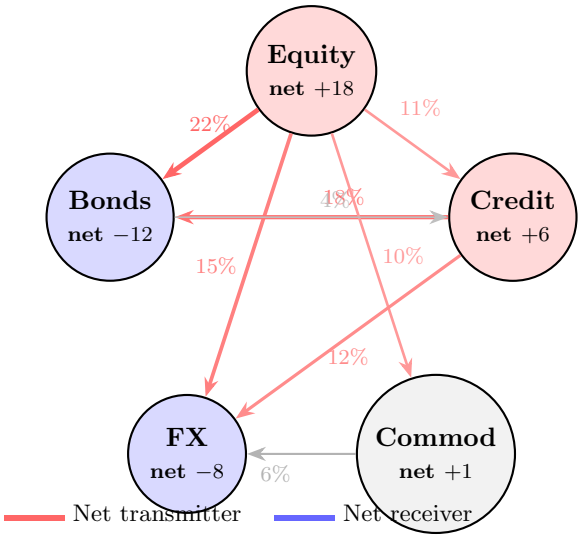


Figure 16.1: Spillover network for five asset classes. Edge labels show the percentage of j's forecast-error variance explained by shocks from k. Node color indicates net transmitter (red) vs. net receiver (blue). Data are illustrative.

$N = 3$:

$$S_{\text{Equity}\leftarrow\bullet} = \frac{25+15}{3} = 13.3\%,$$
$$S_{\text{Bonds}\leftarrow\bullet} = \frac{30+15}{3} = 15.0\%,$$
$$S_{\text{FX}\leftarrow\bullet} = \frac{20+10}{3} = 10.0\%.$$

Step 2: Directional TO. Sum the off-diagonal entries in each column, divided by N:

$$S_{\text{Equity}\rightarrow\bullet} = \frac{30+20}{3} = 16.7\%,$$
$$S_{\text{Bonds}\rightarrow\bullet} = \frac{25+10}{3} = 11.7\%,$$
$$S_{\text{FX}\rightarrow\bullet} = \frac{15+15}{3} = 10.0\%.$$

Step 3: Net spillover.

$$S^{\text{net}}_{\text{Equity}} = 16.7 - 13.3 = +3.4\% \quad \text{(net transmitter)},$$
$$S^{\text{net}}_{\text{Bonds}} = 11.7 - 15.0 = -3.3\% \quad \text{(net receiver)},$$
$$S^{\text{net}}_{\text{FX}} = 10.0 - 10.0 = 0.0\% \quad \text{(neutral)}.$$

Step 4: Total spillover index. Average of all off-diagonal entries, divided by N:

$$S^{(10)} = \frac{25+15+30+15+20+10}{3\times 3} \times 100 = \frac{115}{9} \times 100 \approx 38.3\%.$$

Interpretation: about 38% of the total forecast-error variance in this system comes from cross-asset shocks rather than own shocks. Equity is the dominant transmitter.

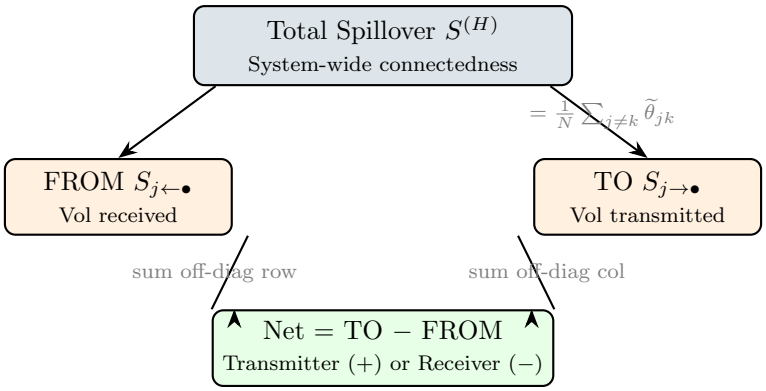


Figure 16.2: Decomposition of the Diebold-Yilmaz spillover measures. The total index summarizes system-wide connectedness. Directional FROM and TO break it down per asset, and the net spillover identifies transmitters vs. receivers.

16.2 Time-Varying Spillovers

The static spillover table in the previous section gives you a single snapshot. In practice, connectedness changes dramatically over time: calm markets have low spillovers, crises have high ones. This section covers two methods to capture the dynamics.

16.2.1 Rolling-Window Approach

The simplest method: re-estimate the VAR and GFEVD on a rolling window of w days (typically $w = 200$) and plot $S_t^{(H)}$ over time.

Rolling-Window Spillover

For each day t, estimate the VAR on $\{t-w+1, \ldots, t\}$, compute the GFEVD, and record $S_t^{(H)}$. The resulting time series reveals spillover regimes:

- Calm periods: $S_t^{(H)} \approx 30$–40%,
- Crises (2008 GFC, 2020 COVID): $S_t^{(H)}$ spikes to 70–85%.

Window-length sensitivity

The rolling-window approach introduces a hyperparameter w that affects both level and smoothness. Too short ($w < 100$): noisy, unstable VAR estimates. Too long ($w > 300$): sluggish, crises get averaged out. Always report results for at least two window lengths to check robustness (Diebold and Yilmaz, 2012).

16.2.2 TVP-VAR Connectedness

Antonakakis et al. (2020) replace the rolling-window VAR with a *time-varying parameter VAR* (TVP-VAR) estimated via the Kalman filter. This eliminates the window-length hyperparameter entirely.

The TVP-VAR model allows coefficients to drift:

$$\mathbf{y}_t = \boldsymbol{c}_t + \sum_{\ell=1}^{p} \boldsymbol{A}_{\ell,t}\, \mathbf{y}_{t-\ell} + \boldsymbol{u}_t, \qquad \operatorname{vec}(\boldsymbol{A}_t) = \operatorname{vec}(\boldsymbol{A}_{t-1}) + \boldsymbol{\eta}_t, \tag{16.9}$$

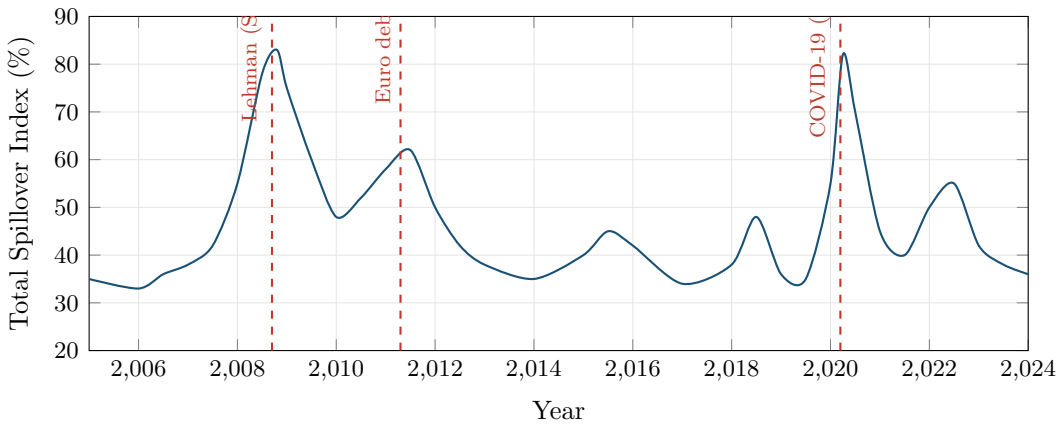


Figure 16.3: Stylized total spillover index (200-day rolling window, $H = 10$) across five major asset classes. The index spikes sharply during the 2008 financial crisis and the March 2020 COVID selloff, reflecting contagion across markets.

In Plain English

This is the same VAR as before, but now the coefficient matrices $\boldsymbol{A}_{\ell,t}$ are allowed to change every day. The second equation says that today's coefficients equal yesterday's plus a small random drift $\boldsymbol{\eta}_t$. The Kalman filter estimates the evolving coefficients without needing a rolling window, so the spillover index updates smoothly each day rather than jumping when observations enter or leave a fixed window.

Why This Matters

TVP-VAR connectedness responds faster to regime changes than rolling-window estimates, making it a better real-time feature for your ML model. When the connectedness index spikes, your model knows that vol dynamics have shifted to "crisis mode" with higher persistence and stronger cross-asset effects. This is a natural conditioning variable for the HAR baseline: interact HAR lags with a high-connectedness indicator to let the model adapt its decay structure in real time.

where:

- $\boldsymbol{A}_{\ell,t}$: time-varying coefficient matrix at lag ℓ and time t,
- $\boldsymbol{\eta}_t \sim (0, \boldsymbol{Q})$: state innovation with covariance $\boldsymbol{Q}$ governing the speed of parameter drift.

At each t, the Kalman filter produces updated coefficient estimates, and you compute the GFEVD exactly as before to get $S_t^{(H)}$.

TVP-VAR vs. Rolling Window

Antonakakis et al. (2020) show that TVP-VAR connectedness is smoother than rolling-window estimates, avoids the abrupt "entry/exit" artifacts when extreme observations enter or leave the window, and responds faster to genuine structural breaks. Both methods agree on the broad pattern: spillovers spike during crises and monetary-policy surprises, but the TVP-VAR resolves timing more sharply.

16.3 Network Visualization and Interpretation

The GFEVD table is a matrix of numbers. Networks make that matrix readable at a glance. This section covers the visual conventions and the patterns you should look for.

16.3.1 Visual Encoding

Four encoding rules produce an interpretable graph:

1. **Node size** proportional to total connectedness ($S_{j\to\bullet} + S_{j\leftarrow\bullet}$). Large nodes are "systemically important" regardless of sign.
2. **Edge thickness** proportional to pairwise spillover $\widetilde{\theta}^{(H)}_{jk}$. Only draw edges above a threshold (e.g., 5%) to avoid clutter.
3. **Edge direction** from shock source k to shock recipient j (arrow points toward the asset whose variance is explained).
4. **Node color** by net spillover sign: red for net transmitters, blue for net receivers. Alternatively, color by sector or asset class.

16.3.2 Calm vs. Crisis Networks

The most striking pattern in spillover networks is the structural shift between tranquil and stressed markets (Demirer et al., 2018).

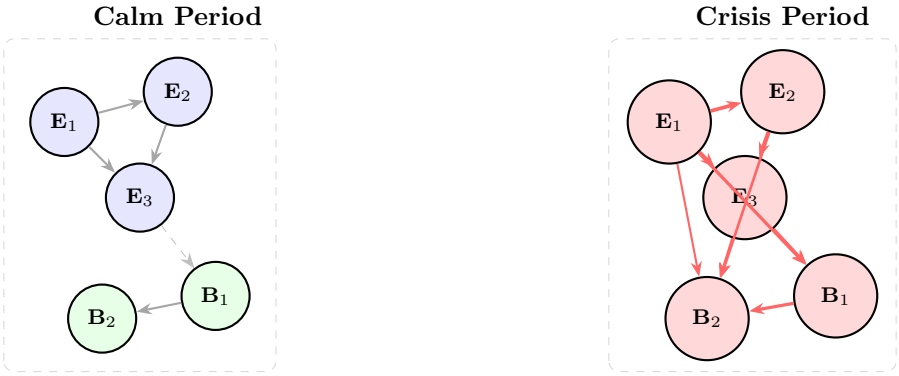


Figure 16.4: Spillover networks during calm (left) and crisis (right) periods. In calm markets, connectedness is largely within-sector (equity-to-equity, bond-to-bond), with sparse cross-sector links. During crises, cross-sector edges thicken and multiply: everything becomes connected to everything. After Demirer et al. (2018).

Calm-Crisis Network Transition

Three stylized facts from the DY network literature:

1. **Calm periods**: within-sector connectedness dominates. Equity shocks stay in equities; bond shocks stay in bonds. Total spillover index is low (30–40%).
2. **Crises**: cross-sector connectedness surges. The network topology shifts from clustered to nearly fully connected. Total spillover jumps to 70–85%.
3. **Net transmitter identity shifts**: in calm markets, commodity shocks are often isolated; during crises, equity and credit become dominant transmitters (Diebold and Yilmaz, 2014).

16.4 Cross-Asset Universality

The high cross-sector connectedness during crises hints at something deeper: maybe volatility dynamics are not just correlated but fundamentally *similar* across asset classes. Two recent papers make this case precisely.

Sirignano and Cont (2019) train a single feedforward network to predict returns and volatility by pooling data across all assets (stocks, currencies, commodities). The pooled ("universal") model performs comparably to individual asset-specific models, implying that the features driving volatility are largely shared.

Universal Volatility Features

Sirignano and Cont (2019) show that a model trained on one asset class transfers well to others with minimal fine-tuning. This "universality" holds for features based on returns, realized volatility, and order-flow imbalance. The implication: spillover networks may reflect common factor exposures rather than direct causal transmission.

Rosenbaum and Zhang (2022) push universality further by estimating the Hurst exponent H across assets using a universal LSTM. Their key finding: $H \approx 0.1$ everywhere (equities, FX, rates, commodities), consistent with the rough volatility theory from Chapter 7.

Why universality matters for spillovers

If volatility dynamics really are "universal" (same roughness, same feature importance across assets), then the strong cross-asset connectedness you see during crises may not require a contagion story. Instead, a single common factor (e.g., dealer risk capacity, funding liquidity) could drive all assets simultaneously. This does not diminish the usefulness of spillover indices as diagnostic tools, but it changes how you interpret them: high spillover may reflect common-factor exposure rather than sequential transmission from one asset to the next.

Why This Matters

Universality has a direct architectural implication for your project. If vol dynamics are truly shared across assets, you can train a single model on pooled data (all assets stacked) rather than fitting N separate models. This dramatically increases your effective sample size and reduces overfitting. Even for a univariate HAR-based forecast, pooling across assets with shared features (same lags, same jump indicators) can improve out-of-sample QLIKE.

16.5 Spillover Indices as Predictive Features

The previous sections treated spillover indices as descriptive diagnostics. This section flips the lens: can you use them as *inputs* to the forecasting models from Chapters 11–14? The answer is a qualified yes, with important caveats.

16.5.1 Three Feature Families

Spillover-Based Features for ML

1. **Total spillover as regime indicator.** High $S_t^{(H)}$ signals "crisis mode" where correlations are elevated and volatility persistence is stronger. Use it as a conditioning variable: for example, a LightGBM model can split on $S_t^{(H)} > 60$ to activate crisis-specific trees.
2. **Directional FROM as vulnerability measure.** An asset with a high and rising $S_{j\leftarrow\bullet}$ is absorbing shocks from many sources. This predicts higher future volatility for asset j, especially in the 5–20 day horizon. See Chapter 10 for how to z-score and lag these features.
3. **Net spillover as contrarian signal.** Extreme net receivers ($S_j^{\text{net}} \ll 0$) tend to mean-revert: assets that have absorbed large cross-market shocks often see volatility revert to lower levels once the contagion subsides. This generates a medium-horizon (1–3 month) signal for volatility forecasting.

16.5.2 Practical Considerations

Spillover features are slow-moving

Spillover indices computed from a 200-day rolling window update slowly. For short-horizon forecasts (1–5 days), they may add little predictive power beyond the VIX or a simple HAR model. Their value is highest for medium-to-long horizons (1 week to 3 months) and for regime-conditional models.
Additionally, the TVP-VAR variant from Antonakakis et al. (2020) responds faster than rolling-window estimates, making it more suitable as an ML input if you need timelier signals.

A practical recipe for incorporating spillover features:

1. Compute the DY spillover index and directional measures using a 200-day rolling window (or TVP-VAR).
2. Construct three features per asset: total spillover $S_t^{(H)}$, directional FROM $S_{j\leftarrow\bullet}^{(H)}$, and net spillover $S_j^{\text{net},(H)}$.
3. Z-score each feature using a 252-day trailing window.
4. Include as additional columns in the feature matrix from Chapter 10, alongside HAR lags, VRP, and other predictors.
5. Check SHAP importance: if spillover features rank below the top 10 in a LightGBM model, they likely add noise rather than signal for your target horizon.

16.6 Summary

- The Diebold–Yilmaz framework measures volatility spillovers using a VAR on realized volatilities and generalized forecast error variance decomposition (GFEVD).
- The **total spillover index** $S^{(H)}$ captures the fraction of forecast-error variance explained by cross-asset shocks; typical values are 30–40% in calm markets, 70–85% during crises.
- **Directional spillovers** (FROM, TO, net) identify which assets transmit and which receive volatility shocks; equity and credit tend to be net transmitters during stress.
- Rolling-window estimation (200-day window) is simple but introduces a window-length hyperparameter; the TVP-VAR approach of Antonakakis et al. (2020) eliminates this choice.

- Network visualization reveals that calm-period connectedness is within-sector (clustered), while crisis-period connectedness is cross-sector (dense, nearly fully connected).
- Demirer et al. (2018) applied the DY framework to a global network of banks and sovereigns, confirming the calm-to-crisis structural shift at institution level.
- Sirignano and Cont (2019) demonstrate "universal" volatility features: a pooled model trained across asset classes performs comparably to asset-specific models, suggesting common underlying dynamics.
- Rosenbaum and Zhang (2022) estimate $H \approx 0.1$ universally across assets using an LSTM, connecting spillover phenomena to rough volatility theory (Chapter 7).
- Spillover indices are useful as ML features for medium-horizon forecasting: total spillover as a regime indicator, directional FROM as a vulnerability measure, and net spillover as a contrarian signal.
- Spillover features are slow-moving (200-day window) and may add limited value for short-horizon forecasts; always verify with SHAP importance analysis.
- The GFEVD approach generalizes the Cholesky decomposition and does not depend on variable ordering, making it suitable for large cross-sections.
- High cross-asset spillovers during crises may reflect common-factor exposure (funding liquidity, dealer balance sheets) rather than sequential causal contagion.

Concept	**Key Result**
Total spillover index	30–40% calm, 70–85% crisis; captures system-wide connectedness
Directional (FROM/TO/net)	Identifies transmitters (equity, credit) vs. receivers (bonds, FX)
Rolling window vs. TVP-VAR	TVP-VAR avoids window-length choice; sharper crisis timing
Network topology shift	Within-sector $\rightarrow$ cross-sector during stress
Universality	Common features and $H \approx 0.1$ across asset classes
Spillover as ML feature	Best for medium horizon; z-score and verify with SHAP

Part VI

Evaluation and Practice

Chapter 17

Forecast Evaluation

Why This Chapter Is Non-Negotiable

This chapter teaches the evaluation methodology used across all project directions. Every volatility forecast you produce must be evaluated with QLIKE (not MSE), compared with the Diebold–Mariano test, and placed in a Model Confidence Set. If you use cross-validation, it must be purged. If you report Sharpe ratios, they must be deflated. These are not optional extras; they are the minimum standard for credible work. Skip this chapter and your results mean nothing.

A good forecast is useless if you cannot prove it is good. This chapter gives you the statistical machinery to distinguish genuine forecasting ability from noise, luck, and overfitting. We cover the right loss function (QLIKE), the right comparison test (Diebold–Mariano), the right multi-model framework (Model Confidence Set), the right cross-validation (purged K-fold with embargo), and the right Sharpe ratio adjustment (Deflated Sharpe Ratio).

17.1 Why Evaluation Methodology Matters

Before diving into specific tools, consider two scenarios that illustrate why evaluation methodology *is* the credibility of your results.

Scenario 1. You build a LightGBM volatility forecast that achieves 5% lower QLIKE than the HAR benchmark (Chapter 6). Your manager asks: "Is that improvement statistically significant, or would it vanish on a different sample?" Without a Diebold–Mariano test, you cannot answer.

Scenario 2. You try 30 different feature sets, pick the best one, and report a backtest Sharpe ratio of 1.5. A colleague asks: "What's the probability that at least one of 30 random strategies would have produced a Sharpe that high?" Without the Deflated Sharpe Ratio, you cannot answer.

The Two Failure Modes

Evaluation errors come in two flavors:

1. **Declaring a winner that isn't one.** A 5% improvement in QLIKE that is not statistically significant is noise, not signal.
2. **Reporting a Sharpe ratio inflated by multiple testing.** A Sharpe of 1.5 from 30 experiments may be pure luck (Bailey and de Prado, 2014).

The evaluation framework is not a final step you tack on after research. It is the infrastructure you build *first*, so that every experiment you run produces honest, comparable numbers from

day one.

17.2 MSE and Its Limitations for Volatility

We start with the loss function you already know, then explain why it is not enough for volatility forecasting.

Loss Functions

A **loss function** $L(\sigma_t^2, h_t)$ measures how far a forecast h_t is from the truth σ_t^2. Lower loss means a better forecast. You have used mean squared error (MSE) in regression problems; it is the default in most ML pipelines.

17.2.1 The MSE Formula

The mean squared error between a sequence of forecasts $\{h_t\}$ and realized values $\{\sigma_t^2\}$ is:

$$\text{MSE} = \frac{1}{T}\sum_{t=1}^{T}\left(\sigma_t^2 - h_t\right)^2 \tag{17.1}$$

where:

- σ_t^2 is the true (unobservable) variance on day t,
- h_t is the forecast variance for day t, produced before day t,
- T is the number of forecast evaluation days.

In Plain English

MSE measures the average squared gap between what you predicted and what actually happened. Squaring means big misses count disproportionately: a forecast that is off by 2 units contributes 4 to the loss, while being off by 4 units contributes 16. It treats over-prediction and under-prediction identically.

17.2.2 The Proxy Problem

We never observe true variance σ_t^2. Instead, we use a proxy: realized variance RV_t (Chapter 2). This proxy is noisy because $\text{RV}_t = \sigma_t^2 + \eta_t$, where η_t is measurement error from finite sampling, microstructure noise (Chapter 3), and jumps (Chapter 4).

The good news: MSE produces correct model *rankings* even when using a noisy proxy, as long as the noise is independent of the forecast (Patton, 2011). If model A has lower MSE than model B when evaluated against RV_t, the same ranking holds against true σ_t^2. This property is called **robustness to noise in the proxy**.

17.2.3 Why MSE Is Still Not Enough

MSE has a deeper problem: it is symmetric and heavily penalizes extreme values.

MSE Penalizes Outliers Disproportionately

Suppose your forecast is $h_t = 1.0$ (in annualized variance units) for every day. On 249 normal days, true variance is 1.0 and MSE contribution is 0. On one crisis day, true variance spikes to 10.0, and MSE contribution is $(10-1)^2 = 81$. That single day dominates the entire loss. This means MSE-optimal forecasts chase outliers: they overweight extreme

days at the expense of forecasting accuracy during normal times, which is the opposite of what most applications need.

MSE Is Necessary but Not Sufficient

MSE is proxy-robust, which is valuable. But its sensitivity to extreme realized variance values makes it a poor primary metric for volatility. Report it as a secondary check alongside QLIKE.

Why This Matters

In your HAR vs. ML comparison, MSE will be dominated by a handful of crisis days (e.g., COVID March 2020, VIX spikes). A model that slightly better predicts those extremes will look dramatically better under MSE, even if it performs worse on the 95% of normal days that matter for daily risk management. Always report MSE as a secondary metric, but never let it drive model selection.

17.3 QLIKE: The Preferred Loss Function

Having seen why MSE over-penalizes extreme days, we now introduce the loss function that the volatility forecasting literature has converged on as the primary metric.

17.3.1 Intuition

QLIKE (quasi-likelihood loss) comes from the negative log-likelihood of a Gaussian distribution with variance h_t. Think of it this way: if returns were exactly normal with variance h_t, the best forecast would minimize QLIKE. Even when returns are not normal (and they never are), QLIKE retains two critical properties that MSE shares one of and lacks the other.

17.3.2 The QLIKE Formula

$$\text{QLIKE} = \frac{1}{T}\sum_{t=1}^{T}\left(\ln h_t + \frac{\sigma_t^2}{h_t}\right) \tag{17.2}$$

where:

- h_t is the forecast variance for day t,
- σ_t^2 is the true variance on day t (in practice, RV_t),
- $\ln h_t$ penalizes forecasts that are too large (over-prediction),
- σ_t^2/h_t penalizes forecasts that are too small (under-prediction),
- T is the number of evaluation days.

In Plain English

QLIKE has two parts pulling in opposite directions. The $\ln h_t$ term punishes you for forecasting too high (wasting capital on unnecessary hedges), while the σ_t^2/h_t term punishes you for forecasting too low (holding unrecognized risk). Critically, the under-prediction penalty (σ_t^2/h_t) explodes as $h_t \to 0$, so QLIKE is far harsher on dangerous under-estimates than on conservative over-estimates. This asymmetry matches real-world priorities: underestimating volatility gets you fired; overestimating it merely costs some opportunity.

Why QLIKE Is Less Sensitive to Outliers

When true variance spikes to $\sigma_t^2 = 10$ and your forecast is $h_t = 1$, the QLIKE contribution is $\ln(1) + 10/1 = 10$. Under MSE, the same day contributes $(10-1)^2 = 81$. QLIKE penalizes the error linearly (through the ratio σ_t^2/h_t) rather than quadratically. Extreme days still matter, but they do not dominate.

Patton (2011): QLIKE and MSE Are the Only Robust Losses

Patton (2011) proves that QLIKE and MSE are the *only* two members of the standard loss function family that produce correct model rankings even when the volatility proxy is noisy. Other common losses (MAE, HMSE, heteroskedasticity-adjusted MSE) can reverse the true ranking when evaluated against RV_t instead of σ_t^2. Of the two robust losses, QLIKE is less sensitive to extreme RV days and is therefore preferred as the primary evaluation metric.

17.3.3 Worked Example: MSE vs. QLIKE Rankings

Worked Example:

MSE vs. QLIKE Can Disagree Consider five days of forecasts from two models, evaluated against realized variance:

Day	RV_t	Model A (h_t^A)	Model B (h_t^B)
1	1.2	1.0	1.3
2	0.9	1.0	0.8
3	1.1	1.0	1.0
4	1.0	1.0	1.1
5	8.0	1.0	3.0

Model A is a constant forecast ($h_t = 1.0$). Model B adapts but is still far from the spike on day 5.

MSE computation:

$$\begin{aligned}\text{MSE}_A &= \frac{1}{5}\left[(1.2-1)^2 + (0.9-1)^2 + (1.1-1)^2 + (1-1)^2 + (8-1)^2\right] \\ &= \frac{1}{5}(0.04 + 0.01 + 0.01 + 0 + 49) = 9.812 \\ \text{MSE}_B &= \frac{1}{5}\left[(1.2-1.3)^2 + (0.9-0.8)^2 + (1.1-1)^2 + (1-1.1)^2 + (8-3)^2\right] \\ &= \frac{1}{5}(0.01 + 0.01 + 0.01 + 0.01 + 25) = 5.008\end{aligned}$$

MSE ranks Model B first (5.008 ¡ 9.812), driven almost entirely by day 5.

QLIKE computation:

$$\begin{aligned}
\text{QLIKE}_A &= \frac{1}{5}\left[\left(\ln 1 + \frac{1.2}{1}\right) + \left(\ln 1 + \frac{0.9}{1}\right) + \left(\ln 1 + \frac{1.1}{1}\right) + \left(\ln 1 + \frac{1.0}{1}\right) + \left(\ln 1 + \frac{8.0}{1}\right)\right] \\
&= \frac{1}{5}(1.2 + 0.9 + 1.1 + 1.0 + 8.0) = 2.440 \\
\text{QLIKE}_B &= \frac{1}{5}\big[(\ln 1.3 + 1.2/1.3) + (\ln 0.8 + 0.9/0.8) \\
&\quad + (\ln 1.0 + 1.1/1.0) + (\ln 1.1 + 1.0/1.1) + (\ln 3.0 + 8.0/3.0)\big] \\
&= \frac{1}{5}(1.185 + 0.902 + 1.100 + 1.004 + 3.766) = 1.591
\end{aligned}$$

QLIKE also ranks Model B first here (1.591 ¡ 2.440), but the gap is much smaller: Model B wins by 35% under QLIKE versus 49% under MSE. In samples where the crisis day is less extreme, MSE and QLIKE rankings can reverse entirely.

Always Report QLIKE as Primary

Use QLIKE as your primary loss function for volatility forecast evaluation. Report MSE as a secondary check. If the two metrics disagree on model rankings, the QLIKE ranking is more reliable for practical forecasting because it is less distorted by a few extreme days.

Why This Matters

QLIKE is THE primary evaluation metric for your project. When you report that your ML model beats HAR, the headline number is the percentage reduction in QLIKE. The asymmetry is critical: QLIKE penalizes you more for underestimating vol than overestimating it (through the σ_t^2/h_t ratio), which aligns with risk management priorities where underestimating vol means holding too much risk. Target a 30–80 bps QLIKE improvement over HAR to claim a meaningful result.

17.3.4 Retransformation Bias

Many volatility models forecast in log space because $\log \text{RV}_t$ is more Gaussian, more homoskedastic, and better behaved for regression than raw RV_t. The HAR-log model, for example, regresses $\log \text{RV}_{t+1}$ on lagged log realized variances. But when you need a level-space forecast (e.g., for portfolio variance targeting or VaR computation), you must exponentiate the log forecast back to levels. This innocent-looking step introduces a systematic downward bias known as **retransformation bias**.

The Problem: Jensen's Inequality

The root cause is **Jensen's inequality**: for any convex function g and non-degenerate random variable X,

$$\mathbb{E}\big[g(X)\big] > g\big(\mathbb{E}[X]\big). \tag{17.3}$$

The exponential function is convex, so $\mathbb{E}[\exp(X)] > \exp(\mathbb{E}[X])$.

Suppose your log-space model produces a point forecast $\widehat{\log \text{RV}}_{t+1}$. The naive level-space forecast is:

$$\widehat{\text{RV}}_{t+1}^{\text{naive}} = \exp\big(\widehat{\log \text{RV}}_{t+1}\big).$$

But because the log-space forecast has estimation error, the true conditional expectation of RV_{t+1} is *larger* than this. Exponentiating the conditional mean of the log gives you something systematically below the conditional mean of the level. Every single forecast is biased low.

In Plain English

Think of it this way. Your log-space model says "the average of log RV tomorrow is 0.5." But the average of RV tomorrow is not $\exp(0.5) = 1.65$. It is higher, because the distribution of log RV has spread around 0.5, and the exponential function amplifies high values more than it shrinks low values. The more uncertain your log-space forecast (wider error distribution), the larger the gap between $\exp(\mathbb{E}[\log \text{RV}])$ and $\mathbb{E}[\text{RV}]$.

The Correction Formula

If log-space forecast errors are approximately Gaussian with variance $\hat{\sigma}_\varepsilon^2$, the bias-corrected level-space forecast is:

$$\widehat{\text{RV}}_{t+1} = \exp\left(\widehat{\log \text{RV}}_{t+1} + \frac{\hat{\sigma}_\varepsilon^2}{2}\right) \tag{17.4}$$

where:

- $\widehat{\log \text{RV}}_{t+1}$ is the log-space point forecast,
- $\hat{\sigma}_\varepsilon^2$ is the estimated variance of the log-space forecast errors $\varepsilon_t = \log \text{RV}_t - \widehat{\log \text{RV}}_t$,
- the $\hat{\sigma}_\varepsilon^2/2$ term is the correction that offsets Jensen's inequality.

This formula comes from the moment generating function of a Gaussian: if $\varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2)$, then $\mathbb{E}[\exp(\varepsilon)] = \exp(\sigma_\varepsilon^2/2)$. The corrected forecast multiplies the naive exponential by this factor.

In Plain English

The correction adds half the forecast error variance back before exponentiating. It says: "My best guess for log RV is $\hat{y}$, but there is uncertainty of $\hat{\sigma}_\varepsilon^2$ around that guess. Because exponentiation amplifies upside errors more than downside errors, I need to nudge my forecast upward by $\hat{\sigma}_\varepsilon^2/2$ to get the right average in level space." When forecast errors are small ($\hat{\sigma}_\varepsilon^2 \approx 0$), the correction is negligible. When they are large, as in long-horizon forecasts or during volatile regimes, it can be substantial.

How Large Is the Bias?

Worked Example:

Retransformation Bias Magnitude Suppose your HAR-log model has log-space forecast error variance $\hat{\sigma}_\varepsilon^2 = 0.20$ (a realistic value for 5-day-ahead forecasts of log daily RV on equity indices).

Correction factor:

$$\exp\left(\frac{0.20}{2}\right) = \exp(0.10) \approx 1.105$$

This means the naive forecast underestimates the true conditional mean by about 10.5%. On a day where $\widehat{\log \text{RV}}_{t+1} = -4.0$ (corresponding to annualized vol of about 14%), the naive forecast is $\exp(-4.0) = 0.0183$, while the corrected forecast is $0.0183 \times 1.105 = 0.0202$.

For 1-day-ahead forecasts with $\hat{\sigma}_\varepsilon^2 \approx 0.08$, the correction factor is $\exp(0.04) \approx 1.04$, a 4% adjustment. For 22-day-ahead forecasts with $\hat{\sigma}_\varepsilon^2 \approx 0.35$, it reaches $\exp(0.175) \approx 1.19$, nearly a 20% adjustment.

Without Correction, Every Forecast Is Biased Low

If you forecast in log space and naively exponentiate, your Mincer–Zarnowitz regression (Section 17.4) will show $a > 0$ (systematic under-prediction) and the bias grows with forecast uncertainty. During high-volatility regimes, when $\hat{\sigma}_\varepsilon^2$ is largest, the bias is at its worst, precisely when accurate forecasts matter most for risk management.

Estimating the Correction Variance

The correction requires $\hat{\sigma}_\varepsilon^2$, the variance of log-space forecast errors. In practice, estimate this from the training sample or from a rolling window of recent out-of-sample errors. Using a rolling window (e.g., the trailing 60 trading days) allows the correction to adapt to regime changes: during calm markets $\hat{\sigma}_\varepsilon^2$ is small and the correction is minor; during crises it grows, appropriately increasing the level-space forecast.

Why This Matters

Your project forecasts $\log \mathrm{RV}_{t+1}$ (because log realized variance is better behaved for HAR and LightGBM regressions), but QLIKE evaluation and downstream applications (volatility targeting, VaR) require level-space forecasts. Apply the retransformation correction from Equation (17.4) whenever you convert back to levels. Without it, you introduce a systematic negative bias that inflates QLIKE loss and makes your MZ regression show $a > 0$. The correction is trivially cheap to compute, so there is no reason to skip it (Patton, 2011).

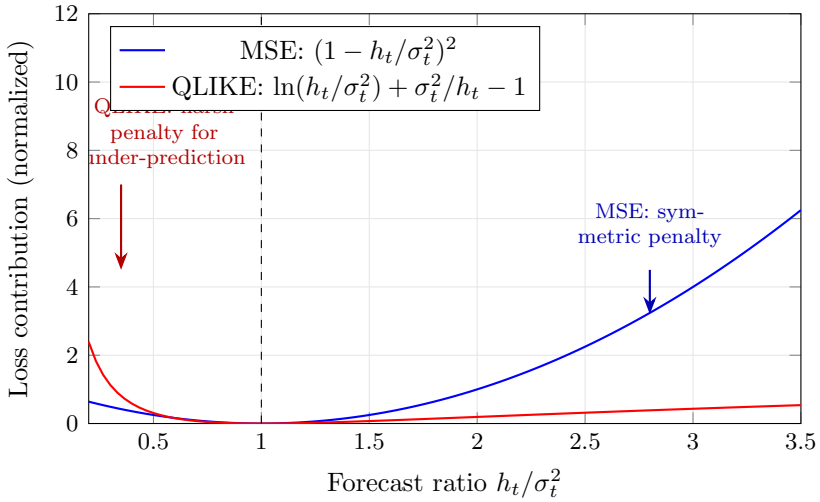


Figure 17.1: QLIKE vs. MSE penalty as a function of the forecast ratio h_t/σ_t^2. Both losses are minimized at the perfect forecast ($h_t = \sigma_t^2$, ratio = 1). MSE penalizes over- and under-prediction symmetrically. QLIKE penalizes under-prediction (ratio < 1) much more harshly than over-prediction (ratio > 1), matching the asymmetric risk preferences in volatility forecasting: underestimating vol means holding too much risk.

17.4 Mincer–Zarnowitz Regressions

QLIKE tells you which model has lower average loss, but it does not tell you *why* a forecast is bad. The Mincer–Zarnowitz regression is a simple diagnostic that decomposes forecast errors into bias and inefficiency.

17.4.1 The Regression

Regress realized variance on the forecast:

$$\sigma_t^2 = a + b \cdot h_t + \varepsilon_t \tag{17.5}$$

where:

- σ_t^2 is realized variance (left-hand side),
- h_t is the forecast (right-hand side),
- a is the intercept (bias term),
- b is the slope (efficiency term),
- ε_t is the residual.

In Plain English

The Mincer–Zarnowitz regression asks: "If I plot realized variance against my forecast, do the points lie along the 45-degree line?" A perfect forecast gives intercept $a = 0$ (no constant bias) and slope $b = 1$ (every unit increase in the forecast corresponds to exactly one unit increase in reality). If $b < 1$, your forecast is too timid; if $b > 1$, it overreacts.

Why This Matters

After fitting your ML model, run the MZ regression before anything else. HAR models typically show b slightly below 1 (they smooth too much in high-vol regimes), and your ML extension should fix this. If your HARQ-ML model still shows $b = 0.85$, you know the improvement needs to come from making the forecast more reactive to recent variance spikes, not from reducing average bias.

Unbiased and Efficient Forecast

A forecast is **unbiased** if $a = 0$ (no systematic over- or under-prediction) and **efficient** if $b = 1$ (the forecast captures the full scale of variance movements). Test the joint hypothesis $H_0 : a = 0, b = 1$ with a standard F-test (Mincer and Zarnowitz, 1969).

17.4.2 Interpreting Deviations

- $a > 0$, $b \approx 1$: the forecast systematically under-predicts by a constant.
- $a \approx 0$, $b < 1$: the forecast under-reacts to variance movements (too smooth).
- $a \approx 0$, $b > 1$: the forecast over-reacts (too volatile).
- R^2: the fraction of realized variance variation explained by the forecast. Higher is better.

Mincer–Zarnowitz as a Diagnostic

Think of the MZ regression as a "bias and calibration check." QLIKE tells you the overall score; MZ tells you what to fix. If $b = 0.7$, your forecast is too smooth: it needs to react more aggressively to recent information. If $a = 0.003$, your forecast systematically under-predicts by about 0.3 variance points.

Use HAC Standard Errors

Volatility forecast errors are serially correlated (today's error predicts tomorrow's error) because volatility clusters (Chapter 5). Use Newey–West (HAC) standard errors in the MZ regression. OLS standard errors will be too small, leading you to reject H_0 too often.

17.5 The Diebold–Mariano Test

You now have a loss function (QLIKE) and a diagnostic (MZ regression). The next question is: given two models, is the difference in loss *statistically significant*, or could it be sampling noise?

17.5.1 Setup

Suppose you have two volatility forecasts, h_t^A and h_t^B, and a loss function L (use QLIKE). Define the **loss differential**:

$$d_t = L(\sigma_t^2, h_t^A) - L(\sigma_t^2, h_t^B) \tag{17.6}$$

where:

- d_t is the difference in loss on day t,
- $L(\sigma_t^2, h_t^A)$ is the loss of model A on day t,
- $L(\sigma_t^2, h_t^B)$ is the loss of model B on day t.

If $d_t > 0$ on average, model B has lower loss (model B wins). The question is whether $\bar{d}$ is significantly different from zero.

In Plain English

The loss differential d_t is simply the daily scorecard: on each day, which model had a worse QLIKE score? Some days model A wins, some days model B wins. You are asking whether model B wins often enough, by enough, that it cannot be explained by random chance.

17.5.2 The Test Statistic

$$\text{DM} = \frac{\bar{d}}{\sqrt{\widehat{\text{Var}}(\bar{d})}} \tag{17.7}$$

where:

- $\bar{d} = \frac{1}{T}\sum_{t=1}^{T} d_t$ is the mean loss differential,
- $\widehat{\text{Var}}(\bar{d})$ is estimated using HAC (Newey–West) standard errors to account for serial correlation in d_t,
- Under $H_0 : \mathbb{E}[d_t] = 0$, the DM statistic is asymptotically standard normal (Diebold and Mariano, 1995).

In Plain English

The DM statistic is a t-test on the average loss difference. The numerator is "how much better is model B on average?" and the denominator is "how uncertain are we about that average, given that consecutive days are correlated?" If the ratio exceeds roughly 2, you have a statistically significant winner at the 5% level.

Why This Matters

The DM test is the formal statistical test you need to claim "my ML model significantly beats HAR." Without it, a reviewer can dismiss any QLIKE improvement as sampling noise. When you report results, the DM p-value goes right next to the QLIKE numbers. If $p > 0.05$, your improvement is not credible regardless of how good the point estimate looks.

HAC Standard Errors

When observations are serially correlated, the usual variance estimator $\widehat{\mathrm{Var}}(\bar{d}) = s_d^2/T$ is biased downward. **Heteroskedasticity and Autocorrelation Consistent (HAC)** estimators, such as Newey–West, correct for this by including autocovariances up to some lag ℓ. A common rule of thumb is $\ell = \lfloor T^{1/3} \rfloor$. For $T = 1{,}000$ days, this gives $\ell = 10$.

17.5.3 Worked Example

Worked Example:

Diebold–Mariano Test You evaluate HAR and LightGBM forecasts over $T = 500$ trading days using QLIKE loss.

Given:

- Mean QLIKE loss differential: $\bar{d} = 0.023$ (HAR has higher loss, so LightGBM wins on average).
- HAC standard error of $\bar{d}$: $\mathrm{SE}_{\mathrm{HAC}} = 0.011$.

Compute:

$$\mathrm{DM} = \frac{0.023}{0.011} = 2.09$$

Interpret: Under H_0, $\mathrm{DM} \sim \mathcal{N}(0,1)$. The two-sided p-value is $2 \times \Phi(-2.09) \approx 0.037$. At the 5% level, you reject H_0: the LightGBM improvement over HAR is statistically significant.

At the 1% level, you would not reject. Report the p-value and let the reader decide.

Small-Sample Correction

Diebold and Mariano (1995) derived the test for large samples. With fewer than 100 observations, use the modified DM statistic from Harvey et al. (1997), which uses a t-distribution with $T - 1$ degrees of freedom and applies a finite-sample correction factor.

17.6 The Model Confidence Set

The Diebold–Mariano test compares models in pairs. With M models, you would need $\binom{M}{2}$ pairwise tests, and the more tests you run, the more likely you are to find a "significant" difference by chance. The Model Confidence Set solves this by comparing all models simultaneously.

17.6.1 Intuition

The Model Confidence Set as a Tournament

Imagine a round-robin tournament. Instead of declaring a single winner, the MCS procedure eliminates models that are *significantly worse* than the others and returns the set of survivors. The survivors are statistically indistinguishable from each other at the chosen confidence level.

17.6.2 The Procedure

The MCS algorithm of Hansen et al. (2011) works as follows:

1. Start with the full set of M models: $\mathcal{M}_0 = \{1, 2, \ldots, M\}$.
2. Test the null hypothesis H_0: all models in the current set have equal expected loss.

3. If H_0 is rejected at significance level α, identify and remove the worst model (the one with the highest average loss).
4. Repeat steps 2–3 on the reduced set until H_0 is not rejected.
5. The surviving set $\widehat{\mathcal{M}}^*_\alpha$ is the **Model Confidence Set** at level α.

Model Confidence Set

The Model Confidence Set $\widehat{\mathcal{M}}^*_\alpha$ at significance level α contains all models whose forecasting performance is not significantly worse than the best model. Formally, it satisfies:

$$\Pr\left(\mathcal{M}^* \subseteq \widehat{\mathcal{M}}^*_\alpha\right) \geq 1 - \alpha$$

where $\mathcal{M}^*$ is the (unknown) set of truly best models.

Hansen, Lunde, and Nason (2011): The Gold Standard for Multi-Model Comparison

Hansen et al. (2011) show that the MCS controls the familywise error rate: the probability of incorrectly excluding any truly best model is at most α. The MCS produces a *set*, not a ranking. Reporting "these 4 models are in the 90% MCS" is more honest than reporting "model X has the lowest QLIKE" when differences are small.

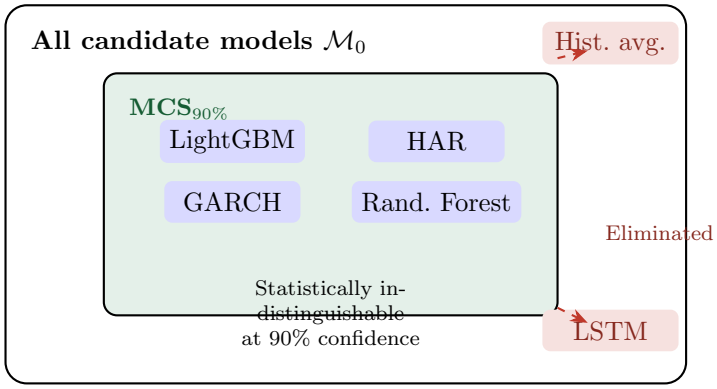


Figure 17.2: The Model Confidence Set retains all models whose performance is statistically indistinguishable from the best (green region) and eliminates models that are significantly worse (red, outside). The surviving models cannot be ranked among themselves with statistical confidence.

17.6.3 Practical Use

Report which models survive at both $\alpha = 0.10$ and $\alpha = 0.05$:

Model	QLIKE	MCS p-value	In MCS$_{90\%}$?
LightGBM + HAR features	1.423	1.000	Yes
HAR (daily, weekly, monthly)	1.441	0.482	Yes
GARCH(1,1)	1.467	0.312	Yes
Random Forest	1.439	0.551	Yes
LSTM	1.502	0.043	No
Historical average	1.589	0.001	No

The MCS p-value for each model is the smallest α at which that model would be excluded. A model with MCS p-value > 0.10 survives in the 90% MCS. In this example, four models are

statistically indistinguishable at the 90% level; the LSTM and historical average are significantly worse.

MCS as a Humility Device

The MCS often reveals an uncomfortable truth: many models that look different in QLIKE are statistically indistinguishable. A 2% QLIKE improvement is rarely significant with 3–5 years of daily data. If your fancy model is in the same MCS as HAR, be honest about it.

Why This Matters

Your model needs to be IN the Model Confidence Set, and ideally, simpler baselines like raw HAR should be excluded. If your LightGBM model and plain HAR both survive in the 90% MCS, you cannot honestly claim superiority. The MCS is also your defense: if a reviewer asks "why not use an LSTM?", you can show it was eliminated from the MCS. Use the `MCS` package in R or the `arch` library in Python to compute MCS p-values.

17.7 Purged K-Fold Cross-Validation with Embargo

The tests above (DM, MCS) evaluate forecasts on a held-out sample. But how do you *select* the model and tune hyperparameters in the first place? Standard K-fold cross-validation fails catastrophically on time series data. This section explains why and introduces the fix.

17.7.1 Why Standard K-Fold Fails

K-Fold Cross-Validation

In standard K-fold CV, you split data into K equally sized folds, train on $K-1$ folds, test on the remaining fold, and rotate. This works when observations are independent (e.g., images, text documents). It does *not* work when observations are serially dependent.

Consider 5-fold CV on 1,250 trading days (5 years). Fold 1 = days 1–250, fold 2 = days 251–500, and so on. When testing on fold 2 (days 251–500), you train on folds 1, 3, 4, 5 (days 1–250 *and* 501–1250).

The problem: volatility on day 501 is highly correlated with volatility on day 500 (the last day of the test set). Training on day 501 while testing on day 500 is using the future to predict the past. Worse, if your labels use multi-day returns (e.g., 5-day forward realized variance), then the label for day 498 overlaps with the label for day 502; the training and test sets share information through the label construction.

17.7.2 The Fix: Purging and Embargo

de Prado (2018) introduces two modifications to standard K-fold CV:

Purging

Purging removes from the training set any observations whose label windows overlap with the test period. If labels are constructed from τ-day forward returns, remove training observations within τ days before the start of the test fold.

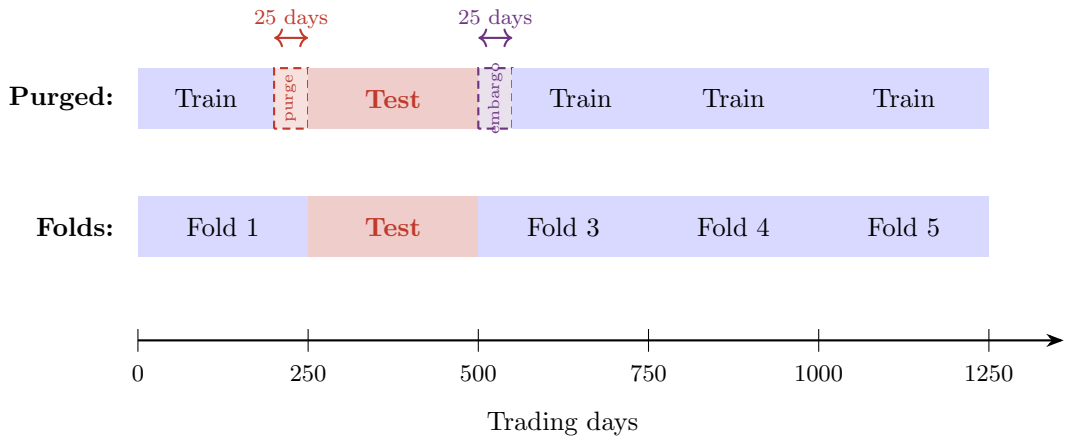


Figure 17.3: Purged K-fold CV with embargo ($K = 5$, $T = 1{,}250$, embargo = 2%). **Top row:** standard fold assignment with fold 2 as the test set. **Bottom row:** after purging and embargo. The 25 days before the test set (purge zone) are removed from training to prevent label overlap. The 25 days after the test set (embargo zone) are removed to prevent information leakage from serial correlation. Training uses only the remaining blue regions.

Embargo

Embargo removes an additional buffer of training observations *after* the end of the test fold. This guards against serial correlation in features: day $t + 1$ features are correlated with day t features, so training on day $t + 1$ while testing on day t leaks information. A typical embargo is 1–2% of total sample size.

17.7.3 Worked Example

Worked Example:

Purged 5-Fold CV **Setup:** $T = 1{,}250$ daily observations (5 years), $K = 5$ folds, labels use 5-day forward realized variance, embargo = 2% of $T = 25$ days.
Fold 2 as test set (days 251–500):

1. **Standard training set:** days 1–250 and 501–1250 (1,000 days).
2. **After purging:** remove days 246–250 from training (their 5-day labels overlap with the test period). Training loses 5 days.
3. **After embargo:** remove days 501–525 from training (serial correlation buffer). Training loses 25 more days.
4. **Final training set:** days 1–245 and 526–1250 (970 days).

You lose 30 training observations per fold (3% of the total). This is a small price for preventing look-ahead bias.

Why This Matters

Your vol forecasting labels use multi-day forward realized variance (typically 1-day or 5-day), which means label windows overlap across consecutive observations. Standard K-fold would train on day 502 (whose label includes returns from days 502–506) while testing on day 500 (whose label includes days 500–504). The overlapping days 502–504 leak test information into training. Purging removes this overlap; embargo handles the residual serial correlation in features like lagged RV. Use `sklearn.model_selection.TimeSeriesSplit` as a starting point, then add purging and embargo manually or use the `purged_cv` imple-

mentation from de Prado (2018).

Random K-Fold on Time Series Is Catastrophic

Random K-fold on time series data (shuffling observations before splitting) is the single most common evaluation error in ML-for-finance papers. A model trained on January and March, tested on February, has literally seen the future. Reported accuracy will be dramatically inflated; out-of-sample performance will collapse. Always use purged CV, expanding-window, or walk-forward evaluation for time series.

17.8 The Deflated Sharpe Ratio

Everything above evaluates *forecasts* of volatility. But volatility forecasts are often embedded in trading strategies (volatility targeting, variance risk premium trading; see Chapters 9 and 18). The standard performance metric for strategies is the Sharpe ratio. This section explains why raw Sharpe ratios are misleading when you have tried multiple strategies, and how to correct them.

17.8.1 The Multiple Testing Problem

Sharpe Ratios and Coin Flips

Suppose you flip 30 fair coins 250 times each and report only the coin with the most heads. That coin will show a "success rate" well above 50%, but it has no skill; you simply selected the luckiest coin. The same logic applies to Sharpe ratios: if you try 30 feature sets and report the best one, the expected maximum Sharpe ratio under the null (no skill) is not zero.

Bailey and de Prado (2014) derive the expected maximum Sharpe ratio under the null when N independent strategies are tested:

$$\mathbb{E}\Big[\max_{i=1,\ldots,N} \mathrm{SR}_i\Big] \approx \sqrt{2\ln N} \tag{17.8}$$

where:

- N is the number of independent strategies (or feature sets, or hyperparameter combinations) tested,
- SR_i is the Sharpe ratio of strategy i under the null (all strategies have true $\mathrm{SR} = 0$),
- The approximation comes from extreme value theory for Gaussian maxima.

For $N = 30$, this gives $\mathbb{E}[\max \mathrm{SR}] \approx \sqrt{2\ln 30} \approx 2.61$. A reported Sharpe of 1.5 after 30 trials is *below* what you would expect from pure luck.

In Plain English

This formula says: the more strategies you try, the higher the Sharpe ratio you should expect from the luckiest one, even if none of them have any real skill. It grows slowly (as $\sqrt{\ln N}$), but 30 trials already pushes the luck threshold to a Sharpe of 2.6. Your observed Sharpe must exceed this threshold to be credible.

Why This Matters

If you test 20 hyperparameter configurations for your vol-timing strategy, the expected maximum Sharpe under pure luck is $\sqrt{2\ln 20} \approx 2.45$. Any backtest Sharpe below this number is entirely consistent with having no skill. This is why you must log every experiment from the start: N only grows, and forgetting trials inflates your apparent performance.

17.8.2 The DSR Formula

The Deflated Sharpe Ratio adjusts the observed Sharpe ratio for the number of trials:

$$\text{DSR} = \Phi\left(\frac{(\widehat{\text{SR}} - \text{SR}_0)\sqrt{T-1}}{\sqrt{1 - \hat{\gamma}_3\widehat{\text{SR}} + \frac{\hat{\gamma}_4 - 1}{4}\widehat{\text{SR}}^2}}\right) \tag{17.9}$$

where:

- $\text{DSR} \in [0, 1]$ is the probability that the observed Sharpe exceeds the multiple-testing threshold (higher is better),
- $\widehat{\text{SR}}$ is the observed (annualized) Sharpe ratio of the best strategy,
- $\text{SR}_0 = \sqrt{2\ln N}$ is the expected maximum Sharpe under the null, with N = number of trials,
- T is the number of return observations,
- $\hat{\gamma}_3$ is the sample skewness of returns,
- $\hat{\gamma}_4$ is the sample kurtosis of returns,
- $\Phi(\cdot)$ is the standard normal CDF.

In Plain English

The DSR converts your observed Sharpe ratio into a probability: "What is the chance that this Sharpe is real, given how many strategies I tried?" It subtracts the luck threshold (SR_0) from your observed Sharpe, scales by sample size, and adjusts for skewness and fat tails in your returns. A DSR near 1 means your Sharpe is almost certainly genuine; a DSR near 0 means it is probably luck.

Why This Matters

If your vol-forecasting project includes a variance risk premium trading strategy (Chapter 9), you will need to report DSR alongside the raw Sharpe. With the typical 10–30 experiments you will run during hyperparameter tuning, even a Sharpe of 1.5 can be entirely consistent with luck. DSR > 0.95 is the bar for a credible backtest result.

Bailey and Lopez de Prado (2014): The Deflated Sharpe Ratio

Bailey and de Prado (2014) show that ignoring the number of trials leads to systematic over-reporting of Sharpe ratios in backtested strategies. The DSR corrects for this by benchmarking the observed Sharpe against the expected maximum under the null. A DSR above 0.95 provides evidence that the strategy's Sharpe ratio is unlikely to have arisen from multiple testing alone.

17.8.3 Worked Example

Worked Example:

Deflated Sharpe Ratio You test $N = 20$ volatility-timing strategies over $T = 1{,}260$ trading days (5 years of daily returns). The best strategy achieves $\widehat{\text{SR}} = 1.8$ (annualized). Returns have skewness $\hat{\gamma}_3 = -0.3$ and kurtosis $\hat{\gamma}_4 = 4.2$.

Step 1: Compute the null benchmark.

$$\text{SR}_0 = \sqrt{2 \ln 20} \approx \sqrt{5.99} \approx 2.45$$

Step 2: Compute the DSR numerator.

$$(\widehat{\text{SR}} - \text{SR}_0)\sqrt{T-1} = (1.8 - 2.45)\sqrt{1259} = (-0.65)(35.48) = -23.06$$

Step 3: Compute the DSR denominator.

$$\sqrt{1 - (-0.3)(1.8) + \frac{4.2-1}{4}(1.8)^2} = \sqrt{1 + 0.54 + 2.592} = \sqrt{4.132} \approx 2.033$$

Step 4: Compute DSR.

$$\text{DSR} = \Phi\left(\frac{-23.06}{2.033}\right) = \Phi(-11.35) \approx 0.000$$

The DSR is essentially zero. Despite an impressive-looking Sharpe of 1.8, the strategy does not survive correction for 20 trials. The expected maximum Sharpe under pure luck ($\text{SR}_0 = 2.45$) exceeds the observed Sharpe. You cannot reject the null that this is the luckiest of 20 random strategies.

Every Experiment Counts as a Trial

The DSR requires you to know N, the total number of strategies tested. This includes every feature set, every hyperparameter grid point, every "quick look" that influenced your final choice. If you do not log experiments, you cannot compute an honest DSR. This is why the experiment tracker (logging every trial) is infrastructure you build *before* you start modeling.

The Haircut Sharpe Ratio

Harvey and Liu (2015) propose a complementary correction. Rather than computing a probability, they "haircut" the Sharpe ratio by the amount attributable to multiple testing. The haircut depends on the number of trials and the correlation among strategies. Both DSR and Haircut Sharpe tell the same story: raw Sharpe ratios from backtests are systematically inflated. Report both if possible.

17.9 What Doesn't Work

This chapter has given you the right tools. This section catalogs the wrong ones, so you can recognize them in other people's work and avoid them in your own.

Evaluation Pitfalls

1. **Random K-fold on time series.** Shuffling observations before splitting destroys temporal structure. Reported accuracy is inflated; real performance collapses. Always use purged CV or walk-forward (Section 17.7).
2. **Naive out-of-sample R^2 without statistical tests.** "Our model achieves OOS $R^2 = 3.2\%$ versus the benchmark's 2.8%." Without a DM test (Section 17.5) or MCS (Section 17.6), you do not know whether 0.4 percentage points is signal or noise.
3. **Training on one regime, testing on another.** Training on 2015–2019 (low volatility) and testing on 2020 (COVID) is not a fair evaluation; it is a regime-change stress test. Useful, but do not confuse it with a general OOS evaluation.
4. **Look-ahead in feature construction.** Using day-t VIX close to predict day-t realized variance is look-ahead bias: VIX is not known until 4:15 PM, while RV accumulates throughout the day. All features must be known *before* the forecast is made.
5. **Reporting tiny improvements without economic significance.** Beating HAR by 0.5% in QLIKE is unlikely to translate to meaningful PnL after transaction costs. Always pair statistical significance (DM test) with economic significance (cost-aware backtest; Chapter 18).
6. **Ignoring forecast variance.** A model that is right on average but has high forecast variance is dangerous for volatility targeting. Two models with identical QLIKE can differ dramatically in how "jumpy" their forecasts are. Report forecast autocorrelation and turnover alongside loss metrics.

17.9.1 Lookahead Bias: A Taxonomy of Four Sources

Item 4 in the list above (look-ahead in feature construction) deserves a full subsection because lookahead bias is the single most destructive error in financial ML. A contaminated model shows excellent in-sample performance that vanishes out of sample, wasting weeks of development time before the bug is identified.

Lookahead bias occurs whenever a feature used at prediction time contains information that would not have been available at the moment the forecast was made. In volatility forecasting, there are four distinct sources, each with its own failure mode. Understanding them concretely, with specific examples of how each one can leak future information into your features, is essential for building a trustworthy pipeline.

Source 1: Realized Measures

Realized variance (Chapter 2) is computed from intraday returns over a trading day. The danger is that the boundary between "today" and "tomorrow" is not always clean.

Realized Measure Leakage

Suppose you forecast RV_{t+1} (tomorrow's realized variance) using features that include RV_t (today's realized variance). If you compute RV_t using returns from 9:30 AM to 4:00 PM, but the last intraday return spans 3:55–4:00 PM, that return reflects information that overlaps with the overnight period leading into day $t+1$. In tick-level data, the problem is worse: the last trade might occur at 4:00:02 PM, technically after the close. Even a few seconds of overlap contaminates the feature with forward-looking information.

Prevention: Strict Temporal Cutoff

All features for forecasting RV_{t+1} must use data from day t or earlier. Define RV_t using a fixed, consistent intraday window (e.g., 9:30 AM to 3:59 PM) and apply this cutoff uniformly across all realized measures: RV, realized quarticity (RQ), bipower variation (BV), and signed components (RV^+, RV^-). Timestamp every data point and assert programmatically that no feature for RV_{t+1} uses data with timestamp $> t$ close. When in doubt, lag by one full day.

Source 2: Microstructure Features

Microstructure features (Chapter 3) are derived from limit order book (LOB) data, trade-and-quote streams, and intraday volume profiles. They are particularly vulnerable to lookahead because they are computed over intraday windows whose boundaries must be carefully aligned with the forecast target.

LOB Feature Leakage

Suppose you compute the **Volume-Synchronized Probability of Informed Trading (VPIN)** over the full trading day from 9:30 AM to 4:00 PM and use it as a feature for forecasting RV_{t+1}. The last hour of VPIN reflects order flow patterns driven by information that will affect the overnight return and the opening of day $t+1$. For example, a large informed seller at 3:45 PM depresses the close and widens the spread; this information is not "known" to a forecaster who must act at 3:00 PM. Full-day microstructure features effectively let you "see" information that accumulates between your forecast time and the close.

Prevention: Truncate LOB Features Before Close

Truncate all intraday microstructure features at a fixed cutoff before the market close. A common choice is 3:00 PM (one hour before close) or even 2:00 PM for conservative pipelines. Apply this cutoff uniformly to VPIN, Kyle's lambda, Amihud illiquidity, bid-ask spread averages, and depth imbalance measures. Document the cutoff time as a pipeline parameter, not a magic number buried in preprocessing code.

Source 3: Options Surface

Implied volatility from the options surface (Chapters 5 and 18) is a forward-looking feature by design: it encodes the market's expectation of future volatility. This makes it powerful but also dangerous, because the surface updates throughout the day in response to new information.

Implied Volatility Leakage

Suppose you use the 3:30 PM VIX level as a feature for forecasting next-day RV. If a company announces earnings at 4:05 PM, the options market will begin pricing in the expected earnings move before the announcement: implied volatility rises during the last hour of trading. The 3:30 PM VIX already reflects this anticipation. A forecaster using this feature has access to information about the expected earnings-day volatility spike that is not "available" in the sense of a genuine real-time forecast made at, say, the previous close. More subtly, the SPX implied volatility surface at 3:30 PM reflects the full day's information flow, including any macro data releases, Fed communications, or geopolitical events that occurred during the day.

Prevention: Use Previous-Day Close or Morning Snapshot

Use the end-of-day implied volatility surface from day $t-1$ (the previous close) or a fixed morning snapshot (e.g., 10:00 AM) as features for forecasting RV_{t+1}. Never use same-day afternoon implied volatility for next-day forecasts. For VIX and VIX term structure features, the same rule applies: use the previous close, not the intraday value. If you need intraday IV features for same-day forecasting (e.g., predicting afternoon RV from morning data), use a morning-only window and document it explicitly.

Source 4: Cross-Asset Features

Cross-asset features (Chapter 18) use data from other markets (e.g., European equities, commodities, currencies, Treasuries) to predict volatility in the target asset (e.g., SPX). The complication is that these markets operate on different schedules, creating timestamp misalignment that can hide lookahead bias.

Cross-Asset Timing Leakage

Suppose you use the EURO STOXX 50 realized variance as a feature for predicting SPX RV_{t+1}. European markets close at 4:30 PM CET (10:30 AM ET). If you label the European close as "day t" data, it is available before the US close on day t at 4:00 PM ET, which is fine. But if the European data is labeled "day t" and the US target is also "day t," you may inadvertently use European close data that overlaps with the US target window. Worse, some cross-asset data sources (e.g., Asian markets) close well before the US open; using "day t" Asian close data for a US "day $t+1$" forecast is correct, but using it for a US "day t" forecast means the Asian data is stale and the time alignment is ambiguous.

Prevention: Align to a Single Information Cutoff

Define a single, global **information cutoff time** for each forecast date (e.g., previous-day US close at 4:00 PM ET). All cross-asset features must use data from before this cutoff. For European data, this means using the European close from the same calendar day (since it precedes the US close). For Asian data, use the Asian close from the same calendar day (which precedes the US open). Build an explicit timezone-aware timestamp column for every data source and assert that all feature timestamps precede the cutoff.

Summary

Table 17.1 collects all four sources with their specific pitfalls and prevention rules.

Why This Matters

Your pipeline ingests data from at least four different source types (tick data, LOB depth, options surface, cross-asset indices), each with its own timestamp conventions. Build the lookahead prevention into your data pipeline as hard constraints, not as documentation that you hope developers will follow. Specifically: (1) add a `max_timestamp` column to every feature table and assert it precedes the forecast cutoff; (2) run a nightly integration test that checks no feature for RV_{t+1} uses data with timestamp $> t$ close; (3) when adding new features, require the contributor to specify the information cutoff in the feature registry. A single lookahead bug can invalidate months of work and is almost impossible to detect from QLIKE numbers alone: the contaminated model simply looks "better than it should."

Table 17.1: Lookahead bias taxonomy for volatility forecasting pipelines. Each source represents a distinct mechanism by which future information can leak into features.

Source	Pitfall	Prevention Rule
Realized measures	Target-day intraday returns leak into features via boundary overlap	Features use data only up to day t close; enforce with programmatic timestamp assertions
Microstructure	Full-day LOB features (VPIN, spreads) include close-period information	Truncate all LOB features at a fixed cutoff (e.g., 3:00 PM) before close
Options surface	Intraday IV changes reflect target-day information (e.g., earnings anticipation)	Use previous-day close IV or a fixed morning snapshot only
Cross-asset	Mixed frequencies and timezone misalignment hide temporal overlap	Align all cross-asset features to a single information cutoff (previous-day US close)

17.10 Putting It All Together: An Evaluation Workflow

You now have all the individual tools. This section assembles them into a practical workflow you should follow for every volatility forecasting experiment.

The Workflow Is the Standard

Following this workflow does not guarantee you will find a good forecast. It guarantees that if you *do* find one, the evidence will survive scrutiny. Skip any step and a careful reader can dismiss your results.

17.11 Summary

- **MSE is proxy-robust** but over-penalizes extreme variance days, making it a poor primary metric for volatility (Section 17.2).
- **QLIKE is the preferred primary loss** for volatility forecast evaluation. It is proxy-robust *and* less sensitive to outliers than MSE (Patton, 2011) (Section 17.3).
- **QLIKE and MSE are the only two robust losses.** Other common losses (MAE, HMSE) can reverse model rankings when the volatility proxy is noisy (Section 17.3).
- **Retransformation bias** arises when exponentiating log-space forecasts back to levels. Apply the correction $\exp(\hat{y} + \hat{\sigma}_\varepsilon^2/2)$ to avoid systematic under-prediction that grows with forecast uncertainty (Patton, 2011) (Section 17.3.4).
- **Mincer–Zarnowitz regressions** diagnose bias ($a \neq 0$) and inefficiency ($b \neq 1$) in forecasts. Use HAC standard errors (Section 17.4).
- **The Diebold–Mariano test** determines whether the difference in loss between two models is statistically significant, accounting for serial correlation via HAC standard errors (Diebold and Mariano, 1995) (Section 17.5).
- **The Model Confidence Set** compares all models simultaneously, returning the set of models that are statistically indistinguishable from the best. It controls familywise error (Hansen et al., 2011) (Section 17.6).
- **Purged K-fold CV with embargo** prevents information leakage in time series cross-validation by removing overlapping labels (purge) and serial-correlation buffers (embargo) (de Prado, 2018) (Section 17.7).
- **Random K-fold on time series is catastrophic.** It inflates reported accuracy by

training on future data (Section 17.7).

- **The Deflated Sharpe Ratio** corrects backtested Sharpe ratios for the number of strategies tested. Every experiment counts as a trial (Bailey and de Prado, 2014) (Section 17.8).
- **Log every experiment.** You cannot compute an honest DSR without knowing N (Section 17.8).
- **Statistical significance is necessary but not sufficient.** Pair DM tests with economic significance: does the improvement survive transaction costs? (Section 17.9).
- **Lookahead bias has four sources** in volatility pipelines: realized measures, microstructure features, options surface, and cross-asset data. Each requires a specific prevention rule; build these as hard constraints in your pipeline, not as documentation (Section 17.9.1).
- **Follow the full workflow** (Figure 17.4): reserve holdout, log experiments, purged CV, QLIKE, MZ, DM, MCS, DSR.

17.11.1 Key Results Recap

Tool	What It Does	Key Reference
QLIKE loss	Primary loss function; proxy-robust, outlier-resistant	Patton (2011)
MSE loss	Secondary loss; proxy-robust but outlier-sensitive	Patton (2011)
Mincer–Zarnowitz	Diagnoses forecast bias and inefficiency	Mincer and Zarnowitz (1969)
Diebold–Mariano	Pairwise statistical test of loss difference	Diebold and Mariano (1995)
Model Confidence Set	Multi-model comparison; returns surviving set	Hansen et al. (2011)
Purged K-fold CV	Time-series CV without look-ahead	de Prado (2018)
Deflated Sharpe Ratio	Corrects Sharpe for multiple testing	Bailey and de Prado (2014)
Haircut Sharpe	Alternative multiple-testing correction	Harvey and Liu (2015)

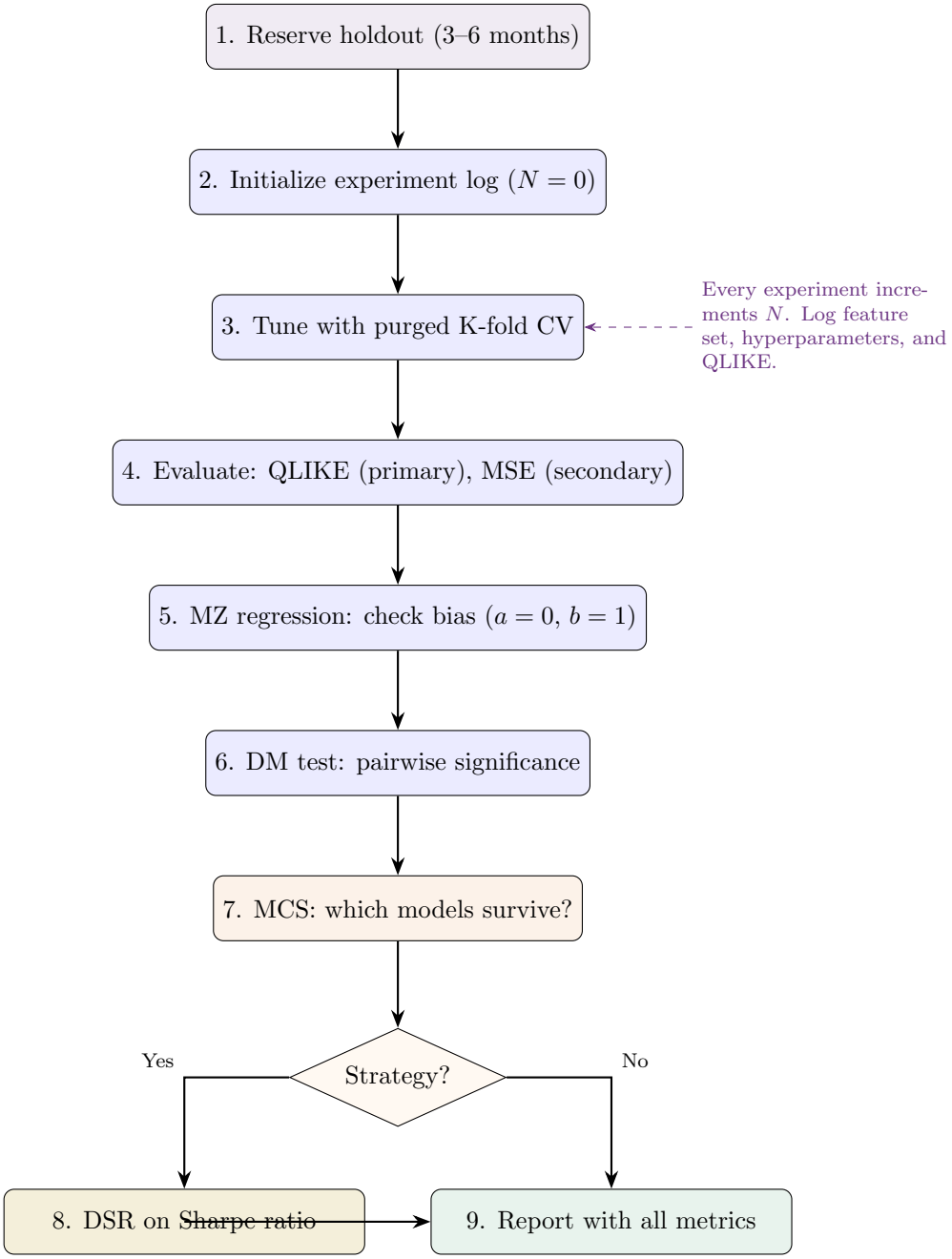


Figure 17.4: Evaluation workflow for volatility forecasting. Reserve the holdout first; log every experiment; use purged CV for tuning; evaluate with QLIKE and MZ; compare with DM and MCS; deflate the Sharpe if the forecast feeds a strategy.

Chapter 18

Practical Applications and Project Directions

Why This Chapter

The preceding chapters built a toolkit: estimators, forecasters, features, and evaluation methods. This chapter answers the question every trading desk asks: "so what?" It translates statistical forecast accuracy into economic value: the language that gets a model deployed.

A model that beats HAR by 8% on QLIKE is scientifically interesting. But a desk head wants to know: how much Sharpe does that buy me? How much P&L does it add to my book? This chapter provides the frameworks for answering those questions.

18.1 Volatility Targeting: The Simplest Economic Value Test

Background

This section requires familiarity with portfolio returns and the Sharpe ratio (any introductory finance text), as well as realized volatility forecasts from Chapters 6–14.

Volatility targeting is the simplest and most widely used application of a vol forecast in systematic investing. The idea: size your position inversely proportional to forecast vol, so that portfolio risk stays roughly constant over time.

18.1.1 The EWMA Baseline

The workhorse volatility estimate used by the vast majority of systematic funds for position sizing is not GARCH, not HAR, but plain **exponentially weighted moving average (EWMA)** smoothing:

$$\hat{\sigma}_t^2 = (1-\delta)\, r_{t-1}^2 + \delta\, \hat{\sigma}_{t-1}^2, \tag{18.1}$$

where δ is chosen so the half-life matches approximately 20 to 60 trading days. This is what most funds actually use for position sizing: not GARCH, not HAR, just exponential smoothing. Its popularity stems from simplicity, low latency, and the fact that it requires exactly one parameter.

In Plain English

Tomorrow's variance estimate is a weighted blend of today's squared return (the "news") and today's variance estimate (the "memory"). A high δ means slow adaptation: the estimate barely moves day-to-day. A low δ means the estimate jumps quickly with every new return, reacting fast but also picking up noise.

Why This Matters

EWMA is the baseline your vol-targeting backtest must beat. If your HAR or ML forecast cannot outperform this one-parameter smoother in a position-sizing test, the model adds no economic value regardless of its QLIKE score.

18.1.2 The Volatility-Targeting Formula

Given a forecast $\hat{\sigma}_t$ (annualized), the **volatility-targeted weight** is:

$$w_t = \frac{\sigma_{\text{target}}}{\hat{\sigma}_t}. \tag{18.2}$$

The portfolio return is then $r_t^{\text{VT}} = w_t \cdot r_t$. When forecast vol is high, $w_t < 1$ (reduce position). When forecast vol is low, $w_t > 1$ (lever up). The result is a return stream with approximately constant realized volatility equal to σ_{target}.

Why Vol-Targeting Adds Sharpe

Moreira and Muir (2017) showed that vol-targeting adds approximately 0.3 Sharpe ratio across equity indices, currencies, and commodities. The mechanism: by reducing exposure before drawdowns, vol-targeting truncates the left tail. A better vol forecast means you cut exposure earlier and more precisely.

Why This Matters

This formula is the direct economic-value test for your internship project (Project Direction 1: HARQ-X + ML residual). Every QLIKE improvement in your forecast translates into a tighter $\hat{\sigma}_t$, which means more precise position sizing and higher Sharpe. The vol-targeting backtest is your primary deliverable for demonstrating that statistical accuracy creates real P&L.

18.1.3 Worked Example: Vol-Targeting the S&P 500

Worked Example: Vol-Targeting the S&P 500

Setup. Target: $\sigma_{\text{target}} = 10\%$ annualized. Compare three forecasts:
(a) **EWMA** (half-life 20 days): the baseline every fund already uses.
(b) **HAR**: three-component model from Chapter 6.
(c) **ML (best model)**: your contribution.

Scenario. Days 1 to 15 are calm (annualized RV $\approx 12\%$). On day 16, a sell-off begins and RV spikes to 35% annualized. The ML model detects the regime change on day 14 (microstructure features shift); HAR reacts on day 16; EWMA reacts gradually through days 17 to 20.

Transition period (days 13 to 20):

Day	True RV (%)	$\hat{\sigma}_{\text{EWMA}}$ (%)	$\hat{\sigma}_{\text{HAR}}$ (%)	$\hat{\sigma}_{\text{ML}}$ (%)	w_{EWMA}	w_{ML}
13	11.8	12.0	12.1	12.2	0.83	0.82
14	12.1	12.0	12.0	18.5	0.83	0.54
15	12.4	12.1	12.2	22.0	0.83	0.45
16	28.5	12.3	18.0	28.0	0.81	0.36
17	33.0	15.8	25.5	32.0	0.63	0.31
18	35.2	19.4	30.0	34.5	0.52	0.29
19	34.0	22.5	32.0	33.8	0.44	0.30
20	30.5	24.8	31.5	30.0	0.40	0.33

Key observation. On day 16 (first big drawdown), EWMA still holds $w = 0.81$ (nearly full position), while the ML model has already cut to $w = 0.36$. The cumulative drawdown reduction from the ML model's early detection is substantial.

Summary statistics (full 252-day backtest):

Metric	EWMA	HAR	ML
Ann. Return (%)	8.2	8.5	8.9
Ann. Volatility (%)	10.8	10.3	10.1
Sharpe Ratio	0.76	0.82	0.88
Max Drawdown (%)	−12.4	−9.8	−7.1
Calmar Ratio	0.66	0.87	1.25

The ML model adds 0.12 Sharpe over the baseline and nearly halves the maximum drawdown, primarily because it detects regime shifts one to two days earlier.

18.1.4 Connection to Time-Series Momentum

Moskowitz et al. (2012) use a related framework for time-series momentum across 58 futures: signal = sign of 12-month return, position size = $40\%/\hat{\sigma}_t$. The vol forecast enters the denominator. Every percentage improvement in your forecast precision tightens the position-sizing, reducing both crash risk and unnecessary leverage. This is the mechanism by which better QLIKE translates to better risk-adjusted returns in a TSMOM book.

Forecast Failure in Crises

Vol targeting assumes volatility is predictable but returns are not. If your forecast systematically underpredicts during crises (all ML models trained on normal data underpredict COVID-style jumps), vol targeting will keep positions too large going into the drawdown. Mitigation: ensemble your ML forecast with a simple max(ML, $1.5 \times$EWMA) floor during high-uncertainty regimes, or cap w_t at a maximum leverage ratio (e.g., $w_t \leq 2.0$).

The Key Deliverable Table

For your internship deliverable, the key table is:

1. Run vol-targeted long-SPX using each model.
2. Report annualized return, vol, Sharpe, max drawdown, and Calmar ratio.
3. Compute Sharpe improvement per 1% QLIKE improvement.

This single table communicates economic value in the language every systematic desk speaks.

18.2 Dealer Gamma and Structured Products Feedback

Beyond statistical forecasting, volatility is shaped by the mechanical hedging behavior of options dealers. When dealers hold large gamma positions, their hedging creates predictable effects on realized vol. Understanding this mechanism gives you both a novel feature and a deeper appreciation for what your model is capturing.

18.2.1 How Dealer Hedging Amplifies or Suppresses Vol

When dealers are **long gamma** (they own options), they delta-hedge by selling into rallies and buying dips. This mean-reversion pressure suppresses realized volatility below what pure news flow would generate. Conversely, when dealers are **short gamma** (common after selling structured products like autocallables), they hedge by buying into rallies and selling into dips, amplifying moves and increasing realized vol.

GEX as a Volatility Feature

Gamma Exposure (GEX) estimates the net dealer gamma across all strikes from publicly available options open interest data. The estimate is approximate but directionally informative:

$$\text{GEX} \approx \sum_{\text{strikes } K} \text{OI}_K \times \Gamma_K \times 100 \times S \tag{18.3}$$

- Positive GEX (dealers long gamma): expect lower-than-forecast RV (mean reversion).
- Negative GEX (dealers short gamma): expect higher-than-forecast RV (momentum/amplification).

Adding sign(GEX) or GEX-quintile as a feature to HAR-X is a novel extension not yet thoroughly explored in the academic literature (Bennett, 2014).

Why This Matters

GEX provides a feature that captures a fundamentally different mechanism from any return-based or RV-based predictor. Including it in your HAR-X specification (Project Direction 1 or 2) demonstrates market microstructure awareness and gives your model an edge that purely statistical approaches miss.

18.2.2 Structured Products and the Short-Gamma Overhang

The primary source of dealer short-gamma exposure is structured products (autocallables, barrier options, worst-of notes). These products embed knock-in/knock-out barriers near current spot levels. As spot approaches a barrier, the product's gamma becomes very negative, forcing dealers to hedge aggressively. The systematic issuance of these products (estimated at hundreds of billions of notional globally) creates a permanent structural short-gamma overhang in equity index markets.

18.2.3 Pin Risk at Expiry

Near options expiry, large open interest at specific strikes creates **pinning** effects: the stock gravitates toward the strike as dealers' gamma hedging intensifies near that level. This suppresses realized vol on expiry days, a calendar effect (Section 10.11) with a mechanical explanation.

Novel HAR-X Features from Dealer Positioning

Dealer positioning features (GEX, distance to nearest barrier level, net open interest by strike) represent a genuinely novel addition to standard HAR-X feature sets. They are publicly computable from options market data and capture a mechanism (mechanical hedging feedback) that is distinct from any feature based on past returns or past RV. For your internship, including a GEX feature and documenting its incremental predictive power would demonstrate awareness of market microstructure that goes beyond the academic literature.

Bibliography

Yacine Aït-Sahalia and Jean Jacod. Testing for jumps in a discretely observed process. *The Annals of Statistics*, 37(1):184–222, 2009. doi: 10.1214/07-AOS568.

Yacine Aït-Sahalia, Per A. Mykland, and Lan Zhang. How often to sample a continuous-time process in the presence of market microstructure noise. *The Review of Financial Studies*, 18 (2):351–416, 2005. doi: 10.1093/rfs/hhi016.

Torben G. Andersen and Tim Bollerslev. Answering the skeptics: Yes, standard volatility models do provide accurate forecasts. *International Economic Review*, 39(4):885–905, 1998. doi: 10.2307/2527343.

Torben G. Andersen, Tim Bollerslev, Francis X. Diebold, and Paul Labys. The distribution of realized exchange rate volatility. *Journal of the American Statistical Association*, 96(453): 42–55, 2001. doi: 10.1198/016214501750332965.

Torben G. Andersen, Tim Bollerslev, Francis X. Diebold, and Paul Labys. Modeling and forecasting realized volatility. *Econometrica*, 71(2):579–625, 2003. doi: 10.1111/1468-0262.00418.

Torben G. Andersen, Tim Bollerslev, and Francis X. Diebold. Roughing it up: Including jump components in the measurement, modeling, and forecasting of return volatility. *The Review of Economics and Statistics*, 89(4):701–720, 2007. doi: 10.1162/rest.89.4.701.

Nikolaos Antonakakis, Ioannis Chatziantoniou, and David Gabauer. Refined measures of dynamic connectedness based on time-varying parameter vector autoregressions. *Journal of Risk and Financial Management*, 13(4):84, 2020. doi: 10.3390/jrfm13040084.

Francesco Audrino and Simon D. Knaus. Lassoing the HAR model: A model selection perspective on realized volatility dynamics. *Econometric Reviews*, 35(8–10):1485–1521, 2016. doi: 10.1080/07474938.2015.1092801.

Francesco Audrino, Fabio Sigrist, and Daniele Ballinari. The impact of sentiment and attention measures on stock market volatility. *International Journal of Forecasting*, 36(2):334–357, 2020.

Varun Babbar, Hayden McTavish, Cynthia Rudin, and Margo Seltzer. Near-optimal decision trees in a SPLIT second. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. Oral presentation; arXiv 2502.15988.

David H. Bailey and Marcos Lopez de Prado. The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *Journal of Portfolio Management*, 40(5):94–107, 2014. doi: 10.3905/jpm.2014.40.5.094.

Richard T. Baillie, Tim Bollerslev, and Hans Ole Mikkelsen. Fractionally integrated generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 74(1):3–30, 1996. doi: 10.1016/S0304-4076(95)01749-6.

Ole E. Barndorff-Nielsen and Neil Shephard. Econometric analysis of realized volatility and its

use in estimating stochastic volatility models. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 64(2):253–280, 2002. doi: 10.1111/1467-9868.00336.

Ole E. Barndorff-Nielsen and Neil Shephard. Power and bipower variation with stochastic volatility and jumps. *Journal of Financial Econometrics*, 2(1):1–37, 2004. doi: 10.1093/jjfinec/nbh001.

Ole E. Barndorff-Nielsen and Neil Shephard. Econometrics of testing for jumps in financial economics using bipower variation. *Journal of Financial Econometrics*, 4(1):1–30, 2006. doi: 10.1093/jjfinec/nbi022.

Ole E. Barndorff-Nielsen, Peter R. Hansen, Asger Lunde, and Neil Shephard. Designing realized kernels to measure the ex post variation of equity prices in the presence of noise. *Econometrica*, 76(6):1481–1536, 2008. doi: 10.3982/ECTA6495.

Ole E. Barndorff-Nielsen, Peter Reinhard Hansen, Asger Lunde, and Neil Shephard. Multivariate realised kernels: Consistent positive semi-definite estimators of the covariation of equity prices with noise and non-synchronous trading. *Journal of Econometrics*, 162(2):149–169, 2011. doi: 10.1016/j.jeconom.2010.07.009.

Christian Bayer, Peter Friz, and Jim Gatheral. Pricing under rough volatility. *Quantitative Finance*, 16(6):887–904, 2016. doi: 10.1080/14697688.2015.1099717.

Geert Bekaert and Marie Hoerova. The VIX, the variance premium and stock market volatility. *Journal of Econometrics*, 183(2):181–192, 2014. doi: 10.1016/j.jeconom.2014.05.008.

Mikkel Bennedsen, Asger Lunde, and Mikko S. Pakkanen. Decoupling the short- and long-term behavior of stochastic volatility. *Journal of Financial Econometrics*, 20(5):961–1006, 2022. doi: 10.1093/jjfinec/nbaa049.

Colin Bennett. *Trading Volatility: Trading Volatility, Correlation, Term Structure and Skew*. Self-published, 2014.

Fischer Black. Studies of stock price volatility changes. *Proceedings of the 1976 Meetings of the American Statistical Association, Business and Economical Statistics Section*, pages 177–181, 1976.

Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973. doi: 10.1086/260062.

Tim Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3):307–327, 1986. doi: 10.1016/0304-4076(86)90063-1.

Tim Bollerslev and Viktor Todorov. Tails, fears, and risk premia. *The Journal of Finance*, 66(6):2165–2211, 2011. doi: 10.1111/j.1540-6261.2011.01695.x. Extended in Bollerslev and Todorov (2015, *Journal of Financial Economics*).

Tim Bollerslev, Uta Kretschmer, Christian Pigorsch, and George Tauchen. A discrete-time model for daily S&P500 returns and realized variations: Jumps and leverage effects. *Journal of Econometrics*, 150(2):151–166, 2009a. doi: 10.1016/j.jeconom.2008.12.001.

Tim Bollerslev, George Tauchen, and Hao Zhou. Expected stock returns and variance risk premia. *The Review of Financial Studies*, 22(11):4463–4492, 2009b. doi: 10.1093/rfs/hhp008.

Tim Bollerslev, Andrew J. Patton, and Rogier Quaedvlieg. Exploiting the errors: A simple approach for improved volatility forecasting. *Journal of Econometrics*, 192(1):1–18, 2016a. doi: 10.1016/j.jeconom.2015.10.007.

Tim Bollerslev, Andrew J. Patton, and Rogier Quaedvlieg. Exploiting the errors: A simple approach for improved volatility forecasting. *Journal of Econometrics*, 192(1):1–18, 2016b.

Tim Bollerslev, Benjamin Hood, John Huss, and Lasse Heje Pedersen. Risk everywhere: Modeling and managing volatility. *The Review of Financial Studies*, 31(7):2729–2773, 2018a. doi: 10.1093/rfs/hhy041.

Tim Bollerslev, Andrew J. Patton, and Rogier Quaedvlieg. Modeling and forecasting (un)reliable realized covariances for more reliable financial decisions. *Journal of Econometrics*, 207(1):71–91, 2018b. doi: 10.1016/j.jeconom.2018.05.004.

Tim Bollerslev, Jia Li, Andrew J. Patton, and Rogier Quaedvlieg. Realized semicovariances. *Econometrica*, 88(4):1515–1551, 2020.

Tim Bollerslev, Marcelo C. Medeiros, Andrew J. Patton, and Rogier Quaedvlieg. HARd to beat: The overlooked impact of rolling windows in the era of machine learning, 2024.

Rafael Branco, Alexandre Rubesam, and Mauricio Zevallos. Forecasting realized volatility: Does anything beat linear models? *Journal of Empirical Finance*, 78:101527, 2024. doi: 10.1016/j.jempfin.2024.101527.

Mark Britten-Jones and Anthony Neuberger. Option prices, implied price processes, and stochastic volatility. *The Journal of Finance*, 55(2):839–866, 2000. doi: 10.1111/0022-1082.00228.

Andrea Bucci. Realized volatility forecasting with neural networks. *Journal of Financial Econometrics*, 18(3):502–531, 2020. doi: 10.1093/jjfinec/nbaa008.

Peter Carr and Liuren Wu. Variance risk premiums. *The Review of Financial Studies*, 22(3): 1311–1341, 2009. doi: 10.1093/rfs/hhn038.

Álvaro Cartea, Sebastian Jaimungal, and José Penalva. *Algorithmic and High-Frequency Trading*. Cambridge University Press, 2015. ISBN 978-1-107-09114-3.

Cboe Exchange. VIX white paper: Cboe volatility index. Technical report, Cboe Global Markets, 2019. URL https://www.cboe.com/tradable_products/vix/vix_white_paper/.

Yuntong Chen and Stephen Roberts. Graph transformers for volatility forecasting. *arXiv preprint arXiv:2209.09014*, 2022.

Roxana Chiriac and Valeri Voev. Modelling and forecasting multivariate realized volatility. *Journal of Applied Econometrics*, 26(6):922–947, 2011. doi: 10.1002/jae.1152.

Kim Christensen, Mathias Siggaard, and Bezirgen Veliyev. A machine learning approach to volatility forecasting. *Journal of Financial Econometrics*, 21(5):1680–1727, 2023. doi: 10.1093/jjfinec/nbac020.

Rama Cont. Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001. doi: 10.1080/713665670.

Rama Cont and José da Fonseca. Dynamics of implied volatility surfaces. *Quantitative Finance*, 2(1):45–60, 2002. doi: 10.1088/1469-7688/2/1/304.

Rama Cont and Purba Das. Rough volatility: Fact or Artefact? *Sankhyā B*, 86(1):191–223, 2024. doi: 10.1007/s13571-024-00322-2.

Fulvio Corsi. A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics*, 7(2):174–196, 2009. doi: 10.1093/jjfinec/nbp001.

Fulvio Corsi, Davide Pirino, and Roberto Renò. Threshold bipower variation and the impact

of jumps on volatility forecasting. *Journal of Econometrics*, 159(2):276–288, 2010. doi: 10.1016/j.jeconom.2010.07.008.

Christa Cuchíero, Jakob Heiss, Wahid Khosrawi, and Peter Spoida. GARCH-informed neural networks for volatility prediction. *arXiv preprint arXiv:2410.00288*, 2024.

Marcos Lopez de Prado. *Advances in Financial Machine Learning*. Wiley, 2018. ISBN 978-1-119-48208-6.

Mert Demirer, Francis X. Diebold, Laura Liu, and Kamil Yilmaz. Estimating global bank network connectedness. *Journal of Applied Econometrics*, 33(1):1–15, 2018. doi: 10.1002/jae.2585.

Francis X. Diebold and Roberto S. Mariano. Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3):253–263, 1995. doi: 10.1080/07350015.1995.10524599.

Francis X. Diebold and Kamil Yilmaz. Measuring financial asset return and volatility spillovers, with application to global equity markets. *The Economic Journal*, 119(534):158–171, 2009. doi: 10.1111/j.1468-0297.2008.02208.x.

Francis X. Diebold and Kamil Yilmaz. Better to give than to receive: Predictive directional measurement of volatility spillovers. *International Journal of Forecasting*, 28(1):57–66, 2012. doi: 10.1016/j.ijforecast.2011.02.006.

Francis X. Diebold and Kamil Yilmaz. On the network topology of variance decompositions: Measuring the connectedness of financial firms. *Journal of Econometrics*, 182(1):119–134, 2014. doi: 10.1016/j.jeconom.2014.04.012.

Yi Ding, Guofu Lu, and Eric C. Cheung. Deep learning for implied volatility surface compression. *Journal of Financial Economics*, 2025. Forthcoming.

Jiayun Dong and Cynthia Rudin. Exploring the cloud of variable importance for the set of all good models, 2020. arXiv 1901.03209; Nature Machine Intelligence.

Jon Donnelly, Srikar Katta, Cynthia Rudin, and Edward P. Browne. Rashomon importance distributions. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Itamar Drechsler and Amir Yaron. What's vol got to do with it. *The Review of Financial Studies*, 24(1):1–45, 2011. doi: 10.1093/rfs/hhq085.

Tianyu Du, Yuki Moriyama, Kei Tanaka, and Shin ichi Ishii. Normalizing flows for realized volatility forecasting. *arXiv preprint arXiv:2304.06985*, 2023.

Bruno Dupire. Pricing with a smile. *Risk*, 7(1):18–20, 1994.

David Easley, Marcos M. Lopez de Prado, and Maureen O'Hara. Flow toxicity and liquidity in a high-frequency world. *The Review of Financial Studies*, 25(5):1457–1493, 2012. doi: 10.1093/rfs/hhs053.

Robert F. Engle. Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4):987–1007, 1982. doi: 10.2307/1912773.

Robert F. Engle. Dynamic conditional correlation: A simple class of multivariate generalized autoregressive conditional heteroskedasticity models. *Journal of Business & Economic Statistics*, 20(3):339–350, 2002. doi: 10.1198/073500102288618487.

Robert F. Engle and Tim Bollerslev. Modelling the persistence of conditional variances. *Econometric Reviews*, 5(1):1–50, 1986. doi: 10.1080/07474938608800095.

Eugene F. Fama. The behavior of stock-market prices. *The Journal of Business*, 38(1):34–105, 1965.

Dylan Fouhy. Forecasting the variance risk premium with machine learning. SSRN Working Paper No. 6570380, 2024. URL https://ssrn.com/abstract=6570380.

Jim Gatheral, Thibault Jaisson, and Mathieu Rosenbaum. Volatility is rough. *Quantitative Finance*, 18(6):933–949, 2018. doi: 10.1080/14697688.2017.1393551.

Lawrence R. Glosten and Paul R. Milgrom. Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1):71–100, 1985. doi: 10.1016/0304-405X(85)90044-3.

Lawrence R. Glosten, Ravi Jagannathan, and David E. Runkle. On the relation between the expected value and the volatility of the nominal excess return on stocks. *The Journal of Finance*, 48(5):1779–1801, 1993. doi: 10.1111/j.1540-6261.1993.tb05128.x.

Shihao Gu, Bryan Kelly, and Dacheng Xiu. Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5):2223–2273, 2020a. doi: 10.1093/rfs/hhaa009.

Shihao Gu, Bryan Kelly, and Dacheng Xiu. Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5):2223–2273, 2020b. doi: 10.1093/rfs/hhaa009.

Peter R. Hansen and Asger Lunde. Realized variance and market microstructure noise. *Journal of Business & Economic Statistics*, 24(2):127–161, 2006. doi: 10.1198/073500106000000071.

Peter R. Hansen, Asger Lunde, and James M. Nason. The model confidence set. *Econometrica*, 79(2):453–497, 2011. doi: 10.3982/ECTA5771.

Peter Reinhard Hansen and Asger Lunde. A forecast comparison of volatility models: Does anything beat a GARCH(1,1)? *Journal of Applied Econometrics*, 20(7):873–889, 2005. doi: 10.1002/jae.800.

Peter Reinhard Hansen, Zhuo Huang, and Howard Howan Shek. Realized GARCH: A joint model for returns and realized measures of volatility. *Journal of Applied Econometrics*, 27(6): 877–906, 2012. doi: 10.1002/jae.1234.

Campbell R. Harvey and Yan Liu. Backtesting. *Journal of Portfolio Management*, 42(1):13–28, 2015. doi: 10.3905/jpm.2015.42.1.013.

David I. Harvey, Stephen J. Leybourne, and Paul Newbold. Testing the equality of prediction mean squared errors. *International Journal of Forecasting*, 13(2):281–291, 1997. doi: 10.1016/S0169-2070(96)00719-4.

Takaki Hayashi and Nakahiro Yoshida. On covariance estimation of non-synchronously observed diffusion processes. *Bernoulli*, 11(2):359–379, 2005. doi: 10.3150/bj/1116340299.

Zakk Heile, Varun Babbar, Hayden McTavish, and Cynthia Rudin. Efficient Rashomon set approximation for decision tree models. In *NeurIPS 2025 Workshop on ML x OR*, 2025.

Xin Huang and George Tauchen. The relative contribution of jumps to total price variance. *Journal of Financial Econometrics*, 3(4):456–499, 2005. doi: 10.1093/jjfinec/nbi025.

Jean Jacod, Yingying Li, Per A. Mykland, Mark Podolskij, and Mathias Vetter. Microstructure noise in the continuous case: The pre-averaging approach. *Stochastic Processes and their Applications*, 119(7):2249–2276, 2009. doi: 10.1016/j.spa.2008.11.004.

Patrick Kidger. *On Neural Differential Equations*. PhD thesis, University of Oxford, 2021.

Albert S. Kyle. Continuous auctions and insider trading. *Econometrica*, 53(6):1315–1335, 1985. doi: 10.2307/1913210.

Suzanne S. Lee and Per A. Mykland. Jumps in financial markets: A new nonparametric test and jump dynamics. *The Review of Financial Studies*, 21(6):2535–2563, 2008. doi: 10.1093/rfs/hhm056.

Lily Y. Liu, Andrew J. Patton, and Kevin Sheppard. Does anything beat 5-minute RV? A comparison of realized measures across multiple asset classes. *Journal of Econometrics*, 187 (1):293–311, 2015. doi: 10.1016/j.jeconom.2015.02.008.

Marcos Lopez de Prado. *Advances in Financial Machine Learning*. Wiley, Hoboken, NJ, 2018.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Paul Malliavin and Maria Elvira Mancino. Fourier series method for measurement of multivariate volatilities. *Finance and Stochastics*, 6(1):49–61, 2002. doi: 10.1007/s780-002-8401-3.

Paul Malliavin and Maria Elvira Mancino. A Fourier transform method for nonparametric estimation of multivariate volatility. *The Annals of Statistics*, 37(4):1983–2010, 2009. doi: 10.1214/08-AOS633.

Cecilia Mancini. Non-parametric threshold estimation for models with stochastic diffusion coefficient and jumps. *Scandinavian Journal of Statistics*, 36(2):270–296, 2009. doi: 10.1111/j.1467-9469.2008.00622.x.

Benoit Mandelbrot. The variation of certain speculative prices. *The Journal of Business*, 36(4): 394–419, 1963.

Benoit B. Mandelbrot and John W. Van Ness. Fractional Brownian motions, fractional noises and applications. *SIAM Review*, 10(4):422–437, 1968. doi: 10.1137/1010093.

Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952. doi: 10.2307/2975974.

Jacob Mincer and Victor Zarnowitz. The evaluation of economic forecasts. *NBER Studies in Business Cycles*, 19:3–46, 1969.

Alan Moreira and Tyler Muir. Volatility-managed portfolios. *The Journal of Finance*, 72(4): 1611–1644, 2017. doi: 10.1111/jofi.12513.

Fernando Moreno-Pino and Stefan Zohren. DeepVol: Volatility forecasting from high-frequency data with dilated causal convolutions. *arXiv preprint arXiv:2209.11761*, 2022.

Tobias J. Moskowitz, Yao Hua Ooi, and Lasse Heje Pedersen. Time series momentum. *Journal of Financial Economics*, 104(2):228–250, 2012. doi: 10.1016/j.jfineco.2011.11.003.

Ulrich A. Müller, Michel M. Dacorogna, Rakhal D. Davé, Richard B. Olsen, Olivier V. Pictet, and Jakob E. von Weizsäcker. Fractals and intrinsic time: A challenge to econometricians. *UAM Research Paper*, 1993. Olsen & Associates working paper; presented at the 39th International Conference of the AEA.

Daniel B. Nelson. Conditional heteroskedasticity in asset returns: A new approach. *Econometrica*, 59(2):347–370, 1991. doi: 10.2307/2938260.

Optiver. Optiver realized volatility prediction, 2021. Kaggle competition. https://www.kaggle.com/c/optiver-realized-volatility-prediction.

Andrew J. Patton. Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*, 160(1):246–256, 2011. doi: 10.1016/j.jeconom.2010.03.034.

Andrew J. Patton and Kevin Sheppard. Good volatility, bad volatility: Signed jumps and the persistence of volatility. *The Review of Economics and Statistics*, 97(3):683–697, 2015a. doi: 10.1162/REST_a_00503.

Andrew J. Patton and Kevin Sheppard. Good volatility, bad volatility: Signed jumps and the persistence of volatility. *Review of Economics and Statistics*, 97(3):683–697, 2015b.

M. Hashem Pesaran and Yongcheol Shin. Generalized impulse response analysis in linear multivariate models. *Economics Letters*, 58(1):17–29, 1998. doi: 10.1016/S0165-1765(97)00214-0.

Eghbal Rahimikia and Ser-Huang Poon. Machine learning for realized volatility forecasting. *Journal of Financial Econometrics*, 2020. doi: 10.2139/ssrn.3707796. Forthcoming.

Eghbal Rahimikia, Stefan Zohren, and Ser-Huang Poon. Realised volatility forecasting: Machine learning via financial word embedding. *Quantitative Finance*, 21(10):1675–1691, 2021a.

Eghbal Rahimikia, Stefan Zohren, and Ser-Huang Poon. Realised volatility forecasting: Machine learning via financial word embedding. *Quantitative Finance*, 21(10):1675–1698, 2021b. doi: 10.1080/14697688.2021.1905658.

Riccardo Rebonato. *Volatility and Correlation: The Perfect Hedger and the Fox*. John Wiley & Sons, 2nd edition, 2004. ISBN 978-0-470-09139-8.

Mathieu Rosenbaum and Jianfei Zhang. On the universality of the volatility formation process: When machine learning and rough volatility agree. 2022. arXiv preprint 2206.14114.

Neil Shephard and Kevin Sheppard. Realising the future: Forecasting with high-frequency-based volatility (HEAVY) models. *Journal of Applied Econometrics*, 25(2):197–231, 2010. doi: 10.1002/jae.1158.

Justin Sirignano and Rama Cont. Universal features of price formation in financial markets: Perspectives from deep learning. *Quantitative Finance*, 19(9):1449–1459, 2019. doi: 10.1080/14697688.2019.1622295.

Rashmi Korlakai Vinayak and Ran Gilad-Bachrach. DART: Dropouts meet multiple additive regression trees. In *Proceedings of the 18th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 489–497, 2015.

Rui Xin, Chudi Zhong, Zhi Chen, Takuya Takagi, Margo Seltzer, and Cynthia Rudin. Exploring the whole Rashomon set of sparse decision trees. In *Advances in Neural Information Processing Systems*, volume 35, 2022. Oral presentation; arXiv 2209.08040.

Dacheng Xiu. Quasi-maximum likelihood estimation of volatility with high frequency data. *Journal of Econometrics*, 159(1):235–250, 2010. doi: 10.1016/j.jeconom.2010.07.002.

Xiuqin Xu and Ying Chen. Deep stochastic volatility model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 10526–10533, 2021.

Chao Zhang, Mihai Cucuringu, and Xiaowen Dong. Graph neural networks for volatility forecasting. *Working Paper*, 2023.

Chao Zhang, Xingyue Pu, Mihai Cucuringu, and Xiaowen Dong. Graph-based methods for forecasting realized covariances. *Journal of Financial Econometrics*, 22(1):1–32, 2024. doi: 10.1093/jjfinec/nbad012.

Lan Zhang. Efficient estimation of stochastic volatility using noisy observations: A multi-scale approach. *Bernoulli*, 12(6):1019–1043, 2006. doi: 10.3150/bj/1165269149.

Lan Zhang, Per A. Mykland, and Yacine Aït-Sahalia. A tale of two time scales: Determining integrated volatility with noisy high-frequency data. *Journal of the American Statistical Association*, 100(472):1394–1411, 2005. doi: 10.1198/016214505000000169.

Zihao Zhang, Stefan Zohren, and Stephen Roberts. DeepLOB: Deep convolutional neural networks for limit order books. In *IEEE Transactions on Signal Processing*, volume 67, pages 3001–3012, 2019. doi: 10.1109/TSP.2019.2907260.